Data Privacy Office (DPO) Request Management System

Source Code Compilation for Copyright Deposit

A web-based system for the management, review, electronic signing, and archival of Data Privacy Office requests, including Non-Disclosure Agreements (NDA) and Data Sharing Agreements.

Submission Date: April 29, 2026
Total Source Files: 65
Backend (Node.js / Express / MongoDB): 16 files
Frontend (React / Vite): 49 files
Total Lines of Code: 13,502
Primary Languages: JavaScript (Node.js), JSX (React)

This document contains only the original, manually authored source code authored by the development team. Auto-generated files, build artifacts, third-party libraries, configuration files, environment variables, and static assets have been excluded.

Table of Contents

40. `client/src/components/SessionWarningModal.jsx`
41. `client/src/components/SignaturePad.jsx`
42. `client/src/components/RequestStepper.jsx`
43. `client/src/components/InPageFeedbackHost.jsx`
44. `client/src/pages/Landing.jsx`
45. `client/src/pages/ActivateAccountPage.jsx`
46. `client/src/pages/VerifyEmailPage.jsx`
47. `client/src/pages/VerifyDocument.jsx`
48. `client/src/pages/public/RepSigningPage.jsx`
49. `client/src/pages/student/StudentDashboard.jsx`
50. `client/src/pages/student/StudentProfile.jsx`
51. `client/src/pages/student/StudentNewRequest.jsx`
52. `client/src/pages/student/StudentAgreementRequest.jsx`
53. `client/src/pages/student/StudentAgreementRequirements.jsx`
54. `client/src/pages/student/StudentNDATypeChooser.jsx`
55. `client/src/pages/student/StudentNDARequest.jsx`
56. `client/src/pages/student/StudentRequestReview.jsx`
57. `client/src/pages/student/StudentResubmitRequest.jsx`
58. `client/src/pages/admin/AdminDashboard.jsx`
59. `client/src/pages/admin/AdminProfile.jsx`
60. `client/src/pages/admin/AdminUsers.jsx`
61. `client/src/pages/admin/AdminRequests.jsx`
62. `client/src/pages/admin/AdminRequestReview.jsx`
63. `client/src/pages/admin/AdminReports.jsx`
64. `client/src/pages/admin/AdminAuditLog.jsx`
65. `client/src/pages/admin/AdminArchives.jsx`

Backend Source Code

1. server/server.js (134 lines)

```
const express = require("express");
const cors = require("cors");
const helmet = require("helmet");
const compression = require("compression");
require("dotenv").config();

const mongoose = require("mongoose");
const dns = require("node:dns");
const cron = require("node-cron");

// Explicitly set DNS servers before connecting
dns.setServers(["8.8.8.8", "8.8.4.4"]);

const connectDB = require("./config/db");

const authRoutes = require("./routes/authRoutes");
const requestRoutes = require("./routes/requestRoutes");
const auditRoutes = require("./routes/auditRoutes");

const app = express();
const PORT = process.env.PORT || 5000;

// Middleware
const allowedOriginsFromEnv = process.env.ALLOWED_ORIGINS
  ? process.env.ALLOWED_ORIGINS.split(",").map((o) => o.trim().replace(/\/+$/, ""))
  : [];

const defaultLocalOrigins = [
  process.env.CLIENT_URL,
  "http://localhost:5173",
  "http://localhost:5174",
  "http://127.0.0.1:5173",
  "http://127.0.0.1:5174",
]
  .filter(Boolean)
  .map((o) => String(o).trim().replace(/\/+$/, ""));

const allowedOrigins = [...new Set([...allowedOriginsFromEnv, ...defaultLocalOrigins])];

const corsOptions = {
  origin: (origin, callback) => {
    // Allow requests with no origin (mobile apps, server-to-server, curl)
    if (!origin) return callback(null, true);
    const normalizedOrigin = origin.replace(/\/+$/, "");
    if (allowedOrigins.length === 0 || allowedOrigins.includes(normalizedOrigin)) {
      return callback(null, true);
    }
    // Return a CORS deny without throwing a generic 500.
    return callback(null, false);
  },
  credentials: true,
  methods: ["GET", "POST", "PATCH", "PUT", "DELETE", "OPTIONS"],
  allowedHeaders: ["Content-Type", "Authorization"],
};

// Handle preflight for all routes explicitly (required by some edge runtimes)
app.options(/.*/, cors(corsOptions));
app.use(cors(corsOptions));
app.use(helmet());
app.use(compression());
app.use(express.json());

// Express 5 compatibility: sanitize request payloads without reassigning req.query.
const sanitizeForNoSql = (value) => {
  if (Array.isArray(value)) return value.map(sanitizeForNoSql);
  if (!value || typeof value !== "object") return value;

  const cleaned = {};
  for (const [key, val] of Object.entries(value)) {
```

```
    if (key.startsWith("$") || key.includes(".")) continue;
    cleaned[key] = sanitizeForNoSql(val);
  }
  return cleaned;
};

app.use((req, res, next) => {
  if (req.body && typeof req.body === "object") {
    req.body = sanitizeForNoSql(req.body);
  }
  if (req.params && typeof req.params === "object") {
    req.params = sanitizeForNoSql(req.params);
  }
  next();
});


// DB Connection
connectDB();

app.use("/api/auth", authRoutes);
app.use("/api/requests", requestRoutes);
app.use("/api/audit", auditRoutes);

// Centralized error handler to avoid opaque 500 responses.
app.use((err, req, res, next) => {
  if (!err) return next();
  console.error("[Server Error]", err.message);
  return res.status(err.status || 500).json({
    message: err.message || "Internal server error",
  });
});

// ■■ 5-year retention cron job (runs daily at 02:00) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Flags ANY request older than 5 years as isArchived=true, regardless of status
cron.schedule("0 2 * * *", async () => {
  try {
    const fiveYearsAgo = new Date();
    fiveYearsAgo.setFullYear(fiveYearsAgo.getFullYear() - 5);

    const Request = require("./models/Request");
    const result = await Request.updateMany(
      {
        isArchived: false,
        createdAt: { $lte: fiveYearsAgo },
      },
      { $set: { isArchived: true, archivedAt: new Date() } }
    );

    if (result.modifiedCount > 0) {
      console.log(`[Archive Job] Archived ${result.modifiedCount} requests (5-year retention policy).`);
    }
  } catch (err) {
    console.error("[Archive Job] Error:", err.message);
  }
});

// Test route
app.get("/", (req, res) => {
  res.send("DPO System API Running...");
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

2. server/config/db.js (13 lines)

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB Connected:", mongoose.connection.name);
  } catch (error) {
    console.error("DB Connection Error:", error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

3. server/models/User.js (76 lines)

```
const mongoose = require("mongoose");

const PASSWORD_REGEX = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z\d])[A-Za-z\d\W]{8,}$/;

const trustedDeviceSchema = new mongoose.Schema(
  {
    ip: { type: String, required: true },
    userAgent: { type: String, default: "" },
    label: { type: String, default: "" },
    lastOtpVerifiedAt: { type: Date, default: null },
  },
  { _id: false }
);

const userSchema = new mongoose.Schema({
  // Structured name fields (preferred)
  firstName: { type: String, trim: true, default: "" },
  middleInitial: { type: String, trim: true, default: "" },
  lastName: { type: String, trim: true, default: "" },
  // Legacy single-name field kept for backward compatibility
  name: {
    type: String,
    required: true,
    trim: true,
  },
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    trim: true,
  },
  password: {
    type: String,
    required: true,
  },
  role: {
    type: String,
    enum: ["student", "admin", "staff"],
    default: "student",
  },
  isActive: {
    type: Boolean,
    default: true,
  },
  // Email verification
  isVerified: { type: Boolean, default: false },
  verificationToken: { type: String, default: null },
  verificationExpires: { type: Date, default: null },
  // OTP for login verification
  otp: { type: String, default: null },
  otpExpiry: { type: Date, default: null },
  otpLastSentAt: { type: Date, default: null },
  // Password reset
  resetToken: { type: String, default: null },
  resetTokenExpiry: { type: Date, default: null },
  // Program/course
  program: { type: String, trim: true, default: "" },
  // Trusted devices for context-aware OTP
  trustedDevices: { type: [trustedDeviceSchema], default: [] },
}, { timestamps: true });

userSchema.statics.isStrongPassword = (password) => PASSWORD_REGEX.test(String(password || ""));

userSchema.path("password").validate(function validatePasswordStrength(value) {
  const v = String(value || "");
  // Bcrypt hashes are already processed and should pass schema validation.
  if (v.startsWith("$2a$") || v.startsWith("$2b$") || v.startsWith("$2y$")) return true;
  return PASSWORD_REGEX.test(v);
}, "Password does not meet complexity policy");

// Query indexes for admin user management and role-based reporting views.
userSchema.index({ role: 1, isActive: 1, createdAt: -1 });
```

```
74  userSchema.index({ isVerified: 1, createdAt: -1 });
75
76  module.exports = mongoose.model("User", userSchema);
```

4. server/models/Request.js (158 lines)

```
const mongoose = require("mongoose");

const predocSchema = new mongoose.Schema(
  {
    origName:         { type: String },
    url:              { type: String },
    path:             { type: String },
    contentType:      { type: String },
    uploadedAt:       { type: String },
    requirementLabel:   { type: String, default: "" },
    requestFolder:      { type: String, default: "" },
    isResubmission:     { type: Boolean, default: false },
    resubmissionRound:  { type: Number, default: 0 },
  },
  { _id: false }
);

const requestSchema = new mongoose.Schema(
  {
    userId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },

    type: {
      type: String,
      enum: ["nda", "agreement"],
      required: true,
    },

    status: {
      type: String,
      enum: [
        // NDA workflow
        "nda_pending", "nda_approved",
        // Shared NDA & Agreement revision
        "stud_revision_requested",
        // Agreement workflow
        "agr_pending_1", "agr_awaiting_rep_signature", "agr_pending_2", "agr_approved",
        // Agreement exception statuses
        "agr_declined", "agr_rep_revision_requested",
      ],
      default: "nda_pending",
    },

    proxyRequestee: {
      isProxy: { type: Boolean, default: false },
      staffUserId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: "User",
        default: null,
      },
      // Structured name fields for proxy requestee
      firstName: { type: String, default: "" },
      middleInitial: { type: String, default: "" },
      lastName: { type: String, default: "" },
      // Legacy single fullName field kept for backward compatibility
      fullName: { type: String, default: "" },
      email: { type: String, default: "" },
      idNumber: { type: String, default: "" },
      departmentOrOrganization: { type: String, default: "" },
    },

    formData: {
      type: Object,
      required: true,
    },

    predocs: {
      type: [predocSchema],
      default: [],
    },
```

```
74
75      postdocs: {
76        url:              { type: String, default: "" },
77        path:             { type: String, default: "" },
78        issuedAt:         { type: String, default: "" },
79        verificationUrl:  { type: String, default: "" },
80      },
81
82      resubmissionCount: {
83        type: Number,
84        default: 0,
85      },
86
87      remarks: {
88        type: String,
89        default: "",
90      },
91
92      serialNo: {
93        type: String,
94        default: "",
95        unique: true,
96        sparse: true,
97      },
98
99      // Agreement e-signature fields
100     signingToken: {
101       type: String,
102       default: "",
103     },
104
105     signingTokenUsed: {
106       type: Boolean,
107       default: false,
108     },
109
110     signingTokenExpiresAt: {
111       type: Date,
112       default: null,
113     },
114
115     // Representative info (submitted on signing page)
116     repInfo: {
117       name: { type: String, default: "" },
118       govIdDoc: {
119         origName:    { type: String, default: "" },
120         url:         { type: String, default: "" },
121         path:        { type: String, default: "" },
122         contentType: { type: String, default: "" },
123         uploadedAt:  { type: String, default: "" },
124       },
125     },
126
127     // Ephemeral signature storage (deleted after final PDF generation)
128     authorizerSigUrl:  { type: String, default: "" },
129     authorizerSigPath: { type: String, default: "" },
130     repSigUrl:         { type: String, default: "" },
131     repSigPath:        { type: String, default: "" },
132
133     // NDA e-signature fields
134     studentSigUrl:  { type: String, default: "" },
135     studentSigPath: { type: String, default: "" },
136     adminSigUrl:    { type: String, default: "" },
137     adminSigPath:   { type: String, default: "" },
138
139     // Signature timestamps
140     studentSignedAt: { type: Date, default: null },
141     adminSignedAt:   { type: Date, default: null },
142     repSignedAt:     { type: Date, default: null },
143
144     // 5-year retention / archiving
145     isArchived: { type: Boolean, default: false },
146     archivedAt: { type: Date, default: null },
147   },
148   { timestamps: true }
149 );
```

```
150
151  // Query-path indexes used by dashboard/report and owner-specific views.
152  requestSchema.index({ userId: 1, createdAt: -1 });
153  requestSchema.index({ status: 1, createdAt: -1 });
154  requestSchema.index({ isArchived: 1, createdAt: -1 });
155  requestSchema.index({ isArchived: 1, status: 1, createdAt: -1 });
156  requestSchema.index({ type: 1, status: 1, createdAt: -1 });
157
158  module.exports = mongoose.model("Request", requestSchema);
```

5. server/models/AuditLog.js (53 lines)

```
const mongoose = require("mongoose");

/**
 * AuditLog Model
 * Tracks system activity for the System Auditor & Analytics Agent (Agent 3).
 * Stores user actions, document routing history, and login events.
 */
const auditLogSchema = new mongoose.Schema(
  {
    userId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      default: null,
    },

    action: {
      type: String,
      required: true,
      // e.g. "login", "login_failed", "request_created", "request_approved",
      //      "request_revision_required", "document_generated"
    },

    resourceType: {
      type: String,
      default: "",
      // e.g. "request", "user", "template"
    },

    resourceId: {
      type: String,
      default: "",
    },

    details: {
      type: Object,
      default: {},
    },

    ipAddress: {
      type: String,
      default: "",
    },
  },
  { timestamps: true }
);

// Index for efficient querying by user and time window (used by Auditor Agent)
auditLogSchema.index({ userId: 1, createdAt: -1 });
auditLogSchema.index({ action: 1, createdAt: -1 });
auditLogSchema.index({ createdAt: -1 });
auditLogSchema.index({ resourceType: 1, createdAt: -1 });

module.exports = mongoose.model("AuditLog", auditLogSchema);
```

6. server/middleware/authMiddleware.js (39 lines)

```
const jwt = require("jsonwebtoken");

exports.protect = (req, res, next) => {
  const header = req.headers.authorization;

  if (!header) {
    return res.status(401).json({ message: "Not authorized" });
  }

  const match = header.match(/^Bearer\s+(.+)$/i);
  if (!match) {
    return res.status(401).json({ message: "Not authorized" });
  }

  try {
    const decoded = jwt.verify(match[1], process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (error) {
    if (error.name === "TokenExpiredError") {
      return res.status(403).json({ message: "Token expired" });
    }
    res.status(401).json({ message: "Token invalid" });
  }
};

exports.authorizeAdmin = (req, res, next) => {
  if (req.user.role !== "admin") {
    return res.status(403).json({ message: "Admin access only" });
  }
  next();
};

exports.authorizeRoles = (...allowedRoles) => (req, res, next) => {
  if (!req.user || !allowedRoles.includes(req.user.role)) {
    return res.status(403).json({ message: "Forbidden" });
  }
  next();
};
```

7. server/middleware/piiRedactMiddleware.js (69 lines)

```
/**
 * PII Redaction Middleware
 * Strips Personally Identifiable Information (PII) before passing data to AI agents.
 * Per Phase 1 security requirements: names, emails, and specific student IDs must
 * be removed before any data is sent to an LLM or AI processing layer.
 */

const EMAIL_REGEX = /[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}/g;
const PHONE_REGEX = /(\+?\d{1,3}[\s\-.]?)?\(?\d{3}\)?[\s\-.]?\d{3}[\s\-.]?\d{4}/g;
const STUDENT_ID_REGEX = /\b\d{4}[-\s]?\d{4,6}\b/g;

// Field names that are considered PII and should be redacted
const PII_FIELD_NAMES = [
  "name", "email", "studentid", "studentno", "phone", "mobile",
  "address", "fullname", "firstname", "lastname", "contactno",
];

/**
 * Checks if a field key is a PII field based on common naming conventions.
 */
const isPiiFieldName = (key) => {
  const lower = key.toLowerCase();
  return PII_FIELD_NAMES.some((pii) => lower.includes(pii));
};

/**
 * Recursively redact PII from a plain object or string.
 * - Known PII field names are replaced with "[REDACTED]"
 * - Email and phone patterns within string values are also replaced
 */
const redactPII = (data) => {
  if (data === null || data === undefined) return data;

  if (typeof data === "string") {
    return data
      .replace(EMAIL_REGEX, "[EMAIL REDACTED]")
      .replace(PHONE_REGEX, "[PHONE REDACTED]")
      .replace(STUDENT_ID_REGEX, "[ID REDACTED]");
  }

  if (Array.isArray(data)) {
    return data.map((item) => redactPII(item));
  }

  if (typeof data === "object") {
    const result = {};
    for (const [key, value] of Object.entries(data)) {
      if (isPiiFieldName(key)) {
        result[key] = "[REDACTED]";
      } else {
        result[key] = redactPII(value);
      }
    }
    return result;
  }

  return data;
};

/**
 * Express middleware that redacts PII from req.body before AI processing.
 * Attaches the redacted copy to req.redactedBody so controllers can use it.
 */
const piiRedactMiddleware = (req, res, next) => {
  req.redactedBody = redactPII(req.body);
  next();
};

module.exports = { piiRedactMiddleware, redactPII };
```

8. server/controllers/authController.js (920 lines)

```
const crypto = require("crypto");
const User = require("../models/User");
const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");
const AuditLog = require("../models/AuditLog");
const {
  sendOtpEmail,
  sendWelcomeEmail,
  sendLoginOtpEmail,
  sendVerificationOtpEmail,
  sendPasswordChangedAlertEmail,
} = require("../utils/emailService");

// Helper to safely log audit events without blocking the response
const logAudit = (data) => {
  AuditLog.create(data).catch(() => {});
};

const generateOtp = () => String(Math.floor(100000 + Math.random() * 900000));

const getClientIp = (req) => {
  const forwarded = req.headers["x-forwarded-for"];
  if (forwarded) return forwarded.split(",")[0].trim();
  return req.socket?.remoteAddress || req.ip || "";
};

const OTP_TRUST_WINDOW_MS = 72 * 60 * 60 * 1000; // 72 hours
const OTP_RESEND_COOLDOWN_MS = 60 * 1000;
const EMAIL_ERROR_MESSAGE = "Failed to send verification email. Please try again later.";
const PASSWORD_POLICY_MESSAGE = "Password must be at least 8 characters and include uppercase, lowercase, number, and special

const isStrongPassword = (password) => User.isStrongPassword(password);

const buildErrorMessage = (error) => {
  if (error?.isEmailError) {
    return error.publicMessage || EMAIL_ERROR_MESSAGE;
  }
  return error?.message || "Internal server error";
};

const issueAuthPayload = (user) => {
  const token = jwt.sign(
    { id: user._id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: "8h" }
  );

  return {
    token,
    user: { id: user._id, name: user.name, email: user.email, role: user.role },
  };
};

const getOtpCooldownState = (user) => {
  const last = user?.otpLastSentAt ? new Date(user.otpLastSentAt).getTime() : 0;
  const elapsed = Date.now() - last;
  const retryAfterMs = OTP_RESEND_COOLDOWN_MS - elapsed;
  return {
    isCoolingDown: last > 0 && retryAfterMs > 0,
    retryAfterSeconds: Math.max(1, Math.ceil(retryAfterMs / 1000)),
  };
};

const sendVerificationOtpToUser = async (user, { enforceCooldown = false } = {}) => {
  if (enforceCooldown) {
    const { isCoolingDown, retryAfterSeconds } = getOtpCooldownState(user);
    if (isCoolingDown) {
      const err = new Error(`Please wait ${retryAfterSeconds}s before requesting another OTP.`);
      err.statusCode = 429;
      err.retryAfterSeconds = retryAfterSeconds;
      throw err;
    }
  }
```

```
74
75    const otp = generateOtp();
76    user.otp = await bcrypt.hash(otp, 10);
77    user.otpExpiry = new Date(Date.now() + 10 * 60 * 1000);
78    user.otpLastSentAt = new Date();
79    await user.save();
80    await sendVerificationOtpEmail(user.email, user.name, otp);
81  };
82
83  const sendLoginOtpToUser = async (user, { enforceCooldown = false } = {}) => {
84    if (enforceCooldown) {
85      const { isCoolingDown, retryAfterSeconds } = getOtpCooldownState(user);
86      if (isCoolingDown) {
87        const err = new Error(`Please wait ${retryAfterSeconds}s before requesting another OTP.`);
88        err.statusCode = 429;
89        err.retryAfterSeconds = retryAfterSeconds;
90        throw err;
91      }
92    }
93
94    const otp = generateOtp();
95    user.otp = await bcrypt.hash(otp, 10);
96    user.otpExpiry = new Date(Date.now() + 10 * 60 * 1000);
97    user.otpLastSentAt = new Date();
98    await user.save();
99    await sendLoginOtpEmail(user.email, otp);
100 };
101
102 const sendResetOtpToUser = async (user, { enforceCooldown = false } = {}) => {
103   if (enforceCooldown) {
104     const { isCoolingDown, retryAfterSeconds } = getOtpCooldownState(user);
105     if (isCoolingDown) {
106       const err = new Error(`Please wait ${retryAfterSeconds}s before requesting another OTP.`);
107       err.statusCode = 429;
108       err.retryAfterSeconds = retryAfterSeconds;
109       throw err;
110     }
111   }
112
113   const otp = generateOtp();
114   user.otp = await bcrypt.hash(otp, 10);
115   user.otpExpiry = new Date(Date.now() + 10 * 60 * 1000); // 10 minutes
116   user.otpLastSentAt = new Date();
117   user.resetToken = null;
118   user.resetTokenExpiry = null;
119   await user.save();
120   await sendOtpEmail(user.email, otp, "reset");
121 };
122
123 // REGISTER
124 exports.register = async (req, res) => {
125   try {
126     const { firstName, middleInitial, lastName, name, email, password, program } = req.body;
127
128     // Support both structured names and legacy single-name field
129     const resolvedFirstName = String(firstName || "").trim();
130     const resolvedLastName = String(lastName || "").trim();
131     const resolvedMiddleInitial = String(middleInitial || "").trim();
132
133     // Build composite name: prefer structured fields, fall back to legacy name
134     // Middle name is stored in full but displayed as initial (e.g. "Santos" → "S.")
135     const mi = resolvedMiddleInitial ? resolvedMiddleInitial.charAt(0).toUpperCase() + "." : "";
136     const compositeName = resolvedFirstName && resolvedLastName
137       ? `${resolvedFirstName}${mi ? " " + mi : ""} ${resolvedLastName}`
138       : String(name || "").trim();
139
140     if (!compositeName || compositeName.length < 2) {
141       return res.status(400).json({ message: "First name and last name are required" });
142     }
143
144     if (!email || !password) {
145       return res.status(400).json({ message: "Email and password are required" });
146     }
147
148     if (!isStrongPassword(password)) {
149       return res.status(400).json({ message: PASSWORD_POLICY_MESSAGE });
```

```
    }

    const existingUser = await User.findOne({ email: email.toLowerCase() });
    if (existingUser)
      return res.status(400).json({ message: "User already exists" });

    const hashedPassword = await bcrypt.hash(password, 12);

    const otp = generateOtp();
    const hashedOtp = await bcrypt.hash(otp, 10);

    const user = await User.create({
      name: compositeName,
      firstName: resolvedFirstName,
      middleInitial: resolvedMiddleInitial,
      lastName: resolvedLastName,
      email: email.toLowerCase(),
      password: hashedPassword,
      program: String(program || "").trim(),
      isVerified: false,
      otp: hashedOtp,
      otpExpiry: new Date(Date.now() + 10 * 60 * 1000), // 10 min
    });

    await sendVerificationOtpEmail(user.email, user.name, otp);

    logAudit({ userId: user._id, action: "account_created", details: { role: user.role, method: "self_register" } });

    res.status(201).json({ requireVerification: true, email: user.email, message: "Account created! Please check your email f
  } catch (error) {
    res.status(500).json({ message: buildErrorMessage(error) });
  }
};

// VERIFY EMAIL (OTP-based)
exports.verifyEmail = async (req, res) => {
  try {
    const { email, otp, token } = req.body;

    // Support legacy token-based verification for existing links
    if (token) {
      const user = await User.findOne({
        verificationToken: token,
        verificationExpires: { $gt: new Date() },
      });
      if (!user) {
        return res.status(400).json({ message: "Invalid or expired verification link. Please register again or contact an adm
      }
      user.isVerified = true;
      user.verificationToken = null;
      user.verificationExpires = null;
      await user.save();
      logAudit({ userId: user._id, action: "email_verified", details: { email: user.email } });
      return res.json({
        message: "Email verified successfully!",
        ...issueAuthPayload(user),
      });
    }

    // OTP-based verification
    if (!email || !otp) return res.status(400).json({ message: "Email and OTP are required" });

    const user = await User.findOne({ email: email.toLowerCase() });
    if (!user || !user.otp || !user.otpExpiry) {
      return res.status(400).json({ message: "Invalid or expired OTP. Please request a new one." });
    }

    if (user.isVerified) {
      return res.json({
        message: "Email is already verified.",
        ...issueAuthPayload(user),
      });
    }

    if (user.otpExpiry < new Date()) {
      return res.status(400).json({ message: "OTP has expired. Please request a new one." });
```

```
    }

    const otpMatch = await bcrypt.compare(String(otp), user.otp);
    if (!otpMatch) return res.status(400).json({ message: "Invalid OTP" });

    user.isVerified = true;
    user.otp = null;
    user.otpExpiry = null;
    await user.save();

    logAudit({ userId: user._id, action: "email_verified", details: { email: user.email } });

    res.json({
      message: "Email verified successfully!",
      ...issueAuthPayload(user),
    });
  } catch (error) {
    res.status(500).json({ message: buildErrorMessage(error) });
  }
};

// RESEND VERIFICATION OTP
exports.resendVerificationOtp = async (req, res) => {
  try {
    const { email } = req.body;
    if (!email) return res.status(400).json({ message: "Email is required" });

    const user = await User.findOne({ email: email.toLowerCase() });
    if (!user) {
      // Don't reveal whether account exists
      return res.json({ message: "If that account exists, a new OTP has been sent." });
    }

    if (user.isVerified) {
      return res.status(400).json({ message: "This account is already verified. You can log in." });
    }

    await sendVerificationOtpToUser(user, { enforceCooldown: true });

    res.json({ message: "A new verification OTP has been sent to your email." });
  } catch (error) {
    if (error?.statusCode === 429) {
      return res.status(429).json({ message: error.message, retryAfterSeconds: error.retryAfterSeconds });
    }
    res.status(500).json({ message: buildErrorMessage(error) });
  }
};

exports.resendLoginOtp = async (req, res) => {
  try {
    const { email } = req.body;
    const clientIp = getClientIp(req);

    if (!email) return res.status(400).json({ message: "Email is required" });

    const user = await User.findOne({ email: email.toLowerCase() });
    if (!user) {
      return res.status(200).json({ message: "If that account exists, a new OTP has been sent." });
    }

    if (!user.isActive) {
      return res.status(403).json({ message: "Account is deactivated. Contact an administrator." });
    }

    if (!user.isVerified) {
      return res.status(403).json({ message: "Please verify your email first." });
    }

    await sendLoginOtpToUser(user, { enforceCooldown: true });

    logAudit({ userId: user._id, action: "login_otp_sent", details: { ip: clientIp, reason: "resend" }, ipAddress: clientIp }
    return res.status(200).json({ message: "A new login OTP has been sent." });
  } catch (error) {
    if (error?.statusCode === 429) {
      return res.status(429).json({ message: error.message, retryAfterSeconds: error.retryAfterSeconds });
    }
```

```
    return res.status(500).json({ message: buildErrorMessage(error) });
  }
};

// VERIFY EMAIL & SET PASSWORD (for admin-created users without a password)
exports.activateAccount = async (req, res) => {
  try {
    const { token, password } = req.body;
    if (!token || !password) {
      return res.status(400).json({ message: "Token and new password are required" });
    }
    if (!isStrongPassword(password)) {
      return res.status(400).json({ message: PASSWORD_POLICY_MESSAGE });
    }

    const user = await User.findOne({
      verificationToken: token,
      verificationExpires: { $gt: new Date() },
    });

    if (!user) {
      return res.status(400).json({ message: "Invalid or expired activation link." });
    }

    user.password = await bcrypt.hash(password, 12);
    user.isVerified = true;
    user.verificationToken = null;
    user.verificationExpires = null;
    await user.save();

    logAudit({ userId: user._id, action: "account_activated", details: { email: user.email } });

    res.json({ message: "Account activated! You can now log in." });
  } catch (error) {
    res.status(500).json({ message: buildErrorMessage(error) });
  }
};

// LOGIN — Step 1: credentials check + OTP challenge if needed
exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;
    const clientIp = getClientIp(req);
    const userAgent = req.headers["user-agent"] || "";

    if (!email || !password) {
      return res.status(400).json({ message: "Email and password are required" });
    }

    const user = await User.findOne({ email: email.toLowerCase() });
    if (!user) {
      logAudit({ action: "login_failed", details: { email: email.toLowerCase(), reason: "user_not_found" }, ipAddress: client
      return res.status(401).json({ message: "Invalid email or password" });
    }

    if (!user.isActive) {
      return res.status(403).json({ message: "Account is deactivated. Contact an administrator." });
    }

    if (!user.isVerified) {
      await sendVerificationOtpToUser(user, { enforceCooldown: true });
      logAudit({ userId: user._id, action: "verification_otp_sent", details: { reason: "login_intercept" }, ipAddress: client
      return res.status(403).json({
        requireVerification: true,
        email: user.email,
        message: "Your email is not verified yet. We sent a verification OTP to your email.",
      });
    }

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      logAudit({ userId: user._id, action: "login_failed", details: { reason: "wrong_password" }, ipAddress: clientIp });
      return res.status(401).json({ message: "Invalid email or password" });
    }

    // Check if this device/IP is trusted and within 72h window
```

```
    const trustedDevice = user.trustedDevices.find((d) => d.ip === clientIp);
    const isWithinWindow = trustedDevice?.lastOtpVerifiedAt &&
      (Date.now() - trustedDevice.lastOtpVerifiedAt.getTime()) < OTP_TRUST_WINDOW_MS;

    if (trustedDevice && isWithinWindow) {
      // Device is trusted → issue token directly
      const token = jwt.sign(
        { id: user._id, role: user.role },
        process.env.JWT_SECRET,
        { expiresIn: "8h" }
      );

      logAudit({ userId: user._id, action: "login", details: { role: user.role, ip: clientIp }, ipAddress: clientIp });

      return res.json({
        token,
        user: { id: user._id, name: user.name, email: user.email, role: user.role },
      });
    }

    // Device not trusted or expired → send OTP
    await sendLoginOtpToUser(user, { enforceCooldown: true });

    logAudit({ userId: user._id, action: "login_otp_sent", details: { ip: clientIp }, ipAddress: clientIp });

    return res.json({
      requireOtp: true,
      message: "A verification code has been sent to your email.",
    });
  } catch (error) {
    if (error?.statusCode === 429) {
      return res.status(429).json({ message: error.message, retryAfterSeconds: error.retryAfterSeconds });
    }
    res.status(500).json({ message: buildErrorMessage(error) });
  }
};

// LOGIN — Step 2: verify login OTP
exports.verifyLoginOtp = async (req, res) => {
  try {
    const { email, otp } = req.body;
    const clientIp = getClientIp(req);
    const userAgent = req.headers["user-agent"] || "";

    if (!email || !otp) return res.status(400).json({ message: "Email and OTP are required" });

    const user = await User.findOne({ email: email.toLowerCase() });
    if (!user || !user.otp || !user.otpExpiry) {
      return res.status(400).json({ message: "Invalid or expired OTP" });
    }

    if (user.otpExpiry < new Date()) {
      return res.status(400).json({ message: "OTP has expired. Please log in again." });
    }

    const otpMatch = await bcrypt.compare(String(otp), user.otp);
    if (!otpMatch) return res.status(400).json({ message: "Invalid OTP" });

    // Clear OTP
    user.otp = null;
    user.otpExpiry = null;

    // Trust this device
    const existingIdx = user.trustedDevices.findIndex((d) => d.ip === clientIp);
    const deviceEntry = {
      ip: clientIp,
      userAgent: userAgent.substring(0, 200),
      label: userAgent.substring(0, 60),
      lastOtpVerifiedAt: new Date(),
    };

    if (existingIdx >= 0) {
      user.trustedDevices[existingIdx] = deviceEntry;
    } else {
      // Keep max 10 trusted devices
      if (user.trustedDevices.length >= 10) {
```

```
        user.trustedDevices.sort((a, b) => (a.lastOtpVerifiedAt || 0) - (b.lastOtpVerifiedAt || 0));
        user.trustedDevices.shift();
      }
      user.trustedDevices.push(deviceEntry);
    }

    await user.save();

    const token = jwt.sign(
      { id: user._id, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: "8h" }
    );

    logAudit({ userId: user._id, action: "login", details: { role: user.role, ip: clientIp, method: "otp" }, ipAddress: clien

    return res.json({
      token,
      user: { id: user._id, name: user.name, email: user.email, role: user.role },
    });
  } catch (error) {
    res.status(500).json({ message: buildErrorMessage(error) });
  }
};

// REFRESH TOKEN (for session extension)
exports.refreshToken = async (req, res) => {
  try {
    const user = await User.findById(req.user.id).select("_id role name email");
    if (!user) return res.status(404).json({ message: "User not found" });

    const token = jwt.sign(
      { id: user._id, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: "8h" }
    );

    res.json({ token, user: { id: user._id, name: user.name, email: user.email, role: user.role } });
  } catch (error) {
    res.status(500).json({ message: buildErrorMessage(error) });
  }
};

// REQUEST OTP FOR PASSWORD CHANGE
exports.requestPasswordChangeOtp = async (req, res) => {
  try {
    const user = await User.findById(req.user.id);
    if (!user) return res.status(404).json({ message: "User not found" });

    const { isCoolingDown, retryAfterSeconds } = getOtpCooldownState(user);
    if (isCoolingDown) {
      return res.status(429).json({ message: `Please wait ${retryAfterSeconds}s before requesting another OTP.`, retryAfterSe
    }

    const otp = generateOtp();
    user.otp = await bcrypt.hash(otp, 10);
    user.otpExpiry = new Date(Date.now() + 10 * 60 * 1000);
    user.otpLastSentAt = new Date();
    await user.save();
    await sendOtpEmail(user.email, otp, "change");

    res.json({ message: "An OTP has been sent to your email." });
  } catch (error) {
    res.status(500).json({ message: buildErrorMessage(error) });
  }
};

// UPDATE PROFILE (name & password)
exports.updateProfile = async (req, res) => {
  try {
    const { firstName, middleInitial, lastName, name, currentPassword, newPassword, otp } = req.body;
    const user = await User.findById(req.user.id);
    if (!user) return res.status(404).json({ message: "User not found" });

    let updated = false;

```

```
    // Update name - support both structured and legacy fields
    const resolvedFirstName = String(firstName || "").trim();
    const resolvedLastName = String(lastName || "").trim();
    const resolvedMiddleInitial = String(middleInitial || "").trim();

    if (resolvedFirstName && resolvedLastName) {
      user.firstName = resolvedFirstName;
      user.middleInitial = resolvedMiddleInitial;
      user.lastName = resolvedLastName;
      const miUp = resolvedMiddleInitial ? resolvedMiddleInitial.charAt(0).toUpperCase() + "." : "";
      user.name = `${resolvedFirstName}${miUp ? " " + miUp : ""} ${resolvedLastName}`;
      updated = true;
    } else if (name && String(name).trim().length >= 2) {
      user.name = String(name).trim();
      updated = true;
    }

    // Update password
    if (newPassword) {
      if (!currentPassword) {
        return res.status(400).json({ message: "Current password is required to set a new password" });
      }
      if (!otp) {
        return res.status(400).json({ message: "OTP verification is required to change password" });
      }
      if (!isStrongPassword(newPassword)) {
        return res.status(400).json({ message: PASSWORD_POLICY_MESSAGE });
      }
      const isMatch = await bcrypt.compare(currentPassword, user.password);
      if (!isMatch) {
        return res.status(400).json({ message: "Current password is incorrect" });
      }
      if (!user.otp || !user.otpExpiry) {
        return res.status(400).json({ message: "No OTP found. Please request a new one." });
      }
      if (user.otpExpiry < new Date()) {
        return res.status(400).json({ message: "OTP has expired. Please request a new one." });
      }
      const otpMatch = await bcrypt.compare(String(otp), user.otp);
      if (!otpMatch) {
        return res.status(400).json({ message: "Invalid OTP. Please try again." });
      }
      user.otp = null;
      user.otpExpiry = null;
      user.password = await bcrypt.hash(newPassword, 12);
      updated = true;
      logAudit({ userId: user._id, action: "password_changed", details: { method: "profile" } });
      await sendPasswordChangedAlertEmail(user.email, user.name, "self-service change");
    }

    if (!updated) {
      return res.status(400).json({ message: "No changes provided" });
    }

    await user.save();

    // Return updated user info (so frontend can update context)
    res.json({
      message: "Profile updated successfully",
      user: { id: user._id, name: user.name, email: user.email, role: user.role },
    });
  } catch (error) {
    res.status(500).json({ message: buildErrorMessage(error) });
  }
};

// FORGOT PASSWORD – sends OTP to email
exports.forgotPassword = async (req, res) => {
  try {
    const { email } = req.body;
    if (!email) return res.status(400).json({ message: "Email is required" });

    const user = await User.findOne({ email: email.toLowerCase() });
    // Always respond 200 to prevent email enumeration
    if (!user) return res.status(200).json({ message: "If that account exists, an OTP has been sent." });

```

```
    await sendResetOtpToUser(user, { enforceCooldown: true });
    logAudit({ userId: user._id, action: "password_reset_requested", details: { email: user.email } });

    res.status(200).json({ message: "If that account exists, an OTP has been sent." });
  } catch (error) {
    if (error?.statusCode === 429) {
      return res.status(429).json({ message: error.message, retryAfterSeconds: error.retryAfterSeconds });
    }
    res.status(500).json({ message: buildErrorMessage(error) });
  }
};

exports.resendResetOtp = async (req, res) => {
  try {
    const { email } = req.body;
    if (!email) return res.status(400).json({ message: "Email is required" });

    const user = await User.findOne({ email: email.toLowerCase() });
    if (!user) {
      return res.status(200).json({ message: "If that account exists, an OTP has been sent." });
    }

    await sendResetOtpToUser(user, { enforceCooldown: true });
    logAudit({ userId: user._id, action: "password_reset_requested", details: { email: user.email, reason: "resend" } });
    return res.status(200).json({ message: "A new password reset OTP has been sent." });
  } catch (error) {
    if (error?.statusCode === 429) {
      return res.status(429).json({ message: error.message, retryAfterSeconds: error.retryAfterSeconds });
    }
    return res.status(500).json({ message: buildErrorMessage(error) });
  }
};

// VERIFY RESET OTP – returns a short-lived reset token
exports.verifyResetOtp = async (req, res) => {
  try {
    const { email, otp } = req.body;
    if (!email || !otp) return res.status(400).json({ message: "Email and OTP are required" });

    const user = await User.findOne({ email: email.toLowerCase() });
    if (!user || !user.otp || !user.otpExpiry)
      return res.status(400).json({ message: "Invalid or expired OTP" });

    if (user.otpExpiry < new Date())
      return res.status(400).json({ message: "OTP has expired. Please request a new one." });

    const otpMatch = await bcrypt.compare(String(otp), user.otp);
    if (!otpMatch) return res.status(400).json({ message: "Invalid OTP" });

    const resetToken = crypto.randomBytes(32).toString("hex");
    user.resetToken = await bcrypt.hash(resetToken, 10);
    user.resetTokenExpiry = new Date(Date.now() + 15 * 60 * 1000); // 15 min
    user.otp = null;
    user.otpExpiry = null;
    await user.save();

    res.json({ resetToken, message: "OTP verified" });
  } catch (error) {
    res.status(500).json({ message: buildErrorMessage(error) });
  }
};

// RESET PASSWORD
exports.resetPassword = async (req, res) => {
  try {
    const { email, resetToken, newPassword } = req.body;
    if (!email || !resetToken || !newPassword)
      return res.status(400).json({ message: "Email, reset token, and new password are required" });

    if (!isStrongPassword(newPassword)) {
      return res.status(400).json({ message: PASSWORD_POLICY_MESSAGE });
    }

    const user = await User.findOne({ email: email.toLowerCase() });
    if (!user || !user.resetToken || !user.resetTokenExpiry)
      return res.status(400).json({ message: "Invalid or expired reset token" });
```

```
682
683     if (user.resetTokenExpiry < new Date())
684       return res.status(400).json({ message: "Reset token has expired" });
685
686     const tokenMatch = await bcrypt.compare(resetToken, user.resetToken);
687     if (!tokenMatch) return res.status(400).json({ message: "Invalid reset token" });
688
689     user.password = await bcrypt.hash(newPassword, 12);
690     user.resetToken = null;
691     user.resetTokenExpiry = null;
692     await user.save();
693
694     logAudit({ userId: user._id, action: "password_changed", details: { method: "reset" } });
695     await sendPasswordChangedAlertEmail(user.email, user.name, "account recovery reset");
696
697     res.json({ message: "Password reset successfully" });
698   } catch (error) {
699     res.status(500).json({ message: buildErrorMessage(error) });
700   }
701 };
702
703 // ADMIN: Get all users
704 exports.getAllUsers = async (req, res) => {
705   try {
706     const users = await User.find()
707       .select("-password -otp -otpExpiry -otpLastSentAt -resetToken -resetTokenExpiry -verificationToken -trustedDevices")
708       .sort({ createdAt: -1 })
709       .lean();
710     res.json(users);
711   } catch (error) {
712     res.status(500).json({ message: buildErrorMessage(error) });
713   }
714 };
715
716 // ADMIN: Create user (with verification email)
717 exports.adminCreateUser = async (req, res) => {
718   try {
719     const { name, email, role, password } = req.body;
720
721     if (!name || !email || !password) return res.status(400).json({ message: "Name, email, role, and temporary password are r
722     if (!["student", "admin", "staff"].includes(role))
723       return res.status(400).json({ message: "Invalid role" });
724     if (!isStrongPassword(password)) {
725       return res.status(400).json({ message: PASSWORD_POLICY_MESSAGE });
726     }
727
728     const existing = await User.findOne({ email: email.toLowerCase() });
729     if (existing) return res.status(400).json({ message: "User with this email already exists" });
730
731     const hashedPassword = await bcrypt.hash(password, 12);
732
733     const user = await User.create({
734       name: String(name).trim(),
735       email: email.toLowerCase(),
736       password: hashedPassword,
737       role: role || "student",
738       isVerified: false,
739       verificationToken: null,
740       verificationExpires: null,
741     });
742
743     await sendVerificationOtpToUser(user);
744     await sendWelcomeEmail(user.email, user.name, password);
745
746     logAudit({
747       userId: req.user.id,
748       action: "account_created",
749       resourceType: "user",
750       resourceId: String(user._id),
751       details: { createdBy: "admin", targetEmail: user.email, role: user.role, tempPasswordSet: true },
752     });
753
754     res.status(201).json({
755       message: "User created. A temporary password and verification OTP were sent by email.",
756       user: { id: user._id, name: user.name, email: user.email, role: user.role },
757     });
```

```
  } catch (error) {
    res.status(500).json({ message: buildErrorMessage(error) });
  }
};

// ADMIN: Toggle user active/inactive
exports.toggleUserActive = async (req, res) => {
  try {
    const user = await User.findById(req.params.id).select("-password -otp -resetToken");
    if (!user) return res.status(404).json({ message: "User not found" });

    if (String(user._id) === String(req.user.id))
      return res.status(400).json({ message: "You cannot deactivate your own account" });

    user.isActive = !user.isActive;
    await user.save();

    logAudit({
      userId: req.user.id,
      action: user.isActive ? "user_activated" : "user_deactivated",
      resourceType: "user",
      resourceId: String(user._id),
      details: { targetEmail: user.email },
    });

    res.json({ message: `User ${user.isActive ? "activated" : "deactivated"}`, isActive: user.isActive });
  } catch (error) {
    res.status(500).json({ message: buildErrorMessage(error) });
  }
};

// ADMIN: Edit user details and role
exports.adminUpdateUser = async (req, res) => {
  try {
    const { name, email, role, isActive } = req.body;
    const user = await User.findById(req.params.id);
    if (!user) return res.status(404).json({ message: "User not found" });

    const updates = {};

    if (name !== undefined) {
      const trimmedName = String(name || "").trim();
      if (trimmedName.length < 2) {
        return res.status(400).json({ message: "Name must be at least 2 characters" });
      }
      updates.name = trimmedName;
    }

    if (email !== undefined) {
      const normalizedEmail = String(email || "").trim().toLowerCase();
      if (!normalizedEmail) return res.status(400).json({ message: "Email is required" });
      const duplicate = await User.findOne({ email: normalizedEmail, _id: { $ne: user._id } });
      if (duplicate) return res.status(400).json({ message: "Another user already uses this email" });
      updates.email = normalizedEmail;
    }

    if (role !== undefined) {
      if (![
        "student",
        "staff",
        "admin",
      ].includes(role)) {
        return res.status(400).json({ message: "Invalid role" });
      }
      updates.role = role;
    }

    if (isActive !== undefined) {
      if (String(user._id) === String(req.user.id) && isActive === false) {
        return res.status(400).json({ message: "You cannot deactivate your own account" });
      }
      updates.isActive = Boolean(isActive);
    }

    Object.assign(user, updates);
    await user.save();
```

```
834
835     logAudit({
836       userId: req.user.id,
837       action: "user_updated",
838       resourceType: "user",
839       resourceId: String(user._id),
840       details: { fields: Object.keys(updates), targetEmail: user.email },
841     });
842
843     return res.json({
844       message: "User updated successfully",
845       user: {
846         id: user._id,
847         name: user.name,
848         email: user.email,
849         role: user.role,
850         isActive: user.isActive,
851         isVerified: user.isVerified,
852       },
853     });
854   } catch (error) {
855     return res.status(500).json({ message: buildErrorMessage(error) });
856   }
857 };
858
859 // ADMIN: Permanently delete user
860 exports.adminDeleteUser = async (req, res) => {
861   try {
862     const user = await User.findById(req.params.id);
863     if (!user) return res.status(404).json({ message: "User not found" });
864     if (String(user._id) === String(req.user.id)) {
865       return res.status(400).json({ message: "You cannot delete your own account" });
866     }
867
868     await User.deleteOne({ _id: user._id });
869
870     logAudit({
871       userId: req.user.id,
872       action: "user_deleted",
873       resourceType: "user",
874       resourceId: String(user._id),
875       details: { targetEmail: user.email },
876     });
877
878     return res.json({ message: "User deleted successfully" });
879   } catch (error) {
880     return res.status(500).json({ message: buildErrorMessage(error) });
881   }
882 };
883
884 // ADMIN: Reset password with admin-defined temporary password
885 exports.adminResetUserPassword = async (req, res) => {
886   try {
887     const { temporaryPassword } = req.body;
888     if (!isStrongPassword(temporaryPassword)) {
889       return res.status(400).json({ message: PASSWORD_POLICY_MESSAGE });
890     }
891
892     const user = await User.findById(req.params.id);
893     if (!user) return res.status(404).json({ message: "User not found" });
894
895     user.password = await bcrypt.hash(temporaryPassword, 12);
896     user.isVerified = false;
897     user.otp = null;
898     user.otpExpiry = null;
899     user.resetToken = null;
900     user.resetTokenExpiry = null;
901     user.trustedDevices = [];
902     await user.save();
903
904     await sendVerificationOtpToUser(user);
905     await sendWelcomeEmail(user.email, user.name, temporaryPassword);
906     await sendPasswordChangedAlertEmail(user.email, user.name, "admin temporary reset");
907
908     logAudit({
909       userId: req.user.id,
```

```
910         action: "password_reset_triggered_by_admin",
911         resourceType: "user",
912         resourceId: String(user._id),
913         details: { targetEmail: user.email, forcedReverify: true },
914       });
915
916       return res.json({ message: `Temporary password set. Security alerts sent to ${user.email}.` });
917     } catch (error) {
918       return res.status(500).json({ message: buildErrorMessage(error) });
919     }
920   };
```

9. server/controllers/requestController.js (996 lines)

```
const crypto = require("crypto");
const mongoose = require("mongoose");
const Request = require("../models/Request");
const User = require("../models/User");
const AuditLog = require("../models/AuditLog");
const { sendStatusUpdateEmail, sendSigningLinkEmail, sendAgreementApprovedEmail } = require("../utils/emailService");

const logAudit = (data) => {
  AuditLog.create(data).catch(() => {});
};

const STATUS_GROUPS = {
  reviewal: ["nda_pending", "agr_pending_1", "agr_pending_2"],
  approved: ["nda_approved", "agr_approved"],
  stud_revision: ["stud_revision_requested"],
  rep_revision: ["agr_rep_revision_requested"],
  rep_declined: ["agr_declined"],
  awaiting_rep: ["agr_awaiting_rep_signature"],
};

const resolveStatusFilter = (statusQuery) => {
  if (!statusQuery) return [];

  const tokens = String(statusQuery)
    .split(",")
    .map((v) => v.trim())
    .filter(Boolean);

  const resolved = tokens.flatMap((token) => STATUS_GROUPS[token] || [token]);
  return [...new Set(resolved)];
};

const escapeRegex = (value) => String(value).replace(/[.*+?^${}()|[\]\\]/g, "\\$&");

const SERIAL_SUFFIX_CHARS = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";

const pad2 = (n) => String(n).padStart(2, "0");

const getSerialDatePart = (date = new Date()) => {
  const year = date.getFullYear();
  const month = pad2(date.getMonth() + 1);
  const day = pad2(date.getDate());
  return `${year}-${month}-${day}`;
};

const generateSerialSuffix = () => {
  let suffix = "";
  for (let i = 0; i < 4; i += 1) {
    const idx = crypto.randomInt(0, SERIAL_SUFFIX_CHARS.length);
    suffix += SERIAL_SUFFIX_CHARS[idx];
  }
  return suffix;
};

const buildSerialNo = (type) => {
  const prefix = type === "nda" ? "NDA" : "DATA";
  return `${prefix}-${getSerialDatePart()}-${generateSerialSuffix()}`;
};

const generateUniqueSerialNo = async (type) => {
  const maxAttempts = 12;
  for (let attempt = 0; attempt < maxAttempts; attempt += 1) {
    const serialNo = buildSerialNo(type);
    const exists = await Request.exists({ serialNo });
    if (!exists) return serialNo;
  }

  throw new Error("Failed to generate a unique serial number");
};

const generateSigningTokenValue = () =>
  crypto.randomBytes(20).toString("hex");

```

```
// ■■■ Student ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

exports.createRequest = async (req, res) => {
  try {
    if (!["student", "staff"].includes(req.user.role)) {
      return res.status(403).json({ message: "Only student and staff accounts can create requests" });
    }

    const {
      type,
      formData,
      predocs,
      authorizerSigUrl,
      authorizerSigPath,
      studentSigUrl,
      studentSigPath,
      proxyRequestee,
    } = req.body;

    if (!type || !["nda", "agreement"].includes(type)) {
      return res.status(400).json({ message: "Invalid type" });
    }

    if (!formData || typeof formData !== "object") {
      return res.status(400).json({ message: "formData must be an object" });
    }

    const initialStatus = type === "agreement" ? "agr_pending_1" : "nda_pending";
    const isStaffProxy = req.user.role === "staff";

    if (isStaffProxy) {
      // Support both structured name fields and legacy fullName
      const hasStructuredName = proxyRequestee &&
        String(proxyRequestee.firstName || "").trim() &&
        String(proxyRequestee.lastName || "").trim();
      const hasLegacyName = proxyRequestee && String(proxyRequestee.fullName || "").trim();

      const requiredNonNameFields = ["email", "idNumber", "departmentOrOrganization"];
      if (!proxyRequestee || typeof proxyRequestee !== "object") {
        return res.status(400).json({ message: "Proxy requestee details are required for staff submissions" });
      }

      if (!hasStructuredName && !hasLegacyName) {
        return res.status(400).json({ message: "Proxy requestee first name and last name are required" });
      }

      for (const field of requiredNonNameFields) {
        if (!String(proxyRequestee[field] || "").trim()) {
          return res.status(400).json({ message: `Proxy requestee ${field} is required` });
        }
      }
    }

    // Compute proxy requestee full name from structured or legacy fields
    const buildProxyFullName = (pr) => {
      const fn = String(pr.firstName || "").trim();
      const miRaw = String(pr.middleInitial || "").trim();
      const mi = miRaw ? miRaw.charAt(0).toUpperCase() + "." : "";
      const ln = String(pr.lastName || "").trim();
      if (fn && ln) return `${fn}${mi ? " " + mi : ""} ${ln}`;
      return String(pr.fullName || "").trim();
    };

    const createPayload = {
      userId: req.user.id,
      type,
      status: initialStatus,
      formData,
      predocs: Array.isArray(predocs) ? predocs : [],
      proxyRequestee: isStaffProxy
        ? {
            isProxy: true,
            staffUserId: req.user.id,
            firstName: String(proxyRequestee.firstName || "").trim(),
            middleInitial: String(proxyRequestee.middleInitial || "").trim(),
            lastName: String(proxyRequestee.lastName || "").trim(),
```

```
            fullName: buildProxyFullName(proxyRequestee),
            email: String(proxyRequestee.email || "").trim(),
            idNumber: String(proxyRequestee.idNumber || "").trim(),
            departmentOrOrganization: String(proxyRequestee.departmentOrOrganization || "").trim(),
          }
        : {
            isProxy: false,
            staffUserId: null,
            firstName: "",
            middleInitial: "",
            lastName: "",
            fullName: "",
            email: "",
            idNumber: "",
            departmentOrOrganization: "",
          },
    };

    if (type === "agreement") {
      createPayload.authorizerSigUrl = authorizerSigUrl || "";
      createPayload.authorizerSigPath = authorizerSigPath || "";
      createPayload.studentSignedAt = new Date();
    }

    if (type === "nda") {
      createPayload.studentSigUrl = studentSigUrl || "";
      createPayload.studentSigPath = studentSigPath || "";
      createPayload.studentSignedAt = new Date();
    }

    const created = await Request.create(createPayload);

    logAudit({
      userId: req.user.id,
      action: "request_created",
      resourceType: "request",
      resourceId: String(created._id),
      details: { type, status: initialStatus, isProxy: isStaffProxy },
    });

    return res.status(201).json({ message: "Request submitted", request: created });
  } catch (err) {
    return res.status(500).json({ message: err.message });
  }
};

exports.getMyRequests = async (req, res) => {
  try {
    const list = await Request.find({ userId: req.user.id, isArchived: { $ne: true } })
      .sort({ createdAt: -1 })
      .select("_id type status formData createdAt proxyRequestee postdocs isArchived")
      .lean();

    return res.json(list);
  } catch (err) {
    return res.status(500).json({ message: err.message });
  }
};

exports.resubmitRequest = async (req, res) => {
  try {
    const { formData, predocs, authorizerSigUrl, authorizerSigPath, studentSigUrl, studentSigPath } = req.body;

    const r = await Request.findById(req.params.id);
    if (!r) return res.status(404).json({ message: "Request not found" });

    if (String(r.userId) !== String(req.user.id)) {
      return res.status(403).json({ message: "Forbidden" });
    }

    if (r.status !== "stud_revision_requested") {
      return res.status(400).json({ message: "Only revision-requested requests can be resubmitted" });
    }

    r.formData = formData && typeof formData === "object" ? formData : r.formData;
    r.resubmissionCount = (r.resubmissionCount || 0) + 1;
```

```
    const markedNew = Array.isArray(predocs)
      ? predocs.map((p) => ({ ...p, isResubmission: true, resubmissionRound: r.resubmissionCount }))
      : [];
    r.predocs = [...(r.predocs || []), ...markedNew];
    r.status = r.type === "agreement" ? "agr_pending_1" : "nda_pending";
    r.remarks = "";
    r.postdocs = { url: "", path: "", issuedAt: "", verificationUrl: "" };

    if (r.type === "agreement" && authorizerSigUrl) {
      r.authorizerSigUrl = authorizerSigUrl;
      r.authorizerSigPath = authorizerSigPath || "";
      r.studentSignedAt = new Date();
    }

    if (r.type === "nda" && studentSigUrl) {
      r.studentSigUrl = studentSigUrl;
      r.studentSigPath = studentSigPath || "";
      r.studentSignedAt = new Date();
    }

    await r.save();

    logAudit({
      userId: req.user.id,
      action: "request_resubmitted",
      resourceType: "request",
      resourceId: String(r._id),
      details: { type: r.type, newStatus: r.status },
    });

    return res.json({ message: "Resubmitted", request: r });
  } catch (err) {
    return res.status(500).json({ message: err.message });
  }
};

// ■■■ Admin ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

exports.getAllRequests = async (req, res) => {
  try {
    const { startDate, endDate, status, includeArchived, search, type, ndaType } = req.query;

    const query = {};

    // Active workflow should exclude archived records unless explicitly requested.
    if (String(includeArchived) !== "true") {
      query.isArchived = { $ne: true };
    }

    const resolvedStatuses = resolveStatusFilter(status);
    if (resolvedStatuses.length) {
      query.status = { $in: resolvedStatuses };
    }

    if (type) {
      query.type = type;
    }
    if (ndaType) {
      query["formData.ndaType"] = ndaType;
    }

    if (startDate || endDate) {
      query.createdAt = {};

      if (startDate) {
        const parsedStart = new Date(startDate);
        if (Number.isNaN(parsedStart.getTime())) {
          return res.status(400).json({ message: "Invalid startDate" });
        }
        parsedStart.setHours(0, 0, 0, 0);
        query.createdAt.$gte = parsedStart;
      }

      if (endDate) {
        const parsedEnd = new Date(endDate);
        if (Number.isNaN(parsedEnd.getTime())) {
```

```
302           return res.status(400).json({ message: "Invalid endDate" });
303         }
304         parsedEnd.setHours(23, 59, 59, 999);
305         query.createdAt.$lte = parsedEnd;
306       }
307     }
308
309     if (search && String(search).trim()) {
310       const term = String(search).trim();
311       const regex = new RegExp(escapeRegex(term), "i");
312
313       const userOr = [
314         { name: regex },
315         { email: regex },
316       ];
317
318       if (mongoose.Types.ObjectId.isValid(term)) {
319         userOr.push({ _id: term });
320       }
321
322       const matchingUsers = await User.find({ $or: userOr }).select("_id");
323       const matchedUserIds = matchingUsers.map((u) => u._id);
324
325       const searchableFormFields = [
326         "formData.purpose",
327         "formData.organizationName",
328         "formData.organization",
329         "formData.orgName",
330         "formData.tags",
331         "formData.title",
332         "formData.projectTitle",
333         "formData.ndaTypeLabel",
334         "proxyRequestee.fullName",
335         "proxyRequestee.email",
336         "proxyRequestee.idNumber",
337         "proxyRequestee.departmentOrOrganization",
338       ];
339
340       const formDataClauses = searchableFormFields.map((field) => ({ [field]: regex }));
341
342       const searchClauses = [
343         { serialNo: regex },
344         ...formDataClauses,
345       ];
346
347       if (matchedUserIds.length) {
348         searchClauses.push({ userId: { $in: matchedUserIds } });
349       }
350
351       if (mongoose.Types.ObjectId.isValid(term)) {
352         searchClauses.push({ _id: term });
353       }
354
355       // Partial Request ID match (last 7 chars of hex _id, case-insensitive)
356       searchClauses.push({
357         $expr: {
358           $regexMatch: {
359             input: { $toString: "$_id" },
360             regex: term,
361             options: "i",
362           },
363         },
364       });
365
366       query.$or = searchClauses;
367     }
368
369     const list = await Request.find(query)
370       .populate("userId", "name email role")
371       .sort({ createdAt: -1 })
372       .select("-__v")
373       .lean();
374
375     return res.json(list);
376   } catch (err) {
377     return res.status(500).json({ message: err.message });
```

```
  }
};

exports.getRequestById = async (req, res) => {
  try {
    const r = await Request.findById(req.params.id)
      .populate("userId", "name email role")
      .select("-__v");

    if (!r) return res.status(404).json({ message: "Request not found" });

    const isPrivilegedViewer = req.user.role === "admin" || req.user.role === "staff";
    const isAdmin = req.user.role === "admin";
    const ownerId = r.userId?._id || r.userId;
    const isOwner = String(ownerId) === String(req.user.id);

    if (!isPrivilegedViewer && !isOwner) return res.status(403).json({ message: "Forbidden" });

    // Strip signing token from non-privileged (student) responses for security
    if (!isPrivilegedViewer) {
      const obj = r.toObject();
      delete obj.signingToken;
      return res.json(obj);
    }

    return res.json(r);
  } catch (err) {
    return res.status(500).json({ message: err.message });
  }
};

// Admin-only workflow updates for NDA and initial Agreement review phases.
exports.updateRequestStatus = async (req, res) => {
  try {
    const { id } = req.params;
    const { status, remarks, adminSigUrl, adminSigPath } = req.body;

    const allowedStatuses = [
      "nda_approved",
      "stud_revision_requested",
    ];
    if (!status || !allowedStatuses.includes(status)) {
      return res.status(400).json({ message: "Invalid status" });
    }

    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({ message: "Invalid request id" });
    }

    const existing = await Request.findById(id).select("type status serialNo");
    if (!existing) return res.status(404).json({ message: "Request not found" });

    const isNda = existing.type === "nda";
    const isAgreement = existing.type === "agreement";

    if (isNda) {
      const allowedFrom = ["nda_pending"];
      if (!allowedFrom.includes(existing.status)) {
        return res.status(400).json({ message: "Only pending NDA requests can be updated here" });
      }
      const allowedTargets = ["nda_approved", "stud_revision_requested"];
      if (!allowedTargets.includes(status)) {
        return res.status(400).json({ message: "Invalid target status for NDA" });
      }
    }

    if (isAgreement) {
      const allowedFrom = ["agr_pending_1"];
      if (!allowedFrom.includes(existing.status)) {
        return res.status(400).json({ message: "Use agreement-specific endpoints for this stage" });
      }
      const allowedTargets = ["stud_revision_requested"];
      if (!allowedTargets.includes(status)) {
        return res.status(400).json({ message: "Invalid target status for Agreement initial review" });
      }
    }
```

```
454
455     const updatePayload = { status, remarks: remarks || "" };
456
457     if (status === "nda_approved" && !existing.serialNo) {
458       updatePayload.serialNo = await generateUniqueSerialNo("nda");
459     }
460
461     // Store admin signature for NDA approvals
462     if (status === "nda_approved" && existing.type === "nda" && adminSigUrl) {
463       updatePayload.adminSigUrl = adminSigUrl;
464       updatePayload.adminSigPath = adminSigPath || "";
465       updatePayload.adminSignedAt = new Date();
466     }
467
468     const updated = await Request.findByIdAndUpdate(id, updatePayload, { new: true }).select("-__v");
469     if (!updated) return res.status(404).json({ message: "Request not found" });
470
471     logAudit({
472       userId: req.user.id,
473       action: status === "nda_approved" ? "request_approved" : "request_revision_required",
474       resourceType: "request",
475       resourceId: String(id),
476       details: { newStatus: status, type: existing.type },
477     });
478
479     // Notify student by email
480     User.findById(updated.userId).then((owner) => {
481       if (owner?.email) {
482         sendStatusUpdateEmail(owner.email, owner.name, updated.type, status, remarks || "").catch((emailErr) => {
483           console.error("Request status notification email failed:", emailErr.message);
484         });
485       }
486     }).catch(() => {});
487
488     return res.json({ message: "Request updated", request: updated });
489   } catch (err) {
490     return res.status(500).json({ message: err.message });
491   }
492 };
493
494 exports.saveApprovedDocument = async (req, res) => {
495   try {
496     const { id } = req.params;
497     const { url, path, issuedAt, verificationUrl } = req.body;
498
499     if (!url || !path) return res.status(400).json({ message: "url and path are required" });
500     if (!mongoose.Types.ObjectId.isValid(id)) return res.status(400).json({ message: "Invalid request id" });
501
502     const r = await Request.findById(id);
503     if (!r) return res.status(404).json({ message: "Request not found" });
504
505     const validFinalStatuses = ["nda_approved", "agr_approved"];
506     if (!validFinalStatuses.includes(r.status)) {
507       return res.status(400).json({ message: "Request is not in a final approved state" });
508     }
509
510     r.postdocs = {
511       url,
512       path,
513       issuedAt: issuedAt || new Date().toISOString(),
514       verificationUrl: verificationUrl || "",
515     };
516
517     // Volatile signatures: immediately nullify signature data after PDF is saved
518     r.studentSigUrl = "";
519     r.studentSigPath = "";
520     r.adminSigUrl = "";
521     r.adminSigPath = "";
522     r.authorizerSigUrl = "";
523     r.authorizerSigPath = "";
524     r.repSigUrl = "";
525     r.repSigPath = "";
526
527     await r.save();
528
529     // Email representative with approved document link (agreement requests only)
```

```
    if (r.type === "agreement") {
      const repEmail = r.formData?.repEmail;
      if (repEmail) {
        const repName = [r.formData?.repFirstName, r.formData?.repLastName].filter(Boolean).join(" ") || "Representative";
        const requestOwner = await User.findById(r.userId).select("name").lean();
        const requestorName = r.proxyRequestee?.isProxy ? r.proxyRequestee.fullName : requestOwner?.name || "Requestor";
        sendAgreementApprovedEmail(repEmail, repName, requestorName, url, verificationUrl || "").catch((err) => {
          console.error("Failed to send approved document email to representative:", err.message);
        });
      }
    }

    return res.json({ message: "Approved document saved", request: r });
  } catch (err) {
    return res.status(500).json({ message: err.message });
  }
};

// ■■■ Agreement Phase Endpoints ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

// Admin: agreement_initial_admin_reviewal -> agreement_awaiting_rep_approval
// Also used for agreement_rep_revision_requested -> generate new token
exports.generateSigningLink = async (req, res) => {
  try {
    const { id } = req.params;

    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({ message: "Invalid request id" });
    }

    const r = await Request.findById(id);
    if (!r) return res.status(404).json({ message: "Request not found" });

    if (r.type !== "agreement") {
      return res.status(400).json({ message: "Only agreement requests use signing links" });
    }

    const allowedFromStatuses = ["agr_pending_1", "agr_rep_revision_requested", "agr_awaiting_rep_signature"];
    if (!allowedFromStatuses.includes(r.status)) {
      return res.status(400).json({
        message: `Cannot generate signing link from status: ${r.status}`,
      });
    }

    r.signingToken = generateSigningTokenValue();
    r.signingTokenUsed = false;
    r.signingTokenExpiresAt = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000); // 7-day TTL
    r.status = "agr_awaiting_rep_signature";
    r.remarks = "";

    await r.save();

    logAudit({
      userId: req.user.id,
      action: "signing_link_generated",
      resourceType: "request",
      resourceId: String(id),
      details: { type: r.type },
    });

    // Send signing link email to representative
    const repEmail = r.formData?.repEmail;
    const repName = [r.formData?.repFirstName, r.formData?.repLastName].filter(Boolean).join(" ") || "Representative";
    const requestOwner = await User.findById(r.userId).select("name").lean();
    const requestorName = r.proxyRequestee?.isProxy ? r.proxyRequestee.fullName : requestOwner?.name;
    const origin = req.headers.origin || req.headers.referer?.replace(/\/$/, "") || process.env.CLIENT_ORIGIN || "http://loca
    const signingLink = `${origin}/sign/${r.signingToken}`;

    if (repEmail) {
      sendSigningLinkEmail(repEmail, repName, requestorName, signingLink, r.signingTokenExpiresAt).catch((err) => {
        console.error("Failed to send signing link email:", err.message);
      });
    }

    return res.json({
      message: "Signing link generated and sent to representative's email.",
```

```
606       signingToken: r.signingToken,
607       signingTokenExpiresAt: r.signingTokenExpiresAt,
608       request: r,
609     });
610   } catch (err) {
611     return res.status(500).json({ message: err.message });
612   }
613 };
614
615 // Public: get request data needed for the signing page
616 exports.getBySigningToken = async (req, res) => {
617   try {
618     const { token } = req.params;
619
620     if (!token) return res.status(400).json({ message: "Missing token" });
621
622     const r = await Request.findOne({ signingToken: token })
623       .populate("userId", "name email")
624       .select("_id formData status signingTokenUsed signingTokenExpiresAt authorizerSigUrl authorizerSigPath remarks repInfo
625       .lean();
626
627     if (!r) return res.status(404).json({ message: "Invalid signing link" });
628
629     // Check for link expiry
630     if (r.signingTokenExpiresAt && new Date(r.signingTokenExpiresAt) < new Date()) {
631       return res.status(410).json({ message: "This signing link has expired. Please request a new one." });
632     }
633
634     return res.json(r);
635   } catch (err) {
636     return res.status(500).json({ message: err.message });
637   }
638 };
639
640 // Public: rep submits on the signing page
641 exports.repSubmit = async (req, res) => {
642   try {
643     const { token } = req.params;
644     const { repName, repGovIdDoc, repSigUrl, repSigPath } = req.body;
645
646     if (!token) return res.status(400).json({ message: "Missing token" });
647
648     const r = await Request.findOne({ signingToken: token });
649     if (!r) return res.status(404).json({ message: "Invalid signing link" });
650
651     if (r.signingTokenUsed) {
652       return res.status(400).json({ message: "This signing link has already been used" });
653     }
654
655     // Check for link expiry
656     if (r.signingTokenExpiresAt && new Date(r.signingTokenExpiresAt) < new Date()) {
657       return res.status(410).json({ message: "This signing link has expired. Please request a new one." });
658     }
659
660     const allowedStatuses = ["agr_awaiting_rep_signature", "agr_rep_revision_requested"];
661     if (!allowedStatuses.includes(r.status)) {
662       return res.status(400).json({ message: "This request is not awaiting representative signature" });
663     }
664
665     if (!repName || !repName.trim()) {
666       return res.status(400).json({ message: "Representative name is required" });
667     }
668     if (!repSigUrl) {
669       return res.status(400).json({ message: "Representative signature is required" });
670     }
671     if (!repGovIdDoc || !repGovIdDoc.url) {
672       return res.status(400).json({ message: "Government ID is required" });
673     }
674
675     r.repInfo = {
676       name: repName.trim(),
677       govIdDoc: repGovIdDoc,
678     };
679     r.repSigUrl = repSigUrl;
680     r.repSigPath = repSigPath || "";
681     r.repSignedAt = new Date();
```

```
    r.signingTokenUsed = true;
    r.status = "agr_pending_2";
    r.remarks = "";

    await r.save();

    logAudit({
      action: "rep_signature_submitted",
      resourceType: "request",
      resourceId: String(r._id),
      details: { repName: repName.trim() },
    });

    return res.json({ message: "Submission received. Thank you." });
  } catch (err) {
    return res.status(500).json({ message: err.message });
  }
};

// Public: rep rejects on the signing page
exports.repReject = async (req, res) => {
  try {
    const { token } = req.params;

    if (!token) return res.status(400).json({ message: "Missing token" });

    const r = await Request.findOne({ signingToken: token });
    if (!r) return res.status(404).json({ message: "Invalid signing link" });

    if (r.signingTokenUsed) {
      return res.status(400).json({ message: "This signing link has already been used" });
    }

    const allowedStatuses = ["agr_awaiting_rep_signature", "agr_rep_revision_requested"];
    if (!allowedStatuses.includes(r.status)) {
      return res.status(400).json({ message: "This request is not awaiting representative signature" });
    }

    r.signingTokenUsed = true;
    r.status = "agr_declined";

    await r.save();

    logAudit({
      action: "rep_signature_declined",
      resourceType: "request",
      resourceId: String(r._id),
    });

    return res.json({ message: "You have declined the signing request." });
  } catch (err) {
    return res.status(500).json({ message: err.message });
  }
};

// Admin: agreement_final_admin_reviewal -> agreement_approved or agreement_rep_revision_requested
exports.adminPhase3Action = async (req, res) => {
  try {
    const { id } = req.params;
    const { action, remarks } = req.body;

    if (!["approve", "rep_revision_requested"].includes(action)) {
      return res.status(400).json({ message: "action must be 'approve' or 'rep_revision_requested'" });
    }

    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({ message: "Invalid request id" });
    }

    const r = await Request.findById(id);
    if (!r) return res.status(404).json({ message: "Request not found" });

    if (r.type !== "agreement") {
      return res.status(400).json({ message: "Only agreement requests use this endpoint" });
    }
    if (r.status !== "agr_pending_2") {
```

```
758       return res.status(400).json({ message: `Cannot perform phase3 action from status: ${r.status}` });
759     }
760
761     if (action === "approve") {
762       r.status = "agr_approved";
763       if (!r.serialNo) r.serialNo = await generateUniqueSerialNo("agreement");
764       r.remarks = "";
765     } else {
766       // rep_revision_requested: generate new signing token
767       r.signingToken = generateSigningTokenValue();
768       r.signingTokenUsed = false;
769       r.signingTokenExpiresAt = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000);
770       r.status = "agr_rep_revision_requested";
771       r.remarks = remarks || "";
772     }
773
774     await r.save();
775
776     logAudit({
777       userId: req.user.id,
778       action: action === "approve" ? "request_approved" : "request_revision_required",
779       resourceType: "request",
780       resourceId: String(id),
781       details: { newStatus: r.status, type: r.type },
782     });
783
784     // Notify student
785     User.findById(r.userId).then((owner) => {
786       if (owner?.email) {
787         sendStatusUpdateEmail(owner.email, owner.name, r.type, r.status, r.remarks).catch((emailErr) => {
788           console.error("Agreement status notification email failed:", emailErr.message);
789         });
790       }
791     }).catch(() => {});
792
793     // Send new signing link to representative on revision
794     if (action === "rep_revision_requested") {
795       const repEmail = r.formData?.repEmail;
796       const repName = [r.formData?.repFirstName, r.formData?.repLastName].filter(Boolean).join(" ") || "Representative";
797       const requestOwner = await User.findById(r.userId).select("name").lean();
798       const requestorName = r.proxyRequestee?.isProxy ? r.proxyRequestee.fullName : requestOwner?.name;
799       const origin = req.headers.origin || process.env.CLIENT_ORIGIN || "http://localhost:5173";
800       const signingLink = `${origin}/sign/${r.signingToken}`;
801       if (repEmail) {
802         sendSigningLinkEmail(repEmail, repName, requestorName, signingLink, r.signingTokenExpiresAt, true).catch((err) => {
803           console.error("Failed to send revision signing link email:", err.message);
804         });
805       }
806     }
807
808     return res.json({
809       message: action === "approve" ? "Agreement approved" : "Rep revision requested",
810       signingToken: action === "rep_revision_requested" ? r.signingToken : undefined,
811       signingTokenExpiresAt: action === "rep_revision_requested" ? r.signingTokenExpiresAt : undefined,
812       request: r,
813     });
814   } catch (err) {
815     return res.status(500).json({ message: err.message });
816   }
817 };
818
819 // ■■■ Admin: proxy Firebase signature images to avoid browser CORS ■■■■■■■■■■■■■■
820
821 exports.getSignatureImages = async (req, res) => {
822   try {
823     const request = await Request.findById(req.params.id)
824       .select("authorizerSigUrl repSigUrl studentSigUrl adminSigUrl type");
825     if (!request) return res.status(404).json({ message: "Request not found" });
826
827     const toDataUrl = async (url) => {
828       if (!url) return null;
829       const response = await fetch(url);
830       const buffer = await response.arrayBuffer();
831       const mimeType = response.headers.get("content-type") || "image/png";
832       const base64 = Buffer.from(buffer).toString("base64");
833       return `data:${mimeType};base64,${base64}`;
```

```
    };

    if (request.type === "nda") {
      const [studentSig, adminSig] = await Promise.all([
        toDataUrl(request.studentSigUrl),
        toDataUrl(request.adminSigUrl),
      ]);
      return res.json({ studentSig, adminSig });
    }

    const [authorizerSig, repSig] = await Promise.all([
      toDataUrl(request.authorizerSigUrl),
      toDataUrl(request.repSigUrl),
    ]);

    return res.json({ authorizerSig, repSig });
  } catch (err) {
    return res.status(500).json({ message: err.message });
  }
};

// ■■■ Public Verify ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

exports.verifyRequestCode = async (req, res) => {
  try {
    const { code } = req.params;

    if (!code) return res.status(400).json({ valid: false, message: "Missing code" });

    const r = await Request.findOne({ serialNo: code })
      .populate("userId", "name email")
      .select("-__v")
      .lean();

    if (!r) return res.status(404).json({ valid: false, message: "Not found" });

    const isValid =
      (r.type === "nda" && r.status === "nda_approved") ||
      (r.type === "agreement" && r.status === "agr_approved");

    return res.json({
      valid: isValid,
      status: r.status,
      type: r.type,
      requestId: r._id,
      issuedAt: r.postdocs?.issuedAt || "",
    });
  } catch (err) {
    return res.status(500).json({ valid: false, message: err.message });
  }
};

// ■■■ Admin: stats for dashboard / reports ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

exports.getRequestStats = async (req, res) => {
  try {
    const now = new Date();
    const startOfMonth = new Date(now.getFullYear(), now.getMonth(), 1);

    const [statusCounts, typeCounts, monthlyTypeCounts, totalArchived] = await Promise.all([
      // Status distribution
      Request.aggregate([
        { $match: { isArchived: false } },
        { $group: { _id: "$status", count: { $sum: 1 } } },
      ]),
      // Type distribution (all time, not archived)
      Request.aggregate([
        { $match: { isArchived: false } },
        { $group: { _id: "$type", count: { $sum: 1 } } },
      ]),
      // Type distribution for current month
      Request.aggregate([
        { $match: { isArchived: false, createdAt: { $gte: startOfMonth } } },
        { $group: { _id: "$type", count: { $sum: 1 } } },
      ]),
      Request.countDocuments({ isArchived: true }),
```

```
    ]);

    res.json({ statusCounts, typeCounts, monthlyTypeCounts, totalArchived });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// ■■■ Admin: get archived requests ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

exports.getArchivedRequests = async (req, res) => {
  try {
    const list = await Request.find({ isArchived: true })
      .populate("userId", "name email role")
      .sort({ archivedAt: -1 })
      .select("-__v")
      .lean();
    res.json(list);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// ■■■ Admin: retention policy stats ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

const RETENTION_YEARS = 5;

exports.getRetentionStats = async (req, res) => {
  try {
    const now = new Date();
    const cutoff = new Date();
    cutoff.setFullYear(cutoff.getFullYear() - RETENTION_YEARS);

    const approachingCutoff = new Date();
    approachingCutoff.setFullYear(approachingCutoff.getFullYear() - (RETENTION_YEARS - 1));

    const [totalArchived, totalActive, approachingRetention, lastArchived] = await Promise.all([
      Request.countDocuments({ isArchived: true }),
      Request.countDocuments({ $or: [{ isArchived: false }, { isArchived: { $exists: false } }] }),
      Request.countDocuments({
        $or: [{ isArchived: false }, { isArchived: { $exists: false } }],
        createdAt: { $lte: approachingCutoff },
      }),
      Request.findOne({ isArchived: true }).sort({ archivedAt: -1 }).select("archivedAt").lean(),
    ]);

    const eligibleForArchive = await Request.countDocuments({
      $or: [{ isArchived: false }, { isArchived: { $exists: false } }],
      createdAt: { $lte: cutoff },
    });

    res.json({
      retentionYears: RETENTION_YEARS,
      totalArchived,
      totalActive,
      approachingRetention,
      eligibleForArchive,
      lastArchivedAt: lastArchived?.archivedAt || null,
    });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

exports.runArchiveNow = async (req, res) => {
  try {
    const cutoff = new Date();
    cutoff.setFullYear(cutoff.getFullYear() - RETENTION_YEARS);

    const result = await Request.updateMany(
      {
        $and: [
          { $or: [{ isArchived: false }, { isArchived: { $exists: false } }] },
          { createdAt: { $lte: cutoff } },
        ],
      },
```

```
986         { $set: { isArchived: true, archivedAt: new Date() } }
987       );
988
989       res.json({
990         message: `Archived ${result.modifiedCount} request(s).`,
991         archivedCount: result.modifiedCount,
992       });
993     } catch (err) {
994       res.status(500).json({ message: err.message });
995     }
996   };
```

10. server/controllers/auditController.js (47 lines)

```
const AuditLog = require("../models/AuditLog");

// GET /api/audit – Admin only, paginated
exports.getAuditLogs = async (req, res) => {
  try {
    const limit = Math.min(parseInt(req.query.limit) || 50, 200);
    const page = Math.max(parseInt(req.query.page) || 1, 1);
    const skip = (page - 1) * limit;
    const { action, userId, resourceId } = req.query;

    const filter = {};
    if (action) filter.action = action;
    if (userId) filter.userId = userId;
    if (resourceId) filter.resourceId = resourceId;

    const [logs, total] = await Promise.all([
      AuditLog.find(filter)
        .populate("userId", "name email role")
        .sort({ createdAt: -1 })
        .skip(skip)
        .limit(limit)
        .select("-__v")
        .lean(),
      AuditLog.countDocuments(filter),
    ]);

    res.json({ logs, total, page, limit });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

// GET /api/audit/recent – last N entries for dashboard widget
exports.getRecentAuditLogs = async (req, res) => {
  try {
    const count = Math.min(parseInt(req.query.count) || 5, 20);
    const logs = await AuditLog.find()
      .populate("userId", "name email role")
      .sort({ createdAt: -1 })
      .limit(count)
      .select("-__v")
      .lean();
    res.json(logs);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

11. server/routes/authRoutes.js (67 lines)

```

const express = require("express");
const router = express.Router();
const rateLimit = require("express-rate-limit");

const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 50,
  standardHeaders: true,
  legacyHeaders: false,
  message: { message: "Too many authentication attempts. Please try again later." },
});

const {
  register,
  login,
  verifyLoginOtp,
  resendLoginOtp,
  verifyEmail,
  resendVerificationOtp,
  activateAccount,
  forgotPassword,
  resendResetOtp,
  verifyResetOtp,
  resetPassword,
  refreshToken,
  updateProfile,
  requestPasswordChangeOtp,
  getAllUsers,
  adminCreateUser,
  toggleUserActive,
  adminUpdateUser,
  adminDeleteUser,
  adminResetUserPassword,
} = require("../controllers/authController");
const { protect, authorizeAdmin } = require("../middleware/authMiddleware");

// Public (rate-limited)
router.post("/register", authLimiter, register);
router.post("/login", authLimiter, login);
router.post("/verify-login-otp", authLimiter, verifyLoginOtp);
router.post("/resend-login-otp", authLimiter, resendLoginOtp);
router.post("/verify-email", authLimiter, verifyEmail);
router.post("/resend-verification-otp", authLimiter, resendVerificationOtp);
router.post("/activate-account", authLimiter, activateAccount);
router.post("/forgot-password", authLimiter, forgotPassword);
router.post("/resend-reset-otp", authLimiter, resendResetOtp);
router.post("/verify-reset-otp", authLimiter, verifyResetOtp);
router.post("/reset-password", authLimiter, resetPassword);

// Authenticated
router.post("/refresh-token", protect, refreshToken);
router.post("/request-password-change-otp", protect, requestPasswordChangeOtp);
router.patch("/profile", protect, updateProfile);

// Admin-only
router.get("/admin", protect, authorizeAdmin, (req, res) => {
  res.json({ message: "Admin access granted" });
});
router.get("/users", protect, authorizeAdmin, getAllUsers);
router.post("/users", protect, authorizeAdmin, adminCreateUser);
router.patch("/users/:id/toggle-active", protect, authorizeAdmin, toggleUserActive);
router.patch("/users/:id", protect, authorizeAdmin, adminUpdateUser);
router.delete("/users/:id", protect, authorizeAdmin, adminDeleteUser);
router.post("/users/:id/reset-password", protect, authorizeAdmin, adminResetUserPassword);

module.exports = router;
```

12. server/routes/requestRoutes.js (58 lines)

```
const express = require("express");
const router = express.Router();

const { protect, authorizeRoles } = require("../middleware/authMiddleware");
const {
  createRequest,
  getMyRequests,
  resubmitRequest,
  getAllRequests,
  saveApprovedDocument,
  updateRequestStatus,
  getRequestById,
  verifyRequestCode,
  generateSigningLink,
  getBySigningToken,
  repSubmit,
  repReject,
  adminPhase3Action,
  getSignatureImages,
  getRequestStats,
  getArchivedRequests,
  getRetentionStats,
  runArchiveNow,
} = require("../controllers/requestController");

// ■■ Public ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
router.get("/verify/:code", verifyRequestCode);

// Agreement signing page (public – no auth required)
router.get("/sign/:token", getBySigningToken);
router.post("/sign/:token/submit", repSubmit);
router.post("/sign/:token/reject", repReject);

// ■■ Student ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
router.post("/", protect, createRequest);
router.get("/my", protect, getMyRequests);
router.patch("/:id/resubmit", protect, resubmitRequest);

// ■■ Admin ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
router.get("/all", protect, authorizeRoles("admin", "staff"), getAllRequests);
router.get("/stats", protect, authorizeRoles("admin", "staff"), getRequestStats);
router.get("/archived", protect, authorizeRoles("admin", "staff"), getArchivedRequests);
router.get("/retention-stats", protect, authorizeRoles("admin"), getRetentionStats);
router.post("/run-archive", protect, authorizeRoles("admin"), runArchiveNow);
router.patch("/:id/approved-document", protect, authorizeRoles("admin", "staff"), saveApprovedDocument);

// Agreement-specific admin actions
router.patch("/:id/generate-signing-link", protect, authorizeRoles("admin", "staff"), generateSigningLink);
router.patch("/:id/admin-phase3", protect, authorizeRoles("admin"), adminPhase3Action);
router.get("/:id/sig-images", protect, authorizeRoles("admin", "staff"), getSignatureImages);

// NDA status update (pending → approved | revision_required)
router.patch("/:id", protect, authorizeRoles("admin"), updateRequestStatus);

router.get("/:id", protect, getRequestById);

module.exports = router;
```

13. server/routes/auditRoutes.js (9 lines)

```
1  const express = require("express");
2  const router = express.Router();
3  const { getAuditLogs, getRecentAuditLogs } = require("../controllers/auditController");
4  const { protect, authorizeRoles } = require("../middleware/authMiddleware");
5
6  router.get("/", protect, authorizeRoles("admin"), getAuditLogs);
7  router.get("/recent", protect, authorizeRoles("admin"), getRecentAuditLogs);
8
9  module.exports = router;
```

14. server/utils/emailService.js (346 lines)

```
const SibApiV3Sdk = require("sib-api-v3-sdk");

const EMAIL_FAILURE_MESSAGE = "Failed to send verification email. Please try again later.";

class EmailDeliveryError extends Error {
  constructor(message, statusCode, details) {
    super(message || EMAIL_FAILURE_MESSAGE);
    this.name = "EmailDeliveryError";
    this.isEmailError = true;
    this.statusCode = statusCode || 500;
    this.publicMessage = EMAIL_FAILURE_MESSAGE;
    this.details = details || null;
  }
}

const getBrevoClient = () => {
  const apiKey = process.env.BREVO_API_KEY;
  if (!apiKey) {
    throw new EmailDeliveryError("Missing BREVO_API_KEY", 500);
  }

  const defaultClient = SibApiV3Sdk.ApiClient.instance;
  defaultClient.authentications["api-key"].apiKey = apiKey;
  return new SibApiV3Sdk.TransactionalEmailsApi();
};

const getSender = () => {
  const email = process.env.BREVO_SENDER_EMAIL;
  const name = process.env.BREVO_SENDER_NAME || "RTU DPO Portal";

  if (!email) {
    throw new EmailDeliveryError("Missing BREVO_SENDER_EMAIL", 500);
  }

  return { email, name };
};

const normalizeRecipients = (to) => {
  if (!to) return [];

  if (Array.isArray(to)) {
    return to
      .filter(Boolean)
      .map((email) => ({ email: String(email).trim() }))
      .filter((recipient) => recipient.email.length > 0);
  }

  return [{ email: String(to).trim() }].filter((recipient) => recipient.email.length > 0);
};

const sendBrevoEmail = async ({ to, subject, htmlContent }) => {
  try {
    const client = getBrevoClient();
    const recipients = normalizeRecipients(to);
    if (!recipients.length) {
      throw new EmailDeliveryError("Recipient email is required", 400);
    }

    const payload = new SibApiV3Sdk.SendSmtpEmail();
    payload.sender = getSender();
    payload.to = recipients;
    payload.subject = subject;
    payload.htmlContent = htmlContent;

    return await client.sendTransacEmail(payload);
  } catch (error) {
    if (error?.isEmailError) {
      throw error;
    }

    const statusCode = error?.status || error?.response?.status || error?.response?.statusCode || 500;
    const details = error?.response?.body || error?.response?.data || error?.message || "Unknown Brevo error";
    const brevoMessage = typeof details === "string" ? details : JSON.stringify(details);
```

```

    console.error(`[Brevo] Email send failed (${statusCode}): ${brevoMessage}`);

    throw new EmailDeliveryError(`Brevo API request failed (${statusCode})`, statusCode, details);
  }
};

/**
 * Send a 6-digit OTP for login verification or password reset.
 */
exports.sendOtpEmail = (email, otp, purpose = "login") => {
  const label = purpose === "reset" ? "Password Reset" : purpose === "change" ? "Password Change" : "Login Verification";
  return sendBrevoEmail({
    to: email,
    subject: `RTU DPO Portal – Your ${label} OTP`,
    htmlContent: `
      <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overfl
        <div style="background:#0f2d6b;padding:24px 32px">
          <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
        </div>
        <div style="padding:32px">
          <p style="color:#0f172a;font-size:15px;margin-top:0">Your <strong>${label} OTP</strong> is:</p>
          <div style="font-size:36px;font-weight:700;letter-spacing:8px;color:#0f2d6b;padding:16px 0">${otp}</div>
          <p style="color:#64748b;font-size:13px">This code expires in <strong>10 minutes</strong>. Do not share it with anyo
          <hr style="border:none;border-top:1px solid #e2e8f0;margin:24px 0">
          <p style="color:#94a3b8;font-size:12px;margin:0">If you did not request this, please ignore this email or contact y
        </div>
      </div>`,
  });
};

/**
 * Send a password-reset link.
 */
exports.sendPasswordResetEmail = (email, resetUrl) =>
  sendBrevoEmail({
    to: email,
    subject: "RTU DPO Portal – Password Reset Request",
    htmlContent: `
      <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overfl
        <div style="background:#0f2d6b;padding:24px 32px">
          <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
        </div>
        <div style="padding:32px">
          <p style="color:#0f172a;font-size:15px;margin-top:0">We received a request to reset your password.</p>
          <a href="${resetUrl}" style="display:inline-block;background:#0f2d6b;color:#fff;padding:12px 24px;border-radius:8px
          <p style="color:#64748b;font-size:13px;margin-top:20px">This link expires in <strong>1 hour</strong>. If you did no
        </div>
      </div>`,
  });

/**
 * Notify a user that their request status has changed.
 */
exports.sendStatusUpdateEmail = (email, name, requestType, newStatus, remarks) => {
  const statusLabels = {
    nda_pending: "Reviewal",
    nda_approved: "Approved",
    stud_revision_requested: "Student Revisions",
    agr_pending_1: "Initial Reviewal",
    agr_awaiting_rep_signature: "Recipient Reviewal",
    agr_pending_2: "Final Reviewal",
    agr_approved: "Approved",
    agr_declined: "Recipient Declined",
    agr_rep_revision_requested: "Recipient Revisions",
  };
  const label = statusLabels[newStatus] || newStatus;
  const remarksBlock = remarks
    ? `<div style="background:#f8fafc;border:1px solid #e2e8f0;border-radius:8px;padding:12px 16px;margin-top:16px"><strong>R
    : "";

  return sendBrevoEmail({
    to: email,
    subject: `RTU DPO Portal – Your ${requestType.toUpperCase()} Request has been updated`,
    htmlContent: `
      <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overfl
```

```
        <div style="background:#0f2d6b;padding:24px 32px">
          <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
        </div>
        <div style="padding:32px">
          <p style="color:#0f172a;font-size:15px;margin-top:0">Hi <strong>${name}</strong>,</p>
          <p style="color:#475569;font-size:14px">Your <strong>${requestType.toUpperCase()}</strong> request status has been
          <div style="font-size:20px;font-weight:700;color:#0f2d6b;padding:8px 0">${label}</div>
          ${remarksBlock}
          <p style="color:#94a3b8;font-size:12px;margin-top:24px">Log in to the RTU DPO Portal for more details.</p>
        </div>
      </div>`,
  });
};

/**
 * Send a welcome / account-created email with a temporary password.
 */
exports.sendWelcomeEmail = (email, name, tempPassword) =>
  sendBrevoEmail({
    to: email,
    subject: "RTU DPO Portal – Your Account Has Been Created",
    htmlContent: `
      <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overfl
        <div style="background:#0f2d6b;padding:24px 32px">
          <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
        </div>
        <div style="padding:32px">
          <p style="color:#0f172a;font-size:15px;margin-top:0">Hi <strong>${name}</strong>, your account has been created.</p
          <p style="color:#475569;font-size:14px">Temporary password: <strong style="color:#0f2d6b">${tempPassword}</strong><
          <p style="color:#64748b;font-size:13px">Please log in and change your password immediately.</p>
        </div>
      </div>`,
  });

/**
 * Send an email verification link.
 */
exports.sendVerificationEmail = (email, name, verifyUrl) =>
  sendBrevoEmail({
    to: email,
    subject: "RTU DPO Portal – Verify Your Email",
    htmlContent: `
      <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overfl
        <div style="background:#0f2d6b;padding:24px 32px">
          <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
        </div>
        <div style="padding:32px">
          <p style="color:#0f172a;font-size:15px;margin-top:0">Hi <strong>${name}</strong>,</p>
          <p style="color:#475569;font-size:14px">Please verify your email address to activate your account.</p>
          <a href="${verifyUrl}" style="display:inline-block;background:#0f2d6b;color:#fff;padding:12px 24px;border-radius:8p
          <p style="color:#64748b;font-size:13px">This link expires in <strong>24 hours</strong>. If you did not create this
        </div>
      </div>`,
  });

/**
 * Send a welcome email with a verification/activation link (admin-created user, no password set).
 */
exports.sendWelcomeVerificationEmail = (email, name, activateUrl) =>
  sendBrevoEmail({
    to: email,
    subject: "RTU DPO Portal – Welcome! Activate Your Account",
    htmlContent: `
      <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overfl
        <div style="background:#0f2d6b;padding:24px 32px">
          <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
        </div>
        <div style="padding:32px">
          <p style="color:#0f172a;font-size:15px;margin-top:0">Hi <strong>${name}</strong>,</p>
          <p style="color:#475569;font-size:14px">An administrator has created an account for you at the RTU DPO Portal. Clic
          <a href="${activateUrl}" style="display:inline-block;background:#0f2d6b;color:#fff;padding:12px 24px;border-radius:
          <p style="color:#64748b;font-size:13px">This link expires in <strong>24 hours</strong>. If you did not expect this,
        </div>
      </div>`,
  });

```

```
/**
 * Send a verification OTP email for new account registration.
 */
exports.sendVerificationOtpEmail = (email, name, otp) =>
  sendBrevoEmail({
    to: email,
    subject: "RTU DPO Portal \u2013 Verify Your Email",
    htmlContent: `
      <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overfl
        <div style="background:#0f2d6b;padding:24px 32px">
          <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
        </div>
        <div style="padding:32px">
          <p style="color:#0f172a;font-size:15px;margin-top:0">Hi <strong>${name}</strong>,</p>
          <p style="color:#475569;font-size:14px">Use the code below to verify your email address and activate your account.<
          <div style="font-size:36px;font-weight:700;letter-spacing:8px;color:#0f2d6b;padding:16px 0">${otp}</div>
          <p style="color:#64748b;font-size:13px">This code expires in <strong>10 minutes</strong>. Do not share it with anyo
          <hr style="border:none;border-top:1px solid #e2e8f0;margin:24px 0">
          <p style="color:#94a3b8;font-size:12px;margin:0">If you did not create this account, please ignore this email.</p>
        </div>
      </div>`,
  });

/**
 * Send a login OTP email for device/IP verification.
 */
exports.sendLoginOtpEmail = (email, otp) =>
  sendBrevoEmail({
    to: email,
    subject: "RTU DPO Portal – Login Verification OTP",
    htmlContent: `
      <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overfl
        <div style="background:#0f2d6b;padding:24px 32px">
          <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
        </div>
        <div style="padding:32px">
          <p style="color:#0f172a;font-size:15px;margin-top:0">A login attempt was made from an unrecognized device or locati
          <p style="color:#475569;font-size:14px">Your verification code is:</p>
          <div style="font-size:36px;font-weight:700;letter-spacing:8px;color:#0f2d6b;padding:16px 0">${otp}</div>
          <p style="color:#64748b;font-size:13px">This code expires in <strong>10 minutes</strong>. If you did not attempt to
        </div>
      </div>`,
  });

exports.sendPasswordChangedAlertEmail = (email, name, reason = "account security event") =>
  sendBrevoEmail({
    to: email,
    subject: "RTU DPO Portal – Security Alert: Password Changed",
    htmlContent: `
      <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overfl
        <div style="background:#7f1d1d;padding:24px 32px">
          <h1 style="color:#fff;margin:0;font-size:18px">Security Alert</h1>
        </div>
        <div style="padding:32px">
          <p style="color:#0f172a;font-size:15px;margin-top:0">Hi <strong>${name || "User"}</strong>,</p>
          <p style="color:#475569;font-size:14px">Your account password was changed due to <strong>${reason}</strong>.</p>
          <p style="color:#475569;font-size:14px">If this was not you, reset your password immediately and contact the DPO of
          <p style="color:#94a3b8;font-size:12px;margin-top:18px">Timestamp: ${new Date().toISOString()}</p>
        </div>
      </div>
    `,
  });
exports.sendSigningLinkEmail = (repEmail, repName, requestorName, signingLink, expiresAt, isRevision = false) => {
  const expiryStr = expiresAt
    ? new Date(expiresAt).toLocaleDateString("en-PH", { year: "numeric", month: "long", day: "numeric" })
    : "7 days";
  const subject = isRevision
    ? "RTU DPO Portal – Agreement Revision Required"
    : "RTU DPO Portal – Agreement Signing Request";
  const headerBg = isRevision ? "#92400e" : "#0f2d6b";
  const bodyText = isRevision
    ? `Your previously submitted agreement for the request by <strong>${requestorName || "an RTU student"}</strong> has been
    : `You have been designated as the authorized representative for an agreement request submitted by <strong>${requestorNam
  return sendBrevoEmail({
    to: repEmail,
    subject,
```

```
    htmlContent: `
      <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overfl
        <div style="background:${headerBg};padding:24px 32px">
          <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
        </div>
        <div style="padding:32px">
          <p style="color:#0f172a;font-size:15px;margin-top:0">Dear <strong>${repName || "Representative"}</strong>,</p>
          <p style="color:#475569;font-size:14px">${bodyText}</p>
          <div style="text-align:center;margin:28px 0">
            <a href="${signingLink}" style="background:${headerBg};color:#fff;padding:14px 28px;border-radius:8px;text-decora
          </div>
          <p style="color:#64748b;font-size:13px">This link is valid until <strong>${expiryStr}</strong> and can only be used
          <hr style="border:none;border-top:1px solid #e2e8f0;margin:24px 0">
          <p style="color:#94a3b8;font-size:12px;margin:0">If you were not expecting this email, please disregard it or conta
        </div>
      </div>`,
  });
};

exports.sendAgreementApprovedEmail = (repEmail, repName, requestorName, documentUrl, verificationUrl) =>
  sendBrevoEmail({
    to: repEmail,
    subject: "RTU DPO Portal – Agreement Approved",
    htmlContent: `
      <div style="font-family:Inter,sans-serif;max-width:480px;margin:auto;border:1px solid #e2e8f0;border-radius:12px;overfl
        <div style="background:#065f46;padding:24px 32px">
          <h1 style="color:#fff;margin:0;font-size:18px">RTU Data Protection Office</h1>
        </div>
        <div style="padding:32px">
          <p style="color:#0f172a;font-size:15px;margin-top:0">Dear <strong>${repName || "Representative"}</strong>,</p>
          <p style="color:#475569;font-size:14px">We are pleased to inform you that the agreement request submitted by <stron
          <p style="color:#475569;font-size:14px">You may access and download the approved agreement document using the butto
          <div style="text-align:center;margin:28px 0">
            <a href="${documentUrl}" style="background:#065f46;color:#fff;padding:14px 28px;border-radius:8px;text-decoration
          </div>
          ${verificationUrl ? `<p style="color:#64748b;font-size:13px;text-align:center">You may also verify document authent
          <hr style="border:none;border-top:1px solid #e2e8f0;margin:24px 0">
          <p style="color:#94a3b8;font-size:12px;margin:0">This is an automated notification from the RTU Data Protection Off
        </div>
      </div>`,
  });

exports.sendBrevoEmail = sendBrevoEmail;
exports.EmailDeliveryError = EmailDeliveryError;
```

15. server/scripts/runArchive.js (48 lines)

```
require('dotenv').config();
const connectDB = require('../config/db');
const mongoose = require('mongoose');
const Request = require('../models/Request');

(async () => {
  try {
    await connectDB();
    const archiveTimestamp = new Date();
    const fiveYearsAgo = new Date();
    fiveYearsAgo.setFullYear(fiveYearsAgo.getFullYear() - 5);

    // Archive ALL requests older than 5 years regardless of status (universal retention policy)
    const filter = {
      $and: [
        {
          $or: [
            { isArchived: false },
            { isArchived: { $exists: false } },
          ],
        },
        { createdAt: { $lte: fiveYearsAgo } },
      ],
    };

    const result = await Request.updateMany(filter, {
      $set: {
        isArchived: true,
        archivedAt: archiveTimestamp,
      },
    });

    console.log(`[runArchive] Archive timestamp: ${archiveTimestamp.toISOString()}`);
    console.log(`[runArchive] Matched ${result.matchedCount || 0} requests.`);
    console.log(`[runArchive] Archived ${result.modifiedCount || 0} requests.`);
    process.exitCode = 0;
  } catch (err) {
    console.error('[runArchive] Error:', err);
    process.exitCode = 1;
  } finally {
    try {
      await mongoose.connection.close();
    } catch (e) {
      // ignore
    }
    process.exit();
  }
})();
```

16. server/scripts/restoreArchive.js (14 lines)

```
require('dotenv').config();
const connectDB = require('../config/db');
const mongoose = require('mongoose');
const Request = require('../models/Request');

(async () => {
  await connectDB();
  const result = await Request.updateMany(
    { isArchived: true },
    { $set: { isArchived: false }, $unset: { archivedAt: '' } }
  );
  console.log(`Restored: ${result.modifiedCount} request(s).`);
  await mongoose.connection.close();
})();
```

Frontend Source Code

17. client/src/main.jsx (26 lines)

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { Buffer } from "buffer";
import "./index.css";
import App from "./App.jsx";
import { notify } from "./utils/inPageFeedback";

import { AuthProvider } from "./context/AuthContext.jsx";

window.Buffer = Buffer;

// Guard against duplicate patching during HMR updates.
if (typeof window !== "undefined" && !window.__dpoAlertPatched) {
  window.__dpoAlertPatched = true;
  window.alert = (message) => {
    notify(String(message ?? ""), { type: "info" });
  };
}

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <AuthProvider>
      <App />
    </AuthProvider>
  </StrictMode>
);
```

18. client/src/App.jsx (346 lines)

```
import { BrowserRouter as Router, Routes, Route, Navigate, Outlet, useLocation } from "react-router-dom";
import { Suspense, lazy, useContext, useMemo } from "react";
import { AnimatePresence, motion } from "framer-motion";
import { AuthContext } from "./context/AuthContext";

import Navbar from "./components/Navbar";
import Headbar from "./components/Headbar";
import SessionWarningModal from "./components/SessionWarningModal";
import InPageFeedbackHost from "./components/InPageFeedbackHost";
import RequireRole from "./guards/RequireRole";

import Landing from "./pages/Landing";
const VerifyDocument = lazy(() => import("./pages/VerifyDocument"));
const VerifyEmailPage = lazy(() => import("./pages/VerifyEmailPage"));
const ActivateAccountPage = lazy(() => import("./pages/ActivateAccountPage"));
const RepSigningPage = lazy(() => import("./pages/public/RepSigningPage"));

// Student pages
const StudentDashboard = lazy(() => import("./pages/student/StudentDashboard"));
const StudentNewRequest = lazy(() => import("./pages/student/StudentNewRequest"));
const StudentNDATypeChooser = lazy(() => import("./pages/student/StudentNDATypeChooser"));
const StudentNDAOrgActivities = lazy(() => import("./pages/student/StudentNDAOrgActivities"));
const StudentNDAResearch = lazy(() => import("./pages/student/StudentNDAResearch"));
const StudentAgreementRequirements = lazy(() => import("./pages/student/StudentAgreementRequirements"));
const StudentAgreementRequest = lazy(() => import("./pages/student/StudentAgreementRequest"));
const StudentRequestReview = lazy(() => import("./pages/student/StudentRequestReview"));
const StudentProfile = lazy(() => import("./pages/student/StudentProfile"));
const StudentResubmitRequest = lazy(() => import("./pages/student/StudentResubmitRequest"));

// Admin pages
const AdminDashboard = lazy(() => import("./pages/admin/AdminDashboard"));
const AdminRequests = lazy(() => import("./pages/admin/AdminRequests"));
const AdminRequestReview = lazy(() => import("./pages/admin/AdminRequestReview"));
const AdminReports = lazy(() => import("./pages/admin/AdminReports"));
const AdminProfile = lazy(() => import("./pages/admin/AdminProfile"));
const AdminUsers = lazy(() => import("./pages/admin/AdminUsers"));
const AdminArchives = lazy(() => import("./pages/admin/AdminArchives"));
const AdminAuditLog = lazy(() => import("./pages/admin/AdminAuditLog"));
const StaffProxyTypeChooser = lazy(() => import("./pages/admin/StaffProxyTypeChooser"));
const StaffProxyNDATypeChooser = lazy(() => import("./pages/admin/StaffProxyNDATypeChooser"));
const StaffProxyNDAOrgActivities = lazy(() => import("./pages/admin/StaffProxyNDAOrgActivities"));
const StaffProxyNDAResearch = lazy(() => import("./pages/admin/StaffProxyNDAResearch"));
const StaffProxyAgreementRequest = lazy(() => import("./pages/admin/StaffProxyAgreementRequest"));

function GlobalLoader() {
  return (
    <div
      className="route-transition-shell"
      style={{
        minHeight: "160px",
        display: "grid",
        placeItems: "center",
      }}
      aria-live="polite"
      aria-busy="true"
    >
      <div
        style={{
          width: "34px",
          height: "34px",
          borderRadius: "999px",
          border: "3px solid rgba(15, 45, 107, 0.2)",
          borderTopColor: "var(--primary)",
          animation: "spin 0.8s linear infinite",
        }}
      />
    </div>
  );
}

const buildTitle = (pathname) => {
  if (pathname === "/student" || pathname === "/admin") return "Dashboard";
  if (pathname.startsWith("/student/new-request")) return "Create Request";
```

```
  if (pathname.startsWith("/student/requests")) return "Request Details";
  if (pathname.startsWith("/student/resubmit")) return "Resubmit Request";
  if (pathname.startsWith("/student/profile")) return "Profile";
  if (pathname === "/admin/requests") return "Request Management";
  if (pathname.startsWith("/admin/requests/")) return "Request Review";
  if (pathname.startsWith("/admin/reports")) return "Transaction Records";
  if (pathname.startsWith("/admin/users")) return "User Management";
  if (pathname.startsWith("/admin/archives")) return "Archives";
  if (pathname.startsWith("/admin/audit")) return "Audit Logs";
  if (pathname.startsWith("/admin/profile")) return "Profile";
  if (pathname.startsWith("/admin/proxy-request")) return "Proxy Request";
  return "Dashboard";
};

function AppLayout() {
  const { user, showSessionWarning, extendSession } = useContext(AuthContext);
  const location = useLocation();
  const title = useMemo(() => buildTitle(location.pathname), [location.pathname]);

  return (
    <div className="app-container">
      {user && <Headbar />}

      <div className="app-body">
        {user && <Navbar />}

        <div className="page-content">
          {user && <h1 className="page-title">{title}</h1>}
          <AnimatePresence mode="wait">
            <motion.div
              key={location.pathname}
              className="route-transition-shell"
              initial={{ opacity: 0, y: 8 }}
              animate={{ opacity: 1, y: 0 }}
              exit={{ opacity: 0, y: -6 }}
              transition={{ duration: 0.22, ease: "easeOut" }}
            >
              <Suspense fallback={<GlobalLoader />}>
                <Outlet />
              </Suspense>
            </motion.div>
          </AnimatePresence>
        </div>
      </div>

      {showSessionWarning && <SessionWarningModal onExtend={extendSession} />}
    </div>
  );
}

function App() {
  const { user } = useContext(AuthContext);

  return (
    <>
      <Router>
        <Suspense fallback={<GlobalLoader />}>
          <Routes>
          {/* Public routes (no auth, no layout) */}
          <Route path="/verify/:code" element={<VerifyDocument />} />
          <Route path="/sign/:token" element={<RepSigningPage />} />
          <Route path="/verify-email" element={<VerifyEmailPage />} />
          <Route path="/activate" element={<ActivateAccountPage />} />

          <Route element={<AppLayout />}>
            <Route
              path="/"
              element={
                user
                  ? <Navigate to={user.role === "student" ? "/student" : "/admin/requests"} replace />
                  : <Landing />
              }
            />
            <Route path="/login" element={<Navigate to="/" replace />} />
            <Route path="/register" element={<Navigate to="/" replace />} />

```

```
            <Route
              path="/student"
              element={
                <RequireRole allowedRoles={["student"]}>
                  <StudentDashboard />
                </RequireRole>
              }
            />
            <Route
              path="/student/new-request"
              element={
                <RequireRole allowedRoles={["student"]}>
                  <StudentNewRequest />
                </RequireRole>
              }
            />
            <Route
              path="/student/new-request/nda"
              element={
                <RequireRole allowedRoles={["student"]}>
                  <StudentNDATypeChooser />
                </RequireRole>
              }
            />
            <Route
              path="/student/new-request/nda/orgactivities"
              element={
                <RequireRole allowedRoles={["student"]}>
                  <StudentNDAOrgActivities />
                </RequireRole>
              }
            />
            <Route
              path="/student/new-request/nda/research"
              element={
                <RequireRole allowedRoles={["student"]}>
                  <StudentNDAResearch />
                </RequireRole>
              }
            />
            <Route
              path="/student/new-request/agreement"
              element={
                <RequireRole allowedRoles={["student"]}>
                  <StudentAgreementRequirements />
                </RequireRole>
              }
            />
            <Route
              path="/student/new-request/agreement/form"
              element={
                <RequireRole allowedRoles={["student"]}>
                  <StudentAgreementRequest />
                </RequireRole>
              }
            />
            <Route
              path="/student/requests/:id"
              element={
                <RequireRole allowedRoles={["student"]}>
                  <StudentRequestReview />
                </RequireRole>
              }
            />
            <Route
              path="/student/profile"
              element={
                <RequireRole allowedRoles={["student"]}>
                  <StudentProfile />
                </RequireRole>
              }
            />
            <Route
              path="/student/resubmit/:id"
              element={
                <RequireRole allowedRoles={["student"]}>
```

```
                <StudentResubmitRequest />
              </RequireRole>
            }
          />

          <Route
            path="/admin"
            element={
              <RequireRole allowedRoles={["admin", "staff"]}>
                <AdminDashboard />
              </RequireRole>
            }
          />
          <Route
            path="/admin/requests"
            element={
              <RequireRole allowedRoles={["admin", "staff"]}>
                <AdminRequests />
              </RequireRole>
            }
          />
          <Route
            path="/admin/requests/:id"
            element={
              <RequireRole allowedRoles={["admin", "staff"]}>
                <AdminRequestReview />
              </RequireRole>
            }
          />
          <Route
            path="/admin/proxy-request"
            element={
              <RequireRole allowedRoles={["staff"]}>
                <StaffProxyTypeChooser />
              </RequireRole>
            }
          />
          <Route
            path="/admin/proxy-request/nda"
            element={
              <RequireRole allowedRoles={["staff"]}>
                <StaffProxyNDATypeChooser />
              </RequireRole>
            }
          />
          <Route
            path="/admin/proxy-request/nda/orgactivities"
            element={
              <RequireRole allowedRoles={["staff"]}>
                <StaffProxyNDAOrgActivities />
              </RequireRole>
            }
          />
          <Route
            path="/admin/proxy-request/nda/research"
            element={
              <RequireRole allowedRoles={["staff"]}>
                <StaffProxyNDAResearch />
              </RequireRole>
            }
          />
          <Route
            path="/admin/proxy-request/agreement"
            element={
              <RequireRole allowedRoles={["staff"]}>
                <StaffProxyAgreementRequest />
              </RequireRole>
            }
          />
          <Route
            path="/admin/reports"
            element={
              <RequireRole allowedRoles={["admin", "staff"]}>
                <AdminReports />
              </RequireRole>
            }
```

```
            />
            <Route
              path="/admin/profile"
              element={
                <RequireRole allowedRoles={["admin", "staff"]}>
                  <AdminProfile />
                </RequireRole>
              }
            />
            <Route
              path="/admin/users"
              element={
                <RequireRole allowedRoles={["admin"]}>
                  <AdminUsers />
                </RequireRole>
              }
            />
            <Route
              path="/admin/archives"
              element={
                <RequireRole allowedRoles={["admin", "staff"]}>
                  <AdminArchives />
                </RequireRole>
              }
            />
            <Route
              path="/admin/audit"
              element={
                <RequireRole allowedRoles={["admin"]}>
                  <AdminAuditLog />
                </RequireRole>
              }
            />

            <Route path="*" element={<Navigate to="/" replace />} />
          </Route>
          </Routes>
        </Suspense>
      </Router>
      <InPageFeedbackHost />
    </>
  );
}

export default App;
```

19. client/src/firebase.js (11 lines)

```
 1  import { initializeApp } from "firebase/app";
 2  import { getStorage } from "firebase/storage";
 3
 4  const firebaseConfig = {
 5    apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
 6    projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
 7    storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
 8  };
 9
10  const app = initializeApp(firebaseConfig);
11  export const storage = getStorage(app);
```

20. client/src/context/AuthContext.jsx (138 lines)

```
/* eslint-disable react-refresh/only-export-components */
import { createContext, useCallback, useEffect, useRef, useState } from "react";
import { logout as logoutService, getCurrentUser, getToken, refreshToken } from "../services/authService";

export const AuthContext = createContext();

const decodeToken = (token) => {
  try {
    const payload = token.split(".")[1];
    return JSON.parse(atob(payload));
  } catch {
    return null;
  }
};

const INACTIVITY_TIMEOUT = 15 * 60 * 1000; // 15 minutes
const WARNING_BEFORE = 2 * 60 * 1000; // Show warning 2 min before timeout
const WARNING_AT = INACTIVITY_TIMEOUT - WARNING_BEFORE; // 13 minutes

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(getCurrentUser());
  const [showSessionWarning, setShowSessionWarning] = useState(false);
  const logoutTimerRef = useRef(null);
  const warningTimerRef = useRef(null);
  const inactivityTimerRef = useRef(null);
  const inactivityWarningRef = useRef(null);

  const clearAllTimers = useCallback(() => {
    [logoutTimerRef, warningTimerRef, inactivityTimerRef, inactivityWarningRef].forEach((ref) => {
      if (ref.current) {
        clearTimeout(ref.current);
        ref.current = null;
      }
    });
  }, []);

  const logout = useCallback(() => {
    logoutService();
    setUser(null);
    setShowSessionWarning(false);
    clearAllTimers();
  }, [clearAllTimers]);

  const scheduleAutoLogout = useCallback(() => {
    if (logoutTimerRef.current) clearTimeout(logoutTimerRef.current);
    const token = getToken();
    if (!token) return;

    const decoded = decodeToken(token);
    if (!decoded?.exp) return;

    const expiresAtMs = decoded.exp * 1000;
    const delay = expiresAtMs - Date.now();

    if (delay <= 0) {
      logout();
      return;
    }

    logoutTimerRef.current = setTimeout(() => {
      logout();
    }, delay);
  }, [logout]);

  // Inactivity timer management
  const resetInactivityTimer = useCallback(() => {
    if (!getToken()) return;

    setShowSessionWarning(false);

    if (inactivityWarningRef.current) clearTimeout(inactivityWarningRef.current);
    if (inactivityTimerRef.current) clearTimeout(inactivityTimerRef.current);

```

```
    // Show warning at 13 min
    inactivityWarningRef.current = setTimeout(() => {
      setShowSessionWarning(true);
    }, WARNING_AT);

    // Force logout at 15 min
    inactivityTimerRef.current = setTimeout(() => {
      setShowSessionWarning(false);
      logout();
    }, INACTIVITY_TIMEOUT);
  }, [logout]);

  const extendSession = useCallback(async () => {
    try {
      const data = await refreshToken();
      if (data.user) {
        setUser(data.user);
      }
      setShowSessionWarning(false);
      scheduleAutoLogout();
      resetInactivityTimer();
    } catch {
      logout();
    }
  }, [logout, scheduleAutoLogout, resetInactivityTimer]);

  const login = useCallback((userData) => {
    setUser(userData);
    scheduleAutoLogout();
    resetInactivityTimer();
  }, [scheduleAutoLogout, resetInactivityTimer]);

  const updateUser = useCallback((userData) => {
    setUser(userData);
    localStorage.setItem("user", JSON.stringify(userData));
  }, []);

  // Activity listeners
  useEffect(() => {
    if (!user) return;

    const events = ["mousemove", "keydown", "scroll", "mousedown", "touchstart"];
    const handler = () => resetInactivityTimer();

    events.forEach((e) => window.addEventListener(e, handler, { passive: true }));
    resetInactivityTimer();

    return () => {
      events.forEach((e) => window.removeEventListener(e, handler));
    };
  }, [user, resetInactivityTimer]);

  useEffect(() => {
    // on load or refresh, check token validity
    scheduleAutoLogout();
    return () => clearAllTimers();
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, []);

  return (
    <AuthContext.Provider value={{ user, login, logout, updateUser, showSessionWarning, extendSession }}>
      {children}
    </AuthContext.Provider>
  );
};
```

21. client/src/guards/RequireRole.jsx (17 lines)

```
import { useContext } from "react";
import { Navigate } from "react-router-dom";
import { AuthContext } from "../context/AuthContext";

const RequireRole = ({ allowedRoles, children }) => {
  const { user } = useContext(AuthContext);

  if (!user) return <Navigate to="/" replace />;

  if (!allowedRoles.includes(user.role)) {
    return <Navigate to={user.role === "student" ? "/student" : "/admin"} replace />;
  }

  return children;
};

export default RequireRole;
```

22. client/src/services/authService.js (148 lines)

```
import axios from "axios";

const API_URL = `${import.meta.env.VITE_API_BASE_URL}/api/auth`;

export const login = async (email, password) => {
  const res = await axios.post(`${API_URL}/login`, { email, password });
  // If OTP required, don't set token/user yet
  if (res.data.requireOtp) {
    return res.data;
  }
  if (res.data.token) {
    localStorage.setItem("token", res.data.token);
    localStorage.setItem("user", JSON.stringify(res.data.user));
  }
  return res.data;
};

export const verifyLoginOtp = async (email, otp) => {
  const res = await axios.post(`${API_URL}/verify-login-otp`, { email, otp });
  if (res.data.token) {
    localStorage.setItem("token", res.data.token);
    localStorage.setItem("user", JSON.stringify(res.data.user));
  }
  return res.data;
};

export const verifyEmail = async (email, otp) => {
  const res = await axios.post(`${API_URL}/verify-email`, { email, otp });
  if (res.data.token) {
    localStorage.setItem("token", res.data.token);
    localStorage.setItem("user", JSON.stringify(res.data.user));
  }
  return res.data;
};

export const verifyEmailByToken = async (token) => {
  const res = await axios.post(`${API_URL}/verify-email`, { token });
  return res.data;
};

export const resendVerificationOtp = async (email) => {
  const res = await axios.post(`${API_URL}/resend-verification-otp`, { email });
  return res.data;
};

export const resendLoginOtp = async (email) => {
  const res = await axios.post(`${API_URL}/resend-login-otp`, { email });
  return res.data;
};

export const activateAccount = async (token, password) => {
  const res = await axios.post(`${API_URL}/activate-account`, { token, password });
  return res.data;
};

export const logout = () => {
  localStorage.removeItem("token");
  localStorage.removeItem("user");
};

export const getCurrentUser = () => {
  const raw = localStorage.getItem("user");
  return raw ? JSON.parse(raw) : null;
};

export const getToken = () => localStorage.getItem("token");

// ■■ Forgot Password / Reset flow ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export const forgotPassword = async (email) => {
  const res = await axios.post(`${API_URL}/forgot-password`, { email });
  return res.data;
};

```

```
export const resendResetOtp = async (email) => {
  const res = await axios.post(`${API_URL}/resend-reset-otp`, { email });
  return res.data;
};

export const verifyResetOtp = async (email, otp) => {
  const res = await axios.post(`${API_URL}/verify-reset-otp`, { email, otp });
  return res.data;
};

export const resetPassword = async (email, resetToken, newPassword) => {
  const res = await axios.post(`${API_URL}/reset-password`, { email, resetToken, newPassword });
  return res.data;
};

// ■■ Session token refresh ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
const authHeader = () => {
  const token = localStorage.getItem("token");
  return token ? { Authorization: `Bearer ${token}` } : {};
};

export const refreshToken = async () => {
  const res = await axios.post(`${API_URL}/refresh-token`, {}, { headers: authHeader() });
  if (res.data.token) {
    localStorage.setItem("token", res.data.token);
    localStorage.setItem("user", JSON.stringify(res.data.user));
  }
  return res.data;
};

// ■■ Password change OTP ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export const requestPasswordChangeOtp = async () => {
  const res = await axios.post(`${API_URL}/request-password-change-otp`, {}, { headers: authHeader() });
  return res.data;
};

// ■■ Profile update ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export const updateProfile = async (payload) => {
  const res = await axios.patch(`${API_URL}/profile`, payload, { headers: authHeader() });
  if (res.data.user) {
    localStorage.setItem("user", JSON.stringify(res.data.user));
  }
  return res.data;
};

// ■■ Admin user management ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export const getAllUsers = async () => {
  const res = await axios.get(`${API_URL}/users`, { headers: authHeader() });
  return res.data;
};

export const adminCreateUser = async (payload) => {
  const res = await axios.post(`${API_URL}/users`, payload, { headers: authHeader() });
  return res.data;
};

export const toggleUserActive = async (id) => {
  const res = await axios.patch(`${API_URL}/users/${id}/toggle-active`, {}, { headers: authHeader() });
  return res.data;
};

export const adminUpdateUser = async (id, payload) => {
  const res = await axios.patch(`${API_URL}/users/${id}`, payload, { headers: authHeader() });
  return res.data;
};

export const adminDeleteUser = async (id) => {
  const res = await axios.delete(`${API_URL}/users/${id}`, { headers: authHeader() });
  return res.data;
};

export const adminResetUserPassword = async (id, temporaryPassword) => {
  const res = await axios.post(`${API_URL}/users/${id}/reset-password`, { temporaryPassword }, { headers: authHeader() });
  return res.data;
};
```

23. client/src/services/requestService.js (110 lines)

```
import axios from "axios";

const API_URL = `${import.meta.env.VITE_API_BASE_URL}/api/requests`;

const authHeader = () => {
  const token = localStorage.getItem("token");
  return token ? { Authorization: `Bearer ${token}` } : {};
};

// Student
export const createRequest = async (payload) => {
  const res = await axios.post(API_URL, payload, { headers: authHeader() });
  return res.data;
};

export const getMyRequests = async () => {
  const res = await axios.get(`${API_URL}/my`, { headers: authHeader() });
  return res.data;
};

export const resubmitRequest = async (id, payload) => {
  const res = await axios.patch(`${API_URL}/${id}/resubmit`, payload, { headers: authHeader() });
  return res.data;
};

// Admin
export const getAllRequests = async (filters = {}) => {
  const params = Object.fromEntries(
    Object.entries(filters).filter(([, value]) => value !== undefined && value !== null && value !== "")
  );

  const res = await axios.get(`${API_URL}/all`, {
    headers: authHeader(),
    params,
  });
  return res.data;
};

export const saveApprovedDocument = async (id, payload) => {
  const res = await axios.patch(`${API_URL}/${id}/approved-document`, payload, { headers: authHeader() });
  return res.data;
};

export const updateRequestStatus = async (id, payload) => {
  const res = await axios.patch(`${API_URL}/${id}`, payload, { headers: authHeader() });
  return res.data;
};

export const getRequestById = async (id) => {
  const res = await axios.get(`${API_URL}/${id}`, { headers: authHeader() });
  return res.data;
};

// Public verify
export const verifyRequestCode = async (code) => {
  const res = await axios.get(`${API_URL}/verify/${code}`);
  return res.data;
};

// ■■ Agreement signing (public – no auth) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export const getBySigningToken = async (token) => {
  const res = await axios.get(`${API_URL}/sign/${token}`);
  return res.data;
};

export const repSubmit = async (token, payload) => {
  const res = await axios.post(`${API_URL}/sign/${token}/submit`, payload);
  return res.data;
};

export const repReject = async (token) => {
  const res = await axios.post(`${API_URL}/sign/${token}/reject`);
  return res.data;
```

```
};

// ■■ Agreement admin endpoints ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export const generateSigningLink = async (id) => {
  const res = await axios.patch(`${API_URL}/${id}/generate-signing-link`, {}, { headers: authHeader() });
  return res.data;
};

export const adminPhase3Action = async (id, payload) => {
  const res = await axios.patch(`${API_URL}/${id}/admin-phase3`, payload, { headers: authHeader() });
  return res.data;
};

export const getSignatureImages = async (id) => {
  const res = await axios.get(`${API_URL}/${id}/sig-images`, { headers: authHeader() });
  return res.data;
};

export const getRequestStats = async () => {
  const res = await axios.get(`${API_URL}/stats`, { headers: authHeader() });
  return res.data;
};

export const getArchivedRequests = async () => {
  const res = await axios.get(`${API_URL}/archived`, { headers: authHeader() });
  return res.data;
};

export const getRetentionStats = async () => {
  const res = await axios.get(`${API_URL}/retention-stats`, { headers: authHeader() });
  return res.data;
};

export const runArchiveNow = async () => {
  const res = await axios.post(`${API_URL}/run-archive`, {}, { headers: authHeader() });
  return res.data;
};
```

24. client/src/services/auditService.js (22 lines)

```
1  import axios from "axios";
2
3  const API_URL = `${import.meta.env.VITE_API_BASE_URL}/api/audit`;
4
5  const authHeader = () => {
6    const token = localStorage.getItem("token");
7    return token ? { Authorization: `Bearer ${token}` } : {};
8  };
9
10 export const getAuditLogs = async ({ page = 1, limit = 50, action = "", userId = "", resourceId = "" } = {}) => {
11   const params = new URLSearchParams({ page, limit });
12   if (action) params.set("action", action);
13   if (userId) params.set("userId", userId);
14   if (resourceId) params.set("resourceId", resourceId);
15   const res = await axios.get(`${API_URL}?${params}`, { headers: authHeader() });
16   return res.data;
17 };
18
19 export const getRecentAuditLogs = async (count = 5) => {
20   const res = await axios.get(`${API_URL}/recent?count=${count}`, { headers: authHeader() });
21   return res.data;
22 };
```

25. client/src/services/firebaseStorageService.js (134 lines)

```
import { ref, uploadBytes, getDownloadURL, deleteObject } from "firebase/storage";
import { storage } from "../firebase";

const slugify = (str = "") =>
  String(str)
    .trim()
    .toLowerCase()
    .replace(/\s+/g, "-")
    .replace(/[^a-z0-9-_]/g, "");

const safeFileName = (name = "") =>
  String(name).replace(/[^a-zA-Z0-9._-]/g, "_");

// date_timestamp folder example: "2-16-2026_1139" or "2-16-2026_2329"
export const getDateRequestFolder = () => {
  const d = new Date();
  const date = d.toLocaleDateString("en-US").replaceAll("/", "-"); // M-D-YYYY
  const hhmm = `${String(d.getHours()).padStart(2, "0")}${String(
    d.getMinutes()
  ).padStart(2, "0")}`;
  return `${date}_${hhmm}`;
};

export async function uploadRequirements(files, type, studentName, requestFolder) {
  const uploaded = [];
  const studentSlug = slugify(studentName);
  const folder = requestFolder || getDateRequestFolder();

  for (const file of Array.from(files || [])) {
    const name = safeFileName(file.name);

    // <slugify_name_folder>/<requests_folder>/<request_type_folder>/<date_timestamp_folder>/<actual file>
    const path = `${studentSlug}/requests/${type}/${folder}/${name}`;

    const fileRef = ref(storage, path);
    const snap = await uploadBytes(fileRef, file);
    const url = await getDownloadURL(snap.ref);

    uploaded.push({
      origName: file.name,
      url,
      path,
      contentType: file.type,
      uploadedAt: new Date().toISOString(),
      requestFolder: folder,
    });
  }

  return uploaded;
}

export async function uploadApprovedForm(pdfBlob, type, studentName, requestFolder, fileName) {
  const studentSlug = slugify(studentName);
  const safeName = safeFileName(fileName);

  const path = `${studentSlug}/requests/${type}/${requestFolder}/approved_form/${safeName}`;

  const typedBlob = new Blob([pdfBlob], { type: "application/pdf" });
  const meta = { contentType: "application/pdf" };

  const fileRef = ref(storage, path);
  const snap = await uploadBytes(fileRef, typedBlob, meta);
  const url = await getDownloadURL(snap.ref);

  return { url, path, issuedAt: new Date().toISOString() };
}

export async function uploadApprovedQrImage(qrDataUrl, type, studentName, requestFolder) {
  const studentSlug = slugify(studentName);
  const path = `${studentSlug}/requests/${type}/${requestFolder}/approved_form/qr_code.png`;

  const qrBlob = await fetch(qrDataUrl).then((res) => res.blob());
  const meta = { contentType: "image/png" };
```

```
74
75    const fileRef = ref(storage, path);
76    const snap = await uploadBytes(fileRef, qrBlob, meta);
77    const url = await getDownloadURL(snap.ref);
78
79    return { url, path };
80  }
81
82  /**
83   * Upload a base64 signature data URL as a PNG to Firebase Storage.
84   * Returns { url, path }.
85   */
86  export async function uploadSignatureImage(dataUrl, type, studentName, requestFolder, sigName) {
87    const studentSlug = slugify(studentName);
88    const path = `${studentSlug}/requests/${type}/${requestFolder}/sigs/${sigName}`;
89
90    const blob = await fetch(dataUrl).then((r) => r.blob());
91    const meta = { contentType: "image/png" };
92
93    const fileRef = ref(storage, path);
94    const snap = await uploadBytes(fileRef, blob, meta);
95    const url = await getDownloadURL(snap.ref);
96
97    return { url, path };
98  }
99
100 /**
101  * Upload a File object for the representative's government ID.
102  * Returns { origName, url, path, contentType, uploadedAt }.
103  */
104 export async function uploadRepGovId(file, type, studentName, requestFolder) {
105   const studentSlug = slugify(studentName);
106   const name = safeFileName(file.name);
107   const path = `${studentSlug}/requests/${type}/${requestFolder}/rep_docs/${name}`;
108
109   const fileRef = ref(storage, path);
110   const snap = await uploadBytes(fileRef, file);
111   const url = await getDownloadURL(snap.ref);
112
113   return {
114     origName: file.name,
115     url,
116     path,
117     contentType: file.type,
118     uploadedAt: new Date().toISOString(),
119   };
120 }
121
122 /**
123  * Delete a file from Firebase Storage by its storage path.
124  * Silently ignores not-found errors.
125  */
126 export async function deleteStorageFile(filePath) {
127   if (!filePath) return;
128   try {
129     const fileRef = ref(storage, filePath);
130     await deleteObject(fileRef);
131   } catch (err) {
132     console.warn("deleteStorageFile:", err.message);
133   }
134 }
```

26. client/src/services/qrService.js (8 lines)

```
1  import QRCode from "qrcode";
2
3  export const buildVerificationUrl = (serialNo) =>
4    `${window.location.origin}/verify/${serialNo}`;
5
6  export const generateQrDataUrl = async (url) => {
7    return QRCode.toDataURL(url, { margin: 1, width: 180 });
8  };
```

27. client/src/utils/passwordPolicy.js (24 lines)

```
export const PASSWORD_POLICY_MESSAGE =
  "Minimum 8 chars, with uppercase, lowercase, number, and at least one special character";

export const getPasswordChecks = (value = "") => {
  const p = String(value || "");
  return [
    { key: "length",  label: "At least 8 characters",          pass: p.length >= 8 },
    { key: "upper",   label: "At least 1 uppercase letter",     pass: /[A-Z]/.test(p) },
    { key: "lower",   label: "At least 1 lowercase letter",     pass: /[a-z]/.test(p) },
    { key: "number",  label: "At least 1 number",               pass: /[0-9]/.test(p) },
    { key: "symbol",  label: "At least 1 special character",    pass: /[^A-Za-z0-9]/.test(p) },
  ];
};

export const isStrongPassword = (value = "") => {
  const p = String(value || "");
  return (
    p.length >= 8 &&
    /[A-Z]/.test(p) &&
    /[a-z]/.test(p) &&
    /[0-9]/.test(p) &&
    /[^A-Za-z0-9]/.test(p)
  );
};
```

28. client/src/utils/inPageFeedback.js (113 lines)

```
const NOTIFY_EVENT = "dpo:notify";
const CONFIRM_EVENT = "dpo:confirm";

const STATUS_LABELS = {
  nda_pending: "Reviewal",
  nda_approved: "Approved",
  stud_revision_requested: "Student Revisions",
  agr_pending_1: "Initial Reviewal",
  agr_awaiting_rep_signature: "Recipient Reviewal",
  agr_pending_2: "Final Reviewal",
  agr_approved: "Approved",
  agr_declined: "Recipient Declined",
  agr_rep_revision_requested: "Recipient Revisions",
};

const toTitleCase = (value) => String(value || "")
  .replace(/_/g, " ")
  .toLowerCase()
  .replace(/\b\w/g, (char) => char.toUpperCase());

const humanizeStatusCode = (status) => STATUS_LABELS[status] || toTitleCase(status);

export function normalizeFeedbackMessage(message = "") {
  const raw = String(message || "").trim();
  if (!raw) return "";

  const updatedMatch = raw.match(/^Updated to\s+([A-Za-z0-9_]+)$/i);
  if (updatedMatch) {
    return `Request status updated to ${humanizeStatusCode(updatedMatch[1])}.`;
  }

  const shortStatusMatch = raw.match(/^Request moved to\s+(.+)\.$/i);
  if (shortStatusMatch) {
    return `Request moved to ${shortStatusMatch[1]}.`;
  }

  if (/^Failed\b/i.test(raw)) {
    return raw.endsWith(".") ? raw : `${raw}.`;
  }

  if (/^Approved\b/i.test(raw) || /^Resubmitted\b/i.test(raw) || /^Copied!?$/i.test(raw)) {
    return raw.endsWith(".") ? raw : `${raw}.`;
  }

  return raw;
}

export function inferFeedbackType(message = "") {
  const text = String(message).toLowerCase();
  if (
    text.includes("fail")
    || text.includes("error")
    || text.includes("invalid")
    || text.includes("missing")
    || text.includes("declined")
  ) {
    return "error";
  }
  if (text.includes("warn") || text.includes("expired") || text.includes("required")) {
    return "warning";
  }
  if (
    text.includes("success")
    || text.includes("approved")
    || text.includes("submitted")
    || text.includes("copied")
    || text.includes("updated")
    || text.includes("generated")
  ) {
    return "success";
  }
  return "info";
}
```

```
74
75  export function notify(message, options = {}) {
76    if (typeof window === "undefined") return;
77    const normalizedMessage = normalizeFeedbackMessage(message);
78    window.dispatchEvent(
79      new CustomEvent(NOTIFY_EVENT, {
80        detail: {
81          message: normalizedMessage,
82          type: options.type || inferFeedbackType(normalizedMessage),
83          title: options.title,
84          duration: options.duration,
85        },
86      })
87    );
88  }
89
90  export function confirmInPage(options) {
91    if (typeof window === "undefined") return Promise.resolve(false);
92    const normalized = typeof options === "string" ? { message: options } : (options || {});
93
94    return new Promise((resolve) => {
95      window.dispatchEvent(
96        new CustomEvent(CONFIRM_EVENT, {
97          detail: {
98            title: normalized.title || "Please confirm",
99            message: normalized.message || "Are you sure you want to continue?",
100           confirmText: normalized.confirmText || "Confirm",
101           cancelText: normalized.cancelText || "Cancel",
102           tone: normalized.tone || "warning",
103           resolve,
104         },
105       })
106     );
107   });
108 }
109
110 export const FEEDBACK_EVENTS = {
111   NOTIFY_EVENT,
112   CONFIRM_EVENT,
113 };
```

29. client/src/utils/requestStatusCharts.js (43 lines)

```
export const APPROVED_STATUSES = ["nda_approved", "agr_approved"];

export const PENDING_STATUSES = [
  "nda_pending",
  "agr_pending_1",
  "agr_pending_2",
  "agr_awaiting_rep_signature",
];

export const REVISION_STATUSES = [
  "stud_revision_requested",
  "agr_rep_revision_requested",
];

export const CHART_LABELS = {
  chart_reviewal: "Reviewal",
  chart_approved: "Approved",
  chart_stud_revision: "Student Revisions",
  chart_rep_revision: "Recipient Revisions",
  chart_rep_declined: "Recipient Declined",
  chart_awaiting_rep: "Recipient Reviewal",
};

export const CHART_COLORS = {
  chart_reviewal: "#ca8a04",
  chart_approved: "#059669",
  chart_stud_revision: "#ea580c",
  chart_rep_revision: "#7c3aed",
  chart_rep_declined: "#dc2626",
  chart_awaiting_rep: "#2563eb",
};

export const normalizeStatusForChart = (status) => {
  if (["nda_pending", "agr_pending_1", "agr_pending_2"].includes(status)) return "chart_reviewal";
  if (APPROVED_STATUSES.includes(status)) return "chart_approved";
  if (status === "stud_revision_requested") return "chart_stud_revision";
  if (status === "agr_rep_revision_requested") return "chart_rep_revision";
  if (status === "agr_declined") return "chart_rep_declined";
  if (status === "agr_awaiting_rep_signature") return "chart_awaiting_rep";

  // Unknown statuses fallback
  return "chart_reviewal";
};
```

30. client/src/config/fieldsFileSlotsConfig.js (45 lines)

```
export const FIELDS_FILE_SLOTS_CONFIG = {
  nda: {
    orgactivities: {
      label: "Student Organization Activities",
      fields: [
        { name: "projectTitle", label: "Project Title", required: true },
      ],
      fileSlots: [
        { label: "Letter of Intent", required: true },
        { label: "School ID", required: false },
        { label: "Registration Form", required: false },
        { label: "Sample Forms or Collaterals (if applicable)", required: false },
      ],
    },

    research: {
      label: "Conduct of Research",
      fields: [
        { name: "researchTitle", label: "Research Title", required: true },
      ],
      fileSlots: [
        { label: "Letter of Intent", required: true },
        { label: "Questionnaire with Data Privacy Consent Form", required: false },
        { label: "School Id", required: false },
        { label: "Registration Form", required: false },
      ],
    },
  },

  agreement: {
    label: "Agreement",
    fields: [
      { name: "repFirstName", label: "First Name", required: true },
      { name: "repMiddleInitial", label: "Middle Name", required: false },
      { name: "repLastName", label: "Last Name", required: true },
      { name: "repEmail", label: "Email", required: true, type: "email" },
    ],
    // Note: Representative's Government Issued Valid ID is now collected directly
    // from the representative on the signing page, not from the student.
    fileSlots: [
      { label: "Notarized Authorization Letter", required: true },
      { label: "Requestor's Government Issued Valid ID", required: false },
    ],
  },
};
```

31. client/src/config/documentTemplates/index.jsx (48 lines)

```jsx
/* eslint-disable react-refresh/only-export-components */
import React from "react";

const renderFallbackDoc = ({ request, renderer, styles }) => {
  const { Document, Page } = renderer;
  const { s, Header, BodyText, Footer } = styles;

  return (
    <Document>
      <Page size="A4" style={s.page}>
        <Header title="APPROVED DOCUMENT" />
        <BodyText>Approved document for request {request?._id}</BodyText>
        <BodyText>(No template found for this request type)</BodyText>
        <Footer />
      </Page>
    </Document>
  );
};

export const generateApprovedPDF = async (request) => {
  const [renderer, styles] = await Promise.all([
    import("@react-pdf/renderer"),
    import("./styles"),
  ]);

  let DocComponent = null;

  if (request.type === "agreement") {
    const mod = await import("./AgreementDoc");
    DocComponent = mod.default;
  } else if (request.type === "nda") {
    const t = request.formData?.ndaType;
    if (t === "orgactivities") {
      const mod = await import("./NDAStudentOrgActivitiesDoc");
      DocComponent = mod.default;
    } else if (t === "research") {
      const mod = await import("./NDAResearchDoc");
      DocComponent = mod.default;
    }
  }

  const doc = DocComponent
    ? <DocComponent request={request} />
    : renderFallbackDoc({ request, renderer, styles });

  const blob = await renderer.pdf(doc).toBlob();
  return new Blob([blob], { type: "application/pdf" });
};
```

32. client/src/config/documentTemplates/styles.jsx (100 lines)

```
/* eslint-disable react-refresh/only-export-components */
import React from "react";
import { Text, View, StyleSheet, Image } from "@react-pdf/renderer";

export const s = StyleSheet.create({
  page: { padding: 40, fontSize: 10, fontFamily: "Helvetica", color: "#222" },

  headerTitle: { fontSize: 18, fontFamily: "Helvetica-Bold", textAlign: "center", marginBottom: 2 },
  headerSubtitle: { fontSize: 12, fontFamily: "Helvetica-Bold", textAlign: "center", marginBottom: 2 },
  headerOffice: { fontSize: 9, textAlign: "center", color: "#555", marginBottom: 12 },

  hr: { borderBottomWidth: 1, borderBottomColor: "#bbb", marginBottom: 12 },

  metaRow: { flexDirection: "row", marginBottom: 4 },
  metaLabel: { width: 105, fontFamily: "Helvetica-Bold", fontSize: 10 },
  metaValue: { flex: 1, fontSize: 10 },

  sectionTitle: { fontSize: 11, fontFamily: "Helvetica-Bold", marginTop: 14, marginBottom: 6, color: "#111" },

  fieldRow: { flexDirection: "row", marginBottom: 4, paddingLeft: 8 },
  fieldLabel: { width: 130, fontFamily: "Helvetica-Bold", fontSize: 10 },
  fieldValue: { flex: 1, fontSize: 10 },

  bodyText: { fontSize: 10, lineHeight: 1.6, paddingLeft: 8, marginBottom: 6 },

  footerWrap: { position: "absolute", bottom: 40, left: 40, right: 40 },
  footerHr: { borderBottomWidth: 1, borderBottomColor: "#bbb", marginBottom: 8 },
  footerRow: { flexDirection: "column", alignItems: "flex-start" },
  footerLeft: { flex: 1, paddingRight: 10 },
  footerTitle: { fontSize: 11, fontFamily: "Helvetica-Bold", textAlign: "left", marginBottom: 4 },
  footerNote: { fontSize: 8, textAlign: "left", color: "#888" },
  footerQrWrap: { alignItems: "flex-start", width: 90, marginTop: 8, paddingLeft: 8 },
  footerQr: { width: 70, height: 70 },
  footerQrLabel: { fontSize: 7, color: "#888", marginTop: 4, textAlign: "left" },
});

export const formatDate = (iso) => (iso ? new Date(iso).toLocaleDateString("en-US", { year: "numeric", month: "short", day: "

export const getTemplateAssetUrl = (assetFileName) => {
  const clean = String(assetFileName || "").replace(/^\/+/, "");
  if (!clean) return "";

  const basePath = typeof import.meta !== "undefined" && import.meta.env?.BASE_URL
    ? import.meta.env.BASE_URL
    : "/";
  const normalizedBase = `${basePath}`.endsWith("/") ? `${basePath}` : `${basePath}/`;
  const relative = `${normalizedBase}${clean}`;

  if (typeof window !== "undefined" && window.location?.origin) {
    return new URL(relative, window.location.origin).toString();
  }

  return `/${clean}`;
};

export const Header = ({ title, subtitle }) => (
  <View>
    <Text style={s.headerTitle}>{title}</Text>
    {subtitle ? <Text style={s.headerSubtitle}>{subtitle}</Text> : null}
    <Text style={s.headerOffice}>Data Protection Office</Text>
    <View style={s.hr} />
  </View>
);

export const MetaRow = ({ label, value }) => (
  <View style={s.metaRow}>
    <Text style={s.metaLabel}>{label}:</Text>
    <Text style={s.metaValue}>{String(value || "N/A")}</Text>
  </View>
);

export const SectionTitle = ({ children }) => <Text style={s.sectionTitle}>{children}</Text>;

```

```
export const FieldRow = ({ label, value }) => (
  <View style={s.fieldRow}>
    <Text style={s.fieldLabel}>{label}:</Text>
    <Text style={s.fieldValue}>{String(value || "N/A")}</Text>
  </View>
);

export const BodyText = ({ children }) => <Text style={s.bodyText}>{children}</Text>;

export const Footer = ({ qrDataUrl, verificationUrl }) => (
  <View style={s.footerWrap} fixed>
    <View style={s.footerHr} />
    <View style={s.footerRow}>
      <View style={s.footerLeft}>
        <Text style={s.footerTitle}>APPROVED — Data Protection Office</Text>
        <Text style={s.footerNote}>(Placeholder — e-signature + QR code will go here)</Text>
        {verificationUrl ? <Text style={s.footerNote}>{verificationUrl}</Text> : null}
      </View>
      {qrDataUrl ? (
        <View style={s.footerQrWrap}>
          <Image style={s.footerQr} src={qrDataUrl} />
          <Text style={s.footerQrLabel}>Scan to verify</Text>
        </View>
      ) : null}
    </View>
  </View>
);
```

33. client/src/config/documentTemplates/AgreementDoc.jsx (191 lines)

```
import React from "react";
import { Document, Page, Image, View, Text, StyleSheet } from "@react-pdf/renderer";
import { formatDate, getTemplateAssetUrl } from "./styles";

const styles = StyleSheet.create({
  page: {
    paddingTop: 18,
    paddingBottom: 18,
    paddingHorizontal: 20,
    fontSize: 8.6,
    fontFamily: "Times-Roman",
    color: "#111",
  },
  copyCard: {
    paddingTop: 8,
    paddingBottom: 8,
    paddingHorizontal: 10,
    minHeight: 348,
  },
  cutLine: {
    textAlign: "center",
    fontSize: 7,
    color: "#666",
    letterSpacing: 1.5,
    marginVertical: 5,
  },
  headerRow: { flexDirection: "row", justifyContent: "space-between", alignItems: "flex-start" },
  headerBlock: { width: "48%" },
  headerLeftTitle: { fontSize: 7.8, fontFamily: "Times-Bold" },
  headerLeftCode: { fontSize: 7, marginTop: 1 },
  headerRight: { alignItems: "flex-end" },
  headerRightLine: { fontSize: 7 },
  logoRow: { flexDirection: "row", justifyContent: "center", alignItems: "center", marginTop: 6, marginBottom: 6 },
  logo: { width: 32, height: 32, objectFit: "contain", marginHorizontal: 6 },
  logoDpo: { width: 28, height: 28, objectFit: "contain", marginHorizontal: 6 },
  centerTextWrap: { alignItems: "center", marginHorizontal: 4 },
  centerLine: { fontSize: 7.4, textAlign: "center" },
  centerOffice: { fontSize: 7.4, textAlign: "center" },
  centerAgreement: { fontSize: 8.6, fontFamily: "Times-Bold", textAlign: "center", marginTop: 1 },
  paragraph: {
    width: "92%",
    alignSelf: "center",
    textAlign: "justify",
    textIndent: 12,
    lineHeight: 1.12,
    marginTop: 10,
  },
  paragraphFinal: {
    width: "92%",
    alignSelf: "center",
    textAlign: "justify",
    textIndent: 12,
    lineHeight: 1.12,
    marginTop: 10,
  },
  afterParagraph: {
    marginTop: 8,
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "flex-start",
    width: "92%",
    alignSelf: "center",
  },
  rightColumn: { alignItems: "flex-end" },
  sigBlock: { width: 158, alignItems: "flex-start", marginBottom: 6 },
  sigImage: { width: 122, height: 28, objectFit: "contain", borderBottomWidth: 0.8, borderBottomColor: "#333" },
  sigPlaceholder: { width: 122, height: 28, borderBottomWidth: 0.8, borderBottomColor: "#333" },
  sigName: { fontSize: 7.4, fontFamily: "Times-Bold", marginTop: 2, textAlign: "left" },
  sigLabel: { fontSize: 6.8, textAlign: "left" },
  dateReceived: { fontSize: 7, alignSelf: "flex-start", marginBottom: 2, textAlign: "left" },
  qrWrap: { alignItems: "flex-start", marginBottom: 4 },
  qrImage: { width: 44, height: 44 },
});
```

```
 74
 75  const SigBlock = ({ label, name, imgUrl, dateReceived }) => (
 76    <View style={styles.sigBlock}>
 77      {dateReceived ? <Text style={styles.dateReceived}>{dateReceived}</Text> : null}
 78      {imgUrl
 79        ? <Image style={styles.sigImage} src={imgUrl} />
 80        : <View style={styles.sigPlaceholder} />
 81      }
 82      <Text style={styles.sigName}>{name || ""}</Text>
 83      <Text style={styles.sigLabel}>{label}</Text>
 84    </View>
 85  );
 86
 87  /**
 88   * AgreementDoc renders progressively based on which signatures are present:
 89   *  - authorizerSigUrl  → shown from phase2 preview onward
 90   *  - repSigUrl         → shown from phase3 review onward
 91   *  - adminSigDataUrl   → only in the final approved PDF
 92   */
 93  export default function AgreementDoc({ request }) {
 94    const student = request.userId || {};
 95    const proxy = request.proxyRequestee || {};
 96    const repInfo = request.repInfo || {};
 97    const { authorizerSigUrl, repSigUrl, adminSigDataUrl } = request;
 98    const approverName = request.approverName || request.adminName || request.approvedByName || "";
 99    const toUpper = (value) => (value ? String(value).toUpperCase() : "");
100    const repNameFromForm = [
101      request.formData?.repFirstName,
102      request.formData?.repMiddleInitial,
103      request.formData?.repLastName,
104    ]
105      .filter(Boolean)
106      .join(" ")
107      .replace(/\s+/g, " ")
108      .trim();
109    const recipientName = repInfo.name || repNameFromForm || request.formData?.repName;
110    const rtuLogoSrc = getTemplateAssetUrl("rtu-logo.png");
111    const dpoLogoSrc = getTemplateAssetUrl("dpo-logo-full.png");
112
113    const hasAnySig = authorizerSigUrl || repSigUrl || adminSigDataUrl;
114
115    const AgreementCopy = () => (
116      <View style={styles.copyCard}>
117        <View style={styles.headerRow}>
118          <View style={styles.headerBlock}>
119            <Text style={styles.headerLeftTitle}>RIZAL TECHNOLOGICAL UNIVERSITY</Text>
120            <Text style={styles.headerLeftCode}>RTU-OP-UDPO-F002</Text>
121          </View>
122          <View style={[styles.headerBlock, styles.headerRight]}>
123            <Text style={styles.headerRightLine}>Effectivity {formatDate(request.updatedAt)}</Text>
124            <Text style={styles.headerRightLine}>Control # {request.serialNo || "N/A"}</Text>
125          </View>
126        </View>
127
128        <View style={styles.logoRow}>
129          <Image style={styles.logo} src={rtuLogoSrc} />
130          <View style={styles.centerTextWrap}>
131            <Text style={styles.centerLine}>Rizal Technological University</Text>
132            <Text style={styles.centerLine}>Boni Avenue, City of Mandaluyong</Text>
133            <Text style={styles.centerOffice}>UNIVERSITY DATA PROTECTION CENTER</Text>
134            <Text style={styles.centerAgreement}>AGREEMENT</Text>
135          </View>
136          <Image style={styles.logoDpo} src={dpoLogoSrc} />
137        </View>
138
139        <Text style={styles.paragraph}>
140          I declare that as the intended recipient, it is my responsibility to safeguard the integrity and
141          confidentiality of the enclosed information. As such, I ensure not to reproduce, share, or in any way
142          disclose any information to anyone without the consent of the data subject. Further, I attest that the
143          herein data shall be used for processing of school credentials.
144        </Text>
145        <Text style={styles.paragraphFinal}>
146          Finally, I understand that I may be held liable for any damage caused by unauthorized processing of
147          information.
148        </Text>
149
```

```
      <View style={styles.afterParagraph}>
        {request.qrDataUrl ? (
          <View style={styles.qrWrap}>
            <Image style={styles.qrImage} src={request.qrDataUrl} />
          </View>
        ) : null}
        <View style={styles.rightColumn}>
          {hasAnySig ? (
            <>
              <SigBlock
                label="Authorizer's signature over printed name"
                name={toUpper(proxy.isProxy ? (proxy.fullName || "") : (student.name || ""))}
                imgUrl={authorizerSigUrl || null}
              />
              <SigBlock
                label="Recipient's signature over printed name"
                name={toUpper(recipientName)}
                imgUrl={repSigUrl || null}
              />
              <SigBlock
                dateReceived={`Date Received: ${formatDate(request.updatedAt)}`}
                label="Approving Officer's Signature over printed name"
                name={toUpper(approverName)}
                imgUrl={adminSigDataUrl || null}
              />
            </>
          ) : null}
        </View>
      </View>
    </View>
  );

  return (
    <Document>
      <Page size="A4" style={styles.page}>
        <AgreementCopy />
        <Text style={styles.cutLine}>{"- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
        <AgreementCopy />
      </Page>
    </Document>
  );
}
```

34. client/src/config/documentTemplates/NDAResearchDoc.jsx (178 lines)

```
import React from "react";
import { Document, Page, View, Image, Text, StyleSheet } from "@react-pdf/renderer";
import { formatDate, getTemplateAssetUrl } from "./styles";

const styles = StyleSheet.create({
  page: {
    paddingTop: 36,
    paddingBottom: 40,
    paddingHorizontal: 36,
    fontSize: 10,
    fontFamily: "Times-Roman",
    color: "#111",
  },
  headerRow: { flexDirection: "row", justifyContent: "space-between" },
  headerBlock: { width: "48%" },
  headerLeftTitle: { fontSize: 10, fontFamily: "Times-Bold" },
  headerLeftCode: { fontSize: 9, marginTop: 2 },
  headerRight: { alignItems: "flex-end" },
  headerRightLine: { fontSize: 9 },
  logoRow: { flexDirection: "row", justifyContent: "center", alignItems: "center", marginTop: 12, marginBottom: 10 },
  logo: { width: 70, height: 70, objectFit: "contain", marginHorizontal: 8 },
  logoDpo: { width: 60, height: 60, objectFit: "contain", marginHorizontal: 8 },
  centerTextWrap: { alignItems: "center", marginHorizontal: 6 },
  centerLine: { fontSize: 10, textAlign: "center" },
  centerOffice: { fontSize: 10, textAlign: "center" },
  centerAgreement: { fontSize: 11, fontFamily: "Times-Bold", textAlign: "center", marginTop: 2 },
  paragraph: {
    width: "85%",
    alignSelf: "center",
    textAlign: "justify",
    textIndent: 24,
    lineHeight: 1.25,
    marginTop: 14,
  },
  paragraphNoIndent: {
    width: "85%",
    alignSelf: "center",
    textAlign: "center",
    marginTop: 8,
    marginBottom: 4,
  },
  titleText: {
    fontFamily: "Times-BoldItalic",
  },
  paragraphFinal: {
    width: "85%",
    alignSelf: "center",
    textAlign: "justify",
    textIndent: 24,
    lineHeight: 1.25,
    marginTop: 14,
  },
  afterParagraph: {
    marginTop: 18,
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "flex-end",
    width: "85%",
    alignSelf: "center",
  },
  rightColumn: { alignItems: "flex-end" },
  sigBlock: { width: 220, alignItems: "flex-start", marginBottom: 14 },
  sigImage: { width: 150, height: 50, objectFit: "contain", borderBottomWidth: 1, borderBottomColor: "#333" },
  sigPlaceholder: { width: 150, height: 50, borderBottomWidth: 1, borderBottomColor: "#333" },
  sigName: { fontSize: 10, fontFamily: "Times-Bold", marginTop: 4, textAlign: "left" },
  sigLabel: { fontSize: 8.5, textAlign: "left" },
  dateReceived: { fontSize: 9, alignSelf: "flex-start", marginBottom: 4, textAlign: "left" },
  qrWrap: { alignItems: "flex-start", marginBottom: 10 },
  qrImage: { width: 70, height: 70 },
});

const SigBlock = ({ label, name, imgUrl, dateReceived }) => (
  <View style={styles.sigBlock}>
```

```
    {dateReceived ? <Text style={styles.dateReceived}>{dateReceived}</Text> : null}
    {imgUrl
      ? <Image style={styles.sigImage} src={imgUrl} />
      : <View style={styles.sigPlaceholder} />
    }
    <Text style={styles.sigName}>{name || ""}</Text>
    <Text style={styles.sigLabel}>{label}</Text>
  </View>
);

export default function NDAResearchDoc({ request }) {
  const fd = request.formData || {};
  const student = request.userId || {};
  const proxy = request.proxyRequestee || {};
  const { studentSigDataUrl, adminSigDataUrl } = request;
  const approverName = request.approverName || request.adminName || request.approvedByName || "";
  const toUpper = (value) => (value ? String(value).toUpperCase() : "");
  const studentNameUpper = toUpper(proxy.isProxy ? (proxy.fullName || "") : (student.name || ""));
  const hasAnySig = studentSigDataUrl || adminSigDataUrl;
  const rtuLogoSrc = getTemplateAssetUrl("rtu-logo.png");
  const dpoLogoSrc = getTemplateAssetUrl("dpo-logo-full.png");

  return (
    <Document>
      <Page size="A4" style={styles.page}>
        <View style={styles.headerRow}>
          <View style={styles.headerBlock}>
            <Text style={styles.headerLeftTitle}>RIZAL TECHNOLOGICAL UNIVERSITY</Text>
            <Text style={styles.headerLeftCode}>RTU-OEVP-UDPC-F005</Text>
          </View>
          <View style={[styles.headerBlock, styles.headerRight]}>
            <Text style={styles.headerRightLine}>Effectivity {formatDate(request.updatedAt)}</Text>
            <Text style={styles.headerRightLine}>Control # {request.serialNo || "N/A"}</Text>
          </View>
        </View>

        <View style={styles.logoRow}>
          <Image style={styles.logo} src={rtuLogoSrc} />
          <View style={styles.centerTextWrap}>
            <Text style={styles.centerLine}>Rizal Technological University</Text>
            <Text style={styles.centerLine}>Boni Avenue, City of Mandaluyong</Text>
            <Text style={styles.centerOffice}>UNIVERSITY DATA PROTECTION CENTER</Text>
            <Text style={styles.centerAgreement}>NON-DISCLOSURE AGREEMENT</Text>
          </View>
          <Image style={styles.logoDpo} src={dpoLogoSrc} />
        </View>

        <Text style={styles.paragraph}>
          I, {studentNameUpper || "_________________"}, as the representative of the group, declare that it is our
          responsibility to protect the integrity and confidentiality of the enclosed information. As the intended
          recipient, we ensure that we will not reproduce, share, or disclose any information to anyone without the
          consent of the data subject. Furthermore, we affirm that the data will only be used to conduct the research
          titled:
        </Text>
        <Text style={styles.paragraphNoIndent}>
          <Text style={styles.titleText}>"{fd.researchTitle || "_________________"}"</Text>
        </Text>
        <Text style={styles.paragraph}>
          We guarantee our commitment to strictly adhere to the Data Privacy Act of 2012, as well as any current and
          applicable Implementing Rules and Regulations of the National Privacy Commission and other relevant
          legislation, rules, and publications.
        </Text>
        <Text style={styles.paragraph}>
          We acknowledge the necessity of implementing the required security measures to protect the data, which
          includes establishing appropriate organizational, physical, and technical security actions. Any
          unauthorized disclosure, risk, or suspicious regarding such matters must be reported immediately to the
          Data Protection Office.
        </Text>
        <Text style={styles.paragraph}>
          We categorically guarantee that we will not reveal any generally Confidential, highly sensitive data, or
          private information obtained, either directly or indirectly, from the respondents to any third party.
        </Text>
        <Text style={styles.paragraphFinal}>
          Finally, we understand that we may be held liable for any damages caused by the unauthorized processing of
          this information.
        </Text>
```

```
150
151         <View style={styles.afterParagraph}>
152           {request.qrDataUrl ? (
153             <View style={styles.qrWrap}>
154               <Image style={styles.qrImage} src={request.qrDataUrl} />
155             </View>
156           ) : null}
157           <View style={styles.rightColumn}>
158             {hasAnySig ? (
159               <>
160                 <SigBlock
161                   label="Researcher's signature over printed name"
162                   name={studentNameUpper}
163                   imgUrl={studentSigDataUrl || null}
164                 />
165                 <SigBlock
166                   dateReceived={`Date Received: ${formatDate(request.updatedAt)}`}
167                   label="Approving Officer's Signature over printed name"
168                   name={toUpper(approverName)}
169                   imgUrl={adminSigDataUrl || null}
170                 />
171               </>
172             ) : null}
173           </View>
174         </View>
175       </Page>
176     </Document>
177   );
178 }
```

35. client/src/config/documentTemplates/NDAStudentOrgActivitiesDoc.jsx (178 lines)

```
import React from "react";
import { Document, Page, View, Image, Text, StyleSheet } from "@react-pdf/renderer";
import { formatDate, getTemplateAssetUrl } from "./styles";

const styles = StyleSheet.create({
  page: {
    paddingTop: 36,
    paddingBottom: 40,
    paddingHorizontal: 36,
    fontSize: 10,
    fontFamily: "Times-Roman",
    color: "#111",
  },
  headerRow: { flexDirection: "row", justifyContent: "space-between" },
  headerBlock: { width: "48%" },
  headerLeftTitle: { fontSize: 10, fontFamily: "Times-Bold" },
  headerLeftCode: { fontSize: 9, marginTop: 2 },
  headerRight: { alignItems: "flex-end" },
  headerRightLine: { fontSize: 9 },
  logoRow: { flexDirection: "row", justifyContent: "center", alignItems: "center", marginTop: 12, marginBottom: 10 },
  logo: { width: 70, height: 70, objectFit: "contain", marginHorizontal: 8 },
  logoDpo: { width: 60, height: 60, objectFit: "contain", marginHorizontal: 8 },
  centerTextWrap: { alignItems: "center", marginHorizontal: 6 },
  centerLine: { fontSize: 10, textAlign: "center" },
  centerOffice: { fontSize: 10, textAlign: "center" },
  centerAgreement: { fontSize: 11, fontFamily: "Times-Bold", textAlign: "center", marginTop: 2 },
  paragraph: {
    width: "85%",
    alignSelf: "center",
    textAlign: "justify",
    textIndent: 24,
    lineHeight: 1.25,
    marginTop: 14,
  },
  paragraphNoIndent: {
    width: "85%",
    alignSelf: "center",
    textAlign: "center",
    marginTop: 8,
    marginBottom: 4,
  },
  titleText: {
    fontFamily: "Times-BoldItalic",
  },
  paragraphFinal: {
    width: "85%",
    alignSelf: "center",
    textAlign: "justify",
    textIndent: 24,
    lineHeight: 1.25,
    marginTop: 14,
  },
  afterParagraph: {
    marginTop: 18,
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "flex-end",
    width: "85%",
    alignSelf: "center",
  },
  rightColumn: { alignItems: "flex-end" },
  sigBlock: { width: 220, alignItems: "flex-start", marginBottom: 14 },
  sigImage: { width: 150, height: 50, objectFit: "contain", borderBottomWidth: 1, borderBottomColor: "#333" },
  sigPlaceholder: { width: 150, height: 50, borderBottomWidth: 1, borderBottomColor: "#333" },
  sigName: { fontSize: 10, fontFamily: "Times-Bold", marginTop: 4, textAlign: "left" },
  sigLabel: { fontSize: 8.5, textAlign: "left" },
  dateReceived: { fontSize: 9, alignSelf: "flex-start", marginBottom: 4, textAlign: "left" },
  qrWrap: { alignItems: "flex-start", marginBottom: 10 },
  qrImage: { width: 70, height: 70 },
});

const SigBlock = ({ label, name, imgUrl, dateReceived }) => (
  <View style={styles.sigBlock}>
```

```
    {dateReceived ? <Text style={styles.dateReceived}>{dateReceived}</Text> : null}
    {imgUrl
      ? <Image style={styles.sigImage} src={imgUrl} />
      : <View style={styles.sigPlaceholder} />
    }
    <Text style={styles.sigName}>{name || ""}</Text>
    <Text style={styles.sigLabel}>{label}</Text>
  </View>
);

export default function NDAStudentOrgActivitiesDoc({ request }) {
  const fd = request.formData || {};
  const student = request.userId || {};
  const proxy = request.proxyRequestee || {};
  const { studentSigDataUrl, adminSigDataUrl } = request;
  const approverName = request.approverName || request.adminName || request.approvedByName || "";
  const toUpper = (value) => (value ? String(value).toUpperCase() : "");
  const studentNameUpper = toUpper(proxy.isProxy ? (proxy.fullName || "") : (student.name || ""));
  const hasAnySig = studentSigDataUrl || adminSigDataUrl;
  const rtuLogoSrc = getTemplateAssetUrl("rtu-logo.png");
  const dpoLogoSrc = getTemplateAssetUrl("dpo-logo-full.png");

  return (
    <Document>
      <Page size="A4" style={styles.page}>
        <View style={styles.headerRow}>
          <View style={styles.headerBlock}>
            <Text style={styles.headerLeftTitle}>RIZAL TECHNOLOGICAL UNIVERSITY</Text>
            <Text style={styles.headerLeftCode}>RTU-OEVP-UDPC-F005</Text>
          </View>
          <View style={[styles.headerBlock, styles.headerRight]}>
            <Text style={styles.headerRightLine}>Effectivity {formatDate(request.updatedAt)}</Text>
            <Text style={styles.headerRightLine}>Control # {request.serialNo || "N/A"}</Text>
          </View>
        </View>

        <View style={styles.logoRow}>
          <Image style={styles.logo} src={rtuLogoSrc} />
          <View style={styles.centerTextWrap}>
            <Text style={styles.centerLine}>Rizal Technological University</Text>
            <Text style={styles.centerLine}>Boni Avenue, City of Mandaluyong</Text>
            <Text style={styles.centerOffice}>UNIVERSITY DATA PROTECTION CENTER</Text>
            <Text style={styles.centerAgreement}>NON-DISCLOSURE AGREEMENT</Text>
          </View>
          <Image style={styles.logoDpo} src={dpoLogoSrc} />
        </View>

        <Text style={styles.paragraph}>
          I, {studentNameUpper || "_________________"}, as the representative of the group, declare that it is our
          responsibility to protect the integrity and confidentiality of the enclosed information. As the intended
          recipient, we ensure that we will not reproduce, share, or disclose any information to anyone without the
          consent of the data subject. Furthermore, we affirm that the data will only be used to conduct the project
          titled:
        </Text>
        <Text style={styles.paragraphNoIndent}>
          <Text style={styles.titleText}>"{fd.projectTitle || "_________________"}"</Text>
        </Text>
        <Text style={styles.paragraph}>
          We guarantee our commitment to strictly adhere to the Data Privacy Act of 2012, as well as any current and
          applicable Implementing Rules and Regulations of the National Privacy Commission and other relevant
          legislation, rules, and publications.
        </Text>
        <Text style={styles.paragraph}>
          We acknowledge the necessity of implementing the required security measures to protect the data, which
          includes establishing appropriate organizational, physical, and technical security actions. Any
          unauthorized disclosure, risk, or suspicious regarding such matters must be reported immediately to the
          Data Protection Office.
        </Text>
        <Text style={styles.paragraph}>
          We categorically guarantee that we will not reveal any generally Confidential, highly sensitive data, or
          private information obtained, either directly or indirectly, from the respondents to any third party.
        </Text>
        <Text style={styles.paragraphFinal}>
          Finally, we understand that we may be held liable for any damages caused by the unauthorized processing of
          this information.
        </Text>
```

```
150
151         <View style={styles.afterParagraph}>
152           {request.qrDataUrl ? (
153             <View style={styles.qrWrap}>
154               <Image style={styles.qrImage} src={request.qrDataUrl} />
155             </View>
156           ) : null}
157           <View style={styles.rightColumn}>
158             {hasAnySig ? (
159               <>
160                 <SigBlock
161                   label="Group Representative's signature over printed name"
162                   name={studentNameUpper}
163                   imgUrl={studentSigDataUrl || null}
164                 />
165                 <SigBlock
166                   dateReceived={`Date Received: ${formatDate(request.updatedAt)}`}
167                   label="Approving Officer's Signature over printed name"
168                   name={toUpper(approverName)}
169                   imgUrl={adminSigDataUrl || null}
170                 />
171               </>
172             ) : null}
173           </View>
174         </View>
175       </Page>
176     </Document>
177   );
178 }
```

36. client/src/components/Navbar.jsx (134 lines)

```
import { NavLink } from "react-router-dom";
import { useContext, useState } from "react";
import { AuthContext } from "../context/AuthContext";
import {
  FilePlus,
  ClipboardList,
  BarChart2,
  History,
  Users,
  Archive,
  Search,
  UserCircle,
  LogOut,
} from "lucide-react";
import { motion, AnimatePresence } from "framer-motion";
import "./Navbar.css";

const Navbar = () => {
  const { user, logout } = useContext(AuthContext);
  const [showLogoutConfirm, setShowLogoutConfirm] = useState(false);

  const linkClass = ({ isActive }) =>
    `navbar-link${isActive ? " active" : ""}`;

  const initials = user?.name
    ? user.name.split(" ").map((w) => w[0]).slice(0, 2).join("").toUpperCase()
    : (user?.firstName ? user.firstName[0].toUpperCase() : "?");

  return (
    <>
      <nav className="navbar">
        {/* Navigation links */}
        <div className="navbar-links">
          {user?.role === "student" && (
            <>
              <NavLink to="/student" className={linkClass} end>
                <ClipboardList size={16} className="nav-icon" />
                Dashboard
              </NavLink>
              <NavLink to="/student/new-request" className={linkClass}>
                <FilePlus size={16} className="nav-icon" />
                Create Request
              </NavLink>
              <NavLink to="/student/profile" className={linkClass}>
                <UserCircle size={16} className="nav-icon" />
                Profile
              </NavLink>
            </>
          )}

          {(user?.role === "admin" || user?.role === "staff") && (
            <>
              <NavLink to="/admin" className={linkClass} end>
                <BarChart2 size={16} className="nav-icon" />
                Dashboard
              </NavLink>
              <NavLink to="/admin/requests" className={linkClass}>
                <ClipboardList size={16} className="nav-icon" />
                Request Management
              </NavLink>
              {user?.role === "staff" && (
                <NavLink to="/admin/proxy-request" className={linkClass}>
                  <FilePlus size={16} className="nav-icon" />
                  Proxy Request
                </NavLink>
              )}
              <NavLink to="/admin/reports" className={linkClass}>
                <History size={16} className="nav-icon" />
                Transaction Records
              </NavLink>
              {user?.role === "admin" && (
                <NavLink to="/admin/audit" className={linkClass}>
                  <Search size={16} className="nav-icon" />
```

```
                  Audit Logs
                </NavLink>
              )}
              {user?.role === "admin" && (
                <NavLink to="/admin/users" className={linkClass}>
                  <Users size={16} className="nav-icon" />
                  User Management
                </NavLink>
              )}
              <NavLink to="/admin/archives" className={linkClass}>
                <Archive size={16} className="nav-icon" />
                Archives
              </NavLink>
              <NavLink to="/admin/profile" className={linkClass}>
                <UserCircle size={16} className="nav-icon" />
                Profile
              </NavLink>
            </>
          )}
        </div>

        {/* User section */}
        {user && (
          <div className="navbar-user">
            <div className="navbar-user-top">
              <div className="navbar-user-avatar">
                {initials}
              </div>
              <div className="navbar-user-info">
                <span className="navbar-user-name">{user.name}</span>
                <span className="navbar-user-role">{user.role}</span>
              </div>
            </div>
            <button className="navbar-logout" onClick={() => setShowLogoutConfirm(true)}>
              <LogOut size={14} style={{ marginRight: 6 }} />
              Log out
            </button>
          </div>
        )}
      </nav>

      {/* Logout Confirmation Modal */}
      <AnimatePresence>
        {showLogoutConfirm && (
          <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
            <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12,
              <h3 className="modal-title">Log Out</h3>
              <p className="modal-text">Are you sure you want to log out?</p>
              <div className="modal-actions">
                <button className="ui-btn ui-btn--secondary" onClick={() => setShowLogoutConfirm(false)}>Cancel</button>
                <button className="ui-btn ui-btn--primary" onClick={logout}>Log Out</button>
              </div>
            </motion.div>
          </motion.div>
        )}
      </AnimatePresence>
    </>
  );
};

export default Navbar;
```

37. client/src/components/Headbar.jsx (39 lines)

```
import { useContext } from "react";
import { useNavigate } from "react-router-dom";
import { AuthContext } from "../context/AuthContext";
import "./Headbar.css";

const Headbar = () => {
  const { user } = useContext(AuthContext);
  const navigate = useNavigate();

  if (!user) return null;

  const dashboardPath = user.role === "student" ? "/student" : "/admin";

  return (
    <div className="headbar">
      <div
        className="headbar-brand"
        onClick={() => navigate(dashboardPath)}
        style={{ cursor: "pointer" }}
      >
        <img
          src="/dpo-logo.webp"
          alt="DPO Logo"
          className="headbar-brand-logo"
          width="40"
          height="40"
          loading="eager"
          decoding="async"
        />
        <div className="headbar-brand-text">
          <span className="headbar-brand-name">Data Protection Office</span>
          <span className="headbar-brand-sub">Rizal Technological University</span>
        </div>
      </div>
    </div>
  );
};

export default Headbar;
```

38. client/src/components/FilterSelect.jsx (49 lines)

```
import { useEffect, useRef, useState } from "react";

const FilterSelect = ({ value, onChange, options, defaultValue = "all", className = "" }) => {
  const [open, setOpen] = useState(false);
  const ref = useRef(null);
  const isActive = value !== defaultValue;
  const selectedLabel = options.find((o) => o.value === value)?.label ?? options[0]?.label;

  useEffect(() => {
    const handler = (e) => {
      if (ref.current && !ref.current.contains(e.target)) setOpen(false);
    };
    document.addEventListener("mousedown", handler);
    return () => document.removeEventListener("mousedown", handler);
  }, []);

  return (
    <div
      className={`filter-select${isActive ? " filter-select--active" : ""}${className ? ` ${className}` : ""}`}
      ref={ref}
      onKeyDown={(e) => e.key === "Escape" && setOpen(false)}
    >
      <button
        type="button"
        className="filter-select__trigger"
        data-open={open}
        onClick={() => setOpen((p) => !p)}
      >
        <span className="filter-select__value">{selectedLabel}</span>
      </button>
      {open && (
        <div className="filter-select__menu">
          {options.map((o) => (
            <button
              key={o.value}
              type="button"
              className={`filter-select__option${o.value === value ? " filter-select__option--active" : ""}`}
              onClick={() => { onChange(o.value); setOpen(false); }}
            >
              {o.label}
            </button>
          ))}
        </div>
      )}
    </div>
  );
};

export default FilterSelect;
```

39. client/src/components/PasswordChecklist.jsx (122 lines)

```
import { useEffect, useLayoutEffect, useState } from "react";
import { createPortal } from "react-dom";

const useIsoLayoutEffect = typeof window !== "undefined" ? useLayoutEffect : useEffect;

const REQUIREMENTS = [
  "8 characters",
  "1 uppercase",
  "1 lowercase",
  "1 number",
  "1 special character",
];

const TOOLTIP_WIDTH = 210;
const GAP = 10;

const BORDER = "#b91c1c";
const BG = "#fff1f2";
const TITLE_COLOR = "#7f1d1d";
const TEXT_COLOR = "#991b1b";

function useAutoDismiss(active, persistent = false, duration = 2000) {
  const [visible, setVisible] = useState(false);
  useEffect(() => {
    if (persistent) return;
    if (!active) { setVisible(false); return; }
    setVisible(true);
    const t = setTimeout(() => setVisible(false), duration);
    return () => clearTimeout(t);
  }, [active, persistent, duration]);
  return persistent ? active : visible;
}

function useTooltipPos(active, anchorRef, side) {
  const [pos, setPos] = useState(null);
  const [actualSide, setActualSide] = useState(side);

  useIsoLayoutEffect(() => {
    if (!active || !anchorRef?.current) { setPos(null); return undefined; }

    const recompute = () => {
      const rect = anchorRef.current.getBoundingClientRect();
      const isMobile = window.innerWidth <= 640;

      if (isMobile) {
        setActualSide("bottom");
        setPos({ position: "fixed", top: rect.bottom + GAP, left: Math.max(8, rect.left), width: Math.min(TOOLTIP_WIDTH, wind
        return;
      }

      if (side === "left" && rect.left - TOOLTIP_WIDTH - GAP >= 8) {
        setActualSide("left");
        setPos({ position: "fixed", top: rect.top + rect.height / 2, left: rect.left - TOOLTIP_WIDTH - GAP, width: TOOLTIP_WI
        return;
      }

      setActualSide("right");
      setPos({ position: "fixed", top: rect.top + rect.height / 2, left: Math.min(rect.right + GAP, window.innerWidth - TOOLT
    };

    recompute();
    window.addEventListener("resize", recompute);
    window.addEventListener("scroll", recompute, true);
    return () => {
      window.removeEventListener("resize", recompute);
      window.removeEventListener("scroll", recompute, true);
    };
  }, [active, anchorRef, side]);

  return [pos, actualSide];
}

function TooltipShell({ pos, actualSide, children }) {
```

```
  const arrowBase = { position: "absolute", width: 8, height: 8, background: BG, transform: "rotate(45deg)" };
  const arrowStyle =
    actualSide === "left"  ? { ...arrowBase, right: -5, top: "50%", marginTop: -4, borderTop: `1px solid ${BORDER}`, borderRi
    : actualSide === "right" ? { ...arrowBase, left: -5, top: "50%", marginTop: -4, borderBottom: `1px solid ${BORDER}`, bord
    : { ...arrowBase, top: -5, left: 16, borderTop: `1px solid ${BORDER}`, borderLeft: `1px solid ${BORDER}` };

  const content = (
    <div style={{ ...pos, zIndex: 9999, background: BG, border: `1px solid ${BORDER}`, borderRadius: 8, padding: "10px 13px",
      <span style={{ position: "absolute", ...arrowStyle }} />
      {children}
    </div>
  );

  return typeof document !== "undefined" ? createPortal(content, document.body) : content;
}

/** Requirements tooltip — shown when the password field is focused */
export default function PasswordChecklist({ focused = false, anchorRef, side = "right", persistent = false }) {
  const visible = useAutoDismiss(focused, persistent);
  const [pos, actualSide] = useTooltipPos(visible, anchorRef, side);
  if (!visible || !pos) return null;

  return (
    <TooltipShell pos={pos} actualSide={actualSide}>
      <div style={{ fontSize: 10.5, fontWeight: 700, color: TITLE_COLOR, marginBottom: 7, textTransform: "uppercase", letterS
        Minimum password requirements:
      </div>
      {REQUIREMENTS.map((req) => (
        <div key={req} style={{ display: "flex", alignItems: "center", gap: 7, fontSize: 12.5, color: TEXT_COLOR, padding: "2
          <span style={{ width: 4, height: 4, borderRadius: "50%", background: BORDER, flexShrink: 0 }} />
          {req}
        </div>
      ))}
    </TooltipShell>
  );
}

/** Error tooltip — shown when show=true with a custom message */
export function ErrorTooltip({ show = false, message, anchorRef, side = "right", persistent = false }) {
  const visible = useAutoDismiss(show, persistent);
  const [pos, actualSide] = useTooltipPos(visible, anchorRef, side);
  if (!visible || !pos) return null;

  return (
    <TooltipShell pos={pos} actualSide={actualSide}>
      <div style={{ fontSize: 12.5, color: TEXT_COLOR, fontWeight: 600 }}>{message}</div>
    </TooltipShell>
  );
}
```

40. client/src/components/SessionWarningModal.jsx (60 lines)

```
import { useEffect, useState } from "react";
import { TimerReset } from "lucide-react";

export default function SessionWarningModal({ onExtend }) {
  const [secondsLeft, setSecondsLeft] = useState(120);

  useEffect(() => {
    const interval = setInterval(() => {
      setSecondsLeft((s) => {
        if (s <= 1) {
          clearInterval(interval);
          return 0;
        }
        return s - 1;
      });
    }, 1000);
    return () => clearInterval(interval);
  }, []);

  const minutes = Math.floor(secondsLeft / 60);
  const seconds = secondsLeft % 60;

  return (
    <div style={{
      position: "fixed", inset: 0, zIndex: 9999,
      background: "rgba(0,0,0,0.45)",
      display: "flex", alignItems: "center", justifyContent: "center",
    }}>
      <div style={{
        background: "#fff", borderRadius: 14, padding: "32px 36px",
        maxWidth: 400, width: "90%", textAlign: "center",
        boxShadow: "0 8px 32px rgba(0,0,0,0.18)",
      }}>
        <div style={{ marginBottom: 12, display: "inline-flex", alignItems: "center", justifyContent: "center" }}>
          <TimerReset size={36} color="#1d4ed8" strokeWidth={1.8} aria-hidden="true" />
        </div>
        <h2 style={{ margin: "0 0 8px", fontSize: 18, fontWeight: 700, color: "#0f172a" }}>
          Session Expiring
        </h2>
        <p style={{ margin: "0 0 20px", fontSize: 14, color: "#475569", lineHeight: 1.5 }}>
          Your session will expire in{" "}
          <strong style={{ color: "#dc2626" }}>
            {minutes}:{seconds.toString().padStart(2, "0")}
          </strong>{" "}
          due to inactivity.
        </p>
        <button
          onClick={onExtend}
          style={{
            padding: "10px 28px", borderRadius: 10, border: "none",
            background: "#0f2d6b", color: "#fff", fontSize: 14,
            fontWeight: 600, cursor: "pointer", fontFamily: "inherit",
          }}
        >
          Stay Logged In
        </button>
      </div>
    </div>
  );
}
```

41. client/src/components/SignaturePad.jsx (143 lines)

```
import { useRef, useEffect, forwardRef, useImperativeHandle } from "react";
import { Paintbrush2 } from "lucide-react";

/**
 * Canvas-based signature pad.
 * Expose ref methods: getDataUrl(), isEmpty(), clear()
 */
const SignaturePad = forwardRef(function SignaturePad(
  { height = 150, disabled = false, style = {} },
  ref
) {
  const canvasRef = useRef(null);
  const drawing = useRef(false);

  useImperativeHandle(ref, () => ({
    getDataUrl: () => canvasRef.current?.toDataURL("image/png") ?? null,
    isEmpty: () => {
      const canvas = canvasRef.current;
      if (!canvas) return true;
      const ctx = canvas.getContext("2d");
      const data = ctx.getImageData(0, 0, canvas.width, canvas.height).data;
      return !Array.from(data).some((v) => v !== 0);
    },
    clear: () => {
      const canvas = canvasRef.current;
      if (!canvas) return;
      canvas.getContext("2d").clearRect(0, 0, canvas.width, canvas.height);
    },
  }));

  useEffect(() => {
    const canvas = canvasRef.current;
    if (!canvas) return;
    const ctx = canvas.getContext("2d");
    ctx.strokeStyle = "#000";
    ctx.lineWidth = 2;
    ctx.lineCap = "round";
    ctx.lineJoin = "round";
  }, []);

  const getPos = (e, canvas) => {
    const rect = canvas.getBoundingClientRect();
    const scaleX = canvas.width / rect.width;
    const scaleY = canvas.height / rect.height;
    const clientX = e.touches ? e.touches[0].clientX : e.clientX;
    const clientY = e.touches ? e.touches[0].clientY : e.clientY;
    return {
      x: (clientX - rect.left) * scaleX,
      y: (clientY - rect.top) * scaleY,
    };
  };

  const onStart = (e) => {
    if (disabled) return;
    e.preventDefault();
    const canvas = canvasRef.current;
    const ctx = canvas.getContext("2d");
    const { x, y } = getPos(e, canvas);
    drawing.current = true;
    ctx.beginPath();
    ctx.moveTo(x, y);
  };

  const onMove = (e) => {
    if (!drawing.current || disabled) return;
    e.preventDefault();
    const canvas = canvasRef.current;
    const ctx = canvas.getContext("2d");
    const { x, y } = getPos(e, canvas);
    ctx.lineTo(x, y);
    ctx.stroke();
  };

```

```
  const onEnd = (e) => {
    e.preventDefault();
    drawing.current = false;
  };

  const handleClear = () => {
    const canvas = canvasRef.current;
    if (!canvas) return;
    canvas.getContext("2d").clearRect(0, 0, canvas.width, canvas.height);
  };

  return (
    <div style={{ position: "relative", display: "inline-block", width: "100%", cursor: disabled ? "default" : "crosshair", u
      <canvas
        ref={canvasRef}
        width={600}
        height={height}
        draggable={false}
        className={disabled ? "sig-pad-canvas--disabled" : "sig-pad-canvas"}
        style={{
          border: "1px solid #ccc",
          borderRadius: 4,
          touchAction: "none",
          userSelect: "none",
          width: "100%",
          display: "block",
          background: "#fafafa",
          ...style,
        }}
        onMouseDown={onStart}
        onMouseMove={onMove}
        onMouseUp={onEnd}
        onMouseLeave={onEnd}
        onTouchStart={onStart}
        onTouchMove={onMove}
        onTouchEnd={onEnd}
      />
      {!disabled && (
        <button
          type="button"
          onClick={handleClear}
          title="Clear signature"
          style={{
            position: "absolute",
            top: 6,
            right: 6,
            width: 28,
            height: 28,
            display: "flex",
            alignItems: "center",
            justifyContent: "center",
            background: "#fafafa",
            border: "none",
            borderRadius: 4,
            cursor: "pointer",
            color: "#64748b",
            padding: 0,
            transition: "background 0.15s ease, color 0.15s ease",
          }}
          onMouseEnter={(e) => { e.currentTarget.style.background = "#fee2e2"; e.currentTarget.style.color = "#dc2626"; }}
          onMouseLeave={(e) => { e.currentTarget.style.background = "rgba(255,255,255,0.85)"; e.currentTarget.style.color = "
        >
          <Paintbrush2 size={14} strokeWidth={1.8} style={{ transform: "scaleY(-1)" }} />
        </button>
      )}
    </div>
  );
});

export default SignaturePad;
```

42. client/src/components/RequestStepper.jsx (123 lines)

```jsx
import { Check, X } from "lucide-react";
import { motion } from "framer-motion";
import "./RequestStepper.css";

const NDA_STEPS = ["Submitted", "Reviewal", "Approved"];
const AGREEMENT_STEPS = [
  "Submitted",
  "Initial Reviewal",
  "Recipient Reviewal",
  "Final Reviewal",
  "Approved",
];

const STATUS_TO_INDEX = {
  // NDA workflow
  nda_pending: 1,
  stud_revision_requested: 1,
  nda_approved: 2,

  // Agreement workflow
  agr_pending_1: 1,
  agr_awaiting_rep_signature: 2,
  agr_rep_revision_requested: 3,
  agr_pending_2: 3,
  agr_approved: 4,
  agr_declined: 4,
};

const REVISION_LABEL = {
  stud_revision_requested: "Student Revisions",
  agr_rep_revision_requested: "Recipient Revisions",
};

export default function RequestStepper({ status, type }) {
  const steps = type === "agreement" ? AGREEMENT_STEPS : NDA_STEPS;
  const isDeclined = status === "agr_declined";
  const isRevision = status in REVISION_LABEL;
  const finalStepIndex = steps.length - 1;
  const declinedIndex = type === "agreement" ? 2 : 0;
  const activeIndex = isDeclined ? declinedIndex : (STATUS_TO_INDEX[status] ?? 0);
  const isFinalApproved = ["nda_approved", "agr_approved", "agreement_approved"].includes(status);

  return (
    <div className="request-stepper-wrap">
      <div className="request-stepper-track">
        {steps.map((step, index) => {
          const isCompleted = index < activeIndex || (isFinalApproved && index <= finalStepIndex);
          const isActive = index === activeIndex;
          const isDeclinedNode = isDeclined && index === declinedIndex;

          return (
            <div key={`${step}-${index}`} className="request-stepper-segment">
              <div className="request-stepper-node-wrap">
                <div
                  className={`request-stepper-node ${
                    isDeclinedNode
                      ? "is-declined"
                      : isCompleted
                      ? "is-completed"
                      : isActive && isRevision
                      ? "is-active-revision"
                      : isActive
                      ? "is-active"
                      : "is-upcoming"
                  }`}
                >
                  {isDeclinedNode ? (
                    <X size={13} strokeWidth={3} />
                  ) : isCompleted ? (
                    <Check size={14} strokeWidth={3} />
                  ) : isActive ? (
                    <motion.span
                      className="request-stepper-node-core"
```

```
                      animate={{ scale: [1, 1.25, 1] }}
                      transition={{ duration: 1.6, repeat: Infinity, ease: "easeInOut" }}
                    />
                  ) : (
                    <span className="request-stepper-node-number">{index + 1}</span>
                  )}
                </div>

                <div
                  className={`request-stepper-label ${
                    isDeclinedNode
                      ? "is-declined"
                      : isCompleted
                      ? "is-completed"
                      : isActive && isRevision
                      ? "is-active-revision"
                      : isActive
                      ? "is-active"
                      : "is-upcoming"
                  }`}
                >
                  {isDeclinedNode
                    ? "Declined"
                    : isActive && isRevision
                    ? REVISION_LABEL[status]
                    : step}
                </div>
              </div>

              {index < steps.length - 1 && (
                <div className="request-stepper-line-shell" aria-hidden="true">
                  <div className="request-stepper-line-base" />
                  <motion.div
                    className="request-stepper-line-fill"
                    initial={false}
                    animate={{
                      width: index < activeIndex ? "100%" : "0%",
                    }}
                    transition={{ duration: 0.45, ease: "easeOut" }}
                  />
                </div>
              )}
            </div>
          );
        })}
      </div>
    </div>
  );
}

```

43. client/src/components/InPageFeedbackHost.jsx (154 lines)

```
import { useEffect, useMemo, useRef, useState } from "react";
import { CircleAlert, CircleCheck, CircleX, Info } from "lucide-react";
import { FEEDBACK_EVENTS, inferFeedbackType, notify } from "../utils/inPageFeedback";

const TYPE_META = {
  success: {
    icon: CircleCheck,
    title: "Success",
  },
  error: {
    icon: CircleX,
    title: "Something went wrong",
  },
  warning: {
    icon: CircleAlert,
    title: "Action needed",
  },
  info: {
    icon: Info,
    title: "Notice",
  },
};

export default function InPageFeedbackHost() {
  const [toasts, setToasts] = useState([]);
  const [confirmState, setConfirmState] = useState(null);
  const nativeAlertRef = useRef(null);

  useEffect(() => {
    const onNotify = (event) => {
      const detail = event.detail || {};
      const id = `${Date.now()}_${Math.random().toString(36).slice(2, 8)}`;
      const toast = {
        id,
        type: detail.type || inferFeedbackType(detail.message),
        title: detail.title,
        message: detail.message || "",
        duration: typeof detail.duration === "number" ? detail.duration : 3800,
      };

      setToasts((prev) => {
        const next = [...prev, toast];
        return next.slice(-4);
      });

      if (toast.duration > 0) {
        window.setTimeout(() => {
          setToasts((prev) => prev.filter((item) => item.id !== id));
        }, toast.duration);
      }
    };

    const onConfirm = (event) => {
      const detail = event.detail || {};
      setConfirmState({
        title: detail.title,
        message: detail.message,
        confirmText: detail.confirmText,
        cancelText: detail.cancelText,
        tone: detail.tone,
        resolve: detail.resolve,
      });
    };

    window.addEventListener(FEEDBACK_EVENTS.NOTIFY_EVENT, onNotify);
    window.addEventListener(FEEDBACK_EVENTS.CONFIRM_EVENT, onConfirm);

    nativeAlertRef.current = window.alert;
    window.alert = (message) => {
      notify(String(message ?? ""));
    };

    return () => {
```

```
      window.removeEventListener(FEEDBACK_EVENTS.NOTIFY_EVENT, onNotify);
      window.removeEventListener(FEEDBACK_EVENTS.CONFIRM_EVENT, onConfirm);
      if (nativeAlertRef.current) {
        window.alert = nativeAlertRef.current;
      }
    };
  }, []);

  const confirmMeta = useMemo(() => {
    const tone = confirmState?.tone || "warning";
    return TYPE_META[tone] || TYPE_META.warning;
  }, [confirmState]);
  const ConfirmIcon = confirmMeta.icon;

  const closeConfirm = (accepted) => {
    if (confirmState?.resolve) confirmState.resolve(Boolean(accepted));
    setConfirmState(null);
  };

  useEffect(() => {
    if (!confirmState) return undefined;
    const onKeyDown = (event) => {
      if (event.key === "Escape") {
        event.preventDefault();
        closeConfirm(false);
      }
    };
    window.addEventListener("keydown", onKeyDown);
    return () => window.removeEventListener("keydown", onKeyDown);
  }, [confirmState]);

  return (
    <>
      <div className="inpage-feedback-stack" role="status" aria-live="polite" aria-atomic="false">
        {toasts.map((toast) => {
          const meta = TYPE_META[toast.type] || TYPE_META.info;
          const Icon = meta.icon;
          return (
            <div key={toast.id} className={`inpage-toast inpage-toast--${toast.type}`}>
              <div className="inpage-toast-icon-wrap" aria-hidden="true">
                <Icon size={18} strokeWidth={2} />
              </div>
              <div className="inpage-toast-content">
                <div className="inpage-toast-title">{toast.title || meta.title}</div>
                <div className="inpage-toast-message">{toast.message}</div>
              </div>
            </div>
          );
        })}
      </div>

      {confirmState && (
        <div className="inpage-confirm-backdrop" role="presentation" onClick={() => closeConfirm(false)}>
          <div
            className="inpage-confirm"
            role="alertdialog"
            aria-modal="true"
            aria-labelledby="inpage-confirm-title"
            onClick={(event) => event.stopPropagation()}
          >
            <div className={`inpage-confirm-icon inpage-confirm-icon--${confirmState.tone || "warning"}`} aria-hidden="true">
              <ConfirmIcon size={22} strokeWidth={2} />
            </div>
            <h3 id="inpage-confirm-title" className="inpage-confirm-title">
              {confirmState.title || "Please confirm"}
            </h3>
            <p className="inpage-confirm-message">{confirmState.message}</p>
            <div className="inpage-confirm-actions">
              <button type="button" className="inpage-btn inpage-btn--ghost" onClick={() => closeConfirm(false)}>
                {confirmState.cancelText || "Cancel"}
              </button>
              <button type="button" className="inpage-btn inpage-btn--primary" onClick={() => closeConfirm(true)}>
                {confirmState.confirmText || "Confirm"}
              </button>
            </div>
          </div>
```

```
150          </div>
151        )}
152      </>
153    );
154  }
```

44. client/src/pages/Landing.jsx (587 lines)

```
import { useState, useContext, useCallback, useEffect, useRef } from "react";
import { useNavigate } from "react-router-dom";
import axios from "axios";
import { motion, AnimatePresence } from "framer-motion";
import { Eye, EyeOff } from "lucide-react";
import { AuthContext } from "../context/AuthContext";
import {
  login as loginService,
  verifyLoginOtp,
  resendLoginOtp,
  forgotPassword,
  resendResetOtp,
  verifyResetOtp,
  resetPassword,
} from "../services/authService";
import PasswordChecklist from "../components/PasswordChecklist";
import { isStrongPassword, PASSWORD_POLICY_MESSAGE } from "../utils/passwordPolicy";
import { notify } from "../utils/inPageFeedback";
import "../components/Landing.css";
import "../components/FloatingLabel.css";

const API_URL = `${import.meta.env.VITE_API_BASE_URL}/api/auth`;

// ■■ Forgot-password steps: "email" → "otp" → "newpass"
function ForgotPasswordFlow({ onCancel }) {
  const [step, setStep] = useState("email");
  const [email, setEmail] = useState("");
  const [otp, setOtp] = useState("");
  const [resetToken, setResetToken] = useState("");
  const [newPassword, setNewPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [loading, setLoading] = useState(false);
  const [resendSeconds, setResendSeconds] = useState(0);
  const [showNewPass, setShowNewPass] = useState(false);
  const [showConfirmPass, setShowConfirmPass] = useState(false);
  const forgotPwRef = useRef(null);

  useEffect(() => {
    if (resendSeconds <= 0) return undefined;
    const timer = window.setInterval(() => {
      setResendSeconds((prev) => (prev > 1 ? prev - 1 : 0));
    }, 1000);
    return () => window.clearInterval(timer);
  }, [resendSeconds]);

  const handleSendOtp = async (e) => {
    e.preventDefault();
    if (!email.trim()) { notify("Please enter your email address.", { type: "error", title: "Failed" }); return; }
    setLoading(true);
    try {
      await forgotPassword(email.trim());
      setStep("otp");
      setResendSeconds(60);
    } catch (err) {
      notify(err.response?.data?.message || "Failed to send OTP. Try again.", { type: "error", title: "Failed" });
    } finally {
      setLoading(false);
    }
  };

  const handleResendOtp = async () => {
    if (resendSeconds > 0) return;
    setLoading(true);
    try {
      await resendResetOtp(email.trim());
      setResendSeconds(60);
      notify("OTP resent successfully.", { type: "success", title: "Sent" });
    } catch (err) {
      notify(err.response?.data?.message || "Failed to resend OTP.", { type: "error", title: "Failed" });
    } finally {
      setLoading(false);
    }
  };
```

```
74
75    const handleVerifyOtp = async (e) => {
76      e.preventDefault();
77      if (!otp.trim()) { notify("Please enter the OTP.", { type: "error", title: "Failed" }); return; }
78      setLoading(true);
79      try {
80        const data = await verifyResetOtp(email.trim(), otp.trim());
81        setResetToken(data.resetToken);
82        setStep("newpass");
83      } catch (err) {
84        notify(err.response?.data?.message || "Invalid or expired OTP.", { type: "error", title: "Failed" });
85      } finally {
86        setLoading(false);
87      }
88    };
89
90    const handleResetPassword = async (e) => {
91      e.preventDefault();
92      if (!isStrongPassword(newPassword)) { notify(PASSWORD_POLICY_MESSAGE, { type: "error", title: "Failed" }); return; }
93      if (newPassword !== confirmPassword) { notify("Passwords do not match.", { type: "error", title: "Failed" }); return; }
94      setLoading(true);
95      try {
96        await resetPassword(email.trim(), resetToken, newPassword);
97        notify("Password reset successfully! Please log in with your new password.", { type: "success" });
98        onCancel();
99      } catch (err) {
100       notify(err.response?.data?.message || "Failed to reset password.", { type: "error", title: "Failed" });
101     } finally {
102       setLoading(false);
103     }
104   };
105
106   return (
107     <div className="landing-card">
108       <h2 className="landing-title">
109         {step === "email" && "Forgot Password"}
110         {step === "otp" && "Enter OTP"}
111         {step === "newpass" && "New Password"}
112       </h2>
113       {step === "email" && (
114         <form onSubmit={handleSendOtp} style={{ display: "flex", flexDirection: "column", gap: 14 }}>
115           <p style={{ margin: 0, fontSize: 13, color: "var(--text-primary)", fontWeight: 600 }}>
116             Enter your registered email and we&apos;ll send you an OTP.
117           </p>
118           <div className="landing-field-group fl-wrap">
119             <input type="email" className="fl-input landing-field" placeholder=" " value={email} onChange={(e) => setEmail(e.
120             <label className="fl-label fl-label--glass">Email address</label>
121           </div>
122           <button type="submit" className="landing-submit" disabled={loading}>
123             {loading ? "Sending…" : "Send OTP"}
124           </button>
125           <button type="button" className="landing-link-btn" onClick={onCancel}>← Back to Login</button>
126         </form>
127       )}
128       {step === "otp" && (
129         <form onSubmit={handleVerifyOtp} style={{ display: "flex", flexDirection: "column", gap: 14 }}>
130                 <p style={{ margin: 0, fontSize: 13, color: "var(--text-primary)", fontWeight: 600 }}>
131                   A 6-digit OTP was sent to <strong>{email}</strong>. It expires in 10 minutes.
132                 </p>
133           <div className="landing-field-group fl-wrap">
134             <input type="text" className="fl-input landing-field" placeholder=" " value={otp} onChange={(e) => setOtp(e.targe
135             <label className="fl-label fl-label--glass">OTP Code</label>
136           </div>
137           <button type="submit" className="landing-submit" disabled={loading}>
138             {loading ? "Verifying…" : "Verify OTP"}
139           </button>
140           <button type="button" className="landing-link-btn" onClick={handleResendOtp} disabled={loading || resendSeconds > 0
141             {resendSeconds > 0 ? `Resend OTP in ${resendSeconds}s` : "Resend OTP"}
142           </button>
143         </form>
144       )}
145       {step === "newpass" && (
146         <form onSubmit={handleResetPassword} style={{ display: "flex", flexDirection: "column", gap: 14 }}>
147           <div ref={forgotPwRef} className="landing-field-group">
148             <div className="landing-password-wrap fl-wrap">
149               <input type={showNewPass ? "text" : "password"} className="fl-input landing-field" placeholder=" " value={newPa
```

```
              <label className="fl-label fl-label--glass fl-label--pw">New Password</label>
              <button type="button" className="landing-password-toggle" onClick={() => setShowNewPass((v) => !v)} tabIndex={-
                {showNewPass ? <Eye size={18} /> : <EyeOff size={18} />}
              </button>
            </div>
            <PasswordChecklist focused={newPassword.length > 0 && !isStrongPassword(newPassword)} anchorRef={forgotPwRef} sid
          </div>
          <div className="landing-field-group">
            <div className="landing-password-wrap fl-wrap">
              <input type={showConfirmPass ? "text" : "password"} className="fl-input landing-field" placeholder=" " value={c
              <label className="fl-label fl-label--glass fl-label--pw">Confirm Password</label>
              <button type="button" className="landing-password-toggle" onClick={() => setShowConfirmPass((v) => !v)} tabInde
                {showConfirmPass ? <Eye size={18} /> : <EyeOff size={18} />}
              </button>
            </div>
          </div>
          <button type="submit" className="landing-submit" disabled={loading}>
            {loading ? "Resetting…" : "Reset Password"}
          </button>
        </form>
      )}
    </div>
  );
}

const Landing = () => {
  const [mode, setMode] = useState("login"); // "login" | "register" | "forgot"
  const [form, setForm] = useState({ firstName: "", middleInitial: "", lastName: "", email: "", password: "", program: "" });

  const PROGRAMS = [
    "BS Accountancy",
    "BS Architecture",
    "BS Business Administration",
    "BS Civil Engineering",
    "BS Computer Science",
    "BS Information Technology",
    "BS Nursing",
    "BS Psychology",
    "BS Tourism Management",
    "Bachelor of Secondary Education",
  ];
  const [loading, setLoading] = useState(false);
  const [showPassword, setShowPassword] = useState(false);
  const [passwordFocused, setPasswordFocused] = useState(false);
  const [programOpen, setProgramOpen] = useState(false);
  const programRef = useRef(null);

  useEffect(() => {
    const handler = (e) => {
      if (programRef.current && !programRef.current.contains(e.target)) setProgramOpen(false);
    };
    document.addEventListener("mousedown", handler);
    return () => document.removeEventListener("mousedown", handler);
  }, []);
  const [hasMounted, setHasMounted] = useState(false);
  const [bgLoaded, setBgLoaded] = useState(false);
  const { login } = useContext(AuthContext);
  const navigate = useNavigate();

  // Parallax state
  const bgRef = useRef(null);
  const passwordHintAnchorRef = useRef(null);
  const handleMouseMove = useCallback((e) => {
    if (!bgRef.current) return;
    const { clientX, clientY } = e;
    const { innerWidth, innerHeight } = window;
    const x = ((clientX / innerWidth) - 0.5) * -20;
    const y = ((clientY / innerHeight) - 0.5) * -20;
    bgRef.current.style.transform = `translate3d(${x}px, ${y}px, 0) scale(1.05)`;
  }, []);

  // OTP step state (for context-aware login OTP)
  const [otpStep, setOtpStep] = useState(false);
  const [loginOtp, setLoginOtp] = useState("");
  const [pendingEmail, setPendingEmail] = useState("");
  const [loginResendSeconds, setLoginResendSeconds] = useState(0);
```

```
226
227   useEffect(() => {
228     if (loginResendSeconds <= 0) return undefined;
229     const timer = window.setInterval(() => {
230       setLoginResendSeconds((prev) => (prev > 1 ? prev - 1 : 0));
231     }, 1000);
232     return () => window.clearInterval(timer);
233   }, [loginResendSeconds]);
234
235   const onChange = (key) => (e) => setForm((p) => ({ ...p, [key]: e.target.value }));
236   const resetForm = () => {
237     setForm({ firstName: "", middleInitial: "", lastName: "", email: "", password: "", program: "" });
238     setOtpStep(false);
239     setLoginOtp("");
240     setPendingEmail("");
241     setLoginResendSeconds(0);
242   };
243
244   const handleSubmit = async (e) => {
245     e.preventDefault();
246     setLoading(true);
247     try {
248       if (mode === "login") {
249         const data = await loginService(form.email, form.password);
250         if (data.requireOtp) {
251           setPendingEmail(form.email);
252           setOtpStep(true);
253           setLoginResendSeconds(60);
254           setLoading(false);
255           return;
256         }
257         if (data.requireVerification) {
258           navigate(`/verify-email?email=${encodeURIComponent(data.email || form.email)}`);
259           return;
260         }
261         login(data.user);
262         navigate(data.user.role === "student" ? "/student" : "/admin/requests");
263         return;
264       }
265       if (!isLogin && !isStrongPassword(form.password)) {
266         setLoading(false);
267         return;
268       }
269       if (!form.program) {
270         notify("Please select your program/course.", { type: "warning" });
271         setLoading(false);
272         return;
273       }
274       const res = await axios.post(`${API_URL}/register`, {
275         firstName: form.firstName,
276         middleInitial: form.middleInitial,
277         lastName: form.lastName,
278         email: form.email,
279         password: form.password,
280         program: form.program,
281       });
282       if (res.data.requireVerification) {
283         navigate(`/verify-email?email=${encodeURIComponent(res.data.email)}`);
284         return;
285       }
286       notify(res.data.message || "Registered! Check your email to verify your account.", { type: "success", title: "Success"
287       resetForm();
288       setMode("login");
289     } catch (err) {
290       if (mode === "login" && err.response?.data?.requireVerification) {
291         navigate(`/verify-email?email=${encodeURIComponent(err.response?.data?.email || form.email)}`);
292         return;
293       }
294       notify(
295         err.response?.data?.message ||
296           (mode === "login" ? "Invalid credentials" : "Registration failed"),
297         { type: "error", title: "Failed" }
298       );
299     } finally {
300       setLoading(false);
301     }
```

```
302   };
303
304   const handleVerifyLoginOtp = async (e) => {
305     e.preventDefault();
306     setLoading(true);
307     try {
308       const data = await verifyLoginOtp(pendingEmail, loginOtp.trim());
309       login(data.user);
310       navigate(data.user.role === "student" ? "/student" : "/admin/requests");
311     } catch (err) {
312       notify(err.response?.data?.message || "Invalid or expired OTP.", { type: "error", title: "Failed" });
313     } finally {
314       setLoading(false);
315     }
316   };
317
318   const handleResendLoginOtp = async () => {
319     if (!pendingEmail || loginResendSeconds > 0) return;
320     setLoading(true);
321     try {
322       const data = await resendLoginOtp(pendingEmail);
323       notify(data.message || "A new OTP has been sent to your email.", { type: "success", title: "Sent" });
324       setLoginResendSeconds(60);
325     } catch (err) {
326       notify(err.response?.data?.message || "Failed to resend OTP.", { type: "error", title: "Failed" });
327     } finally {
328       setLoading(false);
329     }
330   };
331
332   const isLogin = mode === "login";
333
334   useEffect(() => {
335     setHasMounted(true);
336
337     const preload = new Image();
338     preload.src = "/RTU-Background.webp";
339     preload.onload = () => setBgLoaded(true);
340     preload.onerror = () => setBgLoaded(false);
341   }, []);
342
343   useEffect(() => {
344     // Lock page scroll while on the landing/auth route for app-like immersion.
345     const previousBodyOverflow = document.body.style.overflow;
346     const previousHtmlOverflow = document.documentElement.style.overflow;
347
348     document.body.style.overflow = "hidden";
349     document.documentElement.style.overflow = "hidden";
350
351     return () => {
352       document.body.style.overflow = previousBodyOverflow;
353       document.documentElement.style.overflow = previousHtmlOverflow;
354     };
355   }, []);
356
357   return (
358     <div className={`landing ${hasMounted ? "is-mounted" : ""}`} onMouseMove={handleMouseMove}>
359       <div
360         ref={bgRef}
361         className={`landing-parallax-bg ${bgLoaded ? "is-loaded" : ""}`}
362         style={{ backgroundImage: "url('/RTU-Background.webp')" }}
363       />
364       <img
365         src="/RTU-Background.webp"
366         alt=""
367         width="1"
368         height="1"
369         loading="eager"
370         fetchPriority="high"
371         decoding="async"
372         aria-hidden="true"
373         style={{ position: "absolute", width: 1, height: 1, opacity: 0, pointerEvents: "none" }}
374       />
375       <div className="landing-overlay" />
376       <div className="landing-wrapper">
377         {/* ■■ Brand Header (on blue background) ■■ */}
```

```
        <div className="landing-brand">
          <img
            src="/dpo-logo.webp"
            alt="RTU DPO Logo"
            className="landing-brand-logo"
            width="156"
            height="156"
            loading="eager"
            decoding="async"
          />
          <div className="landing-brand-text">
            <div className="landing-brand-title">Data Protection Office</div>
            <div className="landing-brand-subtitle">Rizal Technological University</div>
          </div>
        </div>

        {/* ■■ White Form Panel ■■ */}
        <div className={`landing-panel ${hasMounted ? "is-mounted" : ""}`}>
        {/* ■■ Form Area ■■ */}
        <AnimatePresence mode="wait">
          {/* Login OTP step */}
          {otpStep ? (
            <motion.div
              key="login-otp"
              initial={{ opacity: 0, y: 16 }}
              animate={{ opacity: 1, y: 0 }}
              exit={{ opacity: 0, y: -16 }}
              transition={{ duration: 0.2 }}
              style={{ width: "100%", maxWidth: 380 }}
            >
              <form onSubmit={handleVerifyLoginOtp} className="landing-card">
                <h2 className="landing-title">Verify Your Identity</h2>
                <p style={{ margin: 0, fontSize: 13, color: "var(--text-primary)", fontWeight: 600 }}>
                  A 6-digit OTP was sent to <strong>{pendingEmail}</strong>. Enter it below to continue.
                </p>
                <div className="landing-field-group fl-wrap">
                  <input
                    type="text"
                    className="fl-input landing-field"
                    placeholder=" "
                    value={loginOtp}
                    onChange={(e) => setLoginOtp(e.target.value)}
                    maxLength={6}
                    autoFocus
                    required
                  />
                  <label className="fl-label fl-label--glass">OTP Code</label>
                </div>
                <button type="submit" className="landing-submit" disabled={loading}>
                  {loading ? "Verifying…" : "Verify & Login"}
                </button>
                <button
                  type="button"
                  className="landing-link-btn"
                  onClick={handleResendLoginOtp}
                  disabled={loading || loginResendSeconds > 0}
                >
                  {loginResendSeconds > 0 ? `Resend OTP in ${loginResendSeconds}s` : "Resend OTP"}
                </button>
                <button type="button" className="landing-link-btn" onClick={resetForm}>
                  ← Back to Login
                </button>
              </form>
            </motion.div>
          ) : mode === "forgot" ? (
            <motion.div
              key="forgot"
              initial={{ opacity: 0, y: 16 }}
              animate={{ opacity: 1, y: 0 }}
              exit={{ opacity: 0, y: -16 }}
              transition={{ duration: 0.2 }}
              style={{ width: "100%", maxWidth: 380 }}
            >
              <ForgotPasswordFlow onCancel={() => { setMode("login"); resetForm(); }} />
            </motion.div>
          ) : (
```

```
            <motion.div
              key="auth"
              initial={{ opacity: 0, y: 16 }}
              animate={{ opacity: 1, y: 0 }}
              exit={{ opacity: 0, y: -16 }}
              transition={{ duration: 0.2 }}
              style={{ display: "flex", flexDirection: "column", alignItems: "center", gap: 24, width: "100%" }}
            >
              <div className="landing-toggle">
                <button
                  type="button"
                  className={`landing-toggle-btn ${isLogin ? "active" : ""}`}
                  onClick={() => { setMode("login"); resetForm(); }}
                >
                  Login
                </button>
                <button
                  type="button"
                  className={`landing-toggle-btn ${!isLogin ? "active" : ""}`}
                  onClick={() => { setMode("register"); resetForm(); }}
                >
                  Register
                </button>
                <div className={`landing-toggle-indicator ${isLogin ? "left" : "right"}`} />
              </div>

              <form onSubmit={handleSubmit} className={`landing-card ${!isLogin ? "landing-card--register" : ""}`}>
                <h2 className="landing-title">{isLogin ? "Welcome back" : "Create an account"}</h2>

                {!isLogin && (
                  <div className="landing-name-grid">
                    <div className="landing-field-group fl-wrap">
                      <input id="landing-firstname" type="text" placeholder=" " value={form.firstName} onChange={onChange("fi
                      <label className="fl-label fl-label--glass">First Name</label>
                    </div>
                    <div className="landing-field-group fl-wrap">
                      <input id="landing-mi" type="text" placeholder=" " value={form.middleInitial} onChange={onChange("middl
                      <label className="fl-label fl-label--glass">Middle Name</label>
                    </div>
                    <div className="landing-field-group fl-wrap">
                      <input id="landing-lastname" type="text" placeholder=" " value={form.lastName} onChange={onChange("last
                      <label className="fl-label fl-label--glass">Last Name</label>
                    </div>
                  </div>
                )}

                {!isLogin && (
                  <div className="landing-field-group fl-wrap" ref={programRef}>
                    <button
                      type="button"
                      className="landing-field landing-program-trigger"
                      data-open={programOpen}
                      onClick={() => setProgramOpen((p) => !p)}
                    >
                      <span className="landing-program-value">{form.program || ""}</span>
                    </button>
                    <label className="fl-label fl-label--glass landing-program-label" data-active={!!(form.program || program
                      Program / Course
                    </label>
                    {programOpen && (
                      <div className="landing-program-menu">
                        {PROGRAMS.map((p) => (
                          <button
                            key={p}
                            type="button"
                            className={`filter-select__option${form.program === p ? " filter-select__option--active" : ""}`}
                            onClick={() => { onChange("program")({ target: { value: p } }); setProgramOpen(false); }}
                          >
                            {p}
                          </button>
                        ))}
                      </div>
                    )}
                  </div>
                )}
```

```
                <div className="landing-field-group fl-wrap">
                  <input id="landing-email" type="email" placeholder=" " value={form.email} onChange={onChange("email")} requ
                  <label className="fl-label fl-label--glass">Email address</label>
                </div>

                <div ref={passwordHintAnchorRef} className="landing-field-group password-hint-anchor">
                  <div className="landing-password-wrap fl-wrap">
                    <input
                      id="landing-password"
                      type={showPassword ? "text" : "password"}
                      placeholder=" "
                      value={form.password}
                      onChange={onChange("password")}
                      onFocus={() => setPasswordFocused(true)}
                      onBlur={() => setPasswordFocused(false)}
                      required
                      className="fl-input landing-field"
                    />
                    <label className="fl-label fl-label--glass fl-label--pw">Password</label>
                    <button
                      type="button"
                      className="landing-password-toggle"
                      onClick={() => setShowPassword((v) => !v)}
                      tabIndex={-1}
                      aria-label={showPassword ? "Hide password" : "Show password"}
                    >
                      {showPassword ? <Eye size={18} /> : <EyeOff size={18} />}
                    </button>
                  </div>
                  <PasswordChecklist focused={!isLogin && !isStrongPassword(form.password) && (passwordFocused || form.passwo
                </div>

                {isLogin && (
                  <button
                    type="button"
                    className="landing-link-btn"
                    style={{ textAlign: "right", marginTop: -6 }}
                    onClick={() => { setMode("forgot"); resetForm(); }}
                  >
                    Forgot password?
                  </button>
                )}

                <button type="submit" className="landing-submit" disabled={loading}>
                  {loading ? "Please wait…" : (isLogin ? "Log in" : "Create Account")}
                </button>
              </form>
            </motion.div>
          )}
        </AnimatePresence>
        </div>
      </div>
    </div>
  );
};

export default Landing;
```

45. client/src/pages/ActivateAccountPage.jsx (160 lines)

```
import { useState } from "react";
import { useSearchParams, Link } from "react-router-dom";
import { CheckCircle, XCircle } from "lucide-react";
import { activateAccount } from "../services/authService";
import PasswordChecklist from "../components/PasswordChecklist";
import { isStrongPassword, PASSWORD_POLICY_MESSAGE } from "../utils/passwordPolicy";

export default function ActivateAccountPage() {
  const [params] = useSearchParams();
  const token = params.get("token");
  const [password, setPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [passwordAttempted, setPasswordAttempted] = useState(false);
  const [loading, setLoading] = useState(false);
  const [status, setStatus] = useState("form"); // "form" | "success" | "error"
  const [message, setMessage] = useState("");

  if (!token) {
    return (
      <div style={{
        minHeight: "100vh", display: "flex", alignItems: "center", justifyContent: "center",
        background: "#f1f5f9", fontFamily: "'Inter', system-ui, sans-serif",
      }}>
        <div style={{
          background: "#fff", borderRadius: 14, padding: "40px 36px",
          maxWidth: 420, width: "90%", textAlign: "center",
          boxShadow: "0 1px 3px rgba(15,23,42,0.06), 0 4px 12px rgba(15,23,42,0.06)",
          border: "1px solid #e2e8f0",
        }}>
          <XCircle size={48} color="#dc2626" style={{ marginBottom: 12 }} />
          <h2 style={{ margin: "0 0 8px", fontSize: 20, fontWeight: 700, color: "#0f172a" }}>Invalid Link</h2>
          <p style={{ margin: "0 0 24px", fontSize: 14, color: "#475569" }}>Missing activation token.</p>
          <Link to="/" style={{
            display: "inline-block", padding: "10px 24px", borderRadius: 10,
            background: "#0f2d6b", color: "#fff", textDecoration: "none",
            fontWeight: 600, fontSize: 14,
          }}>Go to Login</Link>
        </div>
      </div>
    );
  }

  const handleSubmit = async (e) => {
    e.preventDefault();
    setMessage("");
    if (!isStrongPassword(password)) { setPasswordAttempted(true); setMessage(PASSWORD_POLICY_MESSAGE); return; }
    if (password !== confirmPassword) { setMessage("Passwords do not match."); return; }
    setLoading(true);
    try {
      const data = await activateAccount(token, password);
      setStatus("success");
      setMessage(data.message || "Account activated!");
    } catch (err) {
      setStatus("error");
      setMessage(err.response?.data?.message || "Activation failed.");
    } finally {
      setLoading(false);
    }
  };

  const containerStyle = {
    minHeight: "100vh", display: "flex", alignItems: "center", justifyContent: "center",
    background: "#f1f5f9", fontFamily: "'Inter', system-ui, sans-serif",
  };

  const cardStyle = {
    background: "#fff", borderRadius: 14, padding: "40px 36px",
    maxWidth: 420, width: "90%", textAlign: "center",
    boxShadow: "0 1px 3px rgba(15,23,42,0.06), 0 4px 12px rgba(15,23,42,0.06)",
    border: "1px solid #e2e8f0",
  };

  if (status === "success") {
```

```
    return (
      <div style={containerStyle}>
        <div style={cardStyle}>
          <CheckCircle size={48} color="#16a34a" style={{ marginBottom: 12 }} />
          <h2 style={{ margin: "0 0 8px", fontSize: 20, fontWeight: 700, color: "#0f172a" }}>Account Activated!</h2>
          <p style={{ margin: "0 0 24px", fontSize: 14, color: "#475569" }}>{message}</p>
          <Link to="/" style={{
            display: "inline-block", padding: "10px 24px", borderRadius: 10,
            background: "#0f2d6b", color: "#fff", textDecoration: "none",
            fontWeight: 600, fontSize: 14,
          }}>Go to Login</Link>
        </div>
      </div>
    );
  }

  if (status === "error") {
    return (
      <div style={containerStyle}>
        <div style={cardStyle}>
          <XCircle size={48} color="#dc2626" style={{ marginBottom: 12 }} />
          <h2 style={{ margin: "0 0 8px", fontSize: 20, fontWeight: 700, color: "#0f172a" }}>Activation Failed</h2>
          <p style={{ margin: "0 0 24px", fontSize: 14, color: "#475569" }}>{message}</p>
          <Link to="/" style={{
            display: "inline-block", padding: "10px 24px", borderRadius: 10,
            background: "#0f2d6b", color: "#fff", textDecoration: "none",
            fontWeight: 600, fontSize: 14,
          }}>Go to Login</Link>
        </div>
      </div>
    );
  }

  return (
    <div style={containerStyle}>
      <div style={{ ...cardStyle, textAlign: "left" }}>
        <h2 style={{ margin: "0 0 4px", fontSize: 20, fontWeight: 700, color: "#0f172a", textAlign: "center" }}>
          Set Your Password
        </h2>
        <p style={{ margin: "0 0 20px", fontSize: 13, color: "#475569", textAlign: "center" }}>
          Choose a secure password to activate your account.
        </p>
        {message && (
          <div style={{
            marginBottom: 14, padding: "8px 12px", borderRadius: 10,
            background: "#fee2e2", color: "#991b1b", border: "1px solid #fca5a5", fontSize: 13,
          }}>{message}</div>
        )}
        <form onSubmit={handleSubmit} style={{ display: "flex", flexDirection: "column", gap: 14 }}>
          <div>
            <label style={{ fontSize: 12.5, fontWeight: 600, color: "#475569", display: "block", marginBottom: 6 }}>New Passw
            <input
              type="password" value={password} onChange={(e) => setPassword(e.target.value)}
              placeholder="••••••••" required minLength={8}
              style={{
                width: "100%", padding: "10px 14px", borderRadius: 10,
                border: "1px solid #cbd5e1", fontSize: 14, fontFamily: "inherit",
                color: "#0f172a", boxSizing: "border-box",
              }}
            />
            {passwordAttempted ? <PasswordChecklist password={password} compact /> : null}
          </div>
          <div>
            <label style={{ fontSize: 12.5, fontWeight: 600, color: "#475569", display: "block", marginBottom: 6 }}>Confirm P
            <input
              type="password" value={confirmPassword} onChange={(e) => setConfirmPassword(e.target.value)}
              placeholder="••••••••" required
              style={{
                width: "100%", padding: "10px 14px", borderRadius: 10,
                border: "1px solid #cbd5e1", fontSize: 14, fontFamily: "inherit",
                color: "#0f172a", boxSizing: "border-box",
              }}
            />
          </div>
          <button type="submit" disabled={loading} style={{
            width: "100%", padding: "10px 16px", borderRadius: 10, border: "none",
```

```
150              background: "#0f2d6b", color: "#fff", fontSize: 14, fontWeight: 600,
151              cursor: loading ? "not-allowed" : "pointer", fontFamily: "inherit",
152              opacity: loading ? 0.65 : 1, marginTop: 4,
153            }}>
154              {loading ? "Activating…" : "Activate Account"}
155            </button>
156          </form>
157        </div>
158      </div>
159    );
160  }
```

46. client/src/pages/VerifyEmailPage.jsx (237 lines)

```
import { useContext, useEffect, useState } from "react";
import { useNavigate, useSearchParams, Link } from "react-router-dom";
import { CheckCircle, XCircle } from "lucide-react";
import { verifyEmail, verifyEmailByToken, resendVerificationOtp } from "../services/authService";
import { AuthContext } from "../context/AuthContext";
import { notify } from "../utils/inPageFeedback";
import "../components/Landing.css";
import "../components/FloatingLabel.css";

export default function VerifyEmailPage() {
  const [params] = useSearchParams();
  const navigate = useNavigate();
  const { login } = useContext(AuthContext);
  const token = params.get("token");
  const emailParam = params.get("email") || "";
  const [otp, setOtp] = useState("");
  const [loading, setLoading] = useState(false);
  const [resending, setResending] = useState(false);
  const [status, setStatus] = useState(token ? "loading" : "form"); // "loading" | "form" | "success" | "error"
  const [resendSeconds, setResendSeconds] = useState(token ? 0 : 60);

  useEffect(() => {
    if (resendSeconds <= 0) return undefined;
    const timer = window.setInterval(() => {
      setResendSeconds((prev) => (prev > 1 ? prev - 1 : 0));
    }, 1000);
    return () => window.clearInterval(timer);
  }, [resendSeconds]);

  useEffect(() => {
    const previousBodyOverflow = document.body.style.overflow;
    const previousHtmlOverflow = document.documentElement.style.overflow;

    document.body.style.overflow = "hidden";
    document.documentElement.style.overflow = "hidden";

    return () => {
      document.body.style.overflow = previousBodyOverflow;
      document.documentElement.style.overflow = previousHtmlOverflow;
    };
  }, []);

  const finalizeLogin = (data) => {
    if (!data?.user) return;
    login(data.user);
    navigate(data.user.role === "student" ? "/student" : "/admin", { replace: true });
  };

  // Legacy token-based auto-verification
  useEffect(() => {
    if (!token) return;
    verifyEmailByToken(token)
      .then((data) => {
        if (data?.token && data?.user) {
          finalizeLogin(data);
          return;
        }
        setStatus("success");
        setMessage(data.message || "Email verified successfully!");
      })
      .catch((err) => {
        setStatus("error");
        setMessage(err.response?.data?.message || "Verification failed.");
      });
  // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [token]);

  const handleVerify = async (e) => {
    e.preventDefault();
    if (!otp.trim()) return;
    setLoading(true);
    try {
      const data = await verifyEmail(emailParam, otp.trim());
```

```
      if (data?.token && data?.user) {
        finalizeLogin(data);
        return;
      }
      setStatus("success");
    } catch (err) {
      notify(err.response?.data?.message || "Invalid or expired OTP.", { type: "error", title: "Failed" });
    } finally {
      setLoading(false);
    }
  };

  const handleResend = async () => {
    if (resendSeconds > 0) return;
    setResending(true);
    try {
      const data = await resendVerificationOtp(emailParam);
      notify(data.message || "A new OTP has been sent to your email.", { type: "success", title: "Sent" });
      setResendSeconds(60);
    } catch (err) {
      notify(err.response?.data?.message || "Failed to resend OTP.", { type: "error", title: "Failed" });
    } finally {
      setResending(false);
    }
  };

  if (status === "loading") {
    return (
      <div className="landing is-mounted">
        <div className="landing-parallax-bg is-loaded" style={{ backgroundImage: "url('/RTU-Background.webp')" }} />
        <div className="landing-overlay" />
        <div className="landing-wrapper">
          <div className="landing-brand">
            <img src="/dpo-logo.webp" alt="RTU DPO Logo" className="landing-brand-logo" width="104" height="104" loading="eag
            <div className="landing-brand-text">
              <div className="landing-brand-title">Data Protection Office</div>
              <div className="landing-brand-subtitle">Rizal Technological University</div>
            </div>
          </div>
          <div className="landing-panel is-mounted">
            <div className="landing-card">
              <h2 className="landing-title">Verifying Your Email</h2>
              <p>Please wait while we verify your account.</p>
            </div>
          </div>
        </div>
      </div>
    );
  }

  if (status === "success") {
    return (
      <div className="landing is-mounted">
        <div className="landing-parallax-bg is-loaded" style={{ backgroundImage: "url('/RTU-Background.webp')" }} />
        <div className="landing-overlay" />
        <div className="landing-wrapper">
          <div className="landing-brand">
            <img src="/dpo-logo.webp" alt="RTU DPO Logo" className="landing-brand-logo" width="104" height="104" loading="eag
            <div className="landing-brand-text">
              <div className="landing-brand-title">Data Protection Office</div>
              <div className="landing-brand-subtitle">Rizal Technological University</div>
            </div>
          </div>
          <div className="landing-panel is-mounted">
            <div className="landing-card">
              <CheckCircle size={42} color="#16a34a" />
              <h2 className="landing-title">Email Verified</h2>
              <p>{message}</p>
              <Link to="/" className="landing-submit" style={{ textDecoration: "none", display: "inline-flex", alignItems: "c
                Go to Login
              </Link>
            </div>
          </div>
        </div>
      </div>
    );
```

```
  }

  if (status === "error") {
    return (
      <div className="landing is-mounted">
        <div className="landing-parallax-bg is-loaded" style={{ backgroundImage: "url('/RTU-Background.webp')" }} />
        <div className="landing-overlay" />
        <div className="landing-wrapper">
          <div className="landing-brand">
            <img src="/dpo-logo.webp" alt="RTU DPO Logo" className="landing-brand-logo" width="104" height="104" loading="eag
            <div className="landing-brand-text">
              <div className="landing-brand-title">Data Protection Office</div>
              <div className="landing-brand-subtitle">Rizal Technological University</div>
            </div>
          </div>
          <div className="landing-panel is-mounted">
            <div className="landing-card">
              <XCircle size={42} color="#dc2626" />
              <h2 className="landing-title">Verification Failed</h2>
              <p>{message}</p>
              <Link to="/" className="landing-submit" style={{ textDecoration: "none", display: "inline-flex", alignItems: "c
                Back to Login
              </Link>
            </div>
          </div>
        </div>
      </div>
    );
  }

  // OTP form
  return (
    <div className="landing is-mounted">
      <div className="landing-parallax-bg is-loaded" style={{ backgroundImage: "url('/RTU-Background.webp')" }} />
      <div className="landing-overlay" />
      <div className="landing-wrapper">
        <div className="landing-brand">
          <img src="/dpo-logo.webp" alt="RTU DPO Logo" className="landing-brand-logo" width="104" height="104" loading="eager
          <div className="landing-brand-text">
            <div className="landing-brand-title">Data Protection Office</div>
            <div className="landing-brand-subtitle">Rizal Technological University</div>
          </div>
        </div>

        <div className="landing-panel is-mounted">
          <form onSubmit={handleVerify} className="landing-card">
            <h2 className="landing-title">Verify Your Email</h2>
            <p>
              A 6-digit OTP was sent to <strong>{emailParam}</strong>. Enter it below to activate your account.
            </p>

            <div className="landing-field-group fl-wrap">
              <input
                type="text"
                className="fl-input landing-field"
                value={otp}
                onChange={(e) => setOtp(e.target.value)}
                placeholder=" "
                required
                maxLength={6}
                autoFocus
                inputMode="numeric"
              />
              <label className="fl-label fl-label--glass">OTP Code</label>
            </div>

            <button type="submit" className="landing-submit" disabled={loading}>
              {loading ? "Verifying..." : "Verify Email"}
            </button>

            <button
              type="button"
              className="landing-link-btn"
              onClick={handleResend}
              disabled={resending || resendSeconds > 0}
            >
```

```
226               {resending ? "Resending..." : resendSeconds > 0 ? `Resend OTP in ${resendSeconds}s` : "Resend OTP"}
227             </button>
228
229             <Link to="/" className="landing-link-btn" style={{ textDecoration: "none" }}>
230               Back to Login
231             </Link>
232           </form>
233         </div>
234       </div>
235     </div>
236   );
237 }
```

47. client/src/pages/VerifyDocument.jsx (60 lines)

```
import { useEffect, useState } from "react";
import { useParams } from "react-router-dom";
import { CheckCircle, XCircle } from "lucide-react";
import { verifyRequestCode } from "../services/requestService";

export default function VerifyDocument() {
  const { code } = useParams();
  const [data, setData] = useState(null);
  const [error, setError] = useState("");

  useEffect(() => {
    const load = async () => {
      try {
        const res = await verifyRequestCode(code);
        setData(res);
      } catch (err) {
        setError(err.response?.data?.message || "Verification failed");
      }
    };
    load();
  }, [code]);

  if (error) {
    return (
      <div className="verify-outer">
        <div className="verify-card">
          <span className="verify-result-icon"><XCircle size={48} color="#dc2626" /></span>
          <h2 className="verify-result-error">Verification Failed</h2>
          <p style={{ color: "var(--text-muted)", fontSize: "14px", margin: 0 }}>{error}</p>
        </div>
      </div>
    );
  }

  if (!data) return null;

  return (
    <div className="verify-outer">
      <div className="verify-card">
        <span className="verify-result-icon">{data.valid ? <CheckCircle size={48} color="#16a34a" /> : <XCircle size={48} colo
        {data.valid ? (
          <h2 className="verify-result-title">Document Verified</h2>
        ) : (
          <h2 className="verify-result-error">Invalid Document</h2>
        )}

        <div className="verify-meta-grid">
          <span className="verify-meta-label">Type</span>
          <span className="verify-meta-value">{data.type || "N/A"}</span>

          <span className="verify-meta-label">Status</span>
          <span className="verify-meta-value">{data.status || "N/A"}</span>

          <span className="verify-meta-label">Issued</span>
          <span className="verify-meta-value">{data.issuedAt || "N/A"}</span>
        </div>
      </div>
    </div>
  );
}
```

48. client/src/pages/public/RepSigningPage.jsx (370 lines)

```
import { useEffect, useRef, useState } from "react";
import { useParams } from "react-router-dom";
import { AnimatePresence, motion } from "framer-motion";
import { getBySigningToken, repSubmit, repReject } from "../../services/requestService";
import { uploadSignatureImage, uploadRepGovId, deleteStorageFile } from "../../services/firebaseStorageService";
import SignaturePad from "../../components/SignaturePad";
import { notify } from "../../utils/inPageFeedback";
import "../../components/RepSigningPage.css";

export default function RepSigningPage() {
  const { token } = useParams();

  const [reqData, setReqData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
  const [done, setDone] = useState(""); // success message

  const [repFirstName, setRepFirstName] = useState("");
  const [repMiddleName, setRepMiddleName] = useState("");
  const [repLastName, setRepLastName] = useState("");
  const [govIdFile, setGovIdFile] = useState(null);
  const [accepted, setAccepted] = useState(false);
  const [submitting, setSubmitting] = useState(false);
  const [showConfirm, setShowConfirm] = useState(null); // "submit" | "decline" | null

  const sigPadRef = useRef(null);
  const fileInputRef = useRef(null);

  useEffect(() => {
    (async () => {
      try {
        const r = await getBySigningToken(token);
        setReqData(r);
      } catch (err) {
        setError(err.response?.data?.message || "Invalid or expired signing link.");
      } finally {
        setLoading(false);
      }
    })();
  }, [token]);

  const sanitizeNamePart = (value) => String(value || "").replace(/[^A-Za-z\s'.-]/g, "");

  // Keep this hook before any conditional return to preserve hook order.
  useEffect(() => {
    if (!reqData) return;
    const fd = reqData.formData || {};
    const first = fd.repFirstName || "";
    const middle = fd.repMiddleInitial || "";
    const last = fd.repLastName || "";

    if (first || middle || last) {
      setRepFirstName(sanitizeNamePart(first));
      setRepMiddleName(sanitizeNamePart(middle));
      setRepLastName(sanitizeNamePart(last));
    }
  }, [reqData]);

  if (loading) return (
    <div className="rep-signing-outer">
      <div className="rep-signing-state"><p>Loading...</p></div>
    </div>
  );

  if (error) return (
    <div className="rep-signing-outer">
      <div className="rep-signing-state">
        <h2>Error</h2>
        <p>{error}</p>
      </div>
    </div>
  );

```

```
  if (done) return (
    <div className="rep-signing-outer">
      <div className="rep-signing-state">
        <h2>Thank you</h2>
        <p>{done}</p>
      </div>
    </div>
  );

  // Guard: token already used
  if (reqData.signingTokenUsed) {
    return (
      <div className="rep-signing-outer">
        <div className="rep-signing-state">
          <h2>Link Already Used</h2>
          <p>This signing link has already been used. Please contact the requestor if you need a new one.</p>
        </div>
      </div>
    );
  }

  // Guard: wrong status
  const signingStatuses = [
    "agr_awaiting_rep_signature",
    "agr_rep_revision_requested",
  ];
  if (!signingStatuses.includes(reqData.status)) {
    return (
      <div className="rep-signing-outer">
        <div className="rep-signing-state">
          <h2>Link Not Active</h2>
          <p>This signing link is no longer active (status: {reqData.status}).</p>
        </div>
      </div>
    );
  }

  const isRevision = reqData.status === "agr_rep_revision_requested";
  const student = reqData.userId || {};
  const fd = reqData.formData || {};
  const storedMiddle = sanitizeNamePart(fd.repMiddleInitial || "").trim();
  const storedMiddleInitial = storedMiddle ? `${storedMiddle[0].toUpperCase()}.` : "";
  const requestedRepName = [fd.repFirstName, storedMiddleInitial, fd.repLastName]
    .filter(Boolean)
    .join(" ")
    .replace(/\s+/g, " ")
    .trim() || fd.repName || "—";
  const normalizedMiddleName = sanitizeNamePart(repMiddleName).replace(/\./g, "").trim();
  const representativeFullName = [
    sanitizeNamePart(repFirstName).trim(),
    normalizedMiddleName ? `${normalizedMiddleName[0].toUpperCase()}.` : "",
    sanitizeNamePart(repLastName).trim(),
  ]
    .filter(Boolean)
    .join(" ")
    .replace(/\s+/g, " ")
    .trim();

  const handleReject = () => {
    setShowConfirm("decline");
  };

  const handleConfirmReject = async () => {
    setShowConfirm(null);
    setSubmitting(true);
    try {
      await repReject(token);
      setDone("You have declined the signing request. The requestor has been notified.");
    } catch (err) {
      notify(err.response?.data?.message || "Failed to reject. Please try again.", { type: "error" });
    } finally {
      setSubmitting(false);
    }
  };

  const handleSubmit = (e) => {
```

```
    e.preventDefault();

    if (!sanitizeNamePart(repFirstName).trim()) { notify("Please enter your first name.", { type: "warning" }); return; }
    if (!sanitizeNamePart(repLastName).trim()) { notify("Please enter your last name.", { type: "warning" }); return; }
    if (!govIdFile) { notify("Please upload your government-issued ID.", { type: "warning" }); return; }
    if (!sigPadRef.current || sigPadRef.current.isEmpty()) { notify("Please draw your signature.", { type: "warning" }); retu
    if (!accepted) { notify("You must check 'I accept' to proceed.", { type: "warning" }); return; }

    setShowConfirm("submit");
  };

  const handleConfirmSubmit = async () => {
    setShowConfirm(null);
    setSubmitting(true);
    try {
      const authSigPath = reqData.authorizerSigPath || "";
      const pathParts = authSigPath.split ? authSigPath.split("/") : [];
      const requestFolder = pathParts[3] || `rep_${Date.now()}`;
      const studentName = student.name || "unknown";

      if (isRevision && reqData.repInfo?.govIdDoc?.path) {
        await deleteStorageFile(reqData.repInfo.govIdDoc.path);
      }

      const govIdDoc = await uploadRepGovId(govIdFile, "agreement", studentName, requestFolder);

      const sigDataUrl = sigPadRef.current.getDataUrl();
      const { url: repSigUrl, path: repSigPath } = await uploadSignatureImage(
        sigDataUrl,
        "agreement",
        studentName,
        requestFolder,
        "rep_sig.png"
      );

      await repSubmit(token, {
        repName: representativeFullName,
        repGovIdDoc: govIdDoc,
        repSigUrl,
        repSigPath,
      });

      setDone("Your submission has been received. Thank you for signing the agreement.");
    } catch (err) {
      notify(err.response?.data?.message || "Submission failed. Please try again.", { type: "error" });
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <div className="rep-signing-outer">
      <div className="rep-signing-card">
        <h2 className="rep-signing-title">Agreement Signing Request</h2>

        {isRevision && (
          <div className="rep-signing-revision-banner">
            <strong>Revision Requested</strong>
            <p>{reqData.remarks || "Please review and resubmit your information."}</p>
          </div>
        )}

        <p className="rep-signing-intro">
          You have been invited to sign an authorization agreement as a representative.
          Please review the details below and complete the form to sign.
        </p>

        {/* Letter-format summary */}
        <div className="rep-signing-summary" style={{ padding: "20px 24px", gap: 0 }}>
          {/* Date — left */}
          <div style={{ fontSize: 13, color: "var(--text-secondary)", marginBottom: 18 }}>
            {reqData.createdAt ? new Date(reqData.createdAt).toLocaleDateString("en-US", { year: "numeric", month: "long", da
          </div>

          {/* Authorization paragraph — centered */}
          <p style={{ textAlign: "center", fontSize: 13.5, color: "var(--text-primary)", lineHeight: 1.75, margin: "0 0 24px"
```

```
              I, <strong>{student.name || "—"}</strong>, hereby authorize{" "}
              <strong>{requestedRepName}</strong> to act as my representative and sign on my behalf
              in connection with this agreement, as governed by the Data Protection Office.
            </p>

            {/* Signature block — centered */}
            <div style={{ display: "flex", flexDirection: "column", alignItems: "center", gap: 6 }}>
              {reqData.authorizerSigUrl && (
                <img
                  src={reqData.authorizerSigUrl}
                  alt="Authorizer signature"
                  style={{ maxHeight: 72, display: "block" }}
                  loading="lazy"
                  decoding="async"
                />
              )}
              <div style={{ width: 220, borderTop: "1px solid var(--border-strong)", textAlign: "center", paddingTop: 6, fontSi
                {student.name || "—"}
              </div>
            </div>
          </div>

          <hr className="rep-signing-divider" />

          {/* Representative's form */}
          <form onSubmit={handleSubmit}>
            <h3 className="rep-signing-section-title">Your Information</h3>

            <div className="rep-signing-field">
              <p className="rep-signing-name-note">Please ensure the name entered matches the name shown on your government-iss
              <label className="rep-signing-label">Full Name *</label>
              <div className="rep-signing-name-grid">
                <div className="rep-signing-name-col">
                  <label className="rep-signing-name-sublabel">First Name</label>
                  <input
                    className="rep-signing-input"
                    value={repFirstName}
                    onChange={(e) => setRepFirstName(sanitizeNamePart(e.target.value))}
                    required
                  />
                </div>
                <div className="rep-signing-name-col">
                  <label className="rep-signing-name-sublabel">Middle Name</label>
                  <input
                    className="rep-signing-input"
                    value={repMiddleName}
                    onChange={(e) => setRepMiddleName(sanitizeNamePart(e.target.value))}
                  />
                </div>
                <div className="rep-signing-name-col">
                  <label className="rep-signing-name-sublabel">Last Name</label>
                  <input
                    className="rep-signing-input"
                    value={repLastName}
                    onChange={(e) => setRepLastName(sanitizeNamePart(e.target.value))}
                    required
                  />
                </div>
              </div>
            </div>

            <div className="rep-signing-field">
              <label className="rep-signing-label">Government-Issued Valid ID *</label>
              <div className="rep-signing-file-row">
                <div className="rep-signing-file-info">
                  <span className="rep-signing-file-title">
                    {govIdFile ? govIdFile.name : "No file chosen"}
                  </span>
                  <span className="rep-signing-file-subtitle">PDF or image accepted</span>
                </div>
                <label className="rep-signing-file-action">
                  {govIdFile ? "Change File" : "Upload File"}
                  <input
                    type="file"
                    accept="application/pdf,image/*"
                    onChange={(e) => setGovIdFile(e.target.files?.[0] || null)}
```

```
                  ref={fileInputRef}
                />
              </label>
            </div>
          </div>

          <div className="rep-signing-field">
            <label className="rep-signing-label">Your E-Signature *</label>
            <p className="rep-signing-sig-hint">Draw your signature in the box below.</p>
            <SignaturePad ref={sigPadRef} height={160} />
          </div>

          <label className="rep-signing-accept">
            <input
              type="checkbox"
              checked={accepted}
              onChange={(e) => setAccepted(e.target.checked)}
            />
            <span>
              I accept and acknowledge that by signing this form I am authorizing the above-named requestor
              and agree to the terms of this agreement as issued by the Data Protection Office.
            </span>
          </label>

          <div className="rep-signing-actions">
            <button type="submit" className="rep-signing-btn-submit" disabled={submitting}>
              {submitting ? "Submitting..." : "Submit & Sign"}
            </button>
            <button
              type="button"
              className="rep-signing-btn-decline"
              onClick={handleReject}
              disabled={submitting}
            >
              Decline
            </button>
          </div>
        </form>
      </div>

      <AnimatePresence>
        {showConfirm && (
          <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
            <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12,
              <h3 className="modal-title">
                {showConfirm === "submit" ? "Submit & Sign Agreement" : "Decline Signing Request"}
              </h3>
              <p className="modal-text">
                {showConfirm === "submit"
                  ? "Please confirm that all your information and signature are accurate before submitting."
                  : "Are you sure you want to decline? This action is final and the requestor will be notified."}
              </p>
              <div className="modal-actions">
                <button className="ui-btn ui-btn--secondary" onClick={() => setShowConfirm(null)} disabled={submitting}>Cance
                <button
                  className="ui-btn ui-btn--primary"
                  onClick={showConfirm === "submit" ? handleConfirmSubmit : handleConfirmReject}
                  disabled={submitting}
                >
                  {showConfirm === "submit" ? "Submit" : "Decline"}
                </button>
              </div>
            </motion.div>
          </motion.div>
        )}
      </AnimatePresence>
    </div>
  );
}
```

49. client/src/pages/student/StudentDashboard.jsx (377 lines)

```
import { useEffect, useMemo, useState } from "react";
import { useNavigate } from "react-router-dom";
import { RefreshCw, Search, Filter, FilePlus } from "lucide-react";
import { getMyRequests } from "../../services/requestService";
import FilterSelect from "../../components/FilterSelect";
import { notify } from "../../utils/inPageFeedback";

const STATUS_LABEL = {
  nda_pending:                "Reviewal",
  nda_approved:               "Approved",
  stud_revision_requested:    "Student Revisions",
  agr_pending_1:              "Initial Reviewal",
  agr_awaiting_rep_signature: "Recipient Reviewal",
  agr_pending_2:              "Final Reviewal",
  agr_approved:               "Approved",
  agr_declined:               "Recipient Declined",
  agr_rep_revision_requested: "Recipient Revisions",
};

const STATUS_FILTER_OPTIONS = [
  { value: "all", label: "All Statuses" },
  { value: "reviewal", label: "Reviewal" },
  { value: "approved", label: "Approved" },
  { value: "stud_revision", label: "Student Revisions" },
  { value: "rep_revision", label: "Recipient Revisions" },
  { value: "rep_declined", label: "Recipient Declined" },
  { value: "awaiting_rep", label: "Recipient Reviewal" },
];

const prettyStatus = (s) =>
  STATUS_LABEL[s] || (s ? s.charAt(0).toUpperCase() + s.slice(1).replace(/_/g, " ") : "—");

const statusClass = (s) => {
  if (["nda_approved", "agr_approved"].includes(s)) return "status-pill status-pill--green";
  if (["nda_pending", "agr_pending_1", "agr_pending_2"].includes(s)) return "status-pill status-pill--yellow";
  if (s === "stud_revision_requested") return "status-pill status-pill--orange";
  if (s === "agr_rep_revision_requested") return "status-pill status-pill--violet";
  if (s === "agr_awaiting_rep_signature") return "status-pill status-pill--blue";
  if (s === "agr_declined") return "status-pill status-pill--red";
  return "status-pill";
};

const normalizeStatusForStudentFilter = (status) => {
  if (["nda_pending", "agr_pending_1", "agr_pending_2"].includes(status)) return "reviewal";
  if (["nda_approved", "agr_approved"].includes(status)) return "approved";
  if (status === "stud_revision_requested") return "stud_revision";
  if (status === "agr_rep_revision_requested") return "rep_revision";
  if (status === "agr_declined") return "rep_declined";
  if (status === "agr_awaiting_rep_signature") return "awaiting_rep";
  return "reviewal";
};

const StudentDashboard = () => {
  const navigate = useNavigate();
  const [requests, setRequests] = useState([]);
  const [loading, setLoading] = useState(true);
  const [refreshing, setRefreshing] = useState(false);

  // Search state
  const [searchTerm, setSearchTerm] = useState("");

  // Filter state
  const [typeFilter, setTypeFilter] = useState("all");
  const [activeFilter, setActiveFilter] = useState("all");
  const [dateMode, setDateMode] = useState("preset");
  const [preset, setPreset] = useState("all");
  const [singleDate, setSingleDate] = useState("");
  const [startDate, setStartDate] = useState("");
  const [endDate, setEndDate] = useState("");

  const [isFilterOpen, setIsFilterOpen] = useState(() =>
    typeof window === "undefined" ? true : window.innerWidth >= 768
  );
```

```
  74
  75    const loadRequests = async ({ withToast = false } = {}) => {
  76      if (withToast) setRefreshing(true);
  77      try {
  78        const data = await getMyRequests();
  79        setRequests(data);
  80      } catch (err) {
  81        notify(err.response?.data?.message || "Failed to load requests", { type: "error" });
  82      } finally {
  83        setLoading(false);
  84        setRefreshing(false);
  85      }
  86    };
  87
  88    useEffect(() => {
  89      loadRequests();
  90    }, []);
  91
  92    useEffect(() => {
  93      const onResize = () => { if (window.innerWidth >= 768) setIsFilterOpen(true); };
  94      window.addEventListener("resize", onResize);
  95      return () => window.removeEventListener("resize", onResize);
  96    }, []);
  97
  98    const clearSearch = () => setSearchTerm("");
  99
 100    const resetAndRefresh = () => {
 101      setSearchTerm("");
 102      setTypeFilter("all");
 103      setActiveFilter("all");
 104      setDateMode("preset");
 105      setPreset("all");
 106      setSingleDate("");
 107      setStartDate("");
 108      setEndDate("");
 109      loadRequests({ withToast: true });
 110    };
 111
 112    const filtered = useMemo(() => {
 113      let result = requests;
 114
 115      // Search filter
 116      if (searchTerm) {
 117        const term = searchTerm.toLowerCase();
 118        result = result.filter((r) => {
 119          const id = (r._id || "").slice(-7).toLowerCase();
 120          const type = r.type === "nda"
 121            ? `nda ${r.formData?.ndaTypeLabel || ""}`.toLowerCase()
 122            : "agreement";
 123          return id.includes(term) || type.includes(term);
 124        });
 125      }
 126
 127      // Type filter
 128      if (typeFilter !== "all") {
 129        if (typeFilter === "agreement") {
 130          result = result.filter((r) => r.type === "agreement");
 131        } else if (typeFilter === "nda_orgactivities") {
 132          result = result.filter((r) => r.type === "nda" && r.formData?.ndaType === "orgactivities");
 133        } else if (typeFilter === "nda_research") {
 134          result = result.filter((r) => r.type === "nda" && r.formData?.ndaType === "research");
 135        }
 136      }
 137
 138      // Status filter
 139      if (activeFilter !== "all") {
 140        result = result.filter((r) => normalizeStatusForStudentFilter(r.status) === activeFilter);
 141      }
 142
 143      // Date filter
 144      if (dateMode === "preset") {
 145        const now = new Date();
 146        if (preset === "today") {
 147          const dayStart = new Date(now.getFullYear(), now.getMonth(), now.getDate());
 148          const dayEnd = new Date(dayStart.getTime() + 86400000 - 1);
 149          result = result.filter((r) => { const d = new Date(r.createdAt); return d >= dayStart && d <= dayEnd; });
```

```
      } else if (preset === "thisWeek") {
        const day = now.getDay();
        const monday = new Date(now);
        monday.setDate(now.getDate() - ((day + 6) % 7));
        monday.setHours(0, 0, 0, 0);
        result = result.filter((r) => new Date(r.createdAt) >= monday);
      } else if (preset === "thisMonth") {
        const monthStart = new Date(now.getFullYear(), now.getMonth(), 1);
        result = result.filter((r) => new Date(r.createdAt) >= monthStart);
      } else if (preset === "thisYear") {
        const yearStart = new Date(now.getFullYear(), 0, 1);
        result = result.filter((r) => new Date(r.createdAt) >= yearStart);
      }
    } else if (dateMode === "single" && singleDate) {
      const dayStart = new Date(singleDate);
      const dayEnd = new Date(singleDate + "T23:59:59");
      result = result.filter((r) => { const d = new Date(r.createdAt); return d >= dayStart && d <= dayEnd; });
    } else if (dateMode === "range") {
      if (startDate) result = result.filter((r) => new Date(r.createdAt) >= new Date(startDate));
      if (endDate) result = result.filter((r) => new Date(r.createdAt) <= new Date(endDate + "T23:59:59"));
    }

    return result;
  }, [requests, searchTerm, typeFilter, activeFilter, dateMode, preset, singleDate, startDate, endDate]);

  return (
    <div className="dashboard-page">
      <div className="page-shell__header">
        <button
          type="button"
          className="req-toolbar__action-btn"
          onClick={() => navigate("/student/new-request")}
        >
          <FilePlus size={14} />
          Create Request
        </button>
      </div>

      {/* ■■ Search + Filter toolbar ■■ */}
      <div className="req-toolbar">
        <div className="req-toolbar__search-row">
          <div className="req-toolbar__search-box">
            <input
              type="text"
              className="req-toolbar__search-input"
              placeholder="Search by request ID or type…"
              value={searchTerm}
              onChange={(e) => setSearchTerm(e.target.value)}
            />
            {searchTerm ? (
              <button type="button" className="req-toolbar__search-clear" onClick={clearSearch} aria-label="Clear search">
                ×
              </button>
            ) : null}
            <span className="req-toolbar__search-inline-btn" style={{ pointerEvents: "none" }}>
              <Search size={14} />
            </span>
          </div>
          <button
            type="button"
            className="ui-btn ui-btn--secondary req-toolbar__filter-toggle"
            onClick={() => setIsFilterOpen((prev) => !prev)}
            aria-expanded={isFilterOpen}
          >
            <Filter size={13} />
            Filters
          </button>
        </div>

        <div className={`req-toolbar__filters${isFilterOpen ? " req-toolbar__filters--open" : ""}`}>
          <div className="req-toolbar__filter-group">
            <label className="req-toolbar__filter-label">Type</label>
            <FilterSelect
              value={typeFilter}
              onChange={setTypeFilter}
              className="filter-select--type"
```

```
              options={[
                { value: "all", label: "All Types" },
                { value: "nda_orgactivities", label: "NDA — Student Organization Activities" },
                { value: "nda_research", label: "NDA — Conduct of Research" },
                { value: "agreement", label: "Agreement" },
              ]}
            />
          </div>

          <div className="req-toolbar__filter-group">
            <label className="req-toolbar__filter-label">Status</label>
            <FilterSelect
              value={activeFilter}
              onChange={setActiveFilter}
              className="filter-select--status"
              options={STATUS_FILTER_OPTIONS}
            />
          </div>

          <div className="req-toolbar__filter-group">
            <label className="req-toolbar__filter-label">Date</label>
            <FilterSelect
              value={dateMode}
              onChange={setDateMode}
              options={[
                { value: "preset", label: "Preset" },
                { value: "single", label: "Specific Date" },
                { value: "range", label: "Date Range" },
              ]}
              defaultValue="preset"
            />
          </div>

          {dateMode === "preset" ? (
            <div className="req-toolbar__filter-group">
              <label className="req-toolbar__filter-label">Period</label>
              <FilterSelect
                value={preset}
                onChange={setPreset}
                options={[
                  { value: "all", label: "All Time" },
                  { value: "today", label: "Today" },
                  { value: "thisWeek", label: "This Week" },
                  { value: "thisMonth", label: "This Month" },
                  { value: "thisYear", label: "This Year" },
                ]}
              />
            </div>
          ) : null}

          {dateMode === "single" ? (
            <div className="req-toolbar__filter-group">
              <label className="req-toolbar__filter-label">Date</label>
              <input className={`req-toolbar__filter-select${singleDate !== "" ? " req-toolbar__filter-select--active" : ""}`
            </div>
          ) : null}

          {dateMode === "range" ? (
            <>
              <div className="req-toolbar__filter-group">
                <label className="req-toolbar__filter-label">From</label>
                <input className={`req-toolbar__filter-select${startDate !== "" ? " req-toolbar__filter-select--active" : ""}
              </div>
              <div className="req-toolbar__filter-group">
                <label className="req-toolbar__filter-label">To</label>
                <input className={`req-toolbar__filter-select${endDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
              </div>
            </>
          ) : null}

          <button
            type="button"
            className="req-toolbar__reset-btn"
            onClick={resetAndRefresh}
            disabled={refreshing}
            title="Reset filters and refresh"
```

```
302           >
303             <RefreshCw size={14} className={refreshing ? "spin-anim" : ""} />
304           </button>
305         </div>
306       </div>
307
308       <div className="dashboard-card">
309         <div className="table-scroll">
310         <table className="dashboard-table">
311           <thead>
312             <tr className="dashboard-table-title-row">
313               <th colSpan={5}>
314                 <div className="dashboard-table-title-wrap">
315                   <span className="dashboard-table-title">My Requests</span>
316                 </div>
317               </th>
318             </tr>
319             <tr>
320               <th>Request Date</th>
321               <th>Request ID</th>
322               <th>Type</th>
323               <th>Status</th>
324               <th />
325             </tr>
326           </thead>
327
328           <tbody>
329             {loading ? (
330               [...Array(4)].map((_, i) => (
331                 <tr key={`sk-${i}`}>
332                   <td><span className="skeleton-block skeleton-text" /></td>
333                   <td><span className="skeleton-block skeleton-text" /></td>
334                   <td><span className="skeleton-block skeleton-text" /></td>
335                   <td><span className="skeleton-block skeleton-pill" /></td>
336                   <td />
337                 </tr>
338               ))
339             ) : filtered.length === 0 ? (
340               <tr>
341                 <td colSpan={5}>
342                   <div className="dashboard-empty">
343                     <p className="dashboard-empty-title">No requests found</p>
344                     <p className="dashboard-empty-text">No requests match the current filters.</p>
345                   </div>
346                 </td>
347               </tr>
348             ) : (
349               filtered.map((r) => (
350                 <tr key={r._id} onClick={() => navigate(`/student/requests/${r._id}`)} style={{ cursor: "pointer" }}>
351                   <td style={{ whiteSpace: "nowrap", color: "var(--text-secondary)" }}>{new Date(r.createdAt).toLocaleDateStr
352                   <td><span style={{ color: "var(--text-secondary)" }}>{r._id?.slice(-7).toUpperCase()}</span></td>
353                   <td>
354                     {r.type === "nda"
355                       ? `NDA${r.formData?.ndaTypeLabel ? ` — ${r.formData.ndaTypeLabel}` : ""}`
356                       : "Agreement"}
357                   </td>
358
359                   <td>
360                     <span className={statusClass(r.status)}>
361                       {prettyStatus(r.status)}
362                     </span>
363                   </td>
364
365                   <td className="row-caret">›</td>
366                 </tr>
367               ))
368             )}
369           </tbody>
370         </table>
371         </div>
372       </div>
373     </div>
374   );
375 };
376
377 export default StudentDashboard;
```

50. client/src/pages/student/StudentProfile.jsx (314 lines)

```
import { useContext, useRef, useState } from "react";
import { Eye, EyeOff } from "lucide-react";
import { motion, AnimatePresence } from "framer-motion";
import { AuthContext } from "../../context/AuthContext";
import { updateProfile, requestPasswordChangeOtp } from "../../services/authService";
import PasswordChecklist, { ErrorTooltip } from "../../components/PasswordChecklist";
import { isStrongPassword } from "../../utils/passwordPolicy";
import { notify } from "../../utils/inPageFeedback";
import "../../components/RequestForm.css";
import "../../components/FloatingLabel.css";

const StudentProfile = () => {
  const { user, updateUser } = useContext(AuthContext);
  const [firstName, setFirstName] = useState(user?.firstName || "");
  const [middleName, setMiddleName] = useState(user?.middleInitial || "");
  const [lastName, setLastName] = useState(user?.lastName || "");
  const [currentPassword, setCurrentPassword] = useState("");
  const [newPassword, setNewPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [loading, setLoading] = useState(false);
  const [showCurrentPw, setShowCurrentPw] = useState(false);
  const [showNewPw, setShowNewPw] = useState(false);
  const [showConfirmPw, setShowConfirmPw] = useState(false);
  const [confirmName, setConfirmName] = useState(false);
  const [confirmPw, setConfirmPw] = useState(false);
  const [otp, setOtp] = useState("");
  const [otpCooldown, setOtpCooldown] = useState(0);

  const [newPwFocused, setNewPwFocused] = useState(false);
  const [confirmPwFocused, setConfirmPwFocused] = useState(false);

  // Refs for tooltip anchoring
  const newPwAnchorRef = useRef(null);
  const confirmPwAnchorRef = useRef(null);
  const cooldownRef = useRef(null);

  const initials = user?.name
    ? user.name.split(" ").map((w) => w[0]).slice(0, 2).join("").toUpperCase()
    : "?";

  const startCooldown = (seconds) => {
    setOtpCooldown(seconds);
    clearInterval(cooldownRef.current);
    cooldownRef.current = setInterval(() => {
      setOtpCooldown((s) => {
        if (s <= 1) { clearInterval(cooldownRef.current); return 0; }
        return s - 1;
      });
    }, 1000);
  };

  const handleNameSubmit = (e) => {
    e.preventDefault();
    if (!firstName.trim() || !lastName.trim()) {
      notify("Failed.", { type: "error", title: "Failed" });
      return;
    }
    const hasChanges =
      firstName.trim() !== (user?.firstName || "") ||
      middleName.trim() !== (user?.middleInitial || "") ||
      lastName.trim() !== (user?.lastName || "");
    if (!hasChanges) {
      notify("No changes to save.", { type: "warning", title: "Notice" });
      return;
    }
    setConfirmName(true);
  };

  const handleConfirmNameUpdate = async () => {
    setLoading(true);
    try {
      const data = await updateProfile({ firstName: firstName.trim(), middleInitial: middleName.trim(), lastName: lastName.tr
      updateUser(data.user);
```

```
      notify("Name updated successfully.", { type: "success", title: "Success" });
      setConfirmName(false);
    } catch {
      notify("Failed.", { type: "error", title: "Failed" });
      setConfirmName(false);
    } finally {
      setLoading(false);
    }
  };

  const handlePasswordSubmit = async (e) => {
    e.preventDefault();
    if (!currentPassword || !isStrongPassword(newPassword) || newPassword !== confirmPassword) return;
    setLoading(true);
    try {
      await requestPasswordChangeOtp();
      setOtp("");
      startCooldown(60);
      setConfirmPw(true);
    } catch (err) {
      const data = err?.response?.data;
      if (err?.response?.status === 429 && data?.retryAfterSeconds) {
        startCooldown(data.retryAfterSeconds);
        setConfirmPw(true);
      } else {
        notify(data?.message || "Failed to send OTP.", { type: "error", title: "Error" });
      }
    } finally {
      setLoading(false);
    }
  };

  const handleResendOtp = async () => {
    if (otpCooldown > 0) return;
    setLoading(true);
    try {
      await requestPasswordChangeOtp();
      setOtp("");
      startCooldown(60);
      notify("A new OTP has been sent to your email.", { type: "info", title: "OTP Sent" });
    } catch (err) {
      const data = err?.response?.data;
      if (err?.response?.status === 429 && data?.retryAfterSeconds) {
        startCooldown(data.retryAfterSeconds);
      }
      notify(data?.message || "Failed to resend OTP.", { type: "error", title: "Error" });
    } finally {
      setLoading(false);
    }
  };

  const handleConfirmPasswordUpdate = async () => {
    if (!otp.trim()) { notify("Please enter the OTP sent to your email.", { type: "warning" }); return; }
    setLoading(true);
    try {
      await updateProfile({ currentPassword, newPassword, otp: otp.trim() });
      setCurrentPassword("");
      setNewPassword("");
      setConfirmPassword("");
      setOtp("");
      setConfirmPwFocused(false);
      setConfirmPw(false);
      notify("Password changed successfully.", { type: "success", title: "Success" });
    } catch (err) {
      notify(err?.response?.data?.message || "Failed to change password.", { type: "error", title: "Failed" });
    } finally {
      setLoading(false);
    }
  };

  const pwToggleStyle = {
    position: "absolute", right: 6, top: "50%", transform: "translateY(-50%)",
    background: "none", border: "none", padding: 4, cursor: "pointer",
    color: "var(--text-muted)", display: "flex", alignItems: "center",
  };

```

```
  return (
    <div style={{ display: "flex", justifyContent: "center" }}>
    <div style={{ maxWidth: 520, width: "100%" }}>

      {/* Profile Header */}
      <div style={{ background: "var(--surface)", border: "1px solid var(--border)", borderRadius: "var(--radius-lg)", paddin
        <div style={{ width: 72, height: 72, borderRadius: "50%", background: "var(--primary)", display: "flex", alignItems:
        <p style={{ margin: "0 0 4px", fontSize: 18, fontWeight: 700, color: "var(--text-primary)" }}>{user?.name}</p>
        <p style={{ margin: "0 0 8px", fontSize: 13, color: "var(--text-muted)" }}>{user?.email}</p>
        <span style={{ padding: "3px 12px", borderRadius: 99, fontSize: 11, fontWeight: 700, background: "#f1f5f9", color: "#
      </div>

      {/* Change Name */}
      <div style={{ background: "var(--surface)", border: "1px solid var(--border)", borderRadius: "var(--radius-lg)", paddin
        <h3 style={{ margin: "0 0 16px", fontSize: 15, fontWeight: 700, color: "var(--text-primary)" }}>Change Name</h3>
        <form onSubmit={handleNameSubmit} style={{ display: "flex", flexDirection: "column", gap: 10 }}>
          <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr 1fr", gap: 10 }}>
            <div className="fl-wrap">
              <input placeholder=" " value={firstName} onChange={(e) => setFirstName(e.target.value)} className="fl-input req
              <label className="fl-label">First Name</label>
            </div>
            <div className="fl-wrap">
              <input placeholder=" " value={middleName} onChange={(e) => setMiddleName(e.target.value)} className="fl-input r
              <label className="fl-label">Middle Name</label>
            </div>
            <div className="fl-wrap">
              <input placeholder=" " value={lastName} onChange={(e) => setLastName(e.target.value)} className="fl-input reque
              <label className="fl-label">Last Name</label>
            </div>
          </div>
          <div style={{ display: "flex", justifyContent: "flex-end" }}>
            <button type="submit" className="ui-btn ui-btn--primary" disabled={loading}>Save Changes</button>
          </div>
        </form>
      </div>

      {/* Change Password */}
      <div style={{ background: "var(--surface)", border: "1px solid var(--border)", borderRadius: "var(--radius-lg)", paddin
        <h3 style={{ margin: "0 0 16px", fontSize: 15, fontWeight: 700, color: "var(--text-primary)" }}>Change Password</h3>
        <form onSubmit={handlePasswordSubmit} style={{ display: "flex", flexDirection: "column", gap: 14 }}>

          {/* Current Password */}
          <div className="fl-wrap">
            <input
              type={showCurrentPw ? "text" : "password"}
              placeholder=" "
              value={currentPassword}
              onChange={(e) => setCurrentPassword(e.target.value)}
              className="fl-input request-input"
              style={{ paddingRight: 42 }}
            />
            <label className="fl-label fl-label--pw">Current Password</label>
            <button type="button" onClick={() => setShowCurrentPw((v) => !v)} tabIndex={-1} aria-label={showCurrentPw ? "Hide
              {showCurrentPw ? <Eye size={18} /> : <EyeOff size={18} />}
            </button>
          </div>

          {/* New Password */}
          <div className="fl-wrap" ref={newPwAnchorRef}>
            <input
              type={showNewPw ? "text" : "password"}
              placeholder=" "
              value={newPassword}
              onChange={(e) => setNewPassword(e.target.value)}
              onFocus={() => setNewPwFocused(true)}
              onBlur={() => setNewPwFocused(false)}
              minLength={8}
              className="fl-input request-input"
              style={{ paddingRight: 42 }}
            />
            <label className="fl-label fl-label--pw">New Password</label>
            <button type="button" onClick={() => setShowNewPw((v) => !v)} tabIndex={-1} aria-label={showNewPw ? "Hide" : "Sho
              {showNewPw ? <Eye size={18} /> : <EyeOff size={18} />}
            </button>
            <PasswordChecklist focused={!isStrongPassword(newPassword) && (newPwFocused || newPassword.length > 0)} anchorRef
          </div>
```

```
226
227           {/* Confirm Password */}
228           <div className="fl-wrap" ref={confirmPwAnchorRef}>
229             <input
230               type={showConfirmPw ? "text" : "password"}
231               placeholder=" "
232               value={confirmPassword}
233               onChange={(e) => setConfirmPassword(e.target.value)}
234               onFocus={() => setConfirmPwFocused(true)}
235               onBlur={() => setConfirmPwFocused(false)}
236               className="fl-input request-input"
237               style={{ paddingRight: 42 }}
238             />
239             <label className="fl-label fl-label--pw">Confirm New Password</label>
240             <button type="button" onClick={() => setShowConfirmPw((v) => !v)} tabIndex={-1} aria-label={showConfirmPw ? "Hide
241               {showConfirmPw ? <Eye size={18} /> : <EyeOff size={18} />}
242             </button>
243             <ErrorTooltip
244               show={confirmPassword !== newPassword && (confirmPwFocused || confirmPassword.length > 0)}
245               message="Passwords do not match"
246               anchorRef={confirmPwAnchorRef}
247               side="right"
248               persistent
249             />
250           </div>
251
252           <button type="submit" className="ui-btn ui-btn--primary" disabled={loading} style={{ alignSelf: "flex-end" }}>
253             {loading ? "Sending OTP…" : "Save Changes"}
254           </button>
255         </form>
256       </div>
257     </div>
258
259     {/* Confirm Name Change Modal */}
260     <AnimatePresence>
261       {confirmName && (
262         <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
263           <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12, op
264             <h3 className="modal-title">Confirm Change Name</h3>
265             <p className="modal-text">Are you sure you want to update your name?</p>
266             <p className="modal-text"><strong>{firstName.trim()} {middleName.trim() ? middleName.trim() + " " : ""}{lastName.
267             <div className="modal-actions">
268               <button className="ui-btn ui-btn--secondary" onClick={() => setConfirmName(false)} disabled={loading}>Cancel</b
269               <button className="ui-btn ui-btn--primary" onClick={handleConfirmNameUpdate} disabled={loading}>{loading ? "Sav
270             </div>
271           </motion.div>
272         </motion.div>
273       )}
274     </AnimatePresence>
275
276     {/* Confirm Password Change Modal */}
277     <AnimatePresence>
278       {confirmPw && (
279         <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
280           <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12, op
281             <h3 className="modal-title">Verify Your Identity</h3>
282             <p className="modal-text">An OTP has been sent to <strong>{user?.email}</strong>. Enter it below to confirm your
283             <div className="fl-wrap" style={{ marginBottom: 4 }}>
284               <input
285                 placeholder=" "
286                 value={otp}
287                 onChange={(e) => setOtp(e.target.value)}
288                 className="fl-input request-input"
289                 maxLength={6}
290                 autoFocus
291               />
292               <label className="fl-label">One-Time Password</label>
293             </div>
294             <button
295               type="button"
296               onClick={handleResendOtp}
297               disabled={otpCooldown > 0 || loading}
298               style={{ fontSize: 12, color: otpCooldown > 0 ? "var(--text-muted)" : "var(--primary)", background: "none", bor
299             >
300               {otpCooldown > 0 ? `Resend OTP in ${otpCooldown}s` : "Resend OTP"}
301             </button>
```

```
302              <div className="modal-actions">
303                <button className="ui-btn ui-btn--secondary" onClick={() => { setConfirmPw(false); setOtp(""); }} disabled={loa
304                <button className="ui-btn ui-btn--primary" onClick={handleConfirmPasswordUpdate} disabled={loading}>{loading ?
305              </div>
306            </motion.div>
307          </motion.div>
308        )}
309      </AnimatePresence>
310      </div>
311    );
312  };
313
314  export default StudentProfile;
```

51. client/src/pages/student/StudentNewRequest.jsx (34 lines)

```jsx
import { useNavigate } from "react-router-dom";
import { FileLock2, FileSignature } from "lucide-react";
import "../../components/TypeChooser.css";

export default function StudentNewRequest({
  basePath = "/student/new-request",
  title = "What type of request would you like to submit?",
}) {
  const navigate = useNavigate();

  return (
    <div className="type-chooser-page">
      <div className="type-chooser-grid">
        <button
          type="button"
          className="type-chooser-card-btn"
          onClick={() => navigate(`${basePath}/nda`)}
        >
          <FileLock2 size={64} strokeWidth={1.5} className="type-chooser-card-icon" />
          <span className="type-chooser-card-label">Non-Disclosure Agreement</span>
        </button>

        <button
          type="button"
          className="type-chooser-card-btn"
          onClick={() => navigate(`${basePath}/agreement`)}
        >
          <FileSignature size={64} strokeWidth={1.5} className="type-chooser-card-icon" />
          <span className="type-chooser-card-label">Agreement</span>
        </button>
      </div>
    </div>
  );
}
```

52. client/src/pages/student/StudentAgreementRequest.jsx (323 lines)

```
import { useMemo, useRef, useState } from "react";
import { useNavigate } from "react-router-dom";
import { AnimatePresence, motion } from "framer-motion";
import { FIELDS_FILE_SLOTS_CONFIG } from "../../config/fieldsFileSlotsConfig";
import { createRequest } from "../../services/requestService";
import {
  uploadRequirements,
  uploadSignatureImage,
  getDateRequestFolder,
} from "../../services/firebaseStorageService";
import SignaturePad from "../../components/SignaturePad";
import { notify } from "../../utils/inPageFeedback";
import "../../components/RequestForm.css";
import "../../components/FloatingLabel.css";

export default function StudentAgreementRequest({ proxyMode = false, fallbackPath = "/student/new-request/agreement" }) {
  const navigate = useNavigate();
  const cfg = useMemo(() => FIELDS_FILE_SLOTS_CONFIG.agreement, []);

  const [formData, setFormData] = useState(() => ({}));
  const [proxyRequestee, setProxyRequestee] = useState({
    firstName: "",
    middleInitial: "",
    lastName: "",
    email: "",
  });
  const [files, setFiles] = useState(() => Array(cfg.fileSlots.length).fill(null));
  const [accepted, setAccepted] = useState(false);
  const [submitting, setSubmitting] = useState(false);
  const [showConfirm, setShowConfirm] = useState(false);
  const sigPadRef = useRef(null);

  const fileSlots = cfg.fileSlots;

  const onChangeField = (name, value) =>
    setFormData((p) => ({ ...p, [name]: value }));

  const onChangeProxy = (name, value) =>
    setProxyRequestee((prev) => ({ ...prev, [name]: value }));

  const handleSubmit = async (e) => {
    e.preventDefault();

    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    for (const f of cfg.fields) {
      if (f.required && !String(formData[f.name] || "").trim()) {
        notify(`${f.label} is required`, { type: "warning" });
        return;
      }
      if (f.type === "email" && String(formData[f.name] || "").trim() && !emailRegex.test(String(formData[f.name]).trim())) {
        notify(`${f.label} must be a valid email address`, { type: "warning" });
        return;
      }
    }

    if (proxyMode) {
      const hasName = String(proxyRequestee.firstName || "").trim() && String(proxyRequestee.lastName || "").trim();
      if (!hasName) {
        notify("Requestor First Name and Last Name are required", { type: "warning" });
        return;
      }
      const requiredNonNameFields = [
        ["email", "Requestor Email"],
      ];
      for (const [key, label] of requiredNonNameFields) {
        if (!String(proxyRequestee[key] || "").trim()) {
          notify(`${label} is required`, { type: "warning" });
          return;
        }
      }
    }

    for (let i = 0; i < fileSlots.length; i++) {
```

```
      if (fileSlots[i].required && !files[i]) {
        notify(`${fileSlots[i].label} is required`, { type: "warning" });
        return;
      }
    }

    const selectedFiles = files.filter(Boolean);
    if (selectedFiles.length === 0) {
      notify("Please upload at least 1 requirement file.", { type: "warning" });
      return;
    }

    if (!proxyMode) {
      if (!sigPadRef.current || sigPadRef.current.isEmpty()) {
        notify("Please draw your e-signature before submitting.", { type: "warning" });
        return;
      }
      if (!accepted) {
        notify("Please confirm the declaration before submitting.", { type: "warning" });
        return;
      }
    }

    setShowConfirm(true);
  };

  const handleConfirmSubmit = async () => {
    setShowConfirm(false);
    setSubmitting(true);
    try {
      const selectedFiles = files.filter(Boolean);
      const user = JSON.parse(localStorage.getItem("user") || "null");
      const proxyFullName = proxyMode
        ? `${proxyRequestee.firstName}${proxyRequestee.middleInitial?.trim() ? " " + proxyRequestee.middleInitial.trim().char
        : "";
      const requestSubjectName = proxyMode
        ? proxyFullName || user?.name || "Proxy Requestor"
        : user?.name || "Unknown Student";
      const requestFolder = getDateRequestFolder();

      const uploaded = await uploadRequirements(selectedFiles, "agreement", requestSubjectName, requestFolder);

      let uploadIndex = 0;
      const predocs = files
        .map((f, i) => {
          if (!f) return null;
          const meta = uploaded[uploadIndex++];
          return { ...meta, requirementLabel: fileSlots[i]?.label || `File ${i + 1}` };
        })
        .filter(Boolean);

      let authorizerSigUrl = "";
      let authorizerSigPath = "";
      if (!proxyMode && !sigPadRef.current.isEmpty()) {
        const sigDataUrl = sigPadRef.current.getDataUrl();
        const sigResult = await uploadSignatureImage(sigDataUrl, "agreement", requestSubjectName, requestFolder, "authorizer_
        authorizerSigUrl = sigResult.url;
        authorizerSigPath = sigResult.path;
      }

      await createRequest({
        type: "agreement",
        formData,
        predocs,
        authorizerSigUrl,
        authorizerSigPath,
        ...(proxyMode ? { proxyRequestee: { ...proxyRequestee, fullName: proxyFullName } } : {}),
      });

      notify(proxyMode ? "Proxy agreement request submitted." : "Agreement request submitted!", { type: "success", title: "Su
      navigate(fallbackPath);
    } catch (err) {
      notify(err.response?.data?.message || "Failed to submit Agreement request", { type: "error" });
    } finally {
      setSubmitting(false);
    }
```

```
  };

  return (
    <div className="request-form-page">
      <form onSubmit={handleSubmit} className="request-form-card">
        <div className="request-form-header">
          <button
            type="button"
            className="request-form-back"
            onClick={() => { if (window.history.length > 1) navigate(-1); else navigate("/student"); }}
          >
            ‹ Back
          </button>
          <h2 className="request-form-title">Request Form</h2>
          <div />
        </div>

        {/* Section 1: Representative Data */}
        <div className="request-section">
          <div className="request-section-title">Representative Information</div>
          {/* Name fields — single row */}
          {(() => {
            const nameKeys = ["repFirstName", "repMiddleInitial", "repLastName"];
            const nameFields = cfg.fields.filter((f) => nameKeys.includes(f.name));
            const otherFields = cfg.fields.filter((f) => !nameKeys.includes(f.name));
            return (
              <>
                {nameFields.length > 0 && (
                  <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr 1fr", gap: 8 }}>
                    {nameFields.map((f) => (
                      <div key={f.name} className="fl-wrap">
                        <input
                          value={formData[f.name] || ""}
                          onChange={(e) => onChangeField(f.name, e.target.value)}
                          required={f.required}
                          placeholder=" "
                          className="fl-input request-input"
                        />
                        <label className="fl-label">{f.label}</label>
                      </div>
                    ))}
                  </div>
                )}
                {otherFields.map((f) => (
                  <div key={f.name} className="fl-wrap">
                    {f.kind === "textarea" ? (
                      <textarea
                        value={formData[f.name] || ""}
                        onChange={(e) => onChangeField(f.name, e.target.value)}
                        rows={f.rows || 4}
                        placeholder=" "
                        className="fl-input request-textarea"
                      />
                    ) : (
                      <input
                        type={f.type || "text"}
                        value={formData[f.name] || ""}
                        onChange={(e) => onChangeField(f.name, e.target.value)}
                        required={f.required}
                        placeholder=" "
                        className="fl-input request-input"
                      />
                    )}
                    <label className={`fl-label${f.kind === "textarea" ? " fl-label--area" : ""}`}>{f.label}</label>
                  </div>
                ))}
              </>
            );
          })()}

          {proxyMode && (
            <>
              <div className="request-section-title" style={{ marginTop: 10 }}>Requestor Details (F2F Walk-in)</div>
              <div style={{ display: "grid", gridTemplateColumns: "1fr 0.55fr 1fr", gap: 8 }}>
                <div className="fl-wrap">
                  <input className="fl-input request-input" placeholder=" " value={proxyRequestee.firstName} onChange={(e) =>
```

```
                  <label className="fl-label">First Name</label>
                </div>
                <div className="fl-wrap">
                  <input className="fl-input request-input" placeholder=" " value={proxyRequestee.middleInitial} onChange={(e
                  <label className="fl-label">Middle Name</label>
                </div>
                <div className="fl-wrap">
                  <input className="fl-input request-input" placeholder=" " value={proxyRequestee.lastName} onChange={(e) =>
                  <label className="fl-label">Last Name</label>
                </div>
              </div>
              <div className="fl-wrap">
                <input className="fl-input request-input" type="email" placeholder=" " value={proxyRequestee.email} onChange=
                <label className="fl-label">Requestor Email</label>
              </div>
            </>
          )}

        </div>

        {/* Section 2: Requirements */}
        <div className="request-section">
          <div className="request-section-title">Requirements (Agreement)</div>
          {files.map((file, index) => (
            <div key={index} className="request-file-row">
              <div className="request-file-info">
                <div className="request-file-title">
                  {fileSlots[index]?.label || `Attach file ${index + 1}`}
                  {fileSlots[index]?.required ? " *" : ""}
                </div>
                <div className="request-file-subtitle">{file ? file.name : "No file selected"}</div>
              </div>
              <label className="request-file-action">
                {file ? "Change File" : "Upload File"}
                <input
                  type="file"
                  accept="application/pdf,image/*"
                  onChange={(e) => {
                    const selected = e.target.files?.[0] || null;
                    setFiles((prev) => { const copy = [...prev]; copy[index] = selected; return copy; });
                  }}
                />
              </label>
            </div>
          ))}
        </div>

        {/* Section 3: E-Signature */}
        <div className="request-section">
          {proxyMode ? (
            <div className="info-banner info-banner--info" style={{ marginBottom: 0 }}>
              <strong>F2F Walk-in (Proxy Request)</strong>
              <p>Digital signatures are bypassed for face-to-face walk-ins. The document will be printed for a physical wet s
            </div>
          ) : (
            <>
              <div className="request-section-title">Your E-Signature *</div>
              <p className="request-sig-hint">Draw your signature below. It will be embedded in the agreement document.</p>
              <SignaturePad ref={sigPadRef} height={160} />
            </>
          )}
        </div>

        {/* Confirmation clause */}
        {!proxyMode && (
          <label className="request-accept">
            <input type="checkbox" checked={accepted} onChange={(e) => setAccepted(e.target.checked)} />
            <span>
              I confirm that all information and attachments I have provided are accurate and complete,
              and I consent to the processing of my personal data by the Data Protection Office for the purpose of this reque
            </span>
          </label>
        )}

        <div className="request-form-actions">
          <button type="submit" className="request-form-submit" disabled={submitting}>
```

```
302              {submitting ? "Submitting..." : "Submit"}
303            </button>
304          </div>
305        </form>
306
307        <AnimatePresence>
308          {showConfirm && (
309            <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
310              <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12,
311                <h3 className="modal-title">Submit Agreement Request</h3>
312                <p className="modal-text">Please confirm that all information and attachments are accurate before submitting.</
313                <div className="modal-actions">
314                  <button className="ui-btn ui-btn--secondary" onClick={() => setShowConfirm(false)} disabled={submitting}>Canc
315                  <button className="ui-btn ui-btn--primary" onClick={handleConfirmSubmit} disabled={submitting}>{submitting ?
316                </div>
317              </motion.div>
318            </motion.div>
319          )}
320        </AnimatePresence>
321      </div>
322    );
323  }
```

53. client/src/pages/student/StudentAgreementRequirements.jsx (70 lines)

```jsx
import { useNavigate } from "react-router-dom";
import { FileSignature, Info } from "lucide-react";
import "../../components/TypeChooser.css";

function LoiTooltip({ content }) {
  return (
    <span
      style={{ position: "relative", display: "inline-flex", alignItems: "center", marginLeft: 6, verticalAlign: "middle" }}
      onClick={(e) => e.stopPropagation()}
    >
      <span className="loi-tooltip-trigger">
        <Info size={13} style={{ color: "var(--text-muted)", cursor: "pointer" }} />
        <span className="loi-tooltip-box">
          <strong style={{ display: "block", marginBottom: 4, fontSize: 11 }}>Authorization Letter should include:</strong>
          {content.map((item, i) => (
            <span key={i} style={{ display: "block", fontSize: 11, lineHeight: 1.5 }}>• {item}</span>
          ))}
        </span>
      </span>
    </span>
  );
}

export default function StudentAgreementRequirements({
  formPath = "/student/new-request/agreement/form",
  fallbackPath = "/student",
}) {
  const navigate = useNavigate();

  return (
    <div className="type-chooser-page">
      <button
        type="button"
        className="type-chooser-back"
        onClick={() => {
          if (window.history.length > 1) navigate(-1);
          else navigate(fallbackPath);
        }}
      >
        ‹ Back
      </button>

      <div className="type-chooser-grid">
        <button
          type="button"
          className="type-chooser-card-btn type-chooser-card-btn--detailed"
          onClick={() => navigate(formPath)}
        >
          <FileSignature size={48} strokeWidth={1.5} className="type-chooser-card-icon" />
          <span className="type-chooser-card-label">Agreement</span>
          <span className="type-chooser-card-subtext">
            For assigning a representative to act on your behalf in data-related agreements with the DPO.
          </span>
          <span className="type-chooser-req-title">Requestor Requirements:</span>
          <ul className="type-chooser-req-list" aria-label="Requestor requirements">
            <li>Notarized Authorization Letter<LoiTooltip content={["Scope of authority granted to the representative", "If lo
            <li>Government Issued Valid ID</li>
          </ul>
          <span className="type-chooser-req-title">Representative Requirements:</span>
          <ul className="type-chooser-req-list" aria-label="Representative requirements">
            <li>Government Issued Valid ID</li>
          </ul>
          <span className="type-chooser-card-subtext" style={{ marginTop: 8, fontStyle: "italic" }}>
            Click to continue to the form →
          </span>
        </button>
      </div>
    </div>
  );
}
```

54. client/src/pages/student/StudentNDATypeChooser.jsx (91 lines)

```jsx
import { useNavigate } from "react-router-dom";
import { Building2, FlaskConical, Info } from "lucide-react";
import "../../components/TypeChooser.css";

function LoiTooltip({ content }) {
  return (
    <span
      style={{ position: "relative", display: "inline-flex", alignItems: "center", marginLeft: 6, verticalAlign: "middle" }}
      onClick={(e) => e.stopPropagation()}
    >
      <span className="loi-tooltip-trigger">
        <Info size={13} style={{ color: "var(--text-muted)", cursor: "pointer" }} />
        <span className="loi-tooltip-box">
          <strong style={{ display: "block", marginBottom: 4, fontSize: 11 }}>Letter of Intent should include:</strong>
          {content.map((item, i) => (
            <span key={i} style={{ display: "block", fontSize: 11, lineHeight: 1.5 }}>• {item}</span>
          ))}
        </span>
      </span>
    </span>
  );
}

export default function StudentNDATypeChooser({
  basePath = "/student/new-request/nda",
  fallbackPath = "/student",
}) {
  const navigate = useNavigate();

  return (
    <div className="type-chooser-page">
      <button
        type="button"
        className="type-chooser-back"
        onClick={() => {
          if (window.history.length > 1) navigate(-1);
          else navigate(fallbackPath);
        }}
      >
        ‹ Back
      </button>


      <div className="type-chooser-grid">
        <button
          type="button"
          className="type-chooser-card-btn type-chooser-card-btn--detailed"
          onClick={() => navigate(`${basePath}/orgactivities`)}
        >
          <Building2 size={48} strokeWidth={1.5} className="type-chooser-card-icon" />
          <span className="type-chooser-card-label">School Organization Activities/Projects</span>
          <span className="type-chooser-card-subtext">
            For student organization events or partner engagements involving confidential documents.
          </span>
          <span className="type-chooser-req-title">Requirements:</span>
          <ul className="type-chooser-req-list" aria-label="School organization NDA requirements">
            <li>Letter of Intent<LoiTooltip content={["Title of the Activity/Project", "Objectives of the Activity/Project", "
            <li>School ID</li>
            <li>Registration Form</li>
            <li>Sample Forms or Collaterals (if applicable)</li>
          </ul>
          <span className="type-chooser-card-subtext" style={{ marginTop: 8, fontStyle: "italic" }}>
            Click to continue to the form →
          </span>
        </button>

        <button
          type="button"
          className="type-chooser-card-btn type-chooser-card-btn--detailed"
          onClick={() => navigate(`${basePath}/research`)}
        >
          <FlaskConical size={48} strokeWidth={1.5} className="type-chooser-card-icon" />
          <span className="type-chooser-card-label">Conduct of Research within the University</span>
```

```
            <span className="type-chooser-card-subtext">
              For thesis, capstone, or research involving sensitive data or unpublished findings.
            </span>
            <span className="type-chooser-req-title">Requirements:</span>
            <ul className="type-chooser-req-list" aria-label="Research NDA requirements">
              <li>Letter of Intent<LoiTooltip content={["Research Title", "Introduction / Background of the Research", "Research
              <li>Questionnaire with Data Privacy Consent Form</li>
              <li>School ID</li>
              <li>Registration Form</li>
            </ul>
            <span className="type-chooser-card-subtext" style={{ marginTop: 8, fontStyle: "italic" }}>
              Click to continue to the form →
            </span>
          </button>
        </div>
      </div>
    );
  }
```

55. client/src/pages/student/StudentNDARequest.jsx (284 lines)

```jsx
import { useMemo, useRef, useState } from "react";
import { useNavigate } from "react-router-dom";
import { AnimatePresence, motion } from "framer-motion";
import { FIELDS_FILE_SLOTS_CONFIG } from "../../config/fieldsFileSlotsConfig";
import { createRequest } from "../../services/requestService";
import { uploadRequirements, uploadSignatureImage, getDateRequestFolder } from "../../services/firebaseStorageService";
import SignaturePad from "../../components/SignaturePad";
import { notify } from "../../utils/inPageFeedback";
import "../../components/RequestForm.css";
import "../../components/FloatingLabel.css";

export default function StudentNDARequest({ ndaType, proxyMode = false, fallbackPath = "/student" }) {
  const navigate = useNavigate();
  const cfg = useMemo(() => FIELDS_FILE_SLOTS_CONFIG.nda[ndaType], [ndaType]);

  const [formData, setFormData] = useState(() => ({}));
  const [proxyRequestee, setProxyRequestee] = useState({
    firstName: "",
    middleInitial: "",
    lastName: "",
    email: "",
  });
  const [files, setFiles] = useState(() => Array(cfg.fileSlots.length).fill(null));
  const [accepted, setAccepted] = useState(false);
  const [submitting, setSubmitting] = useState(false);
  const [showConfirm, setShowConfirm] = useState(false);
  const sigPadRef = useRef(null);

  const onChangeField = (name, value) =>
    setFormData((p) => ({ ...p, [name]: value }));

  const onChangeProxy = (name, value) =>
    setProxyRequestee((prev) => ({ ...prev, [name]: value }));

  const handleSubmit = async (e) => {
    e.preventDefault();

    for (const f of cfg.fields) {
      if (f.required && !String(formData[f.name] || "").trim()) {
        notify(`${f.label} is required`, { type: "warning" });
        return;
      }
    }

    if (proxyMode) {
      const hasName = String(proxyRequestee.firstName || "").trim() && String(proxyRequestee.lastName || "").trim();
      if (!hasName) {
        notify("Requestor First Name and Last Name are required", { type: "warning" });
        return;
      }
      const requiredNonNameFields = [
        ["email", "Requestor Email"],
      ];
      for (const [key, label] of requiredNonNameFields) {
        if (!String(proxyRequestee[key] || "").trim()) {
          notify(`${label} is required`, { type: "warning" });
          return;
        }
      }
    }

    for (let i = 0; i < cfg.fileSlots.length; i++) {
      if (cfg.fileSlots[i].required && !files[i]) {
        notify(`${cfg.fileSlots[i].label} is required`, { type: "warning" });
        return;
      }
    }

    const selectedFiles = files.filter(Boolean);
    if (selectedFiles.length === 0) {
      notify("Please upload at least 1 file.", { type: "warning" });
      return;
    }
```

```
74
75      if (!proxyMode) {
76        if (!sigPadRef.current || sigPadRef.current.isEmpty()) {
77          notify("Please draw your e-signature before submitting.", { type: "warning" });
78          return;
79        }
80        if (!accepted) {
81          notify("Please confirm the declaration before submitting.", { type: "warning" });
82          return;
83        }
84      }
85
86      setShowConfirm(true);
87    };
88
89    const handleConfirmSubmit = async () => {
90      setShowConfirm(false);
91      setSubmitting(true);
92      try {
93        const selectedFiles = files.filter(Boolean);
94        const user = JSON.parse(localStorage.getItem("user") || "null");
95        const proxyFullName = proxyMode
96          ? `${proxyRequestee.firstName}${proxyRequestee.middleInitial?.trim() ? " " + proxyRequestee.middleInitial.trim().char
97          : "";
98        const requestSubjectName = proxyMode
99          ? proxyFullName || user?.name || "Proxy Requestor"
100         : user?.name || "Unknown Student";
101       const requestFolder = getDateRequestFolder();
102
103       const uploaded = await uploadRequirements(selectedFiles, "nda", requestSubjectName, requestFolder);
104
105       let uploadIndex = 0;
106       const predocs = files
107         .map((f, i) => {
108           if (!f) return null;
109           const meta = uploaded[uploadIndex++];
110           return { ...meta, requirementLabel: cfg.fileSlots[i]?.label || `File ${i + 1}` };
111         })
112         .filter(Boolean);
113
114       let studentSigUrl = "";
115       let studentSigPath = "";
116       if (!proxyMode && !sigPadRef.current.isEmpty()) {
117         const sigDataUrl = sigPadRef.current.getDataUrl();
118         const sigResult = await uploadSignatureImage(sigDataUrl, "nda", requestSubjectName, requestFolder, "student_sig.png")
119         studentSigUrl = sigResult.url;
120         studentSigPath = sigResult.path;
121       }
122
123       await createRequest({
124         type: "nda",
125         formData: { ...formData, ndaType, ndaTypeLabel: cfg.label },
126         predocs,
127         studentSigUrl,
128         studentSigPath,
129         ...(proxyMode ? { proxyRequestee: { ...proxyRequestee, fullName: proxyFullName } } : {}),
130       });
131
132       notify(proxyMode ? "Proxy NDA request submitted." : "NDA request submitted!", { type: "success", title: "Submitted" });
133       navigate(fallbackPath);
134     } catch (err) {
135       notify(err.response?.data?.message || "Failed to submit request", { type: "error" });
136     } finally {
137       setSubmitting(false);
138     }
139   };
140
141   return (
142     <div className="request-form-page">
143       <form onSubmit={handleSubmit} className="request-form-card">
144         <div className="request-form-header">
145           <button
146             type="button"
147             className="request-form-back"
148             onClick={() => { if (window.history.length > 1) navigate(-1); else navigate("/student"); }}
149           >
```

```
            ‹ Back
          </button>
          <h2 className="request-form-title">Request Form</h2>
          <div />
        </div>

        {/* Section 1: Requestee Data */}
        <div className="request-section">
          <div className="request-section-title">Requestor Information</div>
          {cfg.fields.map((f) => (
            <div key={f.name} className="fl-wrap">
              {f.kind === "textarea" ? (
                <textarea
                  value={formData[f.name] || ""}
                  onChange={(e) => onChangeField(f.name, e.target.value)}
                  rows={f.rows || 4}
                  placeholder=" "
                  className="fl-input request-textarea"
                />
              ) : (
                <input
                  value={formData[f.name] || ""}
                  onChange={(e) => onChangeField(f.name, e.target.value)}
                  required={f.required}
                  placeholder=" "
                  className="fl-input request-input"
                />
              )}
              <label className={`fl-label${f.kind === "textarea" ? " fl-label--area" : ""}`}>{f.label}</label>
            </div>
          ))}

          {proxyMode && (
            <>
              <div className="request-section-title" style={{ marginTop: 10 }}>Requestor Details (F2F Walk-in)</div>
              <div style={{ display: "grid", gridTemplateColumns: "1fr 0.55fr 1fr", gap: 8 }}>
                <div className="fl-wrap">
                  <input className="fl-input request-input" placeholder=" " value={proxyRequestee.firstName} onChange={(e) =>
                  <label className="fl-label">First Name</label>
                </div>
                <div className="fl-wrap">
                  <input className="fl-input request-input" placeholder=" " value={proxyRequestee.middleInitial} onChange={(e
                  <label className="fl-label">Middle Name</label>
                </div>
                <div className="fl-wrap">
                  <input className="fl-input request-input" placeholder=" " value={proxyRequestee.lastName} onChange={(e) =>
                  <label className="fl-label">Last Name</label>
                </div>
              </div>
              <div className="fl-wrap">
                <input className="fl-input request-input" type="email" placeholder=" " value={proxyRequestee.email} onChange=
                <label className="fl-label">Requestor Email</label>
              </div>
            </>
          )}
        </div>

        {/* Section 2: Requirements */}
        <div className="request-section">
          <div className="request-section-title">Requirements (NDA)</div>
          {files.map((file, index) => (
            <div key={index} className="request-file-row">
              <div className="request-file-info">
                <div className="request-file-title">
                  {cfg.fileSlots[index]?.label || `Attach file ${index + 1}`}
                  {cfg.fileSlots[index]?.required ? " *" : ""}
                </div>
                <div className="request-file-subtitle">{file ? file.name : "No file selected"}</div>
              </div>
              <label className="request-file-action">
                {file ? "Change File" : "Upload File"}
                <input
                  type="file"
                  accept="application/pdf,image/*"
                  onChange={(e) => {
                    const selected = e.target.files?.[0] || null;
```

```
                    setFiles((prev) => { const copy = [...prev]; copy[index] = selected; return copy; });
                  }}
                />
              </label>
            </div>
          ))}
        </div>

        {/* Section 3: E-Signature */}
        <div className="request-section">
          {proxyMode ? (
            <div className="info-banner info-banner--info" style={{ marginBottom: 0 }}>
              <strong>F2F Walk-in (Proxy Request)</strong>
              <p>Digital signatures are bypassed for face-to-face walk-ins. The document will be printed for a physical wet s
            </div>
          ) : (
            <>
              <div className="request-section-title">Your E-Signature *</div>
              <p className="request-sig-hint">Draw your signature below. It will be embedded in the NDA document.</p>
              <SignaturePad ref={sigPadRef} height={160} />
            </>
          )}
        </div>

        {/* Confirmation clause */}
        {!proxyMode && (
          <label className="request-accept">
            <input type="checkbox" checked={accepted} onChange={(e) => setAccepted(e.target.checked)} />
            <span>
              I confirm that all information and attachments I have provided are accurate and complete,
              and I consent to the processing of my personal data by the Data Protection Office for the purpose of this reque
            </span>
          </label>
        )}

        <div className="request-form-actions">
          <button type="submit" className="request-form-submit" disabled={submitting}>
            {submitting ? "Submitting..." : "Submit"}
          </button>
        </div>
      </form>

      <AnimatePresence>
        {showConfirm && (
          <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
            <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12,
              <h3 className="modal-title">Submit NDA Request</h3>
              <p className="modal-text">Please confirm that all information and attachments are accurate before submitting.</
              <div className="modal-actions">
                <button className="ui-btn ui-btn--secondary" onClick={() => setShowConfirm(false)} disabled={submitting}>Canc
                <button className="ui-btn ui-btn--primary" onClick={handleConfirmSubmit} disabled={submitting}>{submitting ?
              </div>
            </motion.div>
          </motion.div>
        )}
      </AnimatePresence>
    </div>
  );
}
```

56. client/src/pages/student/StudentRequestReview.jsx (291 lines)

```
import { useEffect, useMemo, useState } from "react";
import { useNavigate, useParams } from "react-router-dom";
import { AlertTriangle, Copy, Paperclip } from "lucide-react";
import RequestStepper from "../../components/RequestStepper";
import { FIELDS_FILE_SLOTS_CONFIG } from "../../config/fieldsFileSlotsConfig";
import { getRequestById } from "../../services/requestService";
import { notify } from "../../utils/inPageFeedback";
import "../../components/RequestForm.css";

const statusPillClass = (s) => {
  if (["nda_approved", "agr_approved"].includes(s)) return "status-pill status-pill--green";
  if (["nda_pending", "agr_pending_1", "agr_pending_2"].includes(s)) return "status-pill status-pill--yellow";
  if (s === "stud_revision_requested") return "status-pill status-pill--orange";
  if (s === "agr_rep_revision_requested") return "status-pill status-pill--violet";
  if (s === "agr_awaiting_rep_signature") return "status-pill status-pill--blue";
  if (s === "agr_declined") return "status-pill status-pill--red";
  return "status-pill";
};

const prettyStatus = (s) => {
  const map = {
    nda_pending:                "Reviewal",
    nda_approved:               "Approved",
    stud_revision_requested:    "Student Revisions",
    agr_pending_1:              "Initial Reviewal",
    agr_awaiting_rep_signature: "Recipient Reviewal",
    agr_pending_2:              "Final Reviewal",
    agr_approved:               "Approved",
    agr_declined:               "Recipient Declined",
    agr_rep_revision_requested: "Recipient Revisions",
  };
  return map[s] || (s ? s.charAt(0).toUpperCase() + s.slice(1).replace(/_/g, " ") : "—");
};

export default function StudentRequestReview() {
  const { id } = useParams();
  const navigate = useNavigate();

  const [reqData, setReqData] = useState(null);

  const load = async () => {
    try {
      const r = await getRequestById(id);

      // Redirect to resubmit page if revision is requested from student
      if (r.status === "stud_revision_requested") {
        navigate(`/student/resubmit/${id}`, { replace: true });
        return;
      }

      setReqData(r);
    } catch (err) {
      notify(err.response?.data?.message || "Failed to load request", { type: "error" });
      navigate("/student");
    }
  };

  useEffect(() => {
    load();
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [id, navigate]);

  const cfg = useMemo(() => {
    if (!reqData) return null;
    if (reqData.type === "agreement") return FIELDS_FILE_SLOTS_CONFIG.agreement;
    if (reqData.type === "nda") return FIELDS_FILE_SLOTS_CONFIG.nda[reqData.formData?.ndaType];
    return null;
  }, [reqData]);

  if (!reqData) return null;

  const title =
    reqData.type === "agreement"
```

```
      ? "Agreement Request"
      : `NDA Request${reqData.formData?.ndaTypeLabel ? ` — ${reqData.formData.ndaTypeLabel}` : ""}`;

  const isApproved = reqData.status === "nda_approved" || reqData.status === "agr_approved";
  const isAgreement = reqData.type === "agreement";

  return (
    <div className="review-page">
      <div className="review-card">
        <div className="request-form-header">
          <button
            type="button"
            className="request-form-back"
            onClick={() => {
              if (window.history.length > 1) navigate(-1);
              else navigate("/student");
            }}
          >
            ‹ Back
          </button>
          <h2 className="request-form-title">{title}</h2>
          <div />
        </div>

        <div className="review-header">
          <div style={{ display: "flex", alignItems: "center", justifyContent: "space-between", gap: 12, flexWrap: "wrap", wi
            <span className={statusPillClass(reqData.status)}>{prettyStatus(reqData.status)}</span>
            <span className="review-meta-row" style={{ marginLeft: "auto" }}>
              <b>Request Date:</b> {new Date(reqData.createdAt).toLocaleDateString("en-US", { year: "numeric", month: "short"
            </span>
          </div>
        </div>

        <div className="review-section review-section--stepper">
          <RequestStepper status={reqData.status} type={reqData.type} />
        </div>


        {/* Form Data / Representative */}
        <div className="review-section">
          <h4 className="review-section-title">{isAgreement ? "Representative" : "Form Data"}</h4>
          {cfg?.fields?.length ? (
            isAgreement ? (
              <>
                {/* Full Name (bundled) */}
                <div className="review-field">
                  <span className="review-field-label">Full Name</span>
                  <div className="review-info-box">
                    {(() => {
                      const fd = reqData.formData || {};
                      const mid = String(fd.repMiddleInitial || "").trim();
                      const midInitial = mid ? `${mid[0].toUpperCase()}.` : "";
                      const full = [fd.repFirstName, midInitial, fd.repLastName].filter(Boolean).join(" ").replace(/\s+/g, "
                      return full || <span className="review-info-box--muted">—</span>;
                    })()}
                  </div>
                </div>
                {/* Remaining fields */}
                {cfg.fields
                  .filter((f) => !["repFirstName", "repMiddleInitial", "repLastName"].includes(f.name))
                  .map((f) => (
                    <div key={f.name} className="review-field">
                      <span className="review-field-label">{f.label}</span>
                      <div className="review-info-box">
                        {String(reqData.formData?.[f.name] ?? "") || (
                          <span className="review-info-box--muted">—</span>
                        )}
                      </div>
                    </div>
                  ))}
              </>
            ) : (
              cfg.fields.map((f) => (
                <div key={f.name} className="review-field">
                  <span className="review-field-label">{f.label}</span>
                  <div className="review-info-box">
```

```
                    {String(reqData.formData?.[f.name] ?? "") || (
                      <span className="review-info-box--muted">—</span>
                    )}
                  </div>
                </div>
              ))
            )
          ) : (
            <div className="review-info-box">
              <span className="review-info-box--muted">No config fields found for this request.</span>
            </div>
          )}
        </div>

        {/* Attachments */}
        <div className="review-section">
          <h4 className="review-section-title">My Attachments</h4>
          {(() => {
            const initialDocs = reqData.predocs?.filter((f) => !f.isResubmission) ?? [];
            return initialDocs.length ? (
              <div style={{ display: "flex", flexDirection: "column", gap: "8px" }}>
                {initialDocs.map((f, idx) => (
                  <a key={idx} href={f.url} target="_blank" rel="noreferrer" className="review-file-link">
                    <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> {f.requirementLabel || f.origName || `File $
                  </a>
                ))}
              </div>
            ) : (
              <div className="review-info-box">
                <span className="review-info-box--muted">No files.</span>
              </div>
            );
          })()}
        </div>

        {(() => {
          const rounds = {};
          (reqData.predocs || []).filter((f) => f.isResubmission).forEach((f) => {
            const r = f.resubmissionRound || 1;
            if (!rounds[r]) rounds[r] = [];
            rounds[r].push(f);
          });
          return Object.keys(rounds).sort((a, b) => a - b).map((round) => (
            <div key={round} className="review-section">
              <h4 className="review-section-title">
                Resubmitted Attachments{Object.keys(rounds).length > 1 ? ` (Round ${round})` : ""}
              </h4>
              <div style={{ display: "flex", flexDirection: "column", gap: "8px" }}>
                {rounds[round].map((f, idx) => (
                  <a key={idx} href={f.url} target="_blank" rel="noreferrer" className="review-file-link">
                    <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> {f.requirementLabel || f.origName || `File $
                  </a>
                ))}
              </div>
            </div>
          ));
        })()}

        {/* Agreement-specific status banners */}
        {isAgreement && reqData.status === "agr_awaiting_rep_signature" && (
          <div className="info-banner info-banner--info">
            <strong>Awaiting Representative</strong>
            <p>
              The admin has approved your request and a signing link has been generated for
              the representative.
            </p>
          </div>
        )}

        {/* Signing link visibility (only while awaiting rep action) */}
        {isAgreement && reqData.signingToken && (reqData.status === "agr_awaiting_rep_signature" || reqData.status === "agr_r
          <div className="review-section">
            <h4 className="review-section-title">Representative Signing Link</h4>
            <div className="review-info-box" style={{ fontSize: 13 }}>
              <div className="signing-link-row">
                <span className="signing-link-code">Representative signing link ready — click to copy</span>
```

```
                <button
                  className="signing-link-copy-btn"
                  onClick={() => {
                    navigator.clipboard.writeText(`${window.location.origin}/sign/${reqData.signingToken}`);
                    notify("Copied!", { type: "success" });
                  }}
                  title="Copy link"
                >
                  <Copy size={14} strokeWidth={2} />
                </button>
              </div>
              {reqData.signingTokenExpiresAt && (
                <div style={{ marginTop: 6, fontSize: 12, color: new Date(reqData.signingTokenExpiresAt) < new Date() ? "#dc2
                  {new Date(reqData.signingTokenExpiresAt) < new Date()
                    ? <span style={{ display: "inline-flex", alignItems: "center", gap: 6 }}><AlertTriangle size={14} strokeW
                    : `Expires: ${new Date(reqData.signingTokenExpiresAt).toLocaleDateString("en-PH", { year: "numeric", mont
                </div>
              )}
            </div>
          </div>
        )}

        {isAgreement && reqData.status === "agr_pending_2" && (
          <div className="info-banner info-banner--success">
            <strong>Representative Signed</strong>
            <p>
              The representative has submitted their signature. Awaiting final admin approval.
            </p>
          </div>
        )}

        {isAgreement && reqData.status === "agr_declined" && (
          <div className="info-banner info-banner--danger">
            <strong>Recipient Declined</strong>
            <p>The recipient declined to sign this agreement.</p>
          </div>
        )}

        {isAgreement && reqData.status === "agr_rep_revision_requested" && (
          <div className="info-banner info-banner--warning">
            <strong>Recipient Revisions Requested</strong>
            <p>
              The admin has requested the recipient to revise and resubmit their information.
            </p>
          </div>
        )}

        {/* Approved document */}
        {isApproved && (
          <div className="review-section">
            <h4 className="review-section-title">Approved Document</h4>
            {reqData.postdocs?.url ? (
              <a href={reqData.postdocs.url} target="_blank" rel="noreferrer" className="review-file-link">
                <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> Approved Document
              </a>
            ) : (
              <div className="review-info-box">
                <span className="review-info-box--muted">No approved document uploaded.</span>
              </div>
            )}
          </div>
        )}
      </div>
    </div>
  );
}
```

57. client/src/pages/student/StudentResubmitRequest.jsx (340 lines)

```
import { useEffect, useMemo, useRef, useState } from "react";
import { useNavigate, useParams } from "react-router-dom";
import { AnimatePresence, motion } from "framer-motion";
import { FIELDS_FILE_SLOTS_CONFIG } from "../../config/fieldsFileSlotsConfig";
import { getRequestById, resubmitRequest } from "../../services/requestService";
import {
  uploadRequirements,
  uploadSignatureImage,
  getDateRequestFolder,
} from "../../services/firebaseStorageService";
import SignaturePad from "../../components/SignaturePad";
import { notify } from "../../utils/inPageFeedback";
import "../../components/RequestForm.css";
import "../../components/FloatingLabel.css";

const getInitialFolderFromPath = (path = "") => {
  const parts = String(path).split("/");
  return parts[3] || "";
};

export default function StudentResubmitRequest() {
  const { id } = useParams();
  const navigate = useNavigate();

  const [reqData, setReqData] = useState(null);
  const [formData, setFormData] = useState({});
  const [files, setFiles] = useState([]);
  const [accepted, setAccepted] = useState(false);
  const [submitting, setSubmitting] = useState(false);
  const [showConfirm, setShowConfirm] = useState(false);
  const sigPadRef = useRef(null);

  useEffect(() => {
    const load = async () => {
      try {
        const r = await getRequestById(id);
        setReqData(r);
        setFormData(r.formData || {});
      } catch (err) {
        notify(err.response?.data?.message || "Failed to load request", { type: "error" });
        navigate("/student");
      }
    };
    load();
  }, [id, navigate]);

  const cfg = useMemo(() => {
    if (!reqData) return null;
    if (reqData.type === "agreement") return FIELDS_FILE_SLOTS_CONFIG.agreement;
    if (reqData.type === "nda") return FIELDS_FILE_SLOTS_CONFIG.nda[reqData.formData?.ndaType];
    return null;
  }, [reqData]);

  useEffect(() => {
    if (!cfg) return;
    setFiles(Array(cfg.fileSlots.length).fill(null));
  }, [cfg]);

  const onChangeField = (name, value) =>
    setFormData((p) => ({ ...p, [name]: value }));

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!reqData || !cfg) return;

    if (!["stud_revision_requested"].includes(reqData.status)) {
      notify("Only revision requested requests can be resubmitted.", { type: "warning" });
      return;
    }

    const selectedFiles = files.filter(Boolean);
    if (selectedFiles.length === 0) {
      notify("Please upload at least 1 revised file.", { type: "warning" });
```

```
      return;
    }

    if (!sigPadRef.current || sigPadRef.current.isEmpty()) {
      notify("Please draw your e-signature before resubmitting.", { type: "warning" });
      return;
    }

    if (!accepted) {
      notify("Please confirm the declaration before submitting.", { type: "warning" });
      return;
    }

    const basePath = reqData.predocs?.[0]?.path || "";
    const initialFolder = getInitialFolderFromPath(basePath);
    if (!initialFolder) {
      notify("Missing original request folder. (No existing file path found)", { type: "error" });
      return;
    }

    setShowConfirm(true);
  };

  const handleConfirmSubmit = async () => {
    setShowConfirm(false);
    setSubmitting(true);
    try {
      const selectedFiles = files.filter(Boolean);
      const user = JSON.parse(localStorage.getItem("user") || "null");
      const studentName = user?.name || "Unknown Student";
      const initialFolder = getInitialFolderFromPath(reqData.predocs?.[0]?.path || "");
      const resubFolder = `resub${getDateRequestFolder()}`;
      const resubPath = `${initialFolder}/${resubFolder}`;

      const uploaded = await uploadRequirements(selectedFiles, reqData.type, studentName, resubPath);

      let uploadIndex = 0;
      const predocs = files
        .map((f, i) => {
          if (!f) return null;
          const meta = uploaded[uploadIndex++];
          return { ...meta, requirementLabel: cfg.fileSlots[i]?.label || `File ${i + 1}` };
        })
        .filter(Boolean);

      const payload = { formData, predocs };

      if (reqData.type === "agreement") {
        const sigDataUrl = sigPadRef.current.getDataUrl();
        const { url: authorizerSigUrl, path: authorizerSigPath } = await uploadSignatureImage(
          sigDataUrl, "agreement", studentName, resubPath, "authorizer_sig.png"
        );
        payload.authorizerSigUrl = authorizerSigUrl;
        payload.authorizerSigPath = authorizerSigPath;
      }

      if (reqData.type === "nda") {
        const sigDataUrl = sigPadRef.current.getDataUrl();
        const { url: studentSigUrl, path: studentSigPath } = await uploadSignatureImage(
          sigDataUrl, "nda", studentName, resubPath, "student_sig.png"
        );
        payload.studentSigUrl = studentSigUrl;
        payload.studentSigPath = studentSigPath;
      }

      await resubmitRequest(id, payload);
      notify("Your request has been resubmitted successfully.", { type: "success", title: "Resubmitted" });
      navigate("/student");
    } catch (err) {
      notify(err.response?.data?.message || "Failed to resubmit. Please try again.", { type: "error" });
    } finally {
      setSubmitting(false);
    }
  };

  if (!reqData) return null;
```

```
150
151   if (!["stud_revision_requested"].includes(reqData.status)) {
152     return (
153       <div className="review-page">
154         <div className="review-card">
155           <button
156             type="button"
157             className="review-back-btn"
158             onClick={() => { if (window.history.length > 1) navigate(-1); else navigate("/student"); }}
159           >
160             ← Back
161           </button>
162           <div className="review-info-box">
163             <span className="review-info-box--muted">This request is not marked as revision requested.</span>
164           </div>
165         </div>
166       </div>
167     );
168   }
169
170   const dataTitle = reqData.type === "agreement" ? "Revised Representative Data" : "Revised Requestor Data";
171
172   return (
173     <div className="request-form-page">
174       <form onSubmit={handleSubmit} className="request-form-card">
175         <div className="request-form-header">
176           <button
177             type="button"
178             className="request-form-back"
179             onClick={() => { if (window.history.length > 1) navigate(-1); else navigate("/student"); }}
180           >
181             ‹ Back
182           </button>
183           <h2 className="request-form-title">Resubmit Request Form</h2>
184           <div />
185         </div>
186
187         {/* Section 1: Remarks */}
188         <div className="request-section">
189           <div className="request-section-title">Remarks from Admin</div>
190           <div style={{
191             padding: "12px 14px",
192             background: "var(--s-warning-bg)",
193             border: "1px solid var(--s-warning-dot)",
194             borderRadius: "var(--radius-md)",
195             fontSize: "13px",
196             color: "var(--s-warning-text)",
197           }}>
198             {reqData.remarks || <span style={{ opacity: 0.7 }}>No remarks provided.</span>}
199           </div>
200         </div>
201
202         {/* Section 2: Revised Data */}
203         <div className="request-section">
204           <div className="request-section-title">{dataTitle}</div>
205           {(() => {
206             const nameKeys = ["repFirstName", "repMiddleInitial", "repLastName"];
207             const nameFields = cfg.fields.filter((f) => nameKeys.includes(f.name));
208             const otherFields = cfg.fields.filter((f) => !nameKeys.includes(f.name));
209             return (
210               <>
211                 {nameFields.length > 0 && (
212                   <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr 1fr", gap: 8 }}>
213                     {nameFields.map((f) => (
214                       <div key={f.name} className="fl-wrap">
215                         <input
216                           value={formData[f.name] || ""}
217                           onChange={(e) => onChangeField(f.name, e.target.value)}
218                           placeholder=" "
219                           className="fl-input request-input"
220                         />
221                         <label className="fl-label">{f.label}</label>
222                       </div>
223                     ))}
224                   </div>
225                 )}
```

```
                {otherFields.map((f) => (
                  <div key={f.name} className="fl-wrap">
                    {f.kind === "textarea" ? (
                      <textarea
                        value={formData[f.name] || ""}
                        onChange={(e) => onChangeField(f.name, e.target.value)}
                        rows={f.rows || 4}
                        placeholder=" "
                        className="fl-input request-textarea"
                      />
                    ) : (
                      <input
                        value={formData[f.name] || ""}
                        onChange={(e) => onChangeField(f.name, e.target.value)}
                        placeholder=" "
                        className="fl-input request-input"
                      />
                    )}
                    <label className={`fl-label${f.kind === "textarea" ? " fl-label--area" : ""}`}>{f.label}</label>
                  </div>
                ))}
              </>
            );
          })()}
        </div>

        {/* Section 3: Previous Attachments */}
        <div className="request-section">
          <div className="request-section-title">Previous Attachments</div>
          {reqData.predocs?.length ? (
            reqData.predocs.map((f, idx) => (
              <div key={idx} className="request-file-row">
                <div className="request-file-info">
                  <div className="request-file-title">{f.requirementLabel || f.origName || `File ${idx + 1}`}</div>
                </div>
                <a href={f.url} target="_blank" rel="noreferrer" className="request-file-action" style={{ textDecoration: "no
                  View
                </a>
              </div>
            ))
          ) : (
            <div className="request-file-row">
              <div className="request-file-info">
                <div className="request-file-subtitle">No files submitted.</div>
              </div>
            </div>
          )}
        </div>

        {/* Section 4: Revised Requirements */}
        <div className="request-section">
          <div className="request-section-title">Revised Requirements</div>
          {files.map((file, index) => (
            <div key={index} className="request-file-row">
              <div className="request-file-info">
                <div className="request-file-title">
                  {cfg.fileSlots[index]?.label || `Attach file ${index + 1}`}
                </div>
                <div className="request-file-subtitle">{file ? file.name : "No file selected"}</div>
              </div>
              <label className="request-file-action">
                {file ? "Change File" : "Upload File"}
                <input
                  type="file"
                  accept="application/pdf,image/*"
                  onChange={(e) => {
                    const selected = e.target.files?.[0] || null;
                    setFiles((prev) => { const copy = [...prev]; copy[index] = selected; return copy; });
                  }}
                />
              </label>
            </div>
          ))}
        </div>

        {/* Section 5: E-Signature */}
```

```
302           <div className="request-section">
303             <div className="request-section-title">Your E-Signature *</div>
304             <p className="request-sig-hint">Please draw your signature again for this resubmission.</p>
305             <SignaturePad ref={sigPadRef} height={160} />
306           </div>
307
308           {/* Confirmation clause */}
309           <label className="request-accept">
310             <input type="checkbox" checked={accepted} onChange={(e) => setAccepted(e.target.checked)} />
311             <span>
312               I confirm that all revised information and attachments I have provided are accurate and complete,
313               and I consent to the processing of my personal data by the Data Protection Office for the purpose of this request
314             </span>
315           </label>
316
317           <div className="request-form-actions">
318             <button type="submit" className="request-form-submit" disabled={submitting}>
319               {submitting ? "Submitting..." : "Resubmit"}
320             </button>
321           </div>
322         </form>
323
324         <AnimatePresence>
325           {showConfirm && (
326             <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
327               <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12,
328                 <h3 className="modal-title">Resubmit Request</h3>
329                 <p className="modal-text">Please confirm that all revised information and attachments are accurate before resub
330                 <div className="modal-actions">
331                   <button className="ui-btn ui-btn--secondary" onClick={() => setShowConfirm(false)} disabled={submitting}>Canc
332                   <button className="ui-btn ui-btn--primary" onClick={handleConfirmSubmit} disabled={submitting}>{submitting ?
333                 </div>
334               </motion.div>
335             </motion.div>
336           )}
337         </AnimatePresence>
338       </div>
339     );
340   }
```

58. client/src/pages/admin/AdminDashboard.jsx (471 lines)

```
import { useContext, useEffect, useMemo, useState } from "react";
import { Link } from "react-router-dom";
import { motion } from "framer-motion";
import {
  PieChart, Pie, Cell, Tooltip, ResponsiveContainer,
  BarChart, Bar, XAxis, YAxis, CartesianGrid, Legend,
  LineChart, Line,
} from "recharts";
import {
  FileText, Check, X, Archive,
  ArrowRight, RefreshCw, MoreHorizontal,
} from "lucide-react";
import FilterSelect from "../../components/FilterSelect";
import { getAllRequests } from "../../services/requestService";
import { getRecentAuditLogs } from "../../services/auditService";
import { AuthContext } from "../../context/AuthContext";
import {
  APPROVED_STATUSES,
  PENDING_STATUSES,
  REVISION_STATUSES,
  CHART_LABELS,
  CHART_COLORS,
  normalizeStatusForChart,
} from "../../utils/requestStatusCharts";

const DONUT_COLORS = ["#059669", "#ea580c", "#7c3aed", "#ca8a04", "#2563eb", "#dc2626", "#64748b", "#3b82f6"];
const MONTH_NAMES = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"];

const ACTION_LABEL = {
  login: "User logged in",
  login_failed: "Failed login attempt",
  login_otp_sent: "Login OTP sent",
  account_created: "New account created",
  account_activated: "Account activated",
  email_verified: "Email verified",
  request_created: "Request submitted",
  request_approved: "Request approved",
  request_revision_required: "Revision requested",
  request_resubmitted: "Request resubmitted",
  rep_signature_submitted: "Representative signed",
  rep_signature_declined: "Representative declined",
  signing_link_generated: "Signing link generated",
  password_changed: "Password changed",
  password_reset_requested: "Password reset requested",
  user_deactivated: "User deactivated",
  user_activated: "User activated",
};

const formatAction = (action) =>
  ACTION_LABEL[action] ||
  action.replace(/_/g, " ").replace(/\b\w/g, (c) => c.toUpperCase());

const cardVariants = {
  hidden: { opacity: 0, y: 20 },
  visible: (i) => ({ opacity: 1, y: 0, transition: { delay: i * 0.07, duration: 0.35 } }),
};

export default function AdminDashboard() {
  const { user } = useContext(AuthContext);
  const [requests, setRequests] = useState([]);
  const [auditLogs, setAuditLogs] = useState([]);
  const [error, setError] = useState("");
  const [refreshing, setRefreshing] = useState(false);
  const [isCompact, setIsCompact] = useState(() => {
    if (typeof window === "undefined") return false;
    return window.innerWidth < 768;
  });

  // Date filter for charts
  const [dateMode, setDateMode] = useState("preset");
  const [preset, setPreset] = useState("all");
  const [singleDate, setSingleDate] = useState("");
  const [startDate, setStartDate] = useState("");
```

```
  const [endDate, setEndDate] = useState("");

  useEffect(() => {
    const onResize = () => setIsCompact(window.innerWidth < 768);
    window.addEventListener("resize", onResize);
    return () => window.removeEventListener("resize", onResize);
  }, []);

  const load = async ({ withToast = false } = {}) => {
      if (withToast) setRefreshing(true);
      const errors = [];

      // Fetch independently so one failure doesn't block the other
      try {
        const reqData = await getAllRequests();
        setRequests(reqData);
      } catch (err) {
        const msg = err.response?.data?.message || err.message || "Failed to load requests";
        const status = err.response?.status;
        errors.push(`Requests: ${msg}${status ? ` (HTTP ${status})` : ""}`);
        console.error("Dashboard requests error:", err);
      }

      try {
        if (user?.role === "admin") {
          const auditData = await getRecentAuditLogs(6);
          setAuditLogs(auditData);
        } else {
          setAuditLogs([]);
        }
      } catch (err) {
        const msg = err.response?.data?.message || err.message || "Failed to load audit logs";
        const status = err.response?.status;
        errors.push(`Audit logs: ${msg}${status ? ` (HTTP ${status})` : ""}`);
        console.error("Dashboard audit error:", err);
      }

      if (errors.length) setError(errors.join(" | "));
      setRefreshing(false);
    };

  useEffect(() => {
    load();
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, []);

  // Filtered requests for charts based on date filter
  const chartFilteredRequests = useMemo(() => {
    const now = new Date();
    if (dateMode === "preset") {
      if (preset === "all") return requests;
      if (preset === "today") {
        const s = new Date(now.getFullYear(), now.getMonth(), now.getDate());
        const e = new Date(s.getTime() + 86400000 - 1);
        return requests.filter((r) => { const d = new Date(r.createdAt); return d >= s && d <= e; });
      }
      if (preset === "thisWeek") {
        const day = now.getDay();
        const monday = new Date(now);
        monday.setDate(now.getDate() - ((day + 6) % 7));
        monday.setHours(0, 0, 0, 0);
        return requests.filter((r) => new Date(r.createdAt) >= monday);
      }
      if (preset === "thisMonth") {
        const s = new Date(now.getFullYear(), now.getMonth(), 1);
        return requests.filter((r) => new Date(r.createdAt) >= s);
      }
      if (preset === "thisYear") {
        const s = new Date(now.getFullYear(), 0, 1);
        return requests.filter((r) => new Date(r.createdAt) >= s);
      }
    }
    if (dateMode === "single" && singleDate) {
      const s = new Date(singleDate);
      const e = new Date(singleDate + "T23:59:59");
      return requests.filter((r) => { const d = new Date(r.createdAt); return d >= s && d <= e; });
```

```
    }
    if (dateMode === "range") {
      return requests.filter((r) => {
        const d = new Date(r.createdAt);
        if (startDate && d < new Date(startDate)) return false;
        if (endDate && d > new Date(endDate + "T23:59:59")) return false;
        return true;
      });
    }
    return requests;
  }, [requests, dateMode, preset, singleDate, startDate, endDate]);

  // Derived chart data (same approach as Reports page)
  const statusData = useMemo(() => {
    const counts = {};
    chartFilteredRequests.forEach((r) => {
      if (!r.isArchived) {
        const key = normalizeStatusForChart(r.status);
        counts[key] = (counts[key] || 0) + 1;
      }
    });
    return Object.entries(counts).map(([id, value]) => ({
      id,
      name: CHART_LABELS[id] || id,
      value,
    }));
  }, [chartFilteredRequests]);

  const typeData = useMemo(() => [
    { name: "NDA", count: chartFilteredRequests.filter((r) => r.type === "nda").length },
    { name: "Agreement", count: chartFilteredRequests.filter((r) => r.type === "agreement").length },
  ], [chartFilteredRequests]);

  // Monthly trend (last 6 months)
  const trendData = useMemo(() => {
    const now = new Date();
    return Array.from({ length: 6 }, (_, i) => {
      const d = new Date(now.getFullYear(), now.getMonth() - (5 - i), 1);
      const monthStart = new Date(d.getFullYear(), d.getMonth(), 1);
      const monthEnd = new Date(d.getFullYear(), d.getMonth() + 1, 0, 23, 59, 59);
      const inMonth = requests.filter((r) => {
        const c = new Date(r.createdAt);
        return c >= monthStart && c <= monthEnd;
      });
      return {
        month: MONTH_NAMES[d.getMonth()],
        total: inMonth.length,
        approved: inMonth.filter((r) => APPROVED_STATUSES.includes(r.status)).length,
        pending: inMonth.filter((r) => PENDING_STATUSES.includes(r.status)).length,
      };
    });
  }, [requests]);

  const totalActive = requests.filter((r) => !r.isArchived).length;
  const totalPending = requests.filter((r) => PENDING_STATUSES.includes(r.status)).length;
  const totalApproved = requests.filter((r) => APPROVED_STATUSES.includes(r.status)).length;
  const totalRevisions = requests.filter((r) => REVISION_STATUSES.includes(r.status)).length;
  const totalArchived = requests.filter((r) => r.isArchived).length;

  const summaryCards = [
    { icon: FileText, label: "Total Requests", value: totalActive, color: "#3b82f6" },
    { icon: MoreHorizontal, label: "Reviewal", value: totalPending, color: "#f59e0b" },
    { icon: Check, label: "Approved", value: totalApproved, color: "#10b981" },
    { icon: X, label: "Revisions", value: totalRevisions, color: "#ea580c" },
    { icon: Archive, label: "Archived", value: totalArchived, color: "#64748b" },
  ];

  return (
    <div className="dashboard-page">

      {/* ■■ Error Banner ■■ */}
      {error && <div className="admin-flash-banner admin-flash-banner--error">{error}</div>}
      {/* ■■ Summary Cards ■■ */}
      <div className="responsive-grid-5 dashboard-section-gap">
        {summaryCards.map((card, i) => (
          <motion.div
```

```
            key={card.label}
            custom={i}
            variants={cardVariants}
            initial="hidden"
            animate="visible"
            style={{
              background: "var(--surface)",
              border: "1px solid var(--border)",
              borderRadius: "var(--radius-lg)",
              padding: "20px 24px",
              boxShadow: "var(--shadow-sm)",
              display: "flex",
              alignItems: "center",
              gap: 16,
            }}
          >
            <div style={{
              width: 44, height: 44, borderRadius: "var(--radius-md)",
              background: `${card.color}18`, display: "flex", alignItems: "center", justifyContent: "center",
              flexShrink: 0,
            }}>
              <card.icon size={20} color={card.color} strokeWidth={2} />
            </div>
            <div>
              <div style={{ fontSize: 24, fontWeight: 700, color: "var(--text-primary)", lineHeight: 1.2 }}>
                {card.value}
              </div>
              <div style={{ fontSize: 12, color: "var(--text-muted)", marginTop: 2 }}>{card.label}</div>
            </div>
          </motion.div>
        ))}
      </div>

      {/* ■■ Filtered Charts Container ■■ */}
      <motion.div
        initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ delay: 0.28 }}
        className="dashboard-card dashboard-section-gap"
        style={{ padding: "16px 20px 20px" }}
      >
        {/* Toolbar */}
        <div className="req-toolbar" style={{ marginBottom: 16, boxShadow: "none", border: "none", padding: 0, background: "t
          <div className="req-toolbar__filters req-toolbar__filters--open" style={{ borderTop: "none", paddingTop: 0 }}>
            <div className="req-toolbar__filter-group">
              <label className="req-toolbar__filter-label">Date</label>
              <FilterSelect
                value={dateMode}
                onChange={setDateMode}
                options={[
                  { value: "preset", label: "Preset" },
                  { value: "single", label: "Specific Date" },
                  { value: "range", label: "Date Range" },
                ]}
                defaultValue="preset"
              />
            </div>

            {dateMode === "preset" && (
              <div className="req-toolbar__filter-group">
                <label className="req-toolbar__filter-label">Period</label>
                <FilterSelect
                  value={preset}
                  onChange={setPreset}
                  options={[
                    { value: "all", label: "All Time" },
                    { value: "today", label: "Today" },
                    { value: "thisWeek", label: "This Week" },
                    { value: "thisMonth", label: "This Month" },
                    { value: "thisYear", label: "This Year" },
                  ]}
                />
              </div>
            )}

            {dateMode === "single" && (
              <div className="req-toolbar__filter-group">
                <label className="req-toolbar__filter-label">Date</label>
```

```
                <input className={`req-toolbar__filter-select${singleDate !== "" ? " req-toolbar__filter-select--active" : ""
              </div>
            )}

            {dateMode === "range" && (
              <>
                <div className="req-toolbar__filter-group">
                  <label className="req-toolbar__filter-label">From</label>
                  <input className={`req-toolbar__filter-select${startDate !== "" ? " req-toolbar__filter-select--active" : "
                </div>
                <div className="req-toolbar__filter-group">
                  <label className="req-toolbar__filter-label">To</label>
                  <input className={`req-toolbar__filter-select${endDate !== "" ? " req-toolbar__filter-select--active" : ""}
                </div>
              </>
            )}

            <button
              type="button"
              className="req-toolbar__reset-btn"
              onClick={() => {
                setDateMode("preset");
                setPreset("all");
                setSingleDate("");
                setStartDate("");
                setEndDate("");
                load({ withToast: true });
              }}
              disabled={refreshing}
              title="Reset filters and refresh"
              style={{ alignSelf: "flex-start" }}
            >
              <RefreshCw size={14} className={refreshing ? "spin-anim" : ""} />
            </button>
          </div>
        </div>

        {/* Charts */}
        <div className="responsive-grid-2">
          {/* Donut Chart – Status Distribution */}
          <div className="admin-chart-card dashboard-chart-card" style={{ boxShadow: "none", border: "1px solid var(--border)
            <div className="dashboard-chart-title">Request Status Distribution</div>
            <div className="dashboard-chart-subtitle">Current active requests by status</div>
            {statusData.length > 0 ? (
              <div className="dashboard-chart-canvas">
                <ResponsiveContainer width="100%" height="100%">
                  <PieChart margin={{ top: 8, right: 12, bottom: isCompact ? 32 : 8, left: 12 }}>
                    <Pie
                      data={statusData}
                      cx={isCompact ? "50%" : "36%"}
                      cy={isCompact ? "45%" : "50%"}
                      innerRadius={isCompact ? 45 : 55}
                      outerRadius={isCompact ? 68 : 85}
                      paddingAngle={3}
                      dataKey="value"
                    >
                      {statusData.map((entry, index) => (
                        <Cell key={`cell-${index}`} fill={CHART_COLORS[entry.id] || DONUT_COLORS[index % DONUT_COLORS.length]
                      ))}
                    </Pie>
                    <Tooltip formatter={(value, name) => [value, name]} />
                    <Legend
                      iconType="circle"
                      iconSize={8}
                      layout={isCompact ? "horizontal" : "vertical"}
                      align={isCompact ? "center" : "right"}
                      verticalAlign={isCompact ? "bottom" : "middle"}
                      wrapperStyle={{ fontSize: 12, paddingLeft: isCompact ? 0 : 24 }}
                      formatter={(value) => {
                        const total = statusData.reduce((sum, d) => sum + d.value, 0);
                        const entry = statusData.find((d) => d.name === value);
                        const pct = entry && total > 0 ? ((entry.value / total) * 100).toFixed(1) : 0;
                        return `${value} (${pct}%)`;
                      }}
                    />
                  </PieChart>
```

```
                </ResponsiveContainer>
              </div>
            ) : (
              <div className="dashboard-chart-canvas dashboard-chart-empty">No data available</div>
            )}
          </div>

          {/* Bar Chart – Document Types */}
          <div className="admin-chart-card dashboard-chart-card" style={{ boxShadow: "none", border: "1px solid var(--border)
            <div className="dashboard-chart-title">Document Types</div>
            <div className="dashboard-chart-subtitle">NDA vs Agreement requests submitted</div>
            <div className="dashboard-chart-canvas">
              <ResponsiveContainer width="100%" height="100%">
                <BarChart data={typeData} barSize={40}>
                  <CartesianGrid strokeDasharray="3 3" stroke="var(--border)" />
                  <XAxis dataKey="name" tick={{ fontSize: 12, fill: "var(--text-secondary)" }} axisLine={false} tickLine={fal
                  <YAxis tick={{ fontSize: 12, fill: "var(--text-muted)" }} axisLine={false} tickLine={false} allowDecimals={
                  <Tooltip cursor={{ fill: "var(--primary-light)" }} />
                  <Bar dataKey="count" name="Requests" fill="var(--primary)" radius={[6, 6, 0, 0]} />
                </BarChart>
              </ResponsiveContainer>
            </div>
          </div>
        </div>
      </motion.div>

      {/* Monthly Trend Line Chart */}
      <motion.div
        initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ delay: 0.49 }}
        className="admin-chart-card dashboard-chart-card dashboard-section-gap"
      >
        <div className="dashboard-chart-title">Monthly Trend</div>
        <div className="dashboard-chart-subtitle">Request volume over the last 6 months</div>
        <div className="dashboard-chart-canvas">
        <ResponsiveContainer width="100%" height="100%">
          <LineChart data={trendData}>
            <CartesianGrid strokeDasharray="3 3" stroke="var(--border)" />
            <XAxis dataKey="month" tick={{ fontSize: 12, fill: "var(--text-secondary)" }} axisLine={false} tickLine={false} /
            <YAxis tick={{ fontSize: 12, fill: "var(--text-muted)" }} axisLine={false} tickLine={false} allowDecimals={false}
            <Tooltip />
            <Legend wrapperStyle={{ fontSize: 12 }} />
            <Line type="monotone" dataKey="total" stroke="#3b82f6" strokeWidth={2} dot={{ r: 3 }} name="Total" />
            <Line type="monotone" dataKey="approved" stroke="#10b981" strokeWidth={2} dot={{ r: 3 }} name="Approved" />
            <Line type="monotone" dataKey="pending" stroke="#f59e0b" strokeWidth={2} dot={{ r: 3 }} name="Pending" />
          </LineChart>
        </ResponsiveContainer>
        </div>
      </motion.div>

      {/* ■■ Recent Audit Logs (admin-only) ■■ */}
      {user?.role === "admin" ? (
        <motion.div
          initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ delay: 0.56 }}
          className="dashboard-audit-card"
        >
          <div className="dashboard-audit-header">
            <div className="dashboard-audit-header-title">
              <span className="dashboard-audit-title-text">Most Recent Activity</span>
            </div>
            <Link to="/admin/audit" className="dashboard-view-all-link">
              View All <ArrowRight size={13} />
            </Link>
          </div>
          <div className="dashboard-audit-list">
            {auditLogs.length === 0 ? (
              <div className="dashboard-audit-empty">
                No activity yet
              </div>
            ) : (
              auditLogs.map((log, idx) => (
                <div
                  key={log._id}
                  className={`dashboard-audit-item ${idx < auditLogs.length - 1 ? "dashboard-audit-item--bordered" : ""}`}
                >
                  <span className="dashboard-audit-action">
                    {formatAction(log.action)}
```

```
                    </span>
                    <span className="dashboard-audit-meta">
                      {log.userId?.name || "System"}
                      {" · "}
                      {new Date(log.createdAt).toLocaleString("en-US", { month: "short", day: "numeric", hour: "numeric", minut
                    </span>
                  </div>
                ))
              )}
            </div>
          </motion.div>
        ) : null}

      </div>
    );
  }
```

59. client/src/pages/admin/AdminProfile.jsx (314 lines)

```
import { useContext, useRef, useState } from "react";
import { Eye, EyeOff } from "lucide-react";
import { motion, AnimatePresence } from "framer-motion";
import { AuthContext } from "../../context/AuthContext";
import { updateProfile, requestPasswordChangeOtp } from "../../services/authService";
import PasswordChecklist, { ErrorTooltip } from "../../components/PasswordChecklist";
import { isStrongPassword } from "../../utils/passwordPolicy";
import { notify } from "../../utils/inPageFeedback";
import "../../components/RequestForm.css";
import "../../components/FloatingLabel.css";

const AdminProfile = () => {
  const { user, updateUser } = useContext(AuthContext);
  const [firstName, setFirstName] = useState(user?.firstName || "");
  const [middleName, setMiddleName] = useState(user?.middleInitial || "");
  const [lastName, setLastName] = useState(user?.lastName || "");
  const [currentPassword, setCurrentPassword] = useState("");
  const [newPassword, setNewPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [loading, setLoading] = useState(false);
  const [showCurrentPw, setShowCurrentPw] = useState(false);
  const [showNewPw, setShowNewPw] = useState(false);
  const [showConfirmPw, setShowConfirmPw] = useState(false);
  const [confirmName, setConfirmName] = useState(false);
  const [confirmPw, setConfirmPw] = useState(false);
  const [otp, setOtp] = useState("");
  const [otpCooldown, setOtpCooldown] = useState(0);

  const [newPwFocused, setNewPwFocused] = useState(false);
  const [confirmPwFocused, setConfirmPwFocused] = useState(false);

  // Refs for tooltip anchoring
  const newPwAnchorRef = useRef(null);
  const confirmPwAnchorRef = useRef(null);
  const cooldownRef = useRef(null);

  const initials = user?.name
    ? user.name.split(" ").map((w) => w[0]).slice(0, 2).join("").toUpperCase()
    : "?";

  const startCooldown = (seconds) => {
    setOtpCooldown(seconds);
    clearInterval(cooldownRef.current);
    cooldownRef.current = setInterval(() => {
      setOtpCooldown((s) => {
        if (s <= 1) { clearInterval(cooldownRef.current); return 0; }
        return s - 1;
      });
    }, 1000);
  };

  const handleNameSubmit = (e) => {
    e.preventDefault();
    if (!firstName.trim() || !lastName.trim()) {
      notify("Failed.", { type: "error", title: "Failed" });
      return;
    }
    const hasChanges =
      firstName.trim() !== (user?.firstName || "") ||
      middleName.trim() !== (user?.middleInitial || "") ||
      lastName.trim() !== (user?.lastName || "");
    if (!hasChanges) {
      notify("No changes to save.", { type: "warning", title: "Notice" });
      return;
    }
    setConfirmName(true);
  };

  const handleConfirmNameUpdate = async () => {
    setLoading(true);
    try {
      const data = await updateProfile({ firstName: firstName.trim(), middleInitial: middleName.trim(), lastName: lastName.tr
      updateUser(data.user);
```

```
      notify("Name updated successfully.", { type: "success", title: "Success" });
      setConfirmName(false);
    } catch (err) {
      notify("Failed.", { type: "error", title: "Failed" });
      setConfirmName(false);
    } finally {
      setLoading(false);
    }
  };

  const handlePasswordSubmit = async (e) => {
    e.preventDefault();
    if (!currentPassword || !isStrongPassword(newPassword) || newPassword !== confirmPassword) return;
    setLoading(true);
    try {
      await requestPasswordChangeOtp();
      setOtp("");
      startCooldown(60);
      setConfirmPw(true);
    } catch (err) {
      const data = err?.response?.data;
      if (err?.response?.status === 429 && data?.retryAfterSeconds) {
        startCooldown(data.retryAfterSeconds);
        setConfirmPw(true);
      } else {
        notify(data?.message || "Failed to send OTP.", { type: "error", title: "Error" });
      }
    } finally {
      setLoading(false);
    }
  };

  const handleResendOtp = async () => {
    if (otpCooldown > 0) return;
    setLoading(true);
    try {
      await requestPasswordChangeOtp();
      setOtp("");
      startCooldown(60);
      notify("A new OTP has been sent to your email.", { type: "info", title: "OTP Sent" });
    } catch (err) {
      const data = err?.response?.data;
      if (err?.response?.status === 429 && data?.retryAfterSeconds) {
        startCooldown(data.retryAfterSeconds);
      }
      notify(data?.message || "Failed to resend OTP.", { type: "error", title: "Error" });
    } finally {
      setLoading(false);
    }
  };

  const handleConfirmPasswordUpdate = async () => {
    if (!otp.trim()) { notify("Please enter the OTP sent to your email.", { type: "warning" }); return; }
    setLoading(true);
    try {
      await updateProfile({ currentPassword, newPassword, otp: otp.trim() });
      setCurrentPassword("");
      setNewPassword("");
      setConfirmPassword("");
      setOtp("");
      setConfirmPwFocused(false);
      setConfirmPw(false);
      notify("Password changed successfully.", { type: "success", title: "Success" });
    } catch (err) {
      notify(err?.response?.data?.message || "Failed to change password.", { type: "error", title: "Failed" });
    } finally {
      setLoading(false);
    }
  };

  const pwToggleStyle = {
    position: "absolute", right: 6, top: "50%", transform: "translateY(-50%)",
    background: "none", border: "none", padding: 4, cursor: "pointer",
    color: "var(--text-muted)", display: "flex", alignItems: "center",
  };

```

```
  return (
    <div style={{ display: "flex", justifyContent: "center" }}>
    <div style={{ maxWidth: 520, width: "100%" }}>

      {/* Profile Header */}
      <div style={{ background: "var(--surface)", border: "1px solid var(--border)", borderRadius: "var(--radius-lg)", paddin
        <div style={{ width: 72, height: 72, borderRadius: "50%", background: "var(--primary)", display: "flex", alignItems:
        <p style={{ margin: "0 0 4px", fontSize: 18, fontWeight: 700, color: "var(--text-primary)" }}>{user?.name}</p>
        <p style={{ margin: "0 0 8px", fontSize: 13, color: "var(--text-muted)" }}>{user?.email}</p>
        <span style={{ padding: "3px 12px", borderRadius: 99, fontSize: 11, fontWeight: 700, background: "#e0e7ff", color: "#
      </div>

      {/* Change Name */}
      <div style={{ background: "var(--surface)", border: "1px solid var(--border)", borderRadius: "var(--radius-lg)", paddin
        <h3 style={{ margin: "0 0 16px", fontSize: 15, fontWeight: 700, color: "var(--text-primary)" }}>Change Name</h3>
        <form onSubmit={handleNameSubmit} style={{ display: "flex", flexDirection: "column", gap: 10 }}>
          <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr 1fr", gap: 10 }}>
            <div className="fl-wrap">
              <input placeholder=" " value={firstName} onChange={(e) => setFirstName(e.target.value)} className="fl-input req
              <label className="fl-label">First Name</label>
            </div>
            <div className="fl-wrap">
              <input placeholder=" " value={middleName} onChange={(e) => setMiddleName(e.target.value)} className="fl-input r
              <label className="fl-label">Middle Name</label>
            </div>
            <div className="fl-wrap">
              <input placeholder=" " value={lastName} onChange={(e) => setLastName(e.target.value)} className="fl-input reque
              <label className="fl-label">Last Name</label>
            </div>
          </div>
          <div style={{ display: "flex", justifyContent: "flex-end" }}>
            <button type="submit" className="ui-btn ui-btn--primary" disabled={loading}>Save Changes</button>
          </div>
        </form>
      </div>

      {/* Change Password */}
      <div style={{ background: "var(--surface)", border: "1px solid var(--border)", borderRadius: "var(--radius-lg)", paddin
        <h3 style={{ margin: "0 0 16px", fontSize: 15, fontWeight: 700, color: "var(--text-primary)" }}>Change Password</h3>
        <form onSubmit={handlePasswordSubmit} style={{ display: "flex", flexDirection: "column", gap: 14 }}>

          {/* Current Password */}
          <div className="fl-wrap">
            <input
              type={showCurrentPw ? "text" : "password"}
              placeholder=" "
              value={currentPassword}
              onChange={(e) => setCurrentPassword(e.target.value)}
              className="fl-input request-input"
              style={{ paddingRight: 42 }}
            />
            <label className="fl-label fl-label--pw">Current Password</label>
            <button type="button" onClick={() => setShowCurrentPw((v) => !v)} tabIndex={-1} aria-label={showCurrentPw ? "Hide
              {showCurrentPw ? <Eye size={18} /> : <EyeOff size={18} />}
            </button>
          </div>

          {/* New Password */}
          <div className="fl-wrap" ref={newPwAnchorRef}>
            <input
              type={showNewPw ? "text" : "password"}
              placeholder=" "
              value={newPassword}
              onChange={(e) => setNewPassword(e.target.value)}
              onFocus={() => setNewPwFocused(true)}
              onBlur={() => setNewPwFocused(false)}
              minLength={8}
              className="fl-input request-input"
              style={{ paddingRight: 42 }}
            />
            <label className="fl-label fl-label--pw">New Password</label>
            <button type="button" onClick={() => setShowNewPw((v) => !v)} tabIndex={-1} aria-label={showNewPw ? "Hide" : "Sho
              {showNewPw ? <Eye size={18} /> : <EyeOff size={18} />}
            </button>
            <PasswordChecklist focused={!isStrongPassword(newPassword) && (newPwFocused || newPassword.length > 0)} anchorRef
          </div>
```

```
226
227           {/* Confirm Password */}
228           <div className="fl-wrap" ref={confirmPwAnchorRef}>
229             <input
230               type={showConfirmPw ? "text" : "password"}
231               placeholder=" "
232               value={confirmPassword}
233               onChange={(e) => setConfirmPassword(e.target.value)}
234               onFocus={() => setConfirmPwFocused(true)}
235               onBlur={() => setConfirmPwFocused(false)}
236               className="fl-input request-input"
237               style={{ paddingRight: 42 }}
238             />
239             <label className="fl-label fl-label--pw">Confirm New Password</label>
240             <button type="button" onClick={() => setShowConfirmPw((v) => !v)} tabIndex={-1} aria-label={showConfirmPw ? "Hide
241               {showConfirmPw ? <Eye size={18} /> : <EyeOff size={18} />}
242             </button>
243             <ErrorTooltip
244               show={confirmPassword !== newPassword && (confirmPwFocused || confirmPassword.length > 0)}
245               message="Passwords do not match"
246               anchorRef={confirmPwAnchorRef}
247               side="right"
248               persistent
249             />
250           </div>
251
252           <button type="submit" className="ui-btn ui-btn--primary" disabled={loading} style={{ alignSelf: "flex-end" }}>
253             {loading ? "Sending OTP…" : "Save Changes"}
254           </button>
255         </form>
256       </div>
257     </div>
258
259     {/* Confirm Name Change Modal */}
260     <AnimatePresence>
261       {confirmName && (
262         <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
263           <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12, op
264             <h3 className="modal-title">Confirm Change Name</h3>
265             <p className="modal-text">Are you sure you want to update your name?</p>
266             <p className="modal-text"><strong>{firstName.trim()} {middleName.trim() ? middleName.trim() + " " : ""}{lastName.
267             <div className="modal-actions">
268               <button className="ui-btn ui-btn--secondary" onClick={() => setConfirmName(false)} disabled={loading}>Cancel</b
269               <button className="ui-btn ui-btn--primary" onClick={handleConfirmNameUpdate} disabled={loading}>{loading ? "Sav
270             </div>
271           </motion.div>
272         </motion.div>
273       )}
274     </AnimatePresence>
275
276     {/* Confirm Password Change Modal */}
277     <AnimatePresence>
278       {confirmPw && (
279         <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
280           <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12, op
281             <h3 className="modal-title">Verify Your Identity</h3>
282             <p className="modal-text">An OTP has been sent to <strong>{user?.email}</strong>. Enter it below to confirm your
283             <div className="fl-wrap" style={{ marginBottom: 4 }}>
284               <input
285                 placeholder=" "
286                 value={otp}
287                 onChange={(e) => setOtp(e.target.value)}
288                 className="fl-input request-input"
289                 maxLength={6}
290                 autoFocus
291               />
292               <label className="fl-label">One-Time Password</label>
293             </div>
294             <button
295               type="button"
296               onClick={handleResendOtp}
297               disabled={otpCooldown > 0 || loading}
298               style={{ fontSize: 12, color: otpCooldown > 0 ? "var(--text-muted)" : "var(--primary)", background: "none", bor
299             >
300               {otpCooldown > 0 ? `Resend OTP in ${otpCooldown}s` : "Resend OTP"}
301             </button>
```

```
302              <div className="modal-actions">
303                <button className="ui-btn ui-btn--secondary" onClick={() => { setConfirmPw(false); setOtp(""); }} disabled={loa
304                <button className="ui-btn ui-btn--primary" onClick={handleConfirmPasswordUpdate} disabled={loading}>{loading ?
305              </div>
306            </motion.div>
307          </motion.div>
308        )}
309      </AnimatePresence>
310      </div>
311    );
312  };
313
314  export default AdminProfile;
```

60. client/src/pages/admin/AdminUsers.jsx (474 lines)

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { UserPlus, RefreshCw, Search, X, Eye, EyeOff } from "lucide-react";
import FilterSelect from "../../components/FilterSelect";
import {
  getAllUsers,
  adminCreateUser,
  adminUpdateUser,
} from "../../services/authService";
import PasswordChecklist from "../../components/PasswordChecklist";
import "../../components/FloatingLabel.css";
import { isStrongPassword } from "../../utils/passwordPolicy";
import { notify } from "../../utils/inPageFeedback";

const ROLE_LABELS = { student: "Student", admin: "Admin", staff: "Staff" };

const ROLE_COLORS = {
  admin: { bg: "#e0e7ff", color: "#3730a3" },
  staff: { bg: "#d1fae5", color: "#065f46" },
  student: { bg: "#f1f5f9", color: "#475569" },
};

export default function AdminUsers() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [refreshing, setRefreshing] = useState(false);
  const [search, setSearch] = useState("");
  const [roleFilter, setRoleFilter] = useState("all");
  const [statusFilter, setStatusFilter] = useState("all");
  const [showCreate, setShowCreate] = useState(false);
  const [creating, setCreating] = useState(false);
  const [actionId, setActionId] = useState(null);
  const [editingUser, setEditingUser] = useState(null);
  const [editForm, setEditForm] = useState({ role: "student", isActive: true });

  const [form, setForm] = useState({ firstName: "", middleName: "", lastName: "", email: "", role: "student", password: "" })
  const [showCreatePw, setShowCreatePw] = useState(false);
  const [createPwFocused, setCreatePwFocused] = useState(false);
  const createPwAnchorRef = useRef(null);
  const [confirmCreate, setConfirmCreate] = useState(false);
  const [confirmEdit, setConfirmEdit] = useState(false);

  const load = useCallback(async ({ withSpinner = false } = {}) => {
    setLoading(true);
    if (withSpinner) setRefreshing(true);
    try {
      const data = await getAllUsers();
      setUsers(data);
    } catch {
      // silently fail on load
    } finally {
      setLoading(false);
      setRefreshing(false);
    }
  }, []);

  useEffect(() => { load(); }, [load]);

  const handleCreate = (e) => {
    e.preventDefault();
    if (!form.firstName.trim() || !form.lastName.trim() || !form.email.trim()) return;
    if (!isStrongPassword(form.password)) return;
    setConfirmCreate(true);
  };

  const handleConfirmCreate = async () => {
    setCreating(true);
    try {
      const mi = form.middleName.trim() ? form.middleName.trim().charAt(0).toUpperCase() + "." : "";
      const composedName = `${form.firstName.trim()}${mi ? " " + mi : ""} ${form.lastName.trim()}`;
      await adminCreateUser({ name: composedName, email: form.email, role: form.role, password: form.password });
      notify("User created successfully.", { type: "success", title: "Success" });
      setForm({ firstName: "", middleName: "", lastName: "", email: "", role: "student", password: "" });
```

```
      setShowCreatePw(false);
      setShowCreate(false);
      setConfirmCreate(false);
      await load();
    } catch (err) {
      notify(err.response?.data?.message || "Failed to create user.", { type: "error", title: "Failed" });
      setConfirmCreate(false);
    } finally {
      setCreating(false);
    }
  };

  const openEditModal = (user) => {
    setEditingUser(user);
    setEditForm({ role: user.role || "student", isActive: user.isActive !== false });
  };

  const handleSaveEdit = async () => {
    if (!editingUser) return;
    setActionId(editingUser._id);
    try {
      await adminUpdateUser(editingUser._id, { role: editForm.role, isActive: editForm.isActive });
      notify("User updated successfully.", { type: "success", title: "Success" });
      setEditingUser(null);
      await load();
    } catch (err) {
      notify(err.response?.data?.message || "Failed to update user.", { type: "error", title: "Failed" });
    } finally {
      setActionId(null);
    }
  };

  const resetAndRefresh = () => {
    setSearch("");
    setRoleFilter("all");
    setStatusFilter("all");
    load({ withSpinner: true });
  };

  const filtered = useMemo(() => {
    const normalized = search.toLowerCase();
    let result = users.filter(
      (u) =>
        u.name?.toLowerCase().includes(normalized) ||
        u.email?.toLowerCase().includes(normalized)
    );
    if (roleFilter !== "all") result = result.filter((u) => u.role === roleFilter);
    if (statusFilter === "active") result = result.filter((u) => u.isActive !== false);
    if (statusFilter === "inactive") result = result.filter((u) => u.isActive === false);
    return result;
  }, [search, roleFilter, statusFilter, users]);

  return (
    <div className="page-shell">
      <div className="page-shell__header">
        <button
          type="button"
          onClick={() => { setForm({ firstName: "", middleName: "", lastName: "", email: "", role: "student", password: "" })
          className="req-toolbar__action-btn"
        >
          <UserPlus size={14} />
          Create User
        </button>
      </div>

      {/* ■■ Toolbar ■■ */}
      <div className="req-toolbar">
        {/* Search row */}
        <div className="req-toolbar__search-row">
          <div className="req-toolbar__search-box">
            <input
              type="text"
              className="req-toolbar__search-input"
              placeholder="Search by name or email…"
              value={search}
              onChange={(e) => setSearch(e.target.value)}
```

```
150             />
151             {search ? (
152               <button type="button" className="req-toolbar__search-clear" onClick={() => setSearch("")} aria-label="Clear sea
153                 ×
154               </button>
155             ) : null}
156             <span className="req-toolbar__search-inline-btn" style={{ pointerEvents: "none" }}>
157               <Search size={14} />
158             </span>
159           </div>
160         </div>
161
162         {/* Filter row */}
163         <div className="req-toolbar__filters req-toolbar__filters--open">
164           <div className="req-toolbar__filter-group">
165             <label className="req-toolbar__filter-label">Role</label>
166             <FilterSelect
167               value={roleFilter}
168               onChange={setRoleFilter}
169               options={[
170                 { value: "all", label: "All Roles" },
171                 { value: "student", label: "Student" },
172                 { value: "staff", label: "Staff" },
173                 { value: "admin", label: "Admin" },
174               ]}
175             />
176           </div>
177
178           <div className="req-toolbar__filter-group">
179             <label className="req-toolbar__filter-label">Status</label>
180             <FilterSelect
181               value={statusFilter}
182               onChange={setStatusFilter}
183               options={[
184                 { value: "all", label: "All Statuses" },
185                 { value: "active", label: "Active" },
186                 { value: "inactive", label: "Inactive" },
187               ]}
188             />
189           </div>
190
191           <button
192             type="button"
193             className="req-toolbar__reset-btn"
194             onClick={resetAndRefresh}
195             disabled={refreshing}
196             title="Reset filters and refresh"
197           >
198             <RefreshCw size={14} className={refreshing ? "spin-anim" : ""} />
199           </button>
200         </div>
201       </div>
202
203       {/* Create User Modal */}
204       <AnimatePresence>
205         {showCreate && (
206           <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}
207             onClick={(e) => { if (e.target === e.currentTarget) { setShowCreate(false); setShowCreatePw(false); setForm({ nam
208           >
209             <motion.div className="user-edit-modal" initial={{ y: 16, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y:
210
211               {/* Title row */}
212               <div className="user-edit-modal__title-row">
213                 <span className="user-edit-modal__title">Create User</span>
214                 <button
215                   type="button"
216                   className="user-edit-modal__close user-edit-modal__close--danger"
217                   onClick={() => { setShowCreate(false); setShowCreatePw(false); setForm({ name: "", email: "", role: "studen
218                   aria-label="Close"
219                 >
220                   <X size={16} />
221                 </button>
222               </div>
223
224               <div className="user-edit-modal__divider" />
225
```

```
              {/* Form */}
              <form onSubmit={handleCreate}>
                <div className="user-edit-modal__fields" style={{ gridTemplateColumns: "1fr" }}>
                  <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr 1fr", gap: 10 }}>
                    <div className="user-edit-modal__field fl-wrap">
                      <input className="fl-input ui-input" placeholder=" " value={form.firstName} onChange={(e) => setForm((p
                      <label className="fl-label">First Name</label>
                    </div>
                    <div className="user-edit-modal__field fl-wrap">
                      <input className="fl-input ui-input" placeholder=" " value={form.middleName} onChange={(e) => setForm((
                      <label className="fl-label">Middle Name</label>
                    </div>
                    <div className="user-edit-modal__field fl-wrap">
                      <input className="fl-input ui-input" placeholder=" " value={form.lastName} onChange={(e) => setForm((p)
                      <label className="fl-label">Last Name</label>
                    </div>
                  </div>
                  <div className="user-edit-modal__field fl-wrap">
                    <input className="fl-input ui-input" type="email" placeholder=" " value={form.email} onChange={(e) => set
                    <label className="fl-label">Email</label>
                  </div>
                  <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 14 }}>
                    <div className="user-edit-modal__field">
                      <select className="ui-input" value={form.role} onChange={(e) => setForm((p) => ({ ...p, role: e.target.
                        <option value="student">Student</option>
                        <option value="staff">Staff</option>
                        <option value="admin">Admin</option>
                      </select>
                    </div>
                    <div ref={createPwAnchorRef} className="user-edit-modal__field fl-wrap">
                      <input
                        className="fl-input ui-input"
                        type={showCreatePw ? "text" : "password"}
                        placeholder=" "
                        value={form.password}
                        onChange={(e) => setForm((p) => ({ ...p, password: e.target.value }))}
                        onFocus={() => setCreatePwFocused(true)}
                        onBlur={() => setCreatePwFocused(false)}
                        required
                        minLength={8}
                        style={{ paddingRight: 42 }}
                      />
                      <label className="fl-label fl-label--pw">Temporary Password</label>
                      <button type="button" onClick={() => setShowCreatePw((v) => !v)} tabIndex={-1} aria-label={showCreatePw
                        {showCreatePw ? <Eye size={18} /> : <EyeOff size={18} />}
                      </button>
                      <PasswordChecklist focused={!isStrongPassword(form.password) && (createPwFocused || form.password.lengt
                    </div>
                  </div>
                </div>

                {/* Footer */}
                <div className="user-edit-modal__footer" style={{ justifyContent: "flex-end" }}>
                  <button type="submit" className="ui-btn ui-btn--primary" disabled={creating}>
                    {creating ? "Creating…" : "Create User"}
                  </button>
                </div>
              </form>

            </motion.div>
          </motion.div>
        )}
      </AnimatePresence>

      {/* Table */}
      <div className="dashboard-card">
        <div className="table-scroll">
        <table className="dashboard-table">
          <thead>
            <tr className="dashboard-table-title-row">
              <th colSpan={5}>
                <div className="dashboard-table-title-wrap">
                  <span className="dashboard-table-title">Users</span>
                </div>
              </th>
            </tr>
```

```
            <tr>
              {["Joined", "User", "Role", "Status"].map((h) => (
                <th key={h}>{h}</th>
              ))}
              <th />
            </tr>
          </thead>
          <tbody>
            {loading ? (
              [...Array(5)].map((_, i) => (
                <tr key={`sk-${i}`}>
                  <td><span className="skeleton-block skeleton-text" /></td>
                  <td><span className="skeleton-block skeleton-text" /></td>
                  <td><span className="skeleton-block skeleton-pill" /></td>
                  <td><span className="skeleton-block skeleton-pill" /></td>
                  <td />
                </tr>
              ))
            ) : filtered.length === 0 ? (
              <tr>
                <td colSpan={5}>
                  <div className="dashboard-empty">
                    <p className="dashboard-empty-title">No users found</p>
                    <p className="dashboard-empty-text">No accounts match the current filters.</p>
                  </div>
                </td>
              </tr>
            ) : (
              filtered.map((user, i) => {
                const roleStyle = ROLE_COLORS[user.role] || ROLE_COLORS.student;
                return (
                  <motion.tr
                    key={user._id}
                    initial={{ opacity: 0 }} animate={{ opacity: 1 }} transition={{ delay: i * 0.03 }}
                    className={user.isActive === false ? "admin-row-inactive" : ""}
                    onClick={() => openEditModal(user)}
                    style={{ cursor: "pointer" }}
                  >
                    <td style={{ whiteSpace: "nowrap", color: "var(--text-secondary)" }}>
                      {user.createdAt ? new Date(user.createdAt).toLocaleDateString("en-US", { year: "numeric", month: "short
                    </td>
                    <td>
                      <span style={{ fontWeight: 600 }}>{user.name}</span>
                      <span className="dashboard-subtext">{user.email}</span>
                    </td>
                    <td>
                      <span className="tag-pill" style={{ background: roleStyle.bg, color: roleStyle.color }}>
                        {ROLE_LABELS[user.role] || user.role}
                      </span>
                    </td>
                    <td>
                      <span className="tag-pill" style={{
                        background: user.isActive !== false ? "#d1fae5" : "#fee2e2",
                        color: user.isActive !== false ? "#065f46" : "#991b1b",
                      }}>
                        {user.isActive !== false ? "Active" : "Inactive"}
                      </span>
                    </td>
                    <td className="row-caret">›</td>
                  </motion.tr>
                );
              })
            )}
          </tbody>
        </table>
        </div>
      </div>

      <AnimatePresence>
        {editingUser && (() => {
          const initials = (editingUser.name || "?").split(" ").map((w) => w[0]).slice(0, 2).join("").toUpperCase();
          const roleStyle = ROLE_COLORS[editingUser.role] || ROLE_COLORS.student;
          return (
            <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
              <motion.div className="user-edit-modal" initial={{ y: 16, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{
```

```
                {/* Title row */}
                <div className="user-edit-modal__title-row">
                  <span className="user-edit-modal__title">Edit User</span>
                  <button
                    type="button"
                    className="user-edit-modal__close user-edit-modal__close--danger"
                    onClick={() => setEditingUser(null)}
                    aria-label="Close"
                  >
                    <X size={16} />
                  </button>
                </div>

                {/* Profile header */}
                <div className="user-edit-modal__header">
                  <div className="user-edit-modal__avatar" style={{ background: roleStyle.bg, color: roleStyle.color }}>
                    {initials}
                  </div>
                  <div className="user-edit-modal__identity">
                    <span className="user-edit-modal__name">{editingUser.name}</span>
                    <span className="user-edit-modal__email">{editingUser.email}</span>
                  </div>
                </div>

                {/* Divider */}
                <div className="user-edit-modal__divider" />

                {/* Editable fields */}
                <div className="user-edit-modal__fields">
                  <div className="user-edit-modal__field">
                    <label className="user-edit-modal__label">Role</label>
                    <select className="ui-input" value={editForm.role} onChange={(e) => setEditForm((p) => ({ ...p, role: e.t
                      <option value="student">Student</option>
                      <option value="staff">Staff</option>
                      <option value="admin">Admin</option>
                    </select>
                  </div>
                  <div className="user-edit-modal__field">
                    <label className="user-edit-modal__label">Status</label>
                    <select className="ui-input" value={editForm.isActive ? "active" : "inactive"} onChange={(e) => setEditFo
                      <option value="active">Active</option>
                      <option value="inactive">Inactive</option>
                    </select>
                  </div>
                </div>

                {/* Footer */}
                <div className="user-edit-modal__footer" style={{ justifyContent: "flex-end" }}>
                  <button className="ui-btn ui-btn--primary" onClick={() => setConfirmEdit(true)} disabled={actionId === edit
                    {actionId === editingUser._id ? "Saving…" : "Save Changes"}
                  </button>
                </div>

              </motion.div>
            </motion.div>
          );
        })()}
      </AnimatePresence>

      <AnimatePresence>
        {confirmCreate && (
          <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
            <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12,
              <h3 className="modal-title">Confirm Create User</h3>
              <p className="modal-text">Are you sure you want to create this user?</p>
              <p className="modal-text"><strong>{form.firstName.trim()} {form.middleName.trim() ? form.middleName.trim() + "
              <p className="modal-text"><strong>{form.email}</strong></p>
              <div className="modal-actions">
                <button className="ui-btn ui-btn--secondary" onClick={() => setConfirmCreate(false)} disabled={creating}>Canc
                <button className="ui-btn ui-btn--primary" onClick={handleConfirmCreate} disabled={creating}>{creating ? "Cre
              </div>
            </motion.div>
          </motion.div>
        )}
      </AnimatePresence>

```

```
454        <AnimatePresence>
455          {confirmEdit && editingUser && (
456            <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
457              <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12,
458                <h3 className="modal-title">Confirm Save Changes</h3>
459                <p className="modal-text">Are you sure you want to save changes for this user?</p>
460                <p className="modal-text"><strong>{editingUser.name}</strong> &mdash; <strong>{ROLE_LABELS[editForm.role] || ed
461                <p className="modal-text"><strong>{editingUser.email}</strong></p>
462                <div className="modal-actions">
463                  <button className="ui-btn ui-btn--secondary" onClick={() => setConfirmEdit(false)} disabled={actionId === edi
464                  <button className="ui-btn ui-btn--primary" onClick={() => { setConfirmEdit(false); handleSaveEdit(); }} disab
465                </div>
466              </motion.div>
467            </motion.div>
468          )}
469        </AnimatePresence>
470
471      </div>
472    );
473  }
474
```

61. client/src/pages/admin/AdminRequests.jsx (404 lines)

```
import { useEffect, useMemo, useState } from "react";
import { useNavigate } from "react-router-dom";
import { Filter, RefreshCw, Search } from "lucide-react";
import { getAllRequests } from "../../services/requestService";
import FilterSelect from "../../components/FilterSelect";

const statusClass = (s) => {
  if (["nda_approved", "agr_approved"].includes(s)) return "status-pill status-pill--green";
  if (["nda_pending", "agr_pending_1", "agr_pending_2"].includes(s)) return "status-pill status-pill--yellow";
  if (s === "stud_revision_requested") return "status-pill status-pill--orange";
  if (s === "agr_rep_revision_requested") return "status-pill status-pill--violet";
  if (s === "agr_awaiting_rep_signature") return "status-pill status-pill--blue";
  if (s === "agr_declined") return "status-pill status-pill--red";
  return "status-pill";
};

const prettyStatus = (s) => {
  const map = {
    nda_pending:                "Reviewal",
    nda_approved:               "Approved",
    stud_revision_requested:    "Student Revisions",
    agr_pending_1:              "Initial Reviewal",
    agr_awaiting_rep_signature: "Recipient Reviewal",
    agr_pending_2:              "Final Reviewal",
    agr_approved:               "Approved",
    agr_declined:               "Recipient Declined",
    agr_rep_revision_requested: "Recipient Revisions",
  };
  return map[s] || (s ? s.charAt(0).toUpperCase() + s.slice(1).replace(/_/g, " ") : "—");
};

export default function AdminRequests() {
  const navigate = useNavigate();
  const [requests, setRequests] = useState([]);
  const [loading, setLoading] = useState(true);
  const [refreshing, setRefreshing] = useState(false);
  const [error, setError] = useState("");
  const [isFilterOpen, setIsFilterOpen] = useState(() => {
    if (typeof window === "undefined") return true;
    return window.innerWidth >= 768;
  });

  const [statusFilter, setStatusFilter] = useState("all");
  const [typeFilter, setTypeFilter] = useState("all");
  const [dateMode, setDateMode] = useState("preset");
  const [preset, setPreset] = useState("all");
  const [singleDate, setSingleDate] = useState("");
  const [startDate, setStartDate] = useState("");
  const [endDate, setEndDate] = useState("");
  const [searchTerm, setSearchTerm] = useState("");

  const buildFilterParams = () => {
    const params = {};

    if (statusFilter !== "all") {
      params.status = statusFilter;
    }

    if (typeFilter !== "all") {
      if (typeFilter === "agreement") {
        params.type = "agreement";
      } else if (typeFilter === "nda_orgactivities") {
        params.type = "nda";
        params.ndaType = "orgactivities";
      } else if (typeFilter === "nda_research") {
        params.type = "nda";
        params.ndaType = "research";
      }
    }


    if (dateMode === "preset") {
      const now = new Date();
```

```
      if (preset === "today") {
        params.startDate = now.toISOString().slice(0, 10);
        params.endDate = now.toISOString().slice(0, 10);
      } else if (preset === "thisWeek") {
        const day = now.getDay();
        const monday = new Date(now);
        monday.setDate(now.getDate() - ((day + 6) % 7));
        params.startDate = monday.toISOString().slice(0, 10);
      } else if (preset === "thisMonth") {
        const start = new Date(now.getFullYear(), now.getMonth(), 1);
        params.startDate = start.toISOString().slice(0, 10);
      } else if (preset === "thisYear") {
        const start = new Date(now.getFullYear(), 0, 1);
        params.startDate = start.toISOString().slice(0, 10);
      }
    }

    if (dateMode === "single" && singleDate) {
      params.startDate = singleDate;
      params.endDate = singleDate;
    }

    if (dateMode === "range") {
      if (startDate) params.startDate = startDate;
      if (endDate) params.endDate = endDate;
    }

    return params;
  };

  const loadRequests = async ({ withToast = false } = {}) => {
    if (withToast) setRefreshing(true);
    setError("");

    try {
      const data = await getAllRequests(buildFilterParams());
      setRequests(data);

    } catch (err) {
      setError(err.response?.data?.message || "Failed to load requests");
    } finally {
      setLoading(false);
      setRefreshing(false);
    }
  };

  useEffect(() => {
    loadRequests();
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [statusFilter, typeFilter, dateMode, preset, singleDate, startDate, endDate]);

  useEffect(() => {
    const onResize = () => {
      if (window.innerWidth >= 768) setIsFilterOpen(true);
    };
    window.addEventListener("resize", onResize);
    return () => window.removeEventListener("resize", onResize);
  }, []);

  const filtered = useMemo(() => {
    const APPROVED = ["nda_approved", "agr_approved"];
    let result = requests;
    if (searchTerm.trim()) {
      const term = searchTerm.toLowerCase();
      result = result.filter((r) => {
        const name = (r.userId?.name || r.proxyRequestee?.fullName || "").toLowerCase();
        const email = (r.userId?.email || r.proxyRequestee?.email || "").toLowerCase();
        const id = (r._id || "").slice(-7).toLowerCase();
        return name.includes(term) || email.includes(term) || id.includes(term);
      });
    }
    return [...result].sort((a, b) => {
      const aApproved = APPROVED.includes(a.status) ? 1 : 0;
      const bApproved = APPROVED.includes(b.status) ? 1 : 0;
      return aApproved - bApproved;
    });
```

```
  }, [requests, searchTerm]);

  const clearSearch = () => setSearchTerm("");

  const resetAndRefresh = () => {
    setSearchTerm("");
    setTypeFilter("all");
    setStatusFilter("all");
    setDateMode("preset");
    setPreset("all");
    setSingleDate("");
    setStartDate("");
    setEndDate("");
    loadRequests({ withToast: true });
  };

  return (
    <div className="dashboard-page page-shell">

      {error ? (
        <div className="info-banner info-banner--danger" style={{ marginBottom: 12 }}>
          <strong>{error}</strong>
        </div>
      ) : null}

      {/* ■■ Search + Filter toolbar ■■ */}
      <div className="req-toolbar">
        {/* Search row */}
        <div className="req-toolbar__search-row">
          <div className="req-toolbar__search-box">
            <input
              type="text"
              className="req-toolbar__search-input"
              placeholder="Search by requestor, email, or request ID…"
              value={searchTerm}
              onChange={(e) => setSearchTerm(e.target.value)}
            />
            {searchTerm ? (
              <button type="button" className="req-toolbar__search-clear" onClick={clearSearch} aria-label="Clear search">
                ×
              </button>
            ) : null}
            <span className="req-toolbar__search-inline-btn" style={{ pointerEvents: "none" }}>
              <Search size={14} />
            </span>
          </div>
          <button
            type="button"
            className="ui-btn ui-btn--secondary req-toolbar__filter-toggle"
            onClick={() => setIsFilterOpen((prev) => !prev)}
            aria-expanded={isFilterOpen}
          >
            <Filter size={13} />
            Filters
            {isFilterOpen ? null : <span className="req-toolbar__filter-badge" />}
          </button>
        </div>

        {/* Filter chips row */}
        <div className={`req-toolbar__filters${isFilterOpen ? " req-toolbar__filters--open" : ""}`}>
          <div className="req-toolbar__filter-group">
            <label className="req-toolbar__filter-label">Type</label>
            <FilterSelect
              value={typeFilter}
              onChange={setTypeFilter}
              className="filter-select--type"
              options={[
                { value: "all", label: "All Types" },
                { value: "nda_orgactivities", label: "NDA — Student Organization Activities" },
                { value: "nda_research", label: "NDA — Conduct of Research" },
                { value: "agreement", label: "Agreement" },
              ]}
            />
          </div>

          <div className="req-toolbar__filter-group">
```

```
            <label className="req-toolbar__filter-label">Status</label>
            <FilterSelect
              value={statusFilter}
              onChange={setStatusFilter}
              className="filter-select--status"
              options={[
                { value: "all", label: "All Statuses" },
                { value: "reviewal", label: "Reviewal" },
                { value: "approved", label: "Approved" },
                { value: "stud_revision", label: "Student Revisions" },
                { value: "rep_revision", label: "Recipient Revisions" },
                { value: "rep_declined", label: "Recipient Declined" },
                { value: "awaiting_rep", label: "Recipient Reviewal" },
              ]}
            />
          </div>

          <div className="req-toolbar__filter-group">
            <label className="req-toolbar__filter-label">Date</label>
            <FilterSelect
              value={dateMode}
              onChange={setDateMode}
              options={[
                { value: "preset", label: "Preset" },
                { value: "single", label: "Specific Date" },
                { value: "range", label: "Date Range" },
              ]}
              defaultValue="preset"
            />
          </div>

          {dateMode === "preset" ? (
            <div className="req-toolbar__filter-group">
              <label className="req-toolbar__filter-label">Period</label>
              <FilterSelect
                value={preset}
                onChange={setPreset}
                options={[
                  { value: "all", label: "All Time" },
                  { value: "today", label: "Today" },
                  { value: "thisWeek", label: "This Week" },
                  { value: "thisMonth", label: "This Month" },
                  { value: "thisYear", label: "This Year" },
                ]}
              />
            </div>
          ) : null}

          {dateMode === "single" ? (
            <div className="req-toolbar__filter-group">
              <label className="req-toolbar__filter-label">Date</label>
              <input
                className={`req-toolbar__filter-select${singleDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
                type="date"
                value={singleDate}
                onChange={(e) => setSingleDate(e.target.value)}
              />
            </div>
          ) : null}

          {dateMode === "range" ? (
            <>
              <div className="req-toolbar__filter-group">
                <label className="req-toolbar__filter-label">From</label>
                <input
                  className={`req-toolbar__filter-select${startDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
                  type="date"
                  value={startDate}
                  onChange={(e) => setStartDate(e.target.value)}
                />
              </div>
              <div className="req-toolbar__filter-group">
                <label className="req-toolbar__filter-label">To</label>
                <input
                  className={`req-toolbar__filter-select${endDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
                  type="date"
```

```
                  value={endDate}
                  onChange={(e) => setEndDate(e.target.value)}
                />
              </div>
            </>
          ) : null}

          <button
            type="button"
            className="req-toolbar__reset-btn"
            onClick={resetAndRefresh}
            disabled={refreshing}
            title="Reset filters and refresh"
          >
            <RefreshCw size={14} className={refreshing ? "spin-anim" : ""} />
          </button>
        </div>
      </div>

      <div className="dashboard-card">
        <div className="table-scroll">
        <table className="dashboard-table">
          <thead>
            <tr className="dashboard-table-title-row">
              <th colSpan={6}>
                <div className="dashboard-table-title-wrap">
                  <span className="dashboard-table-title">Requests</span>
                </div>
              </th>
            </tr>
            <tr>
              <th>Request Date</th>
              <th>Requestor</th>
              <th>Request ID</th>
              <th>Type</th>
              <th>Status</th>
              <th />
            </tr>
          </thead>

          <tbody>
            {loading ? (
              [...Array(4)].map((_, i) => (
                <tr key={`sk-${i}`}>
                  <td><span className="skeleton-block skeleton-text" /></td>
                  <td><span className="skeleton-block skeleton-text" /></td>
                  <td><span className="skeleton-block skeleton-text" /></td>
                  <td><span className="skeleton-block skeleton-text" /></td>
                  <td><span className="skeleton-block skeleton-pill" /></td>
                  <td />
                </tr>
              ))
            ) : filtered.length === 0 ? (
              <tr>
                <td colSpan={6}>
                  <div className="dashboard-empty">
                    <p className="dashboard-empty-title">No requests found</p>
                    <p className="dashboard-empty-text">No requests match the current filter.</p>
                  </div>
                </td>
              </tr>
            ) : (
              filtered.map((r) => (
                <tr key={r._id} onClick={() => navigate(`/admin/requests/${r._id}`)} style={{ cursor: "pointer" }}>
                  <td style={{ whiteSpace: "nowrap", color: "var(--text-secondary)" }}>{new Date(r.createdAt).toLocaleDateStr

                  <td>
                    <span style={{ fontWeight: 600 }}>
                      {r.proxyRequestee?.isProxy ? (r.proxyRequestee?.fullName || "Proxy Requestor") : (r.userId?.name || "Un
                    </span>
                    <span className="dashboard-subtext">
                      {r.proxyRequestee?.isProxy ? (r.proxyRequestee?.email || "") : (r.userId?.email || "")}
                    </span>
                    {r.proxyRequestee?.isProxy ? (
                      <span className="dashboard-subtext">Submitted by staff: {r.userId?.name || "Unknown"}</span>
                    ) : null}
```

```
                  </td>

                  <td><span style={{ color: "var(--text-secondary)" }}>{r._id?.slice(-7).toUpperCase()}</span></td>

                  <td>
                    {r.type === "nda"
                      ? `NDA${r.formData?.ndaTypeLabel ? ` - ${r.formData.ndaTypeLabel}` : ""}`
                      : "Agreement"}
                  </td>

                  <td>
                    <span className={statusClass(r.status)}>
                      {prettyStatus(r.status)}
                    </span>
                  </td>

                  <td className="row-caret">›</td>
                </tr>
              ))
            )}
          </tbody>
        </table>
        </div>
      </div>
    </div>
  );
}
```

62. client/src/pages/admin/AdminRequestReview.jsx (1054 lines)

```
import { useEffect, useRef, useState, useMemo, useContext } from "react";
import { useNavigate, useParams } from "react-router-dom";
import { AlertTriangle, Copy, Paperclip, RefreshCw } from "lucide-react";
import { AnimatePresence, motion } from "framer-motion";
import RequestStepper from "../../components/RequestStepper";
import { FIELDS_FILE_SLOTS_CONFIG } from "../../config/fieldsFileSlotsConfig";
import { AuthContext } from "../../context/AuthContext";

import {
  getRequestById,
  updateRequestStatus,
  saveApprovedDocument,
  generateSigningLink,
  adminPhase3Action,
  getSignatureImages,
} from "../../services/requestService";
import { confirmInPage, notify } from "../../utils/inPageFeedback";

import {
  uploadApprovedForm,
  uploadApprovedQrImage,
  deleteStorageFile,
  uploadSignatureImage,
} from "../../services/firebaseStorageService";
import { buildVerificationUrl, generateQrDataUrl } from "../../services/qrService";
import SignaturePad from "../../components/SignaturePad";
import "../../components/RequestForm.css";

const statusPillClass = (s) => {
  if (["nda_approved", "agr_approved"].includes(s)) return "status-pill status-pill--green";
  if (["nda_pending", "agr_pending_1", "agr_pending_2"].includes(s)) return "status-pill status-pill--yellow";
  if (s === "stud_revision_requested") return "status-pill status-pill--orange";
  if (s === "agr_rep_revision_requested") return "status-pill status-pill--violet";
  if (s === "agr_awaiting_rep_signature") return "status-pill status-pill--blue";
  if (s === "agr_declined") return "status-pill status-pill--red";
  return "status-pill";
};

const prettyStatus = (s) => {
  const map = {
    nda_pending:                "Reviewal",
    nda_approved:               "Approved",
    stud_revision_requested:    "Student Revisions",
    agr_pending_1:              "Initial Reviewal",
    agr_awaiting_rep_signature: "Recipient Reviewal",
    agr_pending_2:              "Final Reviewal",
    agr_approved:               "Approved",
    agr_declined:               "Recipient Declined",
    agr_rep_revision_requested: "Recipient Revisions",
  };
  return map[s] || (s ? s.charAt(0).toUpperCase() + s.slice(1).replace(/_/g, " ") : "—");
};

const getRequestFolder = (predocs = []) => {
  for (const r of predocs) {
    if (r.path) {
      const parts = String(r.path).split("/");
      return parts[3] || "";
    }
  }
  return "";
};

// ═══════════════════════════════════════════════════════════════════════════
// NDA review panel (flow: pending → approved | revision_requested)
// ═══════════════════════════════════════════════════════════════════════════
function NdaReviewPanel({ reqData, canProgress, onRequestUpdated }) {
  const { user } = useContext(AuthContext);
  const [remarks, setRemarks] = useState(reqData.remarks || "");
  const [activeAction, setActiveAction] = useState(null); // "approve" | "revision" | null
  const adminSigRef = useRef(null);

  const cfg = useMemo(() => {
```

```
    if (reqData.type === "nda") return FIELDS_FILE_SLOTS_CONFIG.nda?.[reqData.formData?.ndaType];
    return null;
  }, [reqData]);

  const isPending = reqData?.status === "nda_pending";
  const isRevision = reqData?.status === "stud_revision_requested";
  const isApproved = reqData?.status === "nda_approved";
  const [confirmAction, setConfirmAction] = useState(null); // "approve" | "revision" | null


  const isProxy = reqData?.proxyRequestee?.isProxy;

  const handleUpdate = async (status) => {
    if (status === "nda_approved") {
      if (!isProxy && (!adminSigRef.current || adminSigRef.current.isEmpty())) {
        notify("Please draw your e-signature before approving.", { type: "warning" });
        return;
      }
    }
    if (status === "stud_revision_requested") {
      if (!remarks.trim()) {
        notify("Please enter remarks before requesting a revision.", { type: "warning" });
        return;
      }
    }
    setConfirmAction(null);
    setActiveAction(status === "nda_approved" ? "approve" : "revision");
    try {
      if (status === "nda_approved") {
        // For F2F proxy requests, skip digital signatures (wet signature will be applied on print)
        if (isProxy) {
          const updateRes = await updateRequestStatus(reqData._id, {
            status,
            remarks,
          });
          const updated = updateRes.request;
          if (!updated?.serialNo) throw new Error("Serial number missing");
          const verificationUrl = buildVerificationUrl(updated.serialNo);
          const qrDataUrl = await generateQrDataUrl(verificationUrl);
          const studentName = reqData.proxyRequestee?.fullName || "Proxy Requestor";
          const requestFolder = getRequestFolder(reqData.predocs);
          if (!requestFolder) throw new Error("Could not determine request folder");
          await uploadApprovedQrImage(qrDataUrl, reqData.type, studentName, requestFolder);
          const docReq = {
            ...updated,
            userId: reqData.userId,
            studentSigDataUrl: null,
            adminSigDataUrl: null,
            approverName: user?.name || "",
            verificationUrl,
            qrDataUrl,
          };
          const { generateApprovedPDF } = await import("../../config/documentTemplates");
          const pdfBlob = await generateApprovedPDF(docReq);
          const t = reqData.formData?.ndaType || "general";
          const approvedFileName = `NDA_${t}_Approved_${Date.now()}.pdf`;
          const uploaded = await uploadApprovedForm(
            pdfBlob,
            reqData.type,
            studentName,
            requestFolder,
            approvedFileName
          );
          await saveApprovedDocument(reqData._id, { ...uploaded, verificationUrl });
          notify("Approved! The document is ready for printing and physical signature.", { type: "success", title: "Approved
        } else {
          const adminSigDataUrl = adminSigRef.current.getDataUrl();

          // Upload admin signature to Firebase
          const studentName = reqData.userId?.name || "Unknown Student";
          const requestFolder = getRequestFolder(reqData.predocs);

          const { url: adminSigFirebaseUrl, path: adminSigFirebasePath } = await uploadSignatureImage(
            adminSigDataUrl,
            "nda",
            studentName,
```

```
            requestFolder || "unknown",
            "admin_sig.png"
          );

          const updateRes = await updateRequestStatus(reqData._id, {
            status,
            remarks,
            adminSigUrl: adminSigFirebaseUrl,
            adminSigPath: adminSigFirebasePath,
          });
          const updated = updateRes.request;

          if (!updated?.serialNo) throw new Error("Serial number missing");

          const verificationUrl = buildVerificationUrl(updated.serialNo);
          const qrDataUrl = await generateQrDataUrl(verificationUrl);

          if (!requestFolder) throw new Error("Could not determine request folder");

          await uploadApprovedQrImage(qrDataUrl, reqData.type, studentName, requestFolder);

          // Fetch signature images via proxy to avoid CORS
          const sigImages = await getSignatureImages(reqData._id);

          const docReq = {
            ...updated,
            userId: reqData.userId,
            studentSigDataUrl: sigImages.studentSig,
            adminSigDataUrl: sigImages.adminSig || adminSigDataUrl,
            approverName: user?.name || "",
            verificationUrl,
            qrDataUrl,
          };
          const { generateApprovedPDF } = await import("../../config/documentTemplates");
          const pdfBlob = await generateApprovedPDF(docReq);
          const t = reqData.formData?.ndaType || "general";
          const approvedFileName = `NDA_${t}_Approved_${Date.now()}.pdf`;
          const uploaded = await uploadApprovedForm(
            pdfBlob,
            reqData.type,
            studentName,
            requestFolder,
            approvedFileName
          );

          await saveApprovedDocument(reqData._id, { ...uploaded, verificationUrl });

          notify("Approved and document generated!", { type: "success", title: "Approved" });
        }
      } else {
        await updateRequestStatus(reqData._id, { status, remarks });
        notify("Student revision requested.", { type: "success", title: "Revision Requested" });
      }
      await onRequestUpdated?.({ withToast: true });
    } catch (err) {
      notify(err.response?.data?.message || err.message || "Failed to update request", { type: "error" });
    } finally {
      setActiveAction(null);
    }
  };

  return (
    <>
      <div className="review-section">
        <h4 className="review-section-title">Form Data</h4>
        {cfg?.fields?.length ? (
          cfg.fields.map((f) => (
            <div key={f.name} className="review-field">
              <span className="review-field-label">{f.label}</span>
              <div className="review-info-box">
                {String(reqData.formData?.[f.name] ?? "") || (
                  <span className="review-info-box--muted">—</span>
                )}
              </div>
            </div>
          ))
```

```
          ) : (
            <pre style={{
              background: "var(--surface-2)",
              padding: "10px",
              borderRadius: "var(--radius-md)",
              fontSize: "12px",
              margin: 0,
            }}>
              {JSON.stringify(reqData.formData || {}, null, 2)}
            </pre>
          )}
        </div>

        <div className="review-section">
          <h4 className="review-section-title">Attachments</h4>
          {(() => {
            const initialDocs = reqData.predocs?.filter((f) => !f.isResubmission) ?? [];
            return initialDocs.length ? (
              <div style={{ display: "flex", flexDirection: "column", gap: "8px" }}>
                {initialDocs.map((f, idx) => (
                  <a key={idx} href={f.url} target="_blank" rel="noreferrer" className="review-file-link">
                    <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> {f.requirementLabel || f.origName || `File ${
                  </a>
                ))}
              </div>
            ) : (
              <div className="review-info-box">
                <span className="review-info-box--muted">No files.</span>
              </div>
            );
          })()}
        </div>

        {(() => {
          const rounds = {};
          (reqData.predocs || []).filter((f) => f.isResubmission).forEach((f) => {
            const r = f.resubmissionRound || 1;
            if (!rounds[r]) rounds[r] = [];
            rounds[r].push(f);
          });
          return Object.keys(rounds).sort((a, b) => a - b).map((round) => (
            <div key={round} className="review-section">
              <h4 className="review-section-title">
                Resubmitted Attachments{Object.keys(rounds).length > 1 ? ` (Round ${round})` : ""}
              </h4>
              <div style={{ display: "flex", flexDirection: "column", gap: "8px" }}>
                {rounds[round].map((f, idx) => (
                  <a key={idx} href={f.url} target="_blank" rel="noreferrer" className="review-file-link">
                    <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> {f.requirementLabel || f.origName || `File ${
                  </a>
                ))}
              </div>
            </div>
          ));
        })()}

        {isRevision && (
          <div className="review-section">
            <h4 className="review-section-title">Remarks</h4>
            <div className="review-info-box">
              {reqData.remarks || <span className="review-info-box--muted">No remarks provided.</span>}
            </div>
          </div>
        )}

        {isApproved && (
          <div className="review-section">
            <h4 className="review-section-title">Approved Request Form</h4>
            {reqData.postdocs?.url ? (
              <a href={reqData.postdocs.url} target="_blank" rel="noreferrer" className="review-file-link">
                <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> Approved NDA Document
              </a>
            ) : (
              <div className="review-info-box">
                <span className="review-info-box--muted">No approved document uploaded.</span>
              </div>
```

```
            )}
            {reqData.adminSignedAt && (
              <span style={{ display: "block", marginTop: 6, fontSize: 11, color: "var(--text-muted)" }}>
                Admin signed on {new Date(reqData.adminSignedAt).toLocaleString("en-US", { year: "numeric", month: "short", da
              </span>
            )}
          </div>
        )}

        {/* Student E-Signature */}
        {reqData.studentSigUrl && (
          <div className="review-section">
            <h4 className="review-section-title">Student E-Signature</h4>
            <div className="review-sig-wrap">
              <img src={reqData.studentSigUrl} alt="Student signature" className="review-sig-img" width="560" height="180" loa
              <span className="review-sig-name">{reqData.userId?.name}</span>
              {reqData.studentSignedAt && (
                <span style={{ fontSize: 11, color: "var(--text-muted)" }}>
                  Signed on {new Date(reqData.studentSignedAt).toLocaleString("en-US", { year: "numeric", month: "short", day:
                </span>
              )}
            </div>
          </div>
        )}

        {canProgress && isPending && !isProxy && (
          <div className="review-section">
            <h4 className="review-section-title">Your E-Signature (Admin) *</h4>
            <p style={{ fontSize: "13px", color: "var(--text-muted)", margin: "0 0 8px 0" }}>
              Draw your signature below to sign off on the NDA approval.
            </p>
            <SignaturePad ref={adminSigRef} height={160} />
          </div>
        )}

        {canProgress && isPending && isProxy && (
          <div className="info-banner info-banner--info">
            <strong>F2F Walk-in (Proxy)</strong>
            <p>This is a face-to-face proxy request. Digital signatures are bypassed. The document will be printed for a physi
          </div>
        )}

        {canProgress && isPending && (
          <div className="review-section">
            <h4 className="review-section-title">Remarks</h4>
            <textarea
              value={remarks}
              onChange={(e) => setRemarks(e.target.value)}
              rows={4}
              className="review-textarea"
            />
          </div>
        )}

        {canProgress && isPending && (
          <div className="review-actions">
            <button
              onClick={() => {
                if (!isProxy && (!adminSigRef.current || adminSigRef.current.isEmpty())) {
                  notify("Please draw your e-signature before approving.", { type: "warning" });
                  return;
                }
                setConfirmAction("approve");
              }}
              disabled={!!activeAction}
              className="review-btn-primary"
            >
              {activeAction === "approve" ? "Generating..." : "Approve"}
            </button>
            <button
              onClick={() => {
                if (!remarks.trim()) {
                  notify("Please enter remarks before requesting a revision.", { type: "warning" });
                  return;
                }
                setConfirmAction("revision");
```

```
            }}
            disabled={!!activeAction}
            className="review-btn-secondary"
          >
            {activeAction === "revision" ? "Submitting..." : "Request Student Revision"}
          </button>
        </div>
      )}

      <AnimatePresence>
        {confirmAction && (
          <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
            <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12,
              <h3 className="modal-title">
                {confirmAction === "approve" ? "Approve NDA Request" : "Request Student Revision"}
              </h3>
              <p className="modal-text">
                {confirmAction === "approve"
                  ? "This will finalize and generate the approved NDA document."
                  : "The student will be notified to revise and resubmit their request."}
              </p>
              <div className="modal-actions">
                <button className="ui-btn ui-btn--secondary" onClick={() => setConfirmAction(null)} disabled={!!activeAction
                <button className="ui-btn ui-btn--primary" onClick={() => handleUpdate(confirmAction === "approve" ? "nda_ap
                  {confirmAction === "approve" ? "Approve" : "Confirm"}
                </button>
              </div>
            </motion.div>
          </motion.div>
        )}
      </AnimatePresence>

      {!canProgress && (
        <div className="info-banner info-banner--info">
          <strong>Read-only access</strong>
          <p>Staff can view request details but only Admin can move, approve, or sign this request.</p>
        </div>
      )}
    </>
  );
}

// ═══════════════════════════════════════════════════════════════════════════
// Agreement review panel (multi-phase)
// ═══════════════════════════════════════════════════════════════════════════
function AgreementReviewPanel({ reqData, canProgress, onRequestUpdated }) {
  const { user } = useContext(AuthContext);
  const [remarks, setRemarks] = useState(reqData.remarks || "");
  const [signingLink, setSigningLink] = useState(() => {
    // Reconstruct signing link from persisted token (always show once generated)
    if (reqData.signingToken) {
      return `${window.location.origin}/sign/${reqData.signingToken}`;
    }
    return "";
  });
  const [signingLinkExpiry, setSigningLinkExpiry] = useState(() => reqData.signingTokenExpiresAt || null);
  const [generating, setGenerating] = useState(false);
  const [regenerating, setRegenerating] = useState(false);
  const [phase1Revising, setPhase1Revising] = useState(false);
  const [approving, setApproving] = useState(false);
  const [phase3Revising, setPhase3Revising] = useState(false);
  const [confirmAction, setConfirmAction] = useState(null); // "phase1_approve" | "phase1_revision" | "phase3_approve" | "ph
  const adminSigRef = useRef(null);

  const { status } = reqData;
  const cfg = FIELDS_FILE_SLOTS_CONFIG.agreement;
  const _fd = reqData.formData || {};
  const _mid = String(_fd.repMiddleInitial || "").trim();
  const _midInitial = _mid ? `${_mid[0].toUpperCase()}.` : "";
  const requestedRepName = [_fd.repFirstName, _midInitial, _fd.repLastName].filter(Boolean).join(" ").replace(/\s+/g, " ").t

  const handlePhase1Approve = async () => {
    setConfirmAction(null);
    setGenerating(true);
    try {
      const res = await generateSigningLink(reqData._id);
```

```
      const link = `${window.location.origin}/sign/${res.signingToken}`;
      setSigningLink(link);
      setSigningLinkExpiry(res.signingTokenExpiresAt || null);
      await onRequestUpdated?.();
      notify("Approved. Signing link sent to the representative's email.", { type: "success", title: "Approved" });
    } catch (err) {
      notify(err.response?.data?.message || err.message || "Failed to generate signing link", { type: "error" });
    } finally {
      setGenerating(false);
    }
  };

  const handlePhase1Revision = async () => {
    setConfirmAction(null);
    setPhase1Revising(true);
    try {
      await updateRequestStatus(reqData._id, { status: "stud_revision_requested", remarks });
      notify("Student revision requested.", { type: "success", title: "Revision Requested" });
      await onRequestUpdated?.({ withToast: true });
    } catch (err) {
      notify(err.response?.data?.message || "Failed", { type: "error" });
    } finally {
      setPhase1Revising(false);
    }
  };

  const handleResendLink = async () => {
    const confirmed = await confirmInPage({
      title: "Resend Signing Link",
      message: "This will generate a new signing link and send it to the representative's email. The old link will be invali
      confirmText: "Resend",
      cancelText: "Cancel",
      tone: "warning",
    });
    if (!confirmed) return;
    setRegenerating(true);
    try {
      const res = await generateSigningLink(reqData._id);
      const link = `${window.location.origin}/sign/${res.signingToken}`;
      setSigningLink(link);
      setSigningLinkExpiry(res.signingTokenExpiresAt || null);
      await onRequestUpdated?.();
      notify("New signing link sent to the representative's email.", { type: "success" });
    } catch (err) {
      notify(err.response?.data?.message || err.message || "Failed to resend signing link", { type: "error" });
    } finally {
      setRegenerating(false);
    }
  };

  const handlePhase3Approve = async () => {
    setConfirmAction(null);
    setApproving(true);
    try {
      const actionRes = await adminPhase3Action(reqData._id, { action: "approve" });
      const updated = actionRes.request;

      if (!updated?.serialNo) throw new Error("Serial number missing");

      const verificationUrl = buildVerificationUrl(updated.serialNo);
      const qrDataUrl = await generateQrDataUrl(verificationUrl);
      const adminSigDataUrl = adminSigRef.current.getDataUrl();
      const studentName = reqData.userId?.name || "Unknown Student";
      const requestFolder = getRequestFolder(reqData.predocs);
      if (!requestFolder) throw new Error("Could not determine request folder");

      await uploadApprovedQrImage(qrDataUrl, "agreement", studentName, requestFolder);

      const { authorizerSig, repSig } = await getSignatureImages(reqData._id);

      const docReq = {
        ...updated,
        userId: reqData.userId,
        repInfo: reqData.repInfo,
        authorizerSigUrl: authorizerSig,
        repSigUrl: repSig,
```

```
        adminSigDataUrl,
        approverName: user?.name || "",
        verificationUrl,
        qrDataUrl,
      };

      const { generateApprovedPDF } = await import("../../config/documentTemplates");
      const pdfBlob = await generateApprovedPDF(docReq);
      const approvedFileName = `Agreement_Approved_${Date.now()}.pdf`;
      const uploaded = await uploadApprovedForm(
        pdfBlob,
        "agreement",
        studentName,
        requestFolder,
        approvedFileName
      );

      await saveApprovedDocument(reqData._id, { ...uploaded, verificationUrl });

      await deleteStorageFile(reqData.authorizerSigPath);
      await deleteStorageFile(reqData.repSigPath);

      notify("Agreement fully approved and document generated!", { type: "success", title: "Approved" });
      await onRequestUpdated?.({ withToast: true });
      setApproving(false);
    } catch (err) {
      notify(err.response?.data?.message || err.message || "Failed to approve", { type: "error" });
    } finally {
      setApproving(false);
    }
  };

  const handlePhase3RepRevision = async () => {
    setConfirmAction(null);
    setPhase3Revising(true);
    try {
      const res = await adminPhase3Action(reqData._id, { action: "rep_revision_requested", remarks });
      const newLink = `${window.location.origin}/sign/${res.signingToken || res.request.signingToken}`;
      setSigningLink(newLink);
      setSigningLinkExpiry(res.signingTokenExpiresAt || null);
      await onRequestUpdated?.({ withToast: true });
      notify("Recipient revision requested. A new signing link has been sent to the representative's email.", { type: "succe
    } catch (err) {
      notify(err.response?.data?.message || "Failed", { type: "error" });
    } finally {
      setPhase3Revising(false);
    }
  };

  return (
    <>
      {/* Requestor Attachments */}
      <div className="review-section">
        <h4 className="review-section-title">Requestor Attachments</h4>
        {(() => {
          const initialDocs = reqData.predocs?.filter((f) => !f.isResubmission) ?? [];
          return initialDocs.length ? (
            <div style={{ display: "flex", flexDirection: "column", gap: "8px" }}>
              {initialDocs.map((f, idx) => (
                <a key={idx} href={f.url} target="_blank" rel="noreferrer" className="review-file-link">
                  <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> {f.requirementLabel || f.origName || `File ${
                </a>
              ))}
            </div>
          ) : (
            <div className="review-info-box">
              <span className="review-info-box--muted">No files.</span>
            </div>
          );
        })()}
      </div>

      {(() => {
        const rounds = {};
        (reqData.predocs || []).filter((f) => f.isResubmission).forEach((f) => {
          const r = f.resubmissionRound || 1;
```

```
            if (!rounds[r]) rounds[r] = [];
            rounds[r].push(f);
          });
          return Object.keys(rounds).sort((a, b) => a - b).map((round) => (
            <div key={round} className="review-section">
              <h4 className="review-section-title">
                Resubmitted Attachments{Object.keys(rounds).length > 1 ? ` (Round ${round})` : ""}
              </h4>
              <div style={{ display: "flex", flexDirection: "column", gap: "8px" }}>
                {rounds[round].map((f, idx) => (
                  <a key={idx} href={f.url} target="_blank" rel="noreferrer" className="review-file-link">
                    <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> {f.requirementLabel || f.origName || `File ${
                  </a>
                ))}
              </div>
            </div>
          ));
        })()}

        {/* Representative */}
        <div className="review-section">
          <h4 className="review-section-title">Representative</h4>
          <div className="review-field">
            <span className="review-field-label">Full Name</span>
            <div className="review-info-box">
              {reqData.repInfo?.name || (requestedRepName !== "—" ? requestedRepName : <span className="review-info-box--muted
            </div>
          </div>
          {cfg.fields
            .filter((f) => !["repFirstName", "repMiddleInitial", "repLastName"].includes(f.name))
            .map((f) => (
              <div key={f.name} className="review-field">
                <span className="review-field-label">{f.label}</span>
                <div className="review-info-box">
                  {String(reqData.formData?.[f.name] ?? "") || (
                    <span className="review-info-box--muted">—</span>
                  )}
                </div>
              </div>
            ))}
        </div>

        {/* Representative Attachments (after rep signs) */}
        {reqData.repInfo?.govIdDoc?.url && (
          <div className="review-section">
            <h4 className="review-section-title">Representative Attachments</h4>
            <a href={reqData.repInfo.govIdDoc.url} target="_blank" rel="noreferrer" className="review-file-link">
              <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> Government-Issued ID
            </a>
          </div>
        )}

        {/* Authorization Letter */}
        {status !== "agr_approved" && (
        <div className="review-section">
          <div style={{ padding: "20px 24px", background: "var(--surface-2)", border: "1px solid var(--border)", borderRadius:
            {/* Requestee date — left */}
            <div style={{ fontSize: 13, color: "var(--text-secondary)", marginBottom: 18 }}>
              {reqData.createdAt ? new Date(reqData.createdAt).toLocaleDateString("en-US", { year: "numeric", month: "long", d
            </div>
            {/* Authorization paragraph — centered */}
            <p style={{ textAlign: "center", fontSize: 13.5, color: "var(--text-primary)", lineHeight: 1.75, margin: "0 0 24px
              I, <strong>{reqData.userId?.name || "—"}</strong>, hereby authorize{" "}
              <strong>{requestedRepName}</strong> to act as my representative and sign on my behalf
              in connection with this agreement, as governed by the Data Protection Office.
            </p>
            {/* Requestee signature block — centered */}
            <div style={{ display: "flex", flexDirection: "column", alignItems: "center", gap: 6 }}>
              {reqData.authorizerSigUrl && (
                <img src={reqData.authorizerSigUrl} alt="Authorizer signature" style={{ maxHeight: 72, display: "block" }} loa
              )}
              <div style={{ width: 220, borderTop: "1px solid var(--border-strong)", textAlign: "center", paddingTop: 6, fontS
                {reqData.userId?.name || "—"}
              </div>
              {reqData.studentSignedAt && (
                <span style={{ fontSize: 11, color: "var(--text-muted)" }}>
```

```
                Signed on {new Date(reqData.studentSignedAt).toLocaleString("en-US", { year: "numeric", month: "short", day:
              </span>
            )}
          </div>

          {/* Representative agreement section — hidden during rep revision, shown once rep re-submits */}
          {reqData.repSigUrl && status !== "agr_rep_revision_requested" && (
            <>
              <hr style={{ border: "none", borderTop: "1px solid var(--border)", margin: "24px 0" }} />
              {/* Rep agreement date — left */}
              <div style={{ fontSize: 13, color: "var(--text-secondary)", marginBottom: 18 }}>
                {reqData.updatedAt ? new Date(reqData.updatedAt).toLocaleDateString("en-US", { year: "numeric", month: "long
              </div>
              {/* Rep agreement paragraph — centered */}
              <p style={{ textAlign: "center", fontSize: 13.5, color: "var(--text-primary)", lineHeight: 1.75, margin: "0 0
                I, <strong>{reqData.repInfo?.name || "—"}</strong>, hereby confirm my agreement to act as the authorized rep
                of <strong>{reqData.userId?.name || "—"}</strong> and sign on their behalf,
                as governed by the Data Protection Office.
              </p>
              {/* Representative signature block — centered */}
              <div style={{ display: "flex", flexDirection: "column", alignItems: "center", gap: 6 }}>
                <img src={reqData.repSigUrl} alt="Representative signature" style={{ maxHeight: 72, display: "block" }} load
                <div style={{ width: 220, borderTop: "1px solid var(--border-strong)", textAlign: "center", paddingTop: 6, f
                  {reqData.repInfo?.name || "—"}
                </div>
                {reqData.repSignedAt && (
                  <span style={{ fontSize: 11, color: "var(--text-muted)" }}>
                    Signed on {new Date(reqData.repSignedAt).toLocaleString("en-US", { year: "numeric", month: "short", day:
                  </span>
                )}
              </div>
            </>
          )}
        </div>
      </div>
      )}

      {/* Remarks display */}
      {(status === "stud_revision_requested" || status === "agr_rep_revision_requested") &&
        reqData.remarks && (
          <div className="review-section">
            <h4 className="review-section-title">
              {status === "agr_rep_revision_requested" ? "Remarks For Representative" : "Remarks"}
            </h4>
            <div className="review-info-box">{reqData.remarks}</div>
          </div>
        )}

      {/* Final approved document */}
      {status === "agr_approved" && (
        <div className="review-section">
          <h4 className="review-section-title">Approved Agreement</h4>
          {reqData.postdocs?.url ? (
            <a href={reqData.postdocs.url} target="_blank" rel="noreferrer" className="review-file-link">
              <Paperclip size={14} strokeWidth={1.8} aria-hidden="true" /> Approved Agreement Document
            </a>
          ) : (
            <div className="review-info-box">
              <span className="review-info-box--muted">No document uploaded yet.</span>
            </div>
          )}
        </div>
      )}

      {/* Rep rejected */}
      {status === "agr_declined" && (
        <div className="info-banner info-banner--danger">
          <strong>Recipient Declined</strong>
          <p>The recipient declined to sign. This request lifecycle has ended.</p>
        </div>
      )}

      {/* Generated signing link (visible only while awaiting rep action) */}
      {signingLink && (status === "agr_awaiting_rep_signature" || status === "agr_rep_revision_requested") && (
        <div className="signing-link-box">
          <p className="signing-link-title">Representative Signing Link</p>
```

```
758           <div className="signing-link-row">
759             <span className="signing-link-code">Representative signing link ready — click to copy</span>
760             <button
761               className="signing-link-copy-btn"
762               onClick={() => {
763                 navigator.clipboard.writeText(signingLink);
764               }}
765               title="Copy link"
766             >
767               <Copy size={14} strokeWidth={2} />
768             </button>
769           </div>
770           {signingLinkExpiry && (
771             <div style={{ marginTop: 6, fontSize: 12, color: new Date(signingLinkExpiry) < new Date() ? "#dc2626" : "var(--t
772               {new Date(signingLinkExpiry) < new Date()
773                 ? <span style={{ display: "inline-flex", alignItems: "center", gap: 6 }}><AlertTriangle size={14} strokeWidt
774                 : `Expires: ${new Date(signingLinkExpiry).toLocaleDateString("en-PH", { year: "numeric", month: "short", day
775             </div>
776           )}
777           {canProgress && (
778             <button
779               onClick={handleResendLink}
780               disabled={regenerating || !signingLinkExpiry || new Date(signingLinkExpiry) >= new Date()}
781               className="review-btn-secondary"
782               style={{ marginTop: 10, opacity: (!signingLinkExpiry || new Date(signingLinkExpiry) >= new Date()) ? 0.5 : 1 }
783               title={(!signingLinkExpiry || new Date(signingLinkExpiry) >= new Date()) ? "Link is still active — not yet exp
784             >
785               {regenerating ? "Sending…" : <span style={{ display: "inline-flex", alignItems: "center", gap: 6 }}><RefreshCw
786             </button>
787           )}
788         </div>
789       )}
790
791       {/* ■■ Phase 1 actions ■■ */}
792       {canProgress && status === "agr_pending_1" && !signingLink && (
793         <>
794           <div className="review-section">
795             <h4 className="review-section-title">Remarks</h4>
796             <textarea
797               value={remarks}
798               onChange={(e) => setRemarks(e.target.value)}
799               rows={3}
800               className="review-textarea"
801               placeholder=""
802             />
803           </div>
804           <div className="review-actions">
805             <button
806               onClick={() => setConfirmAction("phase1_approve")}
807               disabled={generating || phase1Revising}
808               className="review-btn-primary"
809             >
810               {generating ? "Generating link..." : "Approve & Generate Signing Link"}
811             </button>
812             <button
813               onClick={() => {
814                 if (!remarks.trim()) {
815                   notify("Please enter remarks before requesting a revision.", { type: "warning" });
816                   return;
817                 }
818                 setConfirmAction("phase1_revision");
819               }}
820               disabled={generating || phase1Revising}
821               className="review-btn-secondary"
822             >
823               {phase1Revising ? "Submitting..." : "Request Student Revision"}
824             </button>
825           </div>
826         </>
827       )}
828
829       {/* ■■ Phase 2: waiting on rep ■■ */}
830       {status === "agr_awaiting_rep_signature" && !signingLink && (
831         <div className="info-banner info-banner--info">
832           <strong>Waiting for Representative</strong>
833           <p>
```

```
            The signing link has been generated. Share it with the representative.
            This page will update once they submit or decline.
          </p>
        </div>
      )}

      {/* ■■ Phase 3 actions ■■ */}
      {canProgress && status === "agr_pending_2" && (
        <>
          <div className="review-section">
            <h4 className="review-section-title">Your E-Signature (Admin) *</h4>
            <p style={{ fontSize: "13px", color: "var(--text-muted)", margin: "0 0 8px 0" }}>
              Draw your signature below to sign off on the final approval.
            </p>
            <SignaturePad ref={adminSigRef} height={160} />
          </div>

          <div className="review-section">
            <h4 className="review-section-title">Remarks</h4>
            <textarea
              value={remarks}
              onChange={(e) => setRemarks(e.target.value)}
              rows={3}
              className="review-textarea"
              placeholder=""
            />
          </div>

          <div className="review-actions">
            <button
              onClick={() => {
                if (!adminSigRef.current || adminSigRef.current.isEmpty()) {
                  notify("Please draw your e-signature before approving.", { type: "warning" });
                  return;
                }
                setConfirmAction("phase3_approve");
              }}
              disabled={approving || phase3Revising}
              className="review-btn-primary"
            >
              {approving ? "Generating final document..." : "Final Approve & Sign"}
            </button>
            <button
              onClick={() => {
                if (!remarks.trim()) {
                  notify("Please enter remarks explaining what the representative needs to revise.", { type: "warning" });
                  return;
                }
                setConfirmAction("phase3_revision");
              }}
              disabled={approving || phase3Revising}
              className="review-btn-secondary"
            >
              {phase3Revising ? "Submitting..." : "Request Representative Revision"}
            </button>
          </div>
        </>
      )}

      {/* ■■ Rep revision requested ■■ */}
      {status === "agr_rep_revision_requested" && !signingLink && (
        <div className="info-banner info-banner--warning">
          <strong>Recipient Revision Pending</strong>
          <p>
            A new signing link has been sent to the representative's email so they can resubmit.
          </p>
        </div>
      )}

      <AnimatePresence>
        {confirmAction && (
          <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
            <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12,
              <h3 className="modal-title">
                {confirmAction === "phase1_approve" && "Approve & Send for Signing"}
                {confirmAction === "phase1_revision" && "Request Student Revision"}
```

```
                {confirmAction === "phase3_approve" && "Final Approve Agreement"}
                {confirmAction === "phase3_revision" && "Request Recipient Revision"}
              </h3>
              <p className="modal-text">
                {confirmAction === "phase1_approve" && "This will approve the request and generate a signing link for the re
                {confirmAction === "phase1_revision" && "The student will be notified to revise and resubmit their request."
                {confirmAction === "phase3_approve" && "This will finalize the agreement and generate the approved document.
                {confirmAction === "phase3_revision" && "A new signing link will be generated for the representative to resu
              </p>
              <div className="modal-actions">
                <button
                  className="ui-btn ui-btn--secondary"
                  onClick={() => setConfirmAction(null)}
                  disabled={generating || phase1Revising || approving || phase3Revising}
                >
                  Cancel
                </button>
                <button
                  className="ui-btn ui-btn--primary"
                  onClick={() => {
                    if (confirmAction === "phase1_approve") handlePhase1Approve();
                    else if (confirmAction === "phase1_revision") handlePhase1Revision();
                    else if (confirmAction === "phase3_approve") handlePhase3Approve();
                    else if (confirmAction === "phase3_revision") handlePhase3RepRevision();
                  }}
                  disabled={generating || phase1Revising || approving || phase3Revising}
                >
                  {(confirmAction === "phase1_approve" || confirmAction === "phase3_approve") ? "Approve" : "Confirm"}
                </button>
              </div>
            </motion.div>
          </motion.div>
        )}
      </AnimatePresence>

      {!canProgress && (
        <div className="info-banner info-banner--info">
          <strong>Read-only access</strong>
          <p>Staff can view request details but only Admin can move, approve, or sign this request.</p>
        </div>
      )}
    </>
  );
}

// ═══════════════════════════════════════════════════════════════════════════
// Main component
// ═══════════════════════════════════════════════════════════════════════════
export default function AdminRequestReview() {
  const { id } = useParams();
  const navigate = useNavigate();
  const { user } = useContext(AuthContext);

  const [reqData, setReqData] = useState(null);
  const loadRequest = async () => {
    try {
      const r = await getRequestById(id);
      setReqData(r);
    } catch (err) {
      notify(err.response?.data?.message || "Failed to load request", { type: "error" });
      navigate("/admin/requests");
    }
  };

  useEffect(() => {
    loadRequest();
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [id, navigate]);

  if (!reqData) return null;

  const isAgreement = reqData.type === "agreement";
  const canProgress = user?.role === "admin";

  const requestTitle =
    reqData.type === "agreement"
```

```
       ? "Agreement Request"
       : `NDA Request${reqData.formData?.ndaTypeLabel ? ` — ${reqData.formData.ndaTypeLabel}` : ""}`;

  return (
    <div className="review-page">
      <div className="review-card">
        <div className="request-form-header">
          <button
            className="request-form-back"
            onClick={() => {
              if (window.history.length > 1) navigate(-1);
              else navigate("/admin");
            }}
          >
            ‹ Back
          </button>
          <h2 className="request-form-title">{requestTitle}</h2>
          <div />
        </div>

        <div className="review-header">
          <div style={{ display: "flex", alignItems: "center", justifyContent: "space-between", gap: 12, flexWrap: "wrap", w
            <span className={statusPillClass(reqData.status)}>{prettyStatus(reqData.status)}</span>
            <span className="review-meta-row" style={{ marginLeft: "auto" }}>
              <b>Request Date:</b> {new Date(reqData.createdAt).toLocaleDateString("en-US", { year: "numeric", month: "short
            </span>
          </div>
        </div>

        <div className="review-section">
          <RequestStepper status={reqData.status} type={reqData.type} />
        </div>

        <div className="review-section">
          <h4 className="review-section-title">Requestor</h4>
          <div className="review-info-box">
            <div style={{ fontWeight: 600 }}>
              {reqData.proxyRequestee?.isProxy ? (reqData.proxyRequestee?.fullName || "Proxy Requestor") : (reqData.userId?.
            </div>
            <div style={{ fontSize: "12px", color: "var(--text-muted)", marginTop: "2px" }}>
              {reqData.proxyRequestee?.isProxy ? (reqData.proxyRequestee?.email || "") : (reqData.userId?.email || "")}
            </div>
            {reqData.proxyRequestee?.isProxy ? (
              <div style={{ fontSize: "12px", color: "var(--text-muted)", marginTop: "2px" }}>
                Submitted by staff: {reqData.userId?.name || "Unknown"}
              </div>
            ) : null}
            {reqData.proxyRequestee?.isProxy && reqData.proxyRequestee?.idNumber ? (
              <div style={{ fontSize: "12px", color: "var(--text-muted)", marginTop: "2px" }}>
                ID: {reqData.proxyRequestee.idNumber}
              </div>
            ) : null}
            {reqData.proxyRequestee?.isProxy && reqData.proxyRequestee?.departmentOrOrganization ? (
              <div style={{ fontSize: "12px", color: "var(--text-muted)", marginTop: "2px" }}>
                Department/Org: {reqData.proxyRequestee.departmentOrOrganization}
              </div>
            ) : null}
          </div>
        </div>

        {isAgreement ? (
          <AgreementReviewPanel reqData={reqData} canProgress={canProgress} onRequestUpdated={loadRequest} />
        ) : (
          <NdaReviewPanel reqData={reqData} canProgress={canProgress} onRequestUpdated={loadRequest} />
        )}
      </div>
    </div>
  );
}
```

63. client/src/pages/admin/AdminReports.jsx (547 lines)

```
import { useEffect, useMemo, useState } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { RefreshCw, Search, Filter, X } from "lucide-react";
import { getAllRequests } from "../../services/requestService";
import FilterSelect from "../../components/FilterSelect";
import { getAuditLogs } from "../../services/auditService";

const STATUS_LABEL = {
  nda_pending: "Reviewal",
  nda_approved: "Approved",
  stud_revision_requested: "Student Revisions",
  agr_pending_1: "Initial Reviewal",
  agr_awaiting_rep_signature: "Recipient Reviewal",
  agr_pending_2: "Final Reviewal",
  agr_approved: "Approved",
  agr_declined: "Recipient Declined",
  agr_rep_revision_requested: "Recipient Revisions",
};

const prettyStatus = (s) =>
  STATUS_LABEL[s] || (s ? s.charAt(0).toUpperCase() + s.slice(1).replace(/_/g, " ") : "—");

const statusPillClass = (s) => {
  if (["nda_approved", "agr_approved"].includes(s)) return "status-pill status-pill--green";
  if (["nda_pending", "agr_pending_1", "agr_pending_2"].includes(s)) return "status-pill status-pill--yellow";
  if (s === "stud_revision_requested") return "status-pill status-pill--orange";
  if (s === "agr_rep_revision_requested") return "status-pill status-pill--violet";
  if (s === "agr_awaiting_rep_signature") return "status-pill status-pill--blue";
  if (s === "agr_declined") return "status-pill status-pill--red";
  return "status-pill";
};

const STATUS_FILTER_OPTIONS = [
  { value: "", label: "All Statuses" },
  { value: "nda_pending,agr_pending_1,agr_pending_2", label: "Reviewal" },
  { value: "nda_approved,agr_approved", label: "Approved" },
  { value: "stud_revision_requested", label: "Student Revisions" },
  { value: "agr_rep_revision_requested", label: "Recipient Revisions" },
  { value: "agr_declined", label: "Recipient Declined" },
  { value: "agr_awaiting_rep_signature", label: "Recipient Reviewal" },
];



const TIMELINE_STEPS = {
  request_created:           { label: "Submitted",                 color: "#3b82f6" },
  request_resubmitted:       { label: "Re-submitted",              color: "#3b82f6" },
  signing_link_generated:    { label: "Sent to Recipient",         color: "#8b5cf6" },
  rep_signature_submitted:   { label: "Recipient Signed",          color: "#10b981" },
  rep_signature_declined:    { label: "Recipient Declined",        color: "#ef4444" },
  request_approved:          { label: "Approved",                  color: "#10b981" },
  request_revision_required: (log) =>
    log.details?.newStatus === "agr_rep_revision_requested"
      ? { label: "Recipient Revision Requested", color: "#f59e0b" }
      : { label: "Student Revision Requested",   color: "#f59e0b" },
};

const resolveStep = (log) => {
  const entry = TIMELINE_STEPS[log.action];
  if (!entry) return null;
  return typeof entry === "function" ? entry(log) : entry;
};

export default function AdminReports() {
  const [requests, setRequests] = useState([]);
  const [loading, setLoading] = useState(true);
  const [refreshing, setRefreshing] = useState(false);
  const [error, setError] = useState("");

  const [timelineRequest, setTimelineRequest] = useState(null);
  const [timelineLogs, setTimelineLogs] = useState([]);
  const [timelineLoading, setTimelineLoading] = useState(false);

```

```
  // Search state
  const [searchTerm, setSearchTerm] = useState("");

  // Filter state
  const [typeFilter, setTypeFilter] = useState("all");
  const [statusFilter, setStatusFilter] = useState("");
  const [dateMode, setDateMode] = useState("preset");
  const [preset, setPreset] = useState("all");
  const [singleDate, setSingleDate] = useState("");
  const [startDate, setStartDate] = useState("");
  const [endDate, setEndDate] = useState("");

  // Filter panel open/close
  const [isFilterOpen, setIsFilterOpen] = useState(() =>
    typeof window === "undefined" ? true : window.innerWidth >= 768
  );

  // Pagination
  const [page, setPage] = useState(1);
  const PAGE_SIZE = 15;

  const buildDateParams = () => {
    const params = {};
    const now = new Date();
    if (dateMode === "preset") {
      if (preset === "today") {
        params.startDate = now.toISOString().slice(0, 10);
        params.endDate = now.toISOString().slice(0, 10);
      } else if (preset === "thisWeek") {
        const day = now.getDay();
        const monday = new Date(now);
        monday.setDate(now.getDate() - ((day + 6) % 7));
        params.startDate = monday.toISOString().slice(0, 10);
      } else if (preset === "thisMonth") {
        params.startDate = new Date(now.getFullYear(), now.getMonth(), 1).toISOString().slice(0, 10);
      } else if (preset === "thisYear") {
        params.startDate = new Date(now.getFullYear(), 0, 1).toISOString().slice(0, 10);
      }
    } else if (dateMode === "single" && singleDate) {
      params.startDate = singleDate;
      params.endDate = singleDate;
    } else if (dateMode === "range") {
      if (startDate) params.startDate = startDate;
      if (endDate) params.endDate = endDate;
    }
    return params;
  };

  const load = async ({ withToast = false } = {}) => {
    setLoading(true);
    if (withToast) setRefreshing(true);
    setError("");
    try {
      const data = await getAllRequests(buildDateParams());
      setRequests(Array.isArray(data) ? data : []);
    } catch (err) {
      const msg = err.response?.data?.message || err.message || "Failed to load data";
      setError(msg);
    } finally {
      setLoading(false);
      setRefreshing(false);
    }
  };

  useEffect(() => { load(); }, []);
  useEffect(() => { load(); setPage(1); }, [dateMode, preset, singleDate, startDate, endDate]); // eslint-disable-line react-

  useEffect(() => {
    const onResize = () => { if (window.innerWidth >= 768) setIsFilterOpen(true); };
    window.addEventListener("resize", onResize);
    return () => window.removeEventListener("resize", onResize);
  }, []);

  const clearSearch = () => { setSearchTerm(""); setPage(1); };

  const resetAndRefresh = () => {
```

```
    setSearchTerm("");
    setTypeFilter("all");
    setStatusFilter("");
    setDateMode("preset");
    setPreset("all");
    setSingleDate("");
    setStartDate("");
    setEndDate("");
    setPage(1);
    load({ withToast: true });
  };

  const filteredRequests = useMemo(() => {
    let result = [...requests];

    if (searchTerm) {
      const term = searchTerm.toLowerCase();
      result = result.filter((r) => {
        const name = (r.userId?.name || r.proxyRequestee?.fullName || "").toLowerCase();
        const email = (r.userId?.email || r.proxyRequestee?.email || "").toLowerCase();
        const id = (r._id || "").slice(-7).toLowerCase();
        return name.includes(term) || email.includes(term) || id.includes(term.toLowerCase());
      });
    }

    if (statusFilter) {
      const statuses = statusFilter.split(",");
      result = result.filter((r) => statuses.includes(r.status));
    }

    if (typeFilter !== "all") {
      if (typeFilter === "agreement") {
        result = result.filter((r) => r.type === "agreement");
      } else if (typeFilter === "nda_orgactivities") {
        result = result.filter((r) => r.type === "nda" && r.formData?.ndaType === "orgactivities");
      } else if (typeFilter === "nda_research") {
        result = result.filter((r) => r.type === "nda" && r.formData?.ndaType === "research");
      }
    }

    result.sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt));

    return result;
  }, [requests, searchTerm, statusFilter, typeFilter]);

  const totalPages = Math.max(1, Math.ceil(filteredRequests.length / PAGE_SIZE));
  const paginatedRequests = filteredRequests.slice((page - 1) * PAGE_SIZE, page * PAGE_SIZE);

  const openTimeline = async (r) => {
    setTimelineRequest(r);
    setTimelineLoading(true);
    setTimelineLogs([]);
    try {
      const data = await getAuditLogs({ resourceId: r._id, limit: 50 });
      const sorted = (data.logs || []).sort((a, b) => new Date(a.createdAt) - new Date(b.createdAt));
      setTimelineLogs(sorted);
    } catch { /* silent */ } finally {
      setTimelineLoading(false);
    }
  };


  return (
    <div className="page-shell">
      {error && (
        <div className="info-banner info-banner--danger" style={{ marginBottom: 12 }}>{error}</div>
      )}

      {/* ■■ Search + Filter toolbar ■■ */}
      <div className="req-toolbar">
        {/* Search row */}
        <div className="req-toolbar__search-row">
          <div className="req-toolbar__search-box">
            <input
              type="text"
              className="req-toolbar__search-input"
```

```
              placeholder="Search by requestor, email, or request ID…"
              value={searchTerm}
              onChange={(e) => { setSearchTerm(e.target.value); setPage(1); }}
            />
            {searchTerm ? (
              <button type="button" className="req-toolbar__search-clear" onClick={clearSearch} aria-label="Clear search">
                ×
              </button>
            ) : null}
            <span className="req-toolbar__search-inline-btn" style={{ pointerEvents: "none" }}>
              <Search size={14} />
            </span>
          </div>
          <button
            type="button"
            className="ui-btn ui-btn--secondary req-toolbar__filter-toggle"
            onClick={() => setIsFilterOpen((prev) => !prev)}
            aria-expanded={isFilterOpen}
          >
            <Filter size={13} />
            Filters
          </button>
        </div>

        {/* Filter chips row */}
        <div className={`req-toolbar__filters${isFilterOpen ? " req-toolbar__filters--open" : ""}`}>
          <div className="req-toolbar__filter-group">
            <label className="req-toolbar__filter-label">Type</label>
            <FilterSelect
              value={typeFilter}
              onChange={(v) => { setTypeFilter(v); setPage(1); }}
              className="filter-select--type"
              options={[
                { value: "all", label: "All Types" },
                { value: "nda_orgactivities", label: "NDA — Student Organization Activities" },
                { value: "nda_research", label: "NDA — Conduct of Research" },
                { value: "agreement", label: "Agreement" },
              ]}
            />
          </div>

          <div className="req-toolbar__filter-group">
            <label className="req-toolbar__filter-label">Status</label>
            <FilterSelect
              value={statusFilter}
              onChange={(v) => { setStatusFilter(v); setPage(1); }}
              className="filter-select--status"
              options={STATUS_FILTER_OPTIONS}
              defaultValue=""
            />
          </div>

          <div className="req-toolbar__filter-group">
            <label className="req-toolbar__filter-label">Date</label>
            <FilterSelect
              value={dateMode}
              onChange={setDateMode}
              options={[
                { value: "preset", label: "Preset" },
                { value: "single", label: "Specific Date" },
                { value: "range", label: "Date Range" },
              ]}
              defaultValue="preset"
            />
          </div>

          {dateMode === "preset" ? (
            <div className="req-toolbar__filter-group">
              <label className="req-toolbar__filter-label">Period</label>
              <FilterSelect
                value={preset}
                onChange={setPreset}
                options={[
                  { value: "all", label: "All Time" },
                  { value: "today", label: "Today" },
                  { value: "thisWeek", label: "This Week" },
```

```
                  { value: "thisMonth", label: "This Month" },
                  { value: "thisYear", label: "This Year" },
                ]}
              />
            </div>
          ) : null}

          {dateMode === "single" ? (
            <div className="req-toolbar__filter-group">
              <label className="req-toolbar__filter-label">Date</label>
              <input
                className={`req-toolbar__filter-select${singleDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
                type="date"
                value={singleDate}
                onChange={(e) => setSingleDate(e.target.value)}
              />
            </div>
          ) : null}

          {dateMode === "range" ? (
            <>
              <div className="req-toolbar__filter-group">
                <label className="req-toolbar__filter-label">From</label>
                <input
                  className={`req-toolbar__filter-select${startDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
                  type="date"
                  value={startDate}
                  onChange={(e) => setStartDate(e.target.value)}
                />
              </div>
              <div className="req-toolbar__filter-group">
                <label className="req-toolbar__filter-label">To</label>
                <input
                  className={`req-toolbar__filter-select${endDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
                  type="date"
                  value={endDate}
                  onChange={(e) => setEndDate(e.target.value)}
                />
              </div>
            </>
          ) : null}

          <button
            type="button"
            className="req-toolbar__reset-btn"
            onClick={resetAndRefresh}
            disabled={refreshing}
            title="Reset filters and refresh"
          >
            <RefreshCw size={14} className={refreshing ? "spin-anim" : ""} />
          </button>
        </div>
      </div>

      {/* Table */}
      <motion.div
        initial={{ opacity: 0, y: 10 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ delay: 0.1 }}
        className="dashboard-card"
      >
        <div className="table-scroll">
          <table className="dashboard-table">
            <thead>
              <tr className="dashboard-table-title-row">
                <th colSpan={6}>
                  <div className="dashboard-table-title-wrap">
                    <span className="dashboard-table-title">Transactions</span>
                  </div>
                </th>
              </tr>
              <tr>
                <th>Last Updated</th>
                <th>Requestor</th>
                <th>Request ID</th>
                <th>Type</th>
```

```
                <th>Status</th>
                <th />
              </tr>
            </thead>
            <tbody>
              {loading ? (
                [...Array(4)].map((_, i) => (
                  <tr key={i}>
                    <td><span className="skeleton-block skeleton-text" /></td>
                    <td><span className="skeleton-block skeleton-text" /></td>
                    <td><span className="skeleton-block skeleton-text" /></td>
                    <td><span className="skeleton-block skeleton-text" /></td>
                    <td><span className="skeleton-block skeleton-pill" /></td>
                    <td />
                  </tr>
                ))
              ) : paginatedRequests.length === 0 ? (
                <tr>
                  <td colSpan={6}>
                    <div className="dashboard-empty">
                      <p className="dashboard-empty-title">No transactions found</p>
                      <p className="dashboard-empty-text">No records match the current filters.</p>
                    </div>
                  </td>
                </tr>
              ) : (
                paginatedRequests.map((r) => {
                  const name = r.proxyRequestee?.isProxy
                    ? (`${r.proxyRequestee.firstName || ""} ${r.proxyRequestee.lastName || ""}`.trim() || r.proxyRequestee.fu
                    : (r.userId?.name || "—");
                  const lastUpdated = r.updatedAt || r.createdAt;
                  const dateStr = lastUpdated
                    ? `${new Date(lastUpdated).toLocaleDateString("en-PH", { year: "numeric", month: "short", day: "numeric"
                    : "—";
                  const typeLabel = r.type === "nda"
                    ? `NDA${r.formData?.ndaTypeLabel ? ` — ${r.formData.ndaTypeLabel}` : ""}`
                    : r.type === "agreement" ? "Agreement" : (r.type || "—");
                  return (
                    <tr key={r._id} onClick={() => openTimeline(r)} style={{ cursor: "pointer" }}>
                      <td style={{ whiteSpace: "nowrap", color: "var(--text-secondary)" }}>{dateStr}</td>
                      <td>
                        <span style={{ fontWeight: 600 }}>{name}</span>
                        {r.proxyRequestee?.isProxy && (
                          <span className="dashboard-subtext">Proxy (F2F)</span>
                        )}
                      </td>
                      <td><span style={{ color: "var(--text-secondary)" }}>{r._id?.slice(-7).toUpperCase()}</span></td>
                      <td>{typeLabel}</td>
                      <td><span className={statusPillClass(r.status)}>{prettyStatus(r.status)}</span></td>
                      <td className="row-caret">›</td>
                    </tr>
                  );
                })
              )}
            </tbody>
          </table>
        </div>

        {/* Pagination */}
        {totalPages > 1 && (
          <div style={{ display: "flex", alignItems: "center", justifyContent: "space-between", padding: "10px 16px", borderT
            <span style={{ fontSize: 12, color: "var(--text-muted)" }}>
              Page {page} of {totalPages}
            </span>
            <div style={{ display: "flex", gap: 6 }}>
              <button
                className="ui-btn ui-btn--secondary ui-btn--compact"
                onClick={() => setPage((p) => Math.max(1, p - 1))}
                disabled={page === 1}
              >
                ‹ Prev
              </button>
              <button
                className="ui-btn ui-btn--secondary ui-btn--compact"
                onClick={() => setPage((p) => Math.min(totalPages, p + 1))}
                disabled={page === totalPages}
```

```
              >
                Next ›
              </button>
            </div>
          </div>
        )}
      </motion.div>

      {/* ■■ Request Timeline Modal ■■ */}
      <AnimatePresence>
        {timelineRequest && (
          <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
            <motion.div className="req-timeline-modal" initial={{ y: 16, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{

              {/* Header */}
              <div className="req-timeline-modal__header">
                <div style={{ flex: 1, minWidth: 0 }}>
                  <div style={{ display: "flex", alignItems: "center", gap: 8, flexWrap: "wrap" }}>
                    <span className="req-timeline-modal__id">
                      #{timelineRequest._id?.slice(-7).toUpperCase()}
                    </span>
                    <span className={statusPillClass(timelineRequest.status)}>
                      {prettyStatus(timelineRequest.status)}
                    </span>
                  </div>
                  <div style={{ fontSize: 12, color: "var(--text-muted)", marginTop: 3 }}>
                    {timelineRequest.type === "nda"
                      ? `NDA${timelineRequest.formData?.ndaTypeLabel ? ` — ${timelineRequest.formData.ndaTypeLabel}` : ""}`
                      : "Agreement"}
                  </div>
                </div>
                <button
                  className="user-edit-modal__close user-edit-modal__close--danger"
                  onClick={() => setTimelineRequest(null)}
                  aria-label="Close"
                >
                  <X size={16} />
                </button>
              </div>

              {/* Body */}
              <div className="req-timeline-modal__body">
                {timelineLoading ? (
                  <div className="req-timeline-modal__loading">
                    {[...Array(4)].map((_, i) => (
                      <div key={i} className="req-timeline-modal__step">
                        <div className="req-timeline-modal__dot-col">
                          <span className="skeleton-block" style={{ width: 12, height: 12, borderRadius: "50%" }} />
                          {i < 3 && <div className="req-timeline-modal__connector" />}
                        </div>
                        <div style={{ paddingBottom: 20 }}>
                          <span className="skeleton-block skeleton-text" style={{ width: 140 }} />
                        </div>
                      </div>
                    ))}
                  </div>
                ) : timelineLogs.length === 0 ? (
                  <p style={{ color: "var(--text-muted)", fontSize: 13, textAlign: "center", padding: "24px 0" }}>
                    No timeline data available.
                  </p>
                ) : (
                  timelineLogs.map((log, i) => {
                    const step = resolveStep(log);
                    if (!step) return null;
                    const isLast = i === timelineLogs.length - 1;
                    const dateStr = new Date(log.createdAt).toLocaleDateString("en-PH", { year: "numeric", month: "short", da
                    const timeStr = new Date(log.createdAt).toLocaleTimeString("en-PH", { hour: "2-digit", minute: "2-digit"
                    return (
                      <div key={log._id} className="req-timeline-modal__step">
                        <div className="req-timeline-modal__dot-col">
                          <div className="req-timeline-modal__dot" style={{ background: step.color }} />
                          {!isLast && <div className="req-timeline-modal__connector" />}
                        </div>
                        <div className="req-timeline-modal__content" style={{ paddingBottom: isLast ? 4 : 20 }}>
                          <div className="req-timeline-modal__label">{step.label}</div>
                          <div className="req-timeline-modal__time">{dateStr} · {timeStr}</div>
```

```
                          {log.userId?.name && (
                            <div className="req-timeline-modal__who">{log.userId.name}</div>
                          )}
                        </div>
                      </div>
                    );
                  })
                )}
              </div>


            </motion.div>
          </motion.div>
        )}
      </AnimatePresence>
    </div>
  );
}
```

64. client/src/pages/admin/AdminAuditLog.jsx (490 lines)

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { motion } from "framer-motion";
import { ChevronLeft, ChevronRight, RefreshCw, Search } from "lucide-react";
import { getAuditLogs } from "../../services/auditService";
import FilterSelect from "../../components/FilterSelect";

const ACTION_COLORS = {
  CREATE: { bg: "#d1fae5", color: "#065f46" },
  UPDATE: { bg: "#dbeafe", color: "#1e3a8a" },
  DELETE: { bg: "#fee2e2", color: "#991b1b" },
  LOGIN: { bg: "#ede9fe", color: "#4c1d95" },
  LOGOUT: { bg: "#f1f5f9", color: "#475569" },
  ARCHIVE: { bg: "#fef9c3", color: "#78350f" },
  PASSWORD_RESET: { bg: "#ffedd5", color: "#9a3412" },
  REGISTER: { bg: "#d1fae5", color: "#065f46" },
  SIGNATURE: { bg: "#e0e7ff", color: "#3730a3" },
  DECLINED: { bg: "#fee2e2", color: "#991b1b" },
};

const ACTION_LABELS = {
  login: "Login",
  login_failed: "Login Failed",
  login_otp_sent: "Login OTP Sent",
  account_created: "Account Created",
  account_activated: "Account Activated",
  email_verified: "Email Verified",
  request_created: "Request Created",
  request_approved: "Request Approved",
  request_revision_required: "Revision Requested",
  request_resubmitted: "Request Resubmitted",
  password_changed: "Password Changed",
  password_reset_requested: "Password Reset Requested",
  password_reset_triggered_by_admin: "Password Reset (Admin)",
  user_updated: "User Updated",
  signing_link_generated: "Signing Link Generated",
  rep_signature_submitted: "Representative Signature Submitted",
  rep_signature_declined: "Representative Signature Declined",
};

const FIELD_LABELS = {
  role: "role",
  isActive: "active status",
  email: "email address",
  name: "name",
  firstName: "first name",
  lastName: "last name",
  middleInitial: "middle name",
};

const prettyAction = (action) =>
  ACTION_LABELS[action] || (action ? action.replace(/_/g, " ").replace(/\b\w/g, (c) => c.toUpperCase()) : "—");

const actionStyle = (action = "") => {
  // Direct match
  const upper = action.toUpperCase();
  if (ACTION_COLORS[upper]) return ACTION_COLORS[upper];
  // Category match
  if (upper.includes("LOGIN") || upper.includes("OTP")) return ACTION_COLORS.LOGIN;
  if (upper.includes("DECLINED") || upper.includes("FAILED") || upper.includes("DEACTIVATED")) return ACTION_COLORS.DECLINED;
  if (upper.includes("CREATED") || upper.includes("REGISTER") || upper.includes("ACTIVATED") || upper.includes("VERIFIED")) r
  if (upper.includes("APPROVED") || upper.includes("UPDATED") || upper.includes("RESUBMIT")) return ACTION_COLORS.UPDATE;
  if (upper.includes("PASSWORD") || upper.includes("RESET")) return ACTION_COLORS.PASSWORD_RESET;
  if (upper.includes("SIGN")) return ACTION_COLORS.SIGNATURE;
  if (upper.includes("ARCHIVE")) return ACTION_COLORS.ARCHIVE;
  return { bg: "#f1f5f9", color: "#64748b" };
};

const formatActor = (user) => {
  if (!user) return "System";
  const role = user.role ? user.role.charAt(0).toUpperCase() + user.role.slice(1) : "User";
  return `${role} ${user.name || user.email || "User"}`;
};

```

```
const LOGIN_FAIL_REASON = {
  user_not_found: "account does not exist",
  wrong_password: "incorrect password",
  account_inactive: "account is deactivated",
};

const PASSWORD_METHOD_LABEL = {
  profile: "via profile settings",
  reset: "via password reset",
};

const prettyType = (type) =>
  type === "nda" ? "NDA" : type === "agreement" ? "Agreement" : String(type || "document").toUpperCase();

const formatAuditDetails = (log) => {
  const actor = formatActor(log.userId);
  const details = log.details && typeof log.details === "object" ? log.details : {};

  switch (log.action) {
    case "login":
      return details.method === "otp"
        ? `${actor} signed in successfully via OTP verification.`
        : `${actor} signed in successfully.`;
    case "login_failed":
      return `${actor} failed to sign in — ${LOGIN_FAIL_REASON[details.reason] || "invalid credentials"}.`;
    case "login_otp_sent":
      return details.reason === "resend"
        ? `${actor} requested a new login OTP.`
        : `A login OTP was sent to ${actor}.`;
    case "verification_otp_sent":
      return `A verification OTP was sent to ${actor} before account access.`;
    case "account_created":
      if (details.createdBy === "admin") {
        return `${actor} created an account for ${details.targetEmail || "a user"} with role ${details.role || "student"}.`;
      }
      return `${actor} registered a new account.`;
    case "account_activated":
      return `${actor} activated their account.`;
    case "email_verified":
      return `${actor} verified their email address.`;
    case "request_created":
      return details.isProxy
        ? `${actor} submitted a ${prettyType(details.type)} request on behalf of a walk-in requestor.`
        : `${actor} submitted a ${prettyType(details.type)} request.`;
    case "request_resubmitted":
      return `${actor} resubmitted a ${prettyType(details.type)} request.`;
    case "request_approved":
      return `${actor} approved a ${prettyType(details.type)} request.`;
    case "request_revision_required":
      return `${actor} requested revisions on a ${prettyType(details.type)} request.`;
    case "signing_link_generated":
      return `${actor} generated a representative signing link for an ${prettyType(details.type)} request.`;
    case "rep_signature_submitted":
      return `Representative ${details.repName || "user"} submitted their e-signature.`;
    case "rep_signature_declined":
      return `The representative declined the signing request.`;
    case "password_changed":
      return `${actor} changed their password ${PASSWORD_METHOD_LABEL[details.method] || ""}.`.trim();
    case "password_reset_requested":
      return details.reason === "resend"
        ? `${actor} requested a new password reset OTP.`
        : `${actor} requested a password reset OTP.`;
    case "password_reset_triggered_by_admin":
      return `${actor} reset the password for ${details.targetEmail || "a user"}.`;
    case "user_updated": {
      const fields = Array.isArray(details.fields)
        ? details.fields.map((f) => FIELD_LABELS[f] || f.replace(/_/g, " ")).join(", ")
        : "account details";
      return `${actor} updated ${details.targetEmail ? `${details.targetEmail}'s` : "a user's"} ${fields}.`;
    }
    default:
      if (typeof log.details === "string" && log.details.trim()) return log.details;
      return "No additional details were recorded.";
  }
};

```

```
const PAGE_SIZE = 20;

export default function AdminAuditLog() {
  const [logs, setLogs] = useState([]);
  const [loading, setLoading] = useState(true);
  const [refreshing, setRefreshing] = useState(false);
  const [page, setPage] = useState(1);
  const [totalPages, setTotalPages] = useState(1);
  const [actionFilter, setActionFilter] = useState("");
  const [searchQuery, setSearchQuery] = useState("");
  const [error, setError] = useState("");
  const [isPillsOpen, setIsPillsOpen] = useState(false);
  const [dateMode, setDateMode] = useState("preset");
  const [preset, setPreset] = useState("all");
  const [singleDate, setSingleDate] = useState("");
  const [startDate, setStartDate] = useState("");
  const [endDate, setEndDate] = useState("");

  const load = useCallback(async (p = page, action = actionFilter, withToast = false) => {
    setLoading(true);
    if (withToast) setRefreshing(true);
    setError("");
    try {
      const res = await getAuditLogs({ page: p, limit: PAGE_SIZE, action: action || undefined });
      setLogs(res.logs || res);
      if (res.total) setTotalPages(Math.ceil(res.total / PAGE_SIZE));
      else if (res.totalPages) setTotalPages(res.totalPages);
    } catch (err) {
      const msg = err.response?.data?.message || err.message || "Failed to load audit logs";
      const status = err.response?.status;
      setError(`${msg}${status ? ` (HTTP ${status})` : ""}`);
      console.error("Audit log load error:", err);
    } finally {
      setLoading(false);
      setRefreshing(false);
    }
  }, [actionFilter, page]);

  useEffect(() => { load(page, actionFilter); }, [actionFilter, load, page]);

  const handleActionFilter = (v) => {
    setActionFilter(v);
    setPage(1);
  };

  const HIDDEN_ACTIONS = new Set(["user_activated", "user_deactivated", "user_deleted"]);

  const filtered = useMemo(() => {
    let result = logs.filter((l) => !HIDDEN_ACTIONS.has(l.action));

    if (dateMode === "preset" && preset !== "all") {
      const now = new Date();
      let from = null;
      if (preset === "today") {
        from = new Date(now.getFullYear(), now.getMonth(), now.getDate());
      } else if (preset === "thisWeek") {
        const day = now.getDay();
        from = new Date(now); from.setDate(now.getDate() - ((day + 6) % 7)); from.setHours(0,0,0,0);
      } else if (preset === "thisMonth") {
        from = new Date(now.getFullYear(), now.getMonth(), 1);
      } else if (preset === "thisYear") {
        from = new Date(now.getFullYear(), 0, 1);
      }
      if (from) result = result.filter((l) => new Date(l.createdAt) >= from);
    }
    if (dateMode === "single" && singleDate) {
      const d = new Date(singleDate);
      const next = new Date(d); next.setDate(d.getDate() + 1);
      result = result.filter((l) => { const t = new Date(l.createdAt); return t >= d && t < next; });
    }
    if (dateMode === "range") {
      if (startDate) result = result.filter((l) => new Date(l.createdAt) >= new Date(startDate));
      if (endDate) { const end = new Date(endDate); end.setDate(end.getDate() + 1); result = result.filter((l) => new Date(l.
    }

    if (searchQuery.trim()) {
```

```
      const q = searchQuery.toLowerCase();
      result = result.filter((l) => {
        const name = (l.userId?.name || "").toLowerCase();
        const email = (l.userId?.email || "").toLowerCase();
        const resource = (l.resourceType || "").toLowerCase();
        const resourceId = (l.resourceId ? String(l.resourceId).slice(-7).toLowerCase() : "");
        return name.includes(q) || email.includes(q) || resource.includes(q) || resourceId.includes(q);
      });
    }

    return result;
  }, [logs, dateMode, preset, singleDate, startDate, endDate, searchQuery]);

  const ACTION_TYPES = [
    { label: "All Actions", value: "" },
    { label: "Login", value: "login" },
    { label: "Login Failed", value: "login_failed" },
    { label: "Account Created", value: "account_created" },
    { label: "Request Created", value: "request_created" },
    { label: "Request Approved", value: "request_approved" },
    { label: "Revision Requested", value: "request_revision_required" },
    { label: "Request Resubmitted", value: "request_resubmitted" },
    { label: "Password Changed", value: "password_changed" },
    { label: "Password Reset", value: "password_reset_requested" },
    { label: "User Updated", value: "user_updated" },
    { label: "Email Verified", value: "email_verified" },
    { label: "Signing Link Generated", value: "signing_link_generated" },
    { label: "Representative Signature Submitted", value: "rep_signature_submitted" },
    { label: "Representative Signature Declined", value: "rep_signature_declined" },
  ];

  return (
    <div className="page-shell">
      {/* Error Banner */}
      {error && (
        <div className="info-banner info-banner--danger" style={{ marginBottom: 16 }}>
          <strong>{error}</strong>
        </div>
      )}

      {/* Toolbar */}
      <div className="req-toolbar" style={{ borderTop: "none" }}>
        {/* Search row */}
        <div className="req-toolbar__search-row">
          <div className="req-toolbar__search-box">
            <input
              type="text"
              className="req-toolbar__search-input"
              placeholder="Search by name or resource..."
              value={searchQuery}
              onChange={(e) => setSearchQuery(e.target.value)}
            />
            {searchQuery ? (
              <button type="button" className="req-toolbar__search-clear" onClick={() => setSearchQuery("")} aria-label="Clea
                ×
              </button>
            ) : null}
            <span className="req-toolbar__search-inline-btn" style={{ pointerEvents: "none" }}>
              <Search size={14} />
            </span>
          </div>
        </div>

        {/* Filters row: Action + date filters + refresh */}
        <div className="req-toolbar__filters">
          <div className="req-toolbar__filter-group">
            <label className="req-toolbar__filter-label">Action</label>
            <button
              type="button"
              className="req-toolbar__filter-btn"
              data-open={isPillsOpen}
              onClick={() => setIsPillsOpen((p) => !p)}
            >
              <span>{ACTION_TYPES.find((a) => a.value === actionFilter)?.label ?? "All Actions"}</span>
            </button>
          </div>
```

```
302
303           <div className="req-toolbar__filter-group">
304             <label className="req-toolbar__filter-label">Date</label>
305             <FilterSelect
306               value={dateMode}
307               onChange={setDateMode}
308               options={[
309                 { value: "preset", label: "Preset" },
310                 { value: "single", label: "Specific Date" },
311                 { value: "range", label: "Date Range" },
312               ]}
313               defaultValue="preset"
314             />
315           </div>
316
317           {dateMode === "preset" && (
318             <div className="req-toolbar__filter-group">
319               <label className="req-toolbar__filter-label">Period</label>
320               <FilterSelect
321                 value={preset}
322                 onChange={setPreset}
323                 options={[
324                   { value: "all", label: "All Time" },
325                   { value: "today", label: "Today" },
326                   { value: "thisWeek", label: "This Week" },
327                   { value: "thisMonth", label: "This Month" },
328                   { value: "thisYear", label: "This Year" },
329                 ]}
330               />
331             </div>
332           )}
333
334           {dateMode === "single" && (
335             <div className="req-toolbar__filter-group">
336               <label className="req-toolbar__filter-label">Date</label>
337               <input className={`req-toolbar__filter-select${singleDate !== "" ? " req-toolbar__filter-select--active" : ""}`
338             </div>
339           )}
340
341           {dateMode === "range" && (
342             <>
343               <div className="req-toolbar__filter-group">
344                 <label className="req-toolbar__filter-label">From</label>
345                 <input className={`req-toolbar__filter-select${startDate !== "" ? " req-toolbar__filter-select--active" : ""}
346               </div>
347               <div className="req-toolbar__filter-group">
348                 <label className="req-toolbar__filter-label">To</label>
349                 <input className={`req-toolbar__filter-select${endDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
350               </div>
351             </>
352           )}
353
354           <button
355             type="button"
356             className="req-toolbar__reset-btn"
357             onClick={() => { load(page, actionFilter, true); setPreset("all"); setDateMode("preset"); setSingleDate(""); setS
358             disabled={refreshing}
359             title="Reset filters and refresh"
360           >
361             <RefreshCw size={14} className={refreshing ? "spin-anim" : ""} />
362           </button>
363         </div>
364
365         {/* Row 3: Action pills */}
366         {isPillsOpen && (
367           <div className="req-toolbar__pills-row" style={{ borderTop: "1px solid var(--border)", paddingTop: "12px" }}>
368             {ACTION_TYPES.map((a) => (
369               <button
370                 key={a.value || "all"}
371                 type="button"
372                 onClick={() => handleActionFilter(a.value)}
373                 className={`req-toolbar__pill ${actionFilter === a.value ? "req-toolbar__pill--active" : ""}`}
374               >
375                 {a.label}
376               </button>
377             ))}
```

```
          </div>
        )}
      </div>

      {/* Table */}
      <div className="dashboard-card">
        <div className="table-scroll">
          <table className="dashboard-table">
            <thead>
              <tr className="dashboard-table-title-row">
                <th colSpan={5}>
                  <div className="dashboard-table-title-wrap">
                    <span className="dashboard-table-title">Logs</span>
                  </div>
                </th>
              </tr>
              <tr>
                <th>Timestamp</th>
                <th>Action</th>
                <th>User</th>
                <th>Resource</th>
                <th>Details</th>
              </tr>
            </thead>
            <tbody>
              {loading ? (
                [...Array(6)].map((_, i) => (
                  <tr key={`sk-${i}`}>
                    <td><span className="skeleton-block skeleton-text" /></td>
                    <td><span className="skeleton-block skeleton-pill" /></td>
                    <td><span className="skeleton-block skeleton-text" /></td>
                    <td><span className="skeleton-block skeleton-text" /></td>
                    <td><span className="skeleton-block skeleton-text" /></td>
                  </tr>
                ))
              ) : filtered.length === 0 ? (
                <tr>
                  <td colSpan={5}>
                    <div className="dashboard-empty">
                      <p className="dashboard-empty-title">No audit logs found</p>
                      <p className="dashboard-empty-text">No logs match the current filters.</p>
                    </div>
                  </td>
                </tr>
              ) : (
                filtered.map((log, i) => (
                  <motion.tr
                    key={log._id}
                    initial={{ opacity: 0 }} animate={{ opacity: 1 }} transition={{ delay: i * 0.015 }}
                  >
                    <td style={{ whiteSpace: "nowrap", color: "var(--text-secondary)", fontSize: 13 }}>
                      {log.createdAt
                        ? new Date(log.createdAt).toLocaleString("en-US", {
                            month: "short", day: "numeric", year: "numeric",
                            hour: "2-digit", minute: "2-digit",
                          })
                        : "—"}
                    </td>
                    <td style={{ fontWeight: 500 }}>{prettyAction(log.action)}</td>
                    <td>
                      {log.userId ? (
                        <>
                          <div style={{ fontWeight: 600, fontSize: 13 }}>{log.userId.name || "—"}</div>
                          <div style={{ fontSize: 11.5, color: "var(--text-muted)" }}>{log.userId.email || ""}</div>
                        </>
                      ) : (
                        <span style={{ fontStyle: "italic", color: "var(--text-muted)", fontSize: 13 }}>System</span>
                      )}
                    </td>
                    <td style={{ color: "var(--text-secondary)", fontSize: 13 }}>
                      {log.resourceId
                        ? <span>{String(log.resourceId).slice(-7).toUpperCase()}</span>
                        : (log.resourceType ? log.resourceType.charAt(0).toUpperCase() + log.resourceType.slice(1) : "—")}
                    </td>
                    <td style={{ color: "var(--text-secondary)", fontSize: 13, maxWidth: 260 }}>
                      <div style={{ overflow: "hidden", textOverflow: "ellipsis", whiteSpace: "nowrap" }} title={formatAuditD
```

```
                        {formatAuditDetails(log)}
                      </div>
                    </td>
                  </motion.tr>
                ))
              )}
            </tbody>
          </table>
        </div>
      </div>

      {/* Pagination */}
      {totalPages > 1 && (
        <div className="admin-pagination">
          <button
            onClick={() => setPage((p) => Math.max(1, p - 1))}
            disabled={page <= 1}
            className="admin-pagination-btn"
          >
            <ChevronLeft size={14} />
          </button>
          <span className="admin-pagination-text">
            Page {page} of {totalPages}
          </span>
          <button
            onClick={() => setPage((p) => Math.min(totalPages, p + 1))}
            disabled={page >= totalPages}
            className="admin-pagination-btn"
          >
            <ChevronRight size={14} />
          </button>
        </div>
      )}
    </div>
  );
}

```

65. client/src/pages/admin/AdminArchives.jsx (459 lines)

```
import { useContext, useEffect, useState, useMemo } from "react";
import { AnimatePresence, motion } from "framer-motion";
import { Search, Filter, RefreshCw, ShieldCheck, Clock, Archive, AlertTriangle, Play } from "lucide-react";
import { getArchivedRequests, getRetentionStats, runArchiveNow } from "../../services/requestService";
import { AuthContext } from "../../context/AuthContext";
import { notify } from "../../utils/inPageFeedback";
import FilterSelect from "../../components/FilterSelect";

const STATUS_LABEL = {
  nda_approved: "Approved",
  agr_approved: "Approved",
};

const prettyStatus = (s) =>
  STATUS_LABEL[s] || (s ? s.charAt(0).toUpperCase() + s.slice(1).replace(/_/g, " ") : "—");

export default function AdminArchives() {
  const { user } = useContext(AuthContext);
  const isAdmin = user?.role === "admin";
  const [requests, setRequests] = useState([]);
  const [loading, setLoading] = useState(true);
  const [refreshing, setRefreshing] = useState(false);
  const [retentionStats, setRetentionStats] = useState(null);
  const [archiving, setArchiving] = useState(false);
  const [showArchiveConfirm, setShowArchiveConfirm] = useState(false);

  const [searchInput, setSearchInput] = useState("");
  const [searchTerm, setSearchTerm] = useState("");
  const [typeFilter, setTypeFilter] = useState("all");
  const [statusFilter, setStatusFilter] = useState("all");
  const [dateMode, setDateMode] = useState("preset");
  const [preset, setPreset] = useState("all");
  const [singleDate, setSingleDate] = useState("");
  const [startDate, setStartDate] = useState("");
  const [endDate, setEndDate] = useState("");

  const [isFilterOpen, setIsFilterOpen] = useState(() =>
    typeof window === "undefined" ? true : window.innerWidth >= 768
  );

  const load = async ({ withToast = false } = {}) => {
    setLoading(true);
    if (withToast) setRefreshing(true);
    try {
      const [data, stats] = await Promise.all([
        getArchivedRequests(),
        isAdmin ? getRetentionStats() : Promise.resolve(null),
      ]);
      setRequests(data);
      if (stats) setRetentionStats(stats);
    } catch (err) {
      console.error(err);
    } finally {
      setLoading(false);
      setRefreshing(false);
    }
  };

  useEffect(() => { load(); }, []);

  const handleRunArchive = async () => {
    setShowArchiveConfirm(false);
    setArchiving(true);
    try {
      const result = await runArchiveNow();
      notify(result.message, { type: "success", title: "Archive Complete" });
      load();
    } catch (err) {
      notify(err.response?.data?.message || "Failed to run archive.", { type: "error" });
    } finally {
      setArchiving(false);
    }
  };
```

```
74
75    useEffect(() => {
76      const onResize = () => { if (window.innerWidth >= 768) setIsFilterOpen(true); };
77      window.addEventListener("resize", onResize);
78      return () => window.removeEventListener("resize", onResize);
79    }, []);
80
81    const applySearch = () => setSearchTerm(searchInput.trim());
82    const clearSearch = () => { setSearchInput(""); setSearchTerm(""); };
83
84    const resetAndRefresh = () => {
85      setSearchInput("");
86      setSearchTerm("");
87      setTypeFilter("all");
88      setStatusFilter("all");
89      setDateMode("preset");
90      setPreset("all");
91      setSingleDate("");
92      setStartDate("");
93      setEndDate("");
94      load({ withToast: true });
95    };
96
97    const filtered = useMemo(() => {
98      let result = requests;
99
100     if (searchTerm) {
101       const term = searchTerm.toLowerCase();
102       result = result.filter((r) => {
103         const name = (r.userId?.name || r.proxyRequestee?.fullName || "").toLowerCase();
104         const email = (r.userId?.email || r.proxyRequestee?.email || "").toLowerCase();
105         const id = (r._id || "").slice(-7).toLowerCase();
106         return name.includes(term) || email.includes(term) || id.includes(term);
107       });
108     }
109
110     if (typeFilter !== "all") {
111       if (typeFilter === "agreement") {
112         result = result.filter((r) => r.type === "agreement");
113       } else if (typeFilter === "nda_orgactivities") {
114         result = result.filter((r) => r.type === "nda" && r.formData?.ndaType === "orgactivities");
115       } else if (typeFilter === "nda_research") {
116         result = result.filter((r) => r.type === "nda" && r.formData?.ndaType === "research");
117       }
118     }
119
120     if (statusFilter !== "all") {
121       const statusMap = {
122         reviewal:      ["nda_pending", "agr_pending_1", "agr_pending_2"],
123         approved:      ["nda_approved", "agr_approved"],
124         stud_revision: ["stud_revision_requested"],
125         rep_revision:  ["agr_rep_revision_requested"],
126         rep_declined:  ["agr_declined"],
127         awaiting_rep:  ["agr_awaiting_rep_signature"],
128       };
129       const statuses = statusMap[statusFilter] || [];
130       result = result.filter((r) => statuses.includes(r.status));
131     }
132
133     if (dateMode === "preset") {
134       const now = new Date();
135       if (preset === "today") {
136         const dayStart = new Date(now.getFullYear(), now.getMonth(), now.getDate());
137         const dayEnd = new Date(dayStart.getTime() + 86400000 - 1);
138         result = result.filter((r) => { const d = new Date(r.createdAt); return d >= dayStart && d <= dayEnd; });
139       } else if (preset === "thisWeek") {
140         const day = now.getDay();
141         const monday = new Date(now);
142         monday.setDate(now.getDate() - ((day + 6) % 7));
143         monday.setHours(0, 0, 0, 0);
144         result = result.filter((r) => new Date(r.createdAt) >= monday);
145       } else if (preset === "thisMonth") {
146         const monthStart = new Date(now.getFullYear(), now.getMonth(), 1);
147         result = result.filter((r) => new Date(r.createdAt) >= monthStart);
148       } else if (preset === "thisYear") {
149         const yearStart = new Date(now.getFullYear(), 0, 1);
```

```
        result = result.filter((r) => new Date(r.createdAt) >= yearStart);
      }
    } else if (dateMode === "single" && singleDate) {
      const dayStart = new Date(singleDate);
      const dayEnd = new Date(singleDate + "T23:59:59");
      result = result.filter((r) => { const d = new Date(r.createdAt); return d >= dayStart && d <= dayEnd; });
    } else if (dateMode === "range") {
      if (startDate) result = result.filter((r) => new Date(r.createdAt) >= new Date(startDate));
      if (endDate) result = result.filter((r) => new Date(r.createdAt) <= new Date(endDate + "T23:59:59"));
    }

    return result;
  }, [requests, searchTerm, typeFilter, statusFilter, dateMode, preset, singleDate, startDate, endDate]);

  return (
    <div className="page-shell">

      {/* ■■ Retention Policy Panel (admin only) ■■ */}
      {isAdmin && retentionStats && (
        <div className="dashboard-card" style={{ marginBottom: 16, padding: "18px 20px" }}>
          <div style={{ display: "flex", alignItems: "center", gap: 8, marginBottom: 14 }}>
            <ShieldCheck size={18} strokeWidth={1.8} style={{ color: "var(--primary)" }} />
            <h3 style={{ margin: 0, fontSize: 15, fontWeight: 700 }}>Retention Policy</h3>
          </div>
          <div style={{ display: "grid", gridTemplateColumns: "repeat(auto-fit, minmax(160px, 1fr))", gap: 12, marginBottom:
            <div style={{ padding: "12px 14px", background: "var(--bg)", borderRadius: "var(--radius-md)", border: "1px solid
              <div style={{ fontSize: 11, color: "var(--text-muted)", fontWeight: 600, marginBottom: 4 }}>Retention Period</d
              <div style={{ fontSize: 18, fontWeight: 700 }}>{retentionStats.retentionYears} Years</div>
            </div>
            <div style={{ padding: "12px 14px", background: "var(--bg)", borderRadius: "var(--radius-md)", border: "1px solid
              <div style={{ fontSize: 11, color: "var(--text-muted)", fontWeight: 600, marginBottom: 4 }}>Active Requests</di
              <div style={{ fontSize: 18, fontWeight: 700 }}>{retentionStats.totalActive}</div>
            </div>
            <div style={{ padding: "12px 14px", background: "var(--bg)", borderRadius: "var(--radius-md)", border: "1px solid
              <div style={{ fontSize: 11, color: "var(--text-muted)", fontWeight: 600, marginBottom: 4 }}>Total Archived</div
              <div style={{ fontSize: 18, fontWeight: 700 }}>{retentionStats.totalArchived}</div>
            </div>
            <div style={{ padding: "12px 14px", background: "var(--bg)", borderRadius: "var(--radius-md)", border: "1px solid
              <div style={{ fontSize: 11, color: "var(--text-muted)", fontWeight: 600, marginBottom: 4 }}>Last Archived</div>
              <div style={{ fontSize: 14, fontWeight: 600 }}>
                {retentionStats.lastArchivedAt
                  ? new Date(retentionStats.lastArchivedAt).toLocaleDateString("en-US", { year: "numeric", month: "short", da
                  : "Never"}
              </div>
            </div>
          </div>

          {retentionStats.approachingRetention > 0 && (
            <div style={{ display: "flex", alignItems: "center", gap: 8, padding: "10px 12px", background: "var(--s-warning-b
              <AlertTriangle size={14} />
              <span><strong>{retentionStats.approachingRetention}</strong> request(s) approaching the {retentionStats.retenti
            </div>
          )}

          <div style={{ display: "flex", alignItems: "center", justifyContent: "space-between", flexWrap: "wrap", gap: 10 }}>
            <p style={{ margin: 0, fontSize: 12, color: "var(--text-muted)", maxWidth: 480 }}>
              Per institutional policy under RA 10173, requests older than {retentionStats.retentionYears} years are automati
              {retentionStats.eligibleForArchive > 0
                ? ` ${retentionStats.eligibleForArchive} request(s) currently eligible.`
                : " No requests currently eligible for archiving."}
            </p>
            <button
              type="button"
              className="ui-btn ui-btn--primary"
              style={{ fontSize: 13, gap: 6, display: "inline-flex", alignItems: "center" }}
              onClick={() => setShowArchiveConfirm(true)}
              disabled={archiving || retentionStats.eligibleForArchive === 0}
            >
              <Play size={13} />
              {archiving ? "Archiving…" : "Run Archive Now"}
            </button>
          </div>
        </div>
      )}

      {/* ■■ Toolbar ■■ */}
```

```
      <div className="req-toolbar">
        <div className="req-toolbar__search-row">
          <div className="req-toolbar__search-box">
            <input
              type="text"
              className="req-toolbar__search-input"
              placeholder="Search by requestor, email, or request ID…"
              value={searchInput}
              onChange={(e) => setSearchInput(e.target.value)}
              onKeyDown={(e) => { if (e.key === "Enter") applySearch(); }}
            />
            {searchInput ? (
              <button type="button" className="req-toolbar__search-clear" onClick={clearSearch} aria-label="Clear search">
                ×
              </button>
            ) : null}
            <button type="button" className="req-toolbar__search-inline-btn" onClick={applySearch} aria-label="Search">
              <Search size={14} />
            </button>
          </div>
          <button
            type="button"
            className="ui-btn ui-btn--secondary req-toolbar__filter-toggle"
            onClick={() => setIsFilterOpen((prev) => !prev)}
            aria-expanded={isFilterOpen}
          >
            <Filter size={13} />
            Filters
          </button>
        </div>

        <div className={`req-toolbar__filters${isFilterOpen ? " req-toolbar__filters--open" : ""}`}>
          <div className="req-toolbar__filter-group">
            <label className="req-toolbar__filter-label">Type</label>
            <FilterSelect
              value={typeFilter}
              onChange={setTypeFilter}
              className="filter-select--type"
              options={[
                { value: "all", label: "All Types" },
                { value: "nda_orgactivities", label: "NDA — Student Organization Activities" },
                { value: "nda_research", label: "NDA — Conduct of Research" },
                { value: "agreement", label: "Agreement" },
              ]}
            />
          </div>

          <div className="req-toolbar__filter-group">
            <label className="req-toolbar__filter-label">Status</label>
            <FilterSelect
              value={statusFilter}
              onChange={setStatusFilter}
              className="filter-select--status"
              options={[
                { value: "all", label: "All Statuses" },
                { value: "reviewal", label: "Reviewal" },
                { value: "approved", label: "Approved" },
                { value: "stud_revision", label: "Student Revisions" },
                { value: "rep_revision", label: "Recipient Revisions" },
                { value: "rep_declined", label: "Recipient Declined" },
                { value: "awaiting_rep", label: "Recipient Reviewal" },
              ]}
            />
          </div>

          <div className="req-toolbar__filter-group">
            <label className="req-toolbar__filter-label">Date</label>
            <FilterSelect
              value={dateMode}
              onChange={setDateMode}
              options={[
                { value: "preset", label: "Preset" },
                { value: "single", label: "Specific Date" },
                { value: "range", label: "Date Range" },
              ]}
              defaultValue="preset"
```

```
            />
          </div>

          {dateMode === "preset" ? (
            <div className="req-toolbar__filter-group">
              <label className="req-toolbar__filter-label">Period</label>
              <FilterSelect
                value={preset}
                onChange={setPreset}
                options={[
                  { value: "all", label: "All Time" },
                  { value: "today", label: "Today" },
                  { value: "thisWeek", label: "This Week" },
                  { value: "thisMonth", label: "This Month" },
                  { value: "thisYear", label: "This Year" },
                ]}
              />
            </div>
          ) : null}

          {dateMode === "single" ? (
            <div className="req-toolbar__filter-group">
              <label className="req-toolbar__filter-label">Date</label>
              <input className={`req-toolbar__filter-select${singleDate !== "" ? " req-toolbar__filter-select--active" : ""}`
            </div>
          ) : null}

          {dateMode === "range" ? (
            <>
              <div className="req-toolbar__filter-group">
                <label className="req-toolbar__filter-label">From</label>
                <input className={`req-toolbar__filter-select${startDate !== "" ? " req-toolbar__filter-select--active" : ""}
              </div>
              <div className="req-toolbar__filter-group">
                <label className="req-toolbar__filter-label">To</label>
                <input className={`req-toolbar__filter-select${endDate !== "" ? " req-toolbar__filter-select--active" : ""}`}
              </div>
            </>
          ) : null}

          <button
            type="button"
            className="req-toolbar__reset-btn"
            onClick={resetAndRefresh}
            disabled={refreshing}
            title="Reset filters and refresh"
          >
            <RefreshCw size={14} className={refreshing ? "spin-anim" : ""} />
          </button>
        </div>
      </div>

      {/* ■■ Table ■■ */}
      <div className="dashboard-card">
        <div className="table-scroll">
          <table className="dashboard-table">
            <thead>
              <tr className="dashboard-table-title-row">
                <th colSpan={6}>
                  <div className="dashboard-table-title-wrap">
                    <span className="dashboard-table-title">Archives</span>
                  </div>
                </th>
              </tr>
              <tr>
                <th>Request Date</th>
                <th>Archived Date</th>
                <th>Requestor</th>
                <th>Request ID</th>
                <th>Type</th>
                <th>Status</th>
              </tr>
            </thead>
            <tbody>
              {loading ? (
                [...Array(4)].map((_, i) => (
```

```
                  <tr key={`sk-${i}`}>
                    <td><span className="skeleton-block skeleton-text" /></td>
                    <td><span className="skeleton-block skeleton-text" /></td>
                    <td><span className="skeleton-block skeleton-text" /></td>
                    <td><span className="skeleton-block skeleton-text" /></td>
                    <td><span className="skeleton-block skeleton-pill" /></td>
                    <td><span className="skeleton-block skeleton-pill" /></td>
                  </tr>
                ))
              ) : filtered.length === 0 ? (
                <tr>
                  <td colSpan={6}>
                    <div className="dashboard-empty">
                      <p className="dashboard-empty-title">No archived requests found</p>
                      <p className="dashboard-empty-text">No archived requests match the current filters.</p>
                    </div>
                  </td>
                </tr>
              ) : (
                filtered.map((r, i) => {
                  const name = r.proxyRequestee?.isProxy
                    ? (`${r.proxyRequestee.firstName || ""} ${r.proxyRequestee.lastName || ""}`.trim() || r.proxyRequestee.fu
                    : (r.userId?.name || "—");
                  const email = r.proxyRequestee?.isProxy
                    ? (r.proxyRequestee.email || "")
                    : (r.userId?.email || "");
                  const typeLabel = r.type === "nda"
                    ? `NDA${r.formData?.ndaTypeLabel ? ` — ${r.formData.ndaTypeLabel}` : ""}`
                    : "Agreement";
                  return (
                    <motion.tr
                      key={r._id}
                      initial={{ opacity: 0 }} animate={{ opacity: 1 }} transition={{ delay: i * 0.02 }}
                    >
                      <td style={{ whiteSpace: "nowrap", color: "var(--text-secondary)" }}>
                        {r.createdAt ? new Date(r.createdAt).toLocaleDateString("en-US", { year: "numeric", month: "short", d
                      </td>
                      <td style={{ whiteSpace: "nowrap", color: "var(--text-secondary)" }}>
                        {r.archivedAt ? new Date(r.archivedAt).toLocaleDateString("en-US", { year: "numeric", month: "short",
                      </td>
                      <td>
                        <span style={{ fontWeight: 600 }}>{name}</span>
                        {email && <span className="dashboard-subtext">{email}</span>}
                        {r.proxyRequestee?.isProxy && (
                          <span className="dashboard-subtext">Proxy (F2F)</span>
                        )}
                      </td>
                      <td><span style={{ color: "var(--text-secondary)" }}>{r._id?.slice(-7).toUpperCase()}</span></td>
                      <td>{typeLabel}</td>
                      <td>
                        <span className="tag-pill tag-pill--neutral">Archived</span>
                      </td>
                    </motion.tr>
                  );
                })
              )}
            </tbody>
          </table>
        </div>
      </div>

      {/* ■■ Archive Confirmation Modal ■■ */}
      <AnimatePresence>
        {showArchiveConfirm && (
          <motion.div className="modal-overlay" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}>
            <motion.div className="modal-card" initial={{ y: 12, opacity: 0 }} animate={{ y: 0, opacity: 1 }} exit={{ y: 12,
              <h3 className="modal-title">Run Archive</h3>
              <p className="modal-text">
                This will archive <strong>{retentionStats?.eligibleForArchive || 0}</strong> request(s) that have exceeded
                the {retentionStats?.retentionYears || 5}-year retention period. This action cannot be undone from the UI.
              </p>
              <div className="modal-actions">
                <button className="ui-btn ui-btn--secondary" onClick={() => setShowArchiveConfirm(false)} disabled={archiving
                <button className="ui-btn ui-btn--primary" onClick={handleRunArchive} disabled={archiving}>{archiving ? "Arch
              </div>
            </motion.div>
```

```
454            </motion.div>
455          )}
456        </AnimatePresence>
457      </div>
458    );
459  }
```